

# Multi-Cloud Engineering Program

Manual integral del curso

24 partes · 288 clases · 1.288 horas

De fundamentos de computación a arquitectura de sistemas, AWS, Azure, Google Cloud, Kubernetes, IaC, datos e IA, seguridad, SRE, FinOps, operación y capstones por industria.

Contenido íntegro generado desde las fuentes versionadas del repositorio.

# Alcance e integridad

Este manual no es un catálogo. Incluye las guías pedagógicas centrales, los 24 módulos, las 288 lecciones completas y sus 288 evaluaciones. El laboratorio ejecutable y sus plantillas permanecen en el repositorio para conservar su carácter reproducible.

| Componente      | Cobertura                                   |
|-----------------|---------------------------------------------|
| Partes          | 24 de 24                                    |
| Lecciones       | 288 de 288                                  |
| Evaluaciones    | 288 de 288                                  |
| Horas estimadas | 1288                                        |
| Fuentes         | Markdown canónico + curriculum/catalog.json |

# Índice general

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>Alcance e integridad</b>                                                    | <b>2</b>  |
| <b>Índice general</b>                                                          | <b>3</b>  |
| <b>Guías pedagógicas</b>                                                       | <b>12</b> |
| Guía del estudiante . . . . .                                                  | 12        |
| Metodología docente . . . . .                                                  | 13        |
| Syllabus y cronograma . . . . .                                                | 15        |
| Rúbrica de evaluación . . . . .                                                | 17        |
| Especializaciones e industrias . . . . .                                       | 18        |
| Mapa de certificaciones . . . . .                                              | 19        |
| Bibliografía central y mapa de lectura . . . . .                               | 20        |
| <b>Parte 00 — Fundamentos de computación, redes y Linux</b>                    | <b>21</b> |
| 001 — Computación digital y modelo mental de la nube . . . . .                 | 23        |
| 002 — Terminal, sistema de archivos, procesos y variables de entorno . . . . . | 29        |
| 003 — Git, GitHub y trabajo reproducible . . . . .                             | 35        |
| 004 — Python, JSON y automatización mínima . . . . .                           | 41        |
| 005 — Redes por capas, TCP/IP, puertos y sockets . . . . .                     | 47        |
| 006 — DNS, HTTP, HTTPS y TLS de extremo a extremo . . . . .                    | 53        |
| 007 — Linux: usuarios, permisos, servicios y logs . . . . .                    | 59        |
| 008 — Virtualización, hipervisores e imágenes . . . . .                        | 65        |
| 009 — APIs REST, autenticación y contratos . . . . .                           | 72        |
| 010 — Responsabilidad compartida y pensamiento de riesgo . . . . .             | 78        |
| 011 — Costo, energía, capacidad y medición básica . . . . .                    | 85        |
| 012 — Proyecto: servicio local reproducible y observable . . . . .             | 91        |
| <b>Parte 01 — Principios, estrategia y adopción cloud</b>                      | <b>98</b> |
| 013 — Definición NIST y características esenciales de cloud . . . . .          | 100       |
| 014 — Regiones, zonas de disponibilidad, puntos de presencia y edge . . . . .  | 104       |
| 015 — IaaS, PaaS, SaaS, CaaS y FaaS . . . . .                                  | 108       |
| 016 — Elasticidad, escalabilidad, disponibilidad y resiliencia . . . . .       | 112       |
| 017 — Tenancy, cuentas, suscripciones, proyectos y jerarquías . . . . .        | 116       |
| 018 — Identidad, roles, políticas y federación . . . . .                       | 120       |
| 019 — Modelo de responsabilidad compartida por servicio . . . . .              | 124       |
| 020 — TCO, costos variables, unit economics y FinOps . . . . .                 | 128       |
| 021 — Well-Architected y atributos de calidad . . . . .                        | 132       |
| 022 — Cloud Adoption Framework y modelo operativo . . . . .                    | 136       |
| 023 — Descubrimiento y clasificación de workloads . . . . .                    | 140       |
| 024 — Proyecto: decisión de migración sustentada con ADR . . . . .             | 144       |

|                                                                             |                |
|-----------------------------------------------------------------------------|----------------|
| <b>Parte 02 — AWS: plataforma esencial</b>                                  | <b>148</b>     |
| 025 — Organizations, cuentas, OU, SCP y landing zone . . . . .              | 150            |
| 026 — IAM, roles, políticas, STS y federación . . . . .                     | 154            |
| 027 — VPC, subredes, rutas, NAT, endpoints y seguridad . . . . .            | 158            |
| 028 — EC2, AMI, EBS y selección de capacidad . . . . .                      | 162            |
| 029 — Elastic Load Balancing y Auto Scaling . . . . .                       | 166            |
| 030 — S3: objetos, versionado, lifecycle y replicación . . . . .            | 170            |
| 031 — RDS, DynamoDB y ElastiCache: decisión de datos . . . . .              | 174            |
| 032 — Lambda, API Gateway y Step Functions . . . . .                        | 178            |
| 033 — SQS, SNS y EventBridge . . . . .                                      | 182            |
| 034 — CloudWatch, CloudTrail, Config y Systems Manager . . . . .            | 186            |
| 035 — KMS, Secrets Manager, WAF y controles de seguridad . . . . .          | 190            |
| 036 — Proyecto: aplicación de tres capas en AWS . . . . .                   | 194            |
| <br><b>Parte 03 — Azure: plataforma esencial</b>                            | <br><b>198</b> |
| 037 — Tenant, management groups, suscripciones y resource groups . . . . .  | 200            |
| 038 — Microsoft Entra ID, RBAC, managed identities y PIM . . . . .          | 204            |
| 039 — Virtual Network, subredes, NSG, UDR, peering y Private Link . . . . . | 208            |
| 040 — Virtual Machines, Scale Sets y Load Balancer . . . . .                | 212            |
| 041 — Blob Storage, Files, redundancia y lifecycle . . . . .                | 216            |
| 042 — Azure SQL, Cosmos DB y Azure Cache for Redis . . . . .                | 220            |
| 043 — App Service, Functions y Container Apps . . . . .                     | 224            |
| 044 — Service Bus, Event Grid y Event Hubs . . . . .                        | 228            |
| 045 — Azure Monitor, Log Analytics y Application Insights . . . . .         | 232            |
| 046 — Key Vault, Defender for Cloud y Azure Policy . . . . .                | 236            |
| 047 — Bicep, plantillas y despliegues por alcance . . . . .                 | 240            |
| 048 — Proyecto: aplicación de tres capas en Azure . . . . .                 | 244            |
| <br><b>Parte 04 — Google Cloud: plataforma esencial</b>                     | <br><b>248</b> |
| 049 — Organización, folders, proyectos, billing y cuotas . . . . .          | 250            |
| 050 — IAM, service accounts y Workload Identity Federation . . . . .        | 254            |
| 051 — VPC global, subredes regionales, firewall y Cloud NAT . . . . .       | 258            |
| 052 — Compute Engine, managed instance groups y load balancing . . . . .    | 262            |
| 053 — Cloud Storage, clases, lifecycle y replicación . . . . .              | 266            |
| 054 — Cloud SQL, Spanner, Firestore y Memorystore . . . . .                 | 270            |
| 055 — Cloud Run, Cloud Functions y API Gateway . . . . .                    | 274            |
| 056 — Pub/Sub, Cloud Tasks y Workflows . . . . .                            | 278            |
| 057 — Cloud Logging, Monitoring, Trace y Audit Logs . . . . .               | 282            |
| 058 — Cloud KMS, Secret Manager y Security Command Center . . . . .         | 286            |
| 059 — Terraform y despliegues reproducibles en GCP . . . . .                | 290            |
| 060 — Proyecto: aplicación de tres capas en Google Cloud . . . . .          | 294            |
| <br><b>Parte 05 — Contenedores, Docker y OCI</b>                            | <br><b>298</b> |

|                                                                    |     |
|--------------------------------------------------------------------|-----|
| 061 — Imágenes, capas, registros y estándar OCI . . . . .          | 300 |
| 062 — Dockerfile reproducible y builds multi-stage . . . . .       | 304 |
| 063 — Namespaces, cgroups y runtime de contenedores . . . . .      | 308 |
| 064 — Volúmenes, bind mounts y persistencia . . . . .              | 312 |
| 065 — Redes bridge, DNS interno y publicación de puertos . . . . . | 316 |
| 066 — Docker Compose y aplicaciones multiservicio . . . . .        | 320 |
| 067 — Registros, SBOM, firma y procedencia de imágenes . . . . .   | 324 |
| 068 — Límites, health checks y apagado ordenado . . . . .          | 328 |
| 069 — Rootless, capabilities, seccomp y secretos . . . . .         | 332 |
| 070 — Diagnóstico de CPU, memoria, red y filesystem . . . . .      | 336 |
| 071 — Migración de una aplicación legacy a contenedores . . . . .  | 340 |
| 072 — Proyecto: stack OCI endurecido y observable . . . . .        | 344 |

## **Parte 06 — Kubernetes y plataformas administradas 348**

|                                                                    |     |
|--------------------------------------------------------------------|-----|
| 073 — API server, etcd, scheduler, controllers y kubelet . . . . . | 350 |
| 074 — Pods, ReplicaSets, Deployments y Jobs . . . . .              | 354 |
| 075 — Services, DNS, Ingress y Gateway API . . . . .               | 358 |
| 076 — ConfigMaps, Secrets y configuración externa . . . . .        | 362 |
| 077 — Volumes, PersistentVolumes, CSI y StatefulSets . . . . .     | 366 |
| 078 — Requests, limits, scheduling y autoscaling . . . . .         | 370 |
| 079 — Probes, rollouts, rollback y PodDisruptionBudget . . . . .   | 374 |
| 080 — Namespaces, RBAC, NetworkPolicy y admission . . . . .        | 378 |
| 081 — Helm, Kustomize y gestión de paquetes . . . . .              | 382 |
| 082 — Logs, métricas, eventos y depuración . . . . .               | 386 |
| 083 — EKS, AKS y GKE: similitudes y diferencias . . . . .          | 390 |
| 084 — Proyecto: plataforma Kubernetes portable . . . . .           | 394 |

## **Parte 07 — Infraestructura como código y configuración 398**

|                                                                      |     |
|----------------------------------------------------------------------|-----|
| 085 — Declarativo, imperativo, idempotencia y convergencia . . . . . | 400 |
| 086 — Terraform: HCL, providers, resources y grafo . . . . .         | 404 |
| 087 — Estado remoto, locking, cifrado y recuperación . . . . .       | 408 |
| 088 — Módulos, contratos, versiones y composición . . . . .          | 412 |
| 089 — Variables, outputs, locals y data sources . . . . .            | 416 |
| 090 — Plan, apply, drift, import y refactor con moved . . . . .      | 420 |
| 091 — Validación, lint, pruebas y policy as code . . . . .           | 424 |
| 092 — Secretos y datos sensibles en IaC . . . . .                    | 428 |
| 093 — CloudFormation, Bicep, Pulumi y Terraform . . . . .            | 432 |
| 094 — Ansible e imagen dorada para configuración . . . . .           | 436 |
| 095 — Plantillas, golden paths y catálogo interno . . . . .          | 440 |
| 096 — Proyecto: infraestructura multiambiente promovible . . . . .   | 444 |

## **Parte 08 — Entrega continua y platform engineering 448**

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| 097 — Integración continua, trunk-based development y feedback . . . . . | 450 |
|--------------------------------------------------------------------------|-----|

|                                                                      |     |
|----------------------------------------------------------------------|-----|
| 098 — GitHub Actions: workflows, runners, permisos y caché . . . . . | 454 |
| 099 — Artefactos inmutables, semver y promoción . . . . .            | 458 |
| 100 — Pruebas, calidad y puertas de cambio . . . . .                 | 462 |
| 101 — SAST, SCA, secretos, SBOM y firma en pipeline . . . . .        | 466 |
| 102 — Rolling, blue-green, canary y rollback . . . . .               | 470 |
| 103 — GitOps con Argo CD o Flux . . . . .                            | 474 |
| 104 — Ambientes efímeros y promoción entre entornos . . . . .        | 478 |
| 105 — Feature flags y separación deploy-release . . . . .            | 482 |
| 106 — Platform engineering e Internal Developer Platform . . . . .   | 486 |
| 107 — Developer experience, DORA y carga cognitiva . . . . .         | 490 |
| 108 — Proyecto: fábrica de software multi-cloud . . . . .            | 494 |

## **Parte 09 — Datos, mensajería, serverless e integración 498**

|                                                                   |     |
|-------------------------------------------------------------------|-----|
| 109 — Bases relacionales administradas y pooling . . . . .        | 500 |
| 110 — NoSQL: clave-valor, documento, columnar y grafo . . . . .   | 504 |
| 111 — Caché, invalidación, TTL y consistencia . . . . .           | 508 |
| 112 — Object storage, data lake y formatos columnares . . . . .   | 512 |
| 113 — Colas, entrega, reintentos y dead-letter queues . . . . .   | 516 |
| 114 — Pub/sub, streams, particiones y orden . . . . .             | 520 |
| 115 — Arquitectura dirigida por eventos y contratos . . . . .     | 524 |
| 116 — Sagas, outbox, idempotencia y deduplicación . . . . .       | 528 |
| 117 — Serverless: límites, cold starts y concurrencia . . . . .   | 532 |
| 118 — API management, cuotas, versiones y monetización . . . . .  | 536 |
| 119 — Workflows y orquestación durable . . . . .                  | 540 |
| 120 — Proyecto: pipeline de pedidos orientado a eventos . . . . . | 544 |

## **Parte 10 — Observabilidad, SRE y confiabilidad 548**

|                                                                        |     |
|------------------------------------------------------------------------|-----|
| 121 — Logs, métricas, trazas y eventos como señales . . . . .          | 550 |
| 122 — Logging estructurado, correlación y retención . . . . .          | 554 |
| 123 — Métricas, cardinalidad y modelos RED y USE . . . . .             | 558 |
| 124 — Tracing distribuido y OpenTelemetry . . . . .                    | 562 |
| 125 — Dashboards, alertas accionables y fatiga . . . . .               | 566 |
| 126 — SLI, SLO, SLA y presupuesto de error . . . . .                   | 570 |
| 127 — Incidentes, severidad, comando y comunicación . . . . .          | 574 |
| 128 — Runbooks, playbooks y automatización operativa . . . . .         | 578 |
| 129 — Capacidad, rendimiento y pruebas de carga . . . . .              | 582 |
| 130 — Timeouts, retries, backoff, circuit breaker y bulkhead . . . . . | 586 |
| 131 — Chaos engineering y game days . . . . .                          | 590 |
| 132 — Proyecto: operación SRE de CloudShop . . . . .                   | 594 |

## **Parte 11 — Seguridad, gobierno, cumplimiento y FinOps 598**

|                                                                              |     |
|------------------------------------------------------------------------------|-----|
| 133 — Zero Trust y defensa en profundidad . . . . .                          | 600 |
| 134 — Mínimo privilegio, acceso temporal y separación de funciones . . . . . | 604 |

|                                                                   |     |
|-------------------------------------------------------------------|-----|
| 135 — Segmentación, perímetro, WAF, DDoS y egress . . . . .       | 608 |
| 136 — Cifrado, KMS, HSM, rotación y envelope encryption . . . . . | 612 |
| 137 — Gestión de secretos y credenciales de workloads . . . . .   | 616 |
| 138 — Vulnerabilidades, imágenes y cadena de suministro . . . . . | 620 |
| 139 — CSPM, postura, policy as code y remediación . . . . .       | 624 |
| 140 — Threat modeling con STRIDE y attack paths . . . . .         | 628 |
| 141 — Cumplimiento, residencia, privacidad y evidencia . . . . .  | 632 |
| 142 — FinOps: showback, chargeback, budgets y anomalías . . . . . | 636 |
| 143 — Optimización de costo, capacidad y sostenibilidad . . . . . | 640 |
| 144 — Proyecto: landing zone con guardrails . . . . .             | 644 |

## **Parte 12 — Arquitectura cloud-native y sistemas distribuidos 648**

|                                                                  |     |
|------------------------------------------------------------------|-----|
| 145 — Requisitos, restricciones y atributos de calidad . . . . . | 650 |
| 146 — Twelve-Factor App y configuración cloud-native . . . . .   | 654 |
| 147 — DDD, bounded contexts y ownership de datos . . . . .       | 658 |
| 148 — Monolito modular, microservicios y función . . . . .       | 662 |
| 149 — CAP, PACELC y consistencia por operación . . . . .         | 666 |
| 150 — Replicación, particionado y consenso . . . . .             | 670 |
| 151 — Fallos parciales y patrones de resiliencia . . . . .       | 674 |
| 152 — Service discovery, malla y comunicación . . . . .          | 678 |
| 153 — Contratos API, compatibilidad y evolución . . . . .        | 682 |
| 154 — Multi-tenancy, aislamiento y noisy neighbor . . . . .      | 686 |
| 155 — Rendimiento, costo, seguridad y operabilidad . . . . .     | 690 |
| 156 — Proyecto: revisión de arquitectura con ADR . . . . .       | 694 |

## **Parte 13 — Multi-cloud, híbrido, migración y recuperación 698**

|                                                                    |     |
|--------------------------------------------------------------------|-----|
| 157 — Motivaciones y anti-patrones de multi-cloud . . . . .        | 700 |
| 158 — Portabilidad, capas de abstracción y lock-in . . . . .       | 704 |
| 159 — Federación de identidad entre nubes . . . . .                | 708 |
| 160 — Conectividad, tránsito, DNS y service discovery . . . . .    | 712 |
| 161 — Replicación de datos, soberanía y costos de egress . . . . . | 716 |
| 162 — Observabilidad y operación entre proveedores . . . . .       | 720 |
| 163 — Terraform multi-provider y separación de estados . . . . .   | 724 |
| 164 — Flotas Kubernetes y políticas comunes . . . . .              | 728 |
| 165 — Nube híbrida, edge y conectividad privada . . . . .          | 732 |
| 166 — Backup, RTO, RPO y patrones de disaster recovery . . . . .   | 736 |
| 167 — Las 7R de migración y oleadas . . . . .                      | 740 |
| 168 — Proyecto: continuidad activa-pasiva entre nubes . . . . .    | 744 |

## **Parte 14 — Plataformas avanzadas, capstones y carrera 748**

|                                                                  |     |
|------------------------------------------------------------------|-----|
| 169 — Landing zones empresariales y vending de cuentas . . . . . | 750 |
| 170 — Gobierno federado y policy as code a escala . . . . .      | 754 |
| 171 — Platform as a Product y roadmap de capacidades . . . . .   | 758 |

|                                                                  |     |
|------------------------------------------------------------------|-----|
| 172 — Modelo operativo FinOps y economía unitaria . . . . .      | 762 |
| 173 — Madurez SRE y confiabilidad organizacional . . . . .       | 766 |
| 174 — Arquitectura de seguridad cloud empresarial . . . . .      | 770 |
| 175 — Workloads de IA, GPU, datos y MLOps multi-cloud . . . . .  | 774 |
| 176 — Edge, IoT y procesamiento desconectado . . . . .           | 778 |
| 177 — Soberanía digital y confidential computing . . . . .       | 782 |
| 178 — Capstone: descubrimiento y diseño . . . . .                | 786 |
| 179 — Capstone: implementación y operación . . . . .             | 790 |
| 180 — Capstone: defensa, portafolio y plan profesional . . . . . | 794 |

## **Parte 15 — Arquitectura de sistemas e ingeniería de requisitos 798**

|                                                                              |     |
|------------------------------------------------------------------------------|-----|
| 181 — Requisitos funcionales, restricciones y atributos de calidad . . . . . | 800 |
| 182 — Contexto, contenedores, componentes y código con C4 . . . . .          | 804 |
| 183 — Acoplamiento, cohesión, modularidad y fronteras . . . . .              | 808 |
| 184 — Arquitectura monolítica, modular y de microservicios . . . . .         | 812 |
| 185 — Disponibilidad, confiabilidad y análisis de puntos de fallo . . . . .  | 816 |
| 186 — Capacidad, latencia, throughput y teoría de colas . . . . .            | 820 |
| 187 — Consistencia, particiones, relojes y consenso . . . . .                | 824 |
| 188 — Contratos de API, eventos y compatibilidad evolutiva . . . . .         | 828 |
| 189 — Modelado de amenazas y arquitectura de confianza cero . . . . .        | 832 |
| 190 — ADRs, fitness functions y gobierno de decisiones . . . . .             | 836 |
| 191 — Architecture review y comunicación con stakeholders . . . . .          | 840 |
| 192 — Proyecto: arquitectura completa de CloudShop . . . . .                 | 844 |

## **Parte 16 — Redes cloud avanzadas, conectividad híbrida y edge 848**

|                                                                            |     |
|----------------------------------------------------------------------------|-----|
| 193 — CIDR, subnetting y planificación IP a escala . . . . .               | 850 |
| 194 — Routing, BGP, tránsito y propagación de rutas . . . . .              | 854 |
| 195 — DNS autoritativo, recursivo, split-horizon y DNSSEC . . . . .        | 858 |
| 196 — Balanceo L4/L7, proxies, TLS y gestión de certificados . . . . .     | 862 |
| 197 — CDN, caché, origin shielding y edge compute . . . . .                | 866 |
| 198 — VPN, Direct Connect, ExpressRoute e Interconnect . . . . .           | 870 |
| 199 — Transit Gateway, Virtual WAN y Network Connectivity Center . . . . . | 874 |
| 200 — Private endpoints, service networking y egress control . . . . .     | 878 |
| 201 — Service mesh, mTLS y gestión de tráfico este-oeste . . . . .         | 882 |
| 202 — eBPF, flow logs, packet capture y diagnóstico . . . . .              | 886 |
| 203 — SD-WAN, 5G, IoT y operación desconectada . . . . .                   | 890 |
| 204 — Proyecto: red multi-región y multi-cloud . . . . .                   | 894 |

## **Parte 17 — AWS: arquitectura, automatización y operación en producción 898**

|                                                                       |     |
|-----------------------------------------------------------------------|-----|
| 205 — Hosting progresivo con Amplify, S3 y CloudFront . . . . .       | 900 |
| 206 — OIDC de GitHub y GitLab hacia AWS sin secretos . . . . .        | 904 |
| 207 — SAM, Lambda, API Gateway y despliegue serverless . . . . .      | 908 |
| 208 — DynamoDB por patrones de acceso y single-table design . . . . . | 912 |



|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| 209 — Cognito, JWT authorizers, WAF y defensa en profundidad . . . . .   | 916 |
| 210 — EventBridge, SQS, DLQ, replay e idempotencia . . . . .             | 920 |
| 211 — CloudWatch, X-Ray y observabilidad como código . . . . .           | 924 |
| 212 — ECR, ECS Fargate, ALB y autoscaling . . . . .                      | 928 |
| 213 — EKS, IRSA, GitOps y operación de clúster . . . . .                 | 932 |
| 214 — Budgets, Cost Explorer, etiquetado y FinOps automatizado . . . . . | 936 |
| 215 — Multi-región, Route 53, failover y game day . . . . .              | 940 |
| 216 — Proyecto: CloudShop productivo en AWS . . . . .                    | 944 |

## **Parte 18 — Azure: arquitectura empresarial y operación en producción 948**

|                                                                       |     |
|-----------------------------------------------------------------------|-----|
| 217 — Enterprise-scale landing zones y management groups . . . . .    | 950 |
| 218 — Entra ID, workload identity, PIM y Conditional Access . . . . . | 954 |
| 219 — Hub-spoke, Virtual WAN, Private Link y DNS privado . . . . .    | 958 |
| 220 — Bicep, deployment stacks y Azure Verified Modules . . . . .     | 962 |
| 221 — App Service, Functions y Container Apps en producción . . . . . | 966 |
| 222 — AKS, workload identity, ingress y GitOps . . . . .              | 970 |
| 223 — Azure SQL, Cosmos DB y consistencia distribuida . . . . .       | 974 |
| 224 — Service Bus, Event Grid y Event Hubs . . . . .                  | 978 |
| 225 — Azure Monitor, Application Insights y OpenTelemetry . . . . .   | 982 |
| 226 — Defender for Cloud, Policy y Sentinel . . . . .                 | 986 |
| 227 — Cost Management, Advisor, resiliencia y Chaos Studio . . . . .  | 990 |
| 228 — Proyecto: CloudShop productivo en Azure . . . . .               | 994 |

## **Parte 19 — Google Cloud: arquitectura de datos y operación en producción 998**

|                                                                       |      |
|-----------------------------------------------------------------------|------|
| 229 — Resource Manager, folders, Shared VPC y guardrails . . . . .    | 1000 |
| 230 — Workload Identity Federation, IAM Conditions y PAM . . . . .    | 1004 |
| 231 — Red global, load balancing, PSC y Cloud DNS . . . . .           | 1008 |
| 232 — Terraform, Infrastructure Manager y policy validation . . . . . | 1012 |
| 233 — Cloud Run, Functions, API Gateway y Workflows . . . . .         | 1016 |
| 234 — GKE Autopilot, Workload Identity y Config Sync . . . . .        | 1020 |
| 235 — Cloud SQL, Spanner, Firestore y Bigtable . . . . .              | 1024 |
| 236 — BigQuery, Dataflow, Dataproc y gobernanza de datos . . . . .    | 1028 |
| 237 — Pub/Sub, Eventarc y entrega exactamente-una-vez . . . . .       | 1032 |
| 238 — Cloud Operations, Trace y OpenTelemetry . . . . .               | 1036 |
| 239 — SCC, VPC Service Controls, KMS y FinOps . . . . .               | 1040 |
| 240 — Proyecto: CloudShop productivo en Google Cloud . . . . .        | 1044 |

## **Parte 20 — Plataformas cloud de datos, analítica, IA y agentes 1048**

|                                                                          |      |
|--------------------------------------------------------------------------|------|
| 241 — Lakehouse, warehouse, mesh y contratos de datos . . . . .          | 1050 |
| 242 — Ingesta batch, CDC y streaming . . . . .                           | 1054 |
| 243 — Orquestación, calidad, lineage y observabilidad de datos . . . . . | 1058 |
| 244 — Feature stores, training pipelines y experiment tracking . . . . . | 1062 |
| 245 — Serving online, batch inference y escalado de modelos . . . . .    | 1066 |

|                                                                      |      |
|----------------------------------------------------------------------|------|
| 246 — MLOps, registro, promoción, drift y rollback . . . . .         | 1070 |
| 247 — Modelos fundacionales, tokens, embeddings y RAG . . . . .      | 1074 |
| 248 — Bedrock, Azure AI Foundry y Vertex AI . . . . .                | 1078 |
| 249 — Agentes, tools, memoria, permisos y guardrails . . . . .       | 1082 |
| 250 — Evaluación de IA, red teaming y observabilidad . . . . .       | 1086 |
| 251 — Privacidad, gobernanza, sostenibilidad y costo de IA . . . . . | 1090 |
| 252 — Proyecto: asistente operativo de CloudShop . . . . .           | 1094 |

## **Parte 21 — Operación cloud, automatización y respuesta a incidentes 1098**

|                                                                       |      |
|-----------------------------------------------------------------------|------|
| 253 — Inventario, etiquetado, CMDB y ownership . . . . .              | 1100 |
| 254 — Patching, imágenes doradas y gestión de configuración . . . . . | 1104 |
| 255 — Backups, restore testing, vaults e inmutabilidad . . . . .      | 1108 |
| 256 — Administración remota sin SSH permanente . . . . .              | 1112 |
| 257 — Alertas, on-call, escalamiento y comunicación . . . . .         | 1116 |
| 258 — Triage de red, cómputo, datos y dependencias . . . . .          | 1120 |
| 259 — Runbooks ejecutables y auto-remediation . . . . .               | 1124 |
| 260 — Change management, ventanas y rollback . . . . .                | 1128 |
| 261 — Game days, chaos engineering y aprendizaje . . . . .            | 1132 |
| 262 — Capacity planning, cuotas y gestión de demanda . . . . .        | 1136 |
| 263 — AIOps, automatización asistida y límites humanos . . . . .      | 1140 |
| 264 — Proyecto: centro de operaciones de CloudShop . . . . .          | 1144 |

## **Parte 22 — Especializaciones, certificaciones y práctica profesional 1148**

|                                                                        |      |
|------------------------------------------------------------------------|------|
| 265 — Ruta Cloud Engineer y mapa de competencias . . . . .             | 1150 |
| 266 — Ruta DevOps y Delivery Engineer . . . . .                        | 1154 |
| 267 — Ruta Platform Engineer . . . . .                                 | 1158 |
| 268 — Ruta Site Reliability Engineer . . . . .                         | 1162 |
| 269 — Ruta Cloud Security Engineer . . . . .                           | 1166 |
| 270 — Ruta FinOps Practitioner . . . . .                               | 1170 |
| 271 — Ruta Cloud Data y AI Engineer . . . . .                          | 1174 |
| 272 — Ruta Cloud Solutions Architect . . . . .                         | 1178 |
| 273 — Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps . . . . .    | 1182 |
| 274 — Preguntas de escenario y estrategia de examen . . . . .          | 1186 |
| 275 — Portafolio, evidencia, README y entrevista de sistemas . . . . . | 1190 |
| 276 — Proyecto: defensa técnica ante panel . . . . .                   | 1194 |

## **Parte 23 — Capstones por industria y defensa final 1198**

|                                                                        |      |
|------------------------------------------------------------------------|------|
| 277 — Capstone retail: comercio multi-región . . . . .                 | 1200 |
| 278 — Capstone financiero: pagos, auditoría y recuperación . . . . .   | 1204 |
| 279 — Capstone salud: privacidad e interoperabilidad . . . . .         | 1208 |
| 280 — Capstone sector público: soberanía y continuidad . . . . .       | 1212 |
| 281 — Capstone media: streaming y distribución global . . . . .        | 1216 |
| 282 — Capstone industria: IoT, edge y operación desconectada . . . . . | 1220 |

|                                                                   |      |
|-------------------------------------------------------------------|------|
| 283 — Capstone SaaS: multi-tenancy y unit economics . . . . .     | 1224 |
| 284 — Capstone datos e IA: plataforma gobernada . . . . .         | 1228 |
| 285 — Game day integrado y respuesta a incidentes . . . . .       | 1232 |
| 286 — Revisión Well-Architected multi-proveedor . . . . .         | 1236 |
| 287 — Paquete de evidencia, costos y riesgos residuales . . . . . | 1240 |
| 288 — Defensa final, retrospectiva y plan de 12 meses . . . . .   | 1244 |

# Guías pedagógicas

## Guía del estudiante

1. Ejecuta el diagnóstico y elige una ruta; no omitas prerequisites que no puedas demostrar.
2. En cada clase: predice, ejecuta, provoca fallo, recupera y conserva evidencia.
3. Cada seis partes presenta un checkpoint. Repite la práctica si una dimensión queda bajo 70%.
4. Usa cuentas autorizadas, identidad temporal, presupuesto y destrucción obligatoria.
5. La graduación requiere capstone, game day, paquete de evidencia y defensa oral.

# Metodología docente

## Propósito

El programa convierte servicios cloud en decisiones de ingeniería comprensibles. Cada clase debe dejar al estudiante pudiendo explicar, ejecutar, observar y defender una decisión.

## Secuencia central

1. Problema: escenario, usuario, restricciones y consecuencias del fallo.
2. Modelo mental: mecanismo neutral antes del nombre comercial.
3. Demostración: el docente verbaliza decisiones y señales.
4. Práctica guiada: laboratorio local, determinista y sin costo.
5. Transferencia: adaptación a AWS, Azure, Google Cloud o entorno híbrido.
6. Evidencia: artefacto, salida, prueba negativa y limitación.
7. Reflexión: trade-off, error frecuente y condición de revisión.

## Modelo de sesión de 120 minutos

| Momento         | Minutos | Actividad                                         |
|-----------------|---------|---------------------------------------------------|
| Apertura        | 10      | Objetivo, caso y activación de prerrequisitos     |
| Modelo mental   | 20      | Una idea fuerte y sus límites                     |
| Demostración    | 20      | Recorrido corto con decisiones verbalizadas       |
| Práctica guiada | 30      | Ejecución y lectura de evidencia                  |
| Reto            | 25      | Variación, fallo o alternativa                    |
| Revisión        | 10      | Pares aplican criterio de aceptación              |
| Cierre          | 5       | Síntesis, error frecuente y siguiente dependencia |

## Regla de herramientas e IA

El estudiante puede consultar documentación, buscadores y asistentes, pero debe seguir:

1. formular el problema;
2. declarar el supuesto;
3. consultar;
4. ejecutar o contrastar;
5. explicar con evidencia;
6. registrar límites y fuente.

Una respuesta generada sin verificación no es evidencia de dominio.

## Intervenciones docentes

Cuando el grupo va lento, se reduce el escenario y se conserva el mismo contrato de evidencia. Cuando va rápido, se añade fallo parcial, límite de costo o segundo proveedor. Cuando aparece ansiedad por la cantidad de servicios, se vuelve al mecanismo neutral y a la pregunta que resuelve. Cuando el laboratorio falla, el error se usa para practicar lectura de logs, hipótesis y descarte sistemático.

## Evidencias mínimas

- comando o cambio ejecutado;
- salida o estado observado;
- interpretación en palabras propias;
- prueba negativa;
- limitación;
- decisión y condición de rollback.

# Syllabus y cronograma

## Carga total

El programa contiene 288 clases. Una clase regular estima 4 horas entre lectura, sesión, laboratorio y reto; los cierres y las tres clases finales de capstone estiman 8 horas. Carga total aproximada: 1.288 h.

| Parte | Tema                                                               | Clases | Horas aprox. | Semanas a 10 h |
|-------|--------------------------------------------------------------------|--------|--------------|----------------|
| 00    | Fundamentos técnicos                                               | 12     | 52           | 5–6            |
| 01    | Estrategia y adopción                                              | 12     | 52           | 5–6            |
| 02    | AWS                                                                | 12     | 52           | 5–6            |
| 03    | Azure                                                              | 12     | 52           | 5–6            |
| 04    | Google Cloud                                                       | 12     | 52           | 5–6            |
| 05    | Contenedores                                                       | 12     | 52           | 5–6            |
| 06    | Kubernetes                                                         | 12     | 52           | 5–6            |
| 07    | IaC                                                                | 12     | 52           | 5–6            |
| 08    | Entrega y plataforma                                               | 12     | 52           | 5–6            |
| 09    | Datos e integración                                                | 12     | 52           | 5–6            |
| 10    | Observabilidad y SRE                                               | 12     | 52           | 5–6            |
| 11    | Seguridad, gobierno y FinOps                                       | 12     | 52           | 5–6            |
| 12    | Arquitectura distribuida                                           | 12     | 52           | 5–6            |
| 13    | Multi-cloud, híbrido y DR                                          | 12     | 52           | 5–6            |
| 14    | Avanzado y capstones                                               | 12     | 60           | 6              |
| 15–23 | Arquitectura, proveedores productivos, IA, operaciones y capstones | 108    | 500          | 50             |
|       | Total                                                              | 288    | 1.288        | 129            |

## Ritmos sugeridos

| Modalidad                          | Dedicación    | Duración  |
|------------------------------------|---------------|-----------|
| Autodidacta compatible con trabajo | 10 h/semana   | 18 meses  |
| Bootcamp parcial                   | 20 h/semana   | 9 meses   |
| Inmersivo                          | 35 h/semana   | 5–6 meses |
| Ruta especializada                 | 8–12 h/semana | 4–8 meses |

## Dependencias

- Parte 00 precede todo trabajo práctico.
- Parte 01 precede el estudio de proveedores.
- Se requiere dominar al menos uno de 02, 03 o 04 antes de 05–08.
- Partes 05 y 07 preceden 06 y 08 respectivamente.
- Partes 09–11 pueden estudiarse en paralelo después de 08.
- Parte 12 integra 09–11; 13 asume 12; 14 cierra todas las rutas.

## Evaluación

Cada clase se aprueba con nivel B o superior. Una parte requiere 80 % de clases aprobadas y su proyecto integrador reproducible. El capstone final pondera arquitectura, implementación, operación, seguridad/costo y defensa profesional.



# Rúbrica de evaluación

## Reto por clase

| Nivel             | Descripción                                                              | Puntos |
|-------------------|--------------------------------------------------------------------------|--------|
| A — competente    | Cumple el criterio completo, reproduce resultados y defiende trade-offs. | 3      |
| B — en desarrollo | Cumple lo esencial; una dimensión requiere mayor profundidad.            | 2      |
| C — inicial       | Entrega parcial, no reproducible o con errores conceptuales.             | 1      |
| 0                 | Sin evidencia.                                                           | 0      |

Se aprueba una clase con B o superior. La evidencia debe provenir de ejecución, inspección o fuente primaria; una captura sin contexto no demuestra por sí sola el resultado.

## Laboratorio

| Criterio           | Peso | A — competente                                    |
|--------------------|------|---------------------------------------------------|
| Corrección técnica | 25   | Configuración y conclusiones correctas            |
| Evidencia          | 20   | Afirmaciones conectadas a salidas observables     |
| Reproducibilidad   | 15   | Otra persona repite el recorrido                  |
| Prueba negativa    | 15   | Denegación o fallo demuestra la frontera          |
| Seguridad y costo  | 15   | Permisos, datos y unidades explícitas             |
| Comunicación       | 10   | Decisión clara para audiencia técnica y ejecutiva |

## Capstone final

| Dimensión                                  | Peso |
|--------------------------------------------|------|
| Requisitos, arquitectura y ADR             | 20   |
| Infraestructura y entrega reproducible     | 20   |
| Confiabilidad, observabilidad e incidentes | 20   |
| Seguridad, gobierno y FinOps               | 20   |
| Pruebas, documentación y defensa           | 20   |

Un capstone aprobado debe sobrevivir una demostración de fallo, reconstruirse desde el repositorio y distinguir claramente evidencia local, sandbox y producción.

## Integridad académica

Se permite colaboración y asistencia de IA declarada. El estudiante debe poder explicar, modificar y defender todo artefacto entregado. Secretos reales, datos personales o recursos no autorizados invalidan la entrega.

## Especializaciones e industrias

Las rutas de Cloud Engineer, DevOps, Platform, SRE, Security, FinOps, Data/AI y Architect terminan en evidencia pública: diseño, código, prueba de fallo, costo, runbook y defensa. Los capstones cubren retail, finanzas, salud, sector público, media, industria, SaaS y datos/IA.

# Mapa de certificaciones

El programa alinea objetivos, no garantiza aprobación. Los dominios se versionan y deben contrastarse con las guías oficiales vigentes.

| Ruta                | Partes principales |
|---------------------|--------------------|
| AWS SAA/DVA/SOA     | 02, 08, 10, 11, 17 |
| Azure AZ-104/AZ-305 | 03, 10, 11, 18     |
| Google ACE/PCA      | 04, 10, 11, 19     |
| CKA/CKAD/CKS        | 05, 06, 08, 11     |
| FinOps Practitioner | 01, 11, 21, 22     |

# Bibliografía central y mapa de lectura

La bibliografía pauta secuencia y profundidad; no se reproduce su contenido. Las ediciones y enlaces comerciales pueden cambiar, por lo que se recomienda verificar catálogo editorial.

| Partes | Área                           | Obras principales                                                                                                     |
|--------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 00     | Sistemas, redes y herramientas | Ward, How Linux Works · Kurose/Ross, Computer Networking · Chacon/Straub, Pro Git                                     |
| 01, 13 | Estrategia y multi-cloud       | Hohpe, Cloud Strategy · Storment/Fuller, Cloud FinOps                                                                 |
| 02     | AWS                            | AWS Well-Architected Framework · Culkin/Zazon, AWS Cookbook · Wittig/Wittig, Amazon Web Services in Action            |
| 03     | Azure                          | Azure Architecture Center · Modi, Azure for Architects · Microsoft Press, Exam Ref AZ-305                             |
| 04     | Google Cloud                   | Google Cloud Architecture Framework · Sullivan, Professional Cloud Architect Study Guide                              |
| 05     | Contenedores                   | Poulton, Docker Deep Dive · Rice, Container Security · Davis, Cloud Native Patterns                                   |
| 06     | Kubernetes                     | Burns/Beda/Hightower, Kubernetes: Up and Running · Ibryam/Huß, Kubernetes Patterns                                    |
| 07     | IaC                            | Morris, Infrastructure as Code · Brikman, Terraform: Up & Running · Geerling, Ansible for DevOps                      |
| 08     | Entrega y plataforma           | Humble/Farley, Continuous Delivery · Forsgren/Humble/Kim, Accelerate · Skelton/Pais, Team Topologies                  |
| 09, 12 | Datos y distribuidos           | Kleppmann, Designing Data-Intensive Applications · Hohpe/Woolf, Enterprise Integration Patterns                       |
| 10     | SRE                            | Beyer et al., Site Reliability Engineering y The Site Reliability Workbook · Majors et al., Observability Engineering |
| 11     | Seguridad y FinOps             | Dotson, Practical Cloud Security · Estrin, Cloud Security Handbook · Storment/Fuller, Cloud FinOps                    |
| 12     | Arquitectura                   | Davis, Cloud Native Patterns · Newman, Building Microservices · Nygard, Release It!                                   |
| 14     | Organización y carrera         | Skelton/Pais, Team Topologies · Richards/Ford, Fundamentals of Software Architecture                                  |

## Fuentes primarias obligatorias

Además de libros, cada implementación debe citar documentación oficial vigente de AWS, Microsoft Azure, Google Cloud, CNCF/Kubernetes, OpenTelemetry, Terraform/OpenTofu y el estándar OCI correspondiente. Registra URL, versión o fecha de consulta y evita asumir que un límite, precio o nombre comercial permanece estable.

## Método de lectura

1. Lee la clase para obtener vocabulario y preguntas.
2. Consulta el capítulo señalado por la parte.
3. Contrasta el mecanismo con documentación oficial.
4. Ejecuta el laboratorio y registra evidencia.
5. Escribe una conclusión propia y una limitación.

# Parte 00 — Fundamentos de computación, redes y Linux

Programa · Índice completo · Parte 01 →

Nivel: inicial · Clases: 12 · Duración sugerida: 6–8 semanas

Construir el vocabulario y las destrezas técnicas que la nube da por supuestas.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

## Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                          | Laboratorio    | Horas |
|-----|----------------------------------------------------------------|----------------|-------|
| 001 | Computación digital y modelo mental de la nube                 | foundation     | 4     |
| 002 | Terminal, sistema de archivos, procesos y variables de entorno | shell          | 4     |
| 003 | Git, GitHub y trabajo reproducible                             | git            | 4     |
| 004 | Python, JSON y automatización mínima                           | automation     | 4     |
| 005 | Redes por capas, TCP/IP, puertos y sockets                     | network        | 4     |
| 006 | DNS, HTTP, HTTPS y TLS de extremo a extremo                    | network        | 4     |
| 007 | Linux: usuarios, permisos, servicios y logs                    | linux          | 4     |
| 008 | Virtualización, hipervisores e imágenes                        | virtualization | 4     |
| 009 | APIs REST, autenticación y contratos                           | api            | 4     |
| 010 | Responsabilidad compartida y pensamiento de riesgo             | security       | 4     |
| 011 | Costo, energía, capacidad y medición básica                    | finops         | 4     |
| 012 | Proyecto: servicio local reproducible y observable             | capstone       | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- How Linux Works — Brian Ward.
- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Pro Git — Chacon y Straub.

# 001 — Computación digital y modelo mental de la nube

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: foundation · Estado: EXECUTABLECORE

## Propósito

Construir el modelo mental que sostiene todo el programa: qué ejecuta realmente una máquina, por qué la latencia y no la velocidad del procesador domina el diseño de un sistema distribuido, y qué cambia exactamente cuando ese cómputo se alquila en lugar de comprarse. Sin este modelo, «la nube» se aprende como un catálogo de nombres comerciales que caduca cada dieciocho meses.

## Resultados de aprendizaje

Al finalizar podrás:

1. Situar una operación en la jerarquía de memoria y estimar su coste en órdenes de magnitud, de 1 ns a 150 ms.
2. Aplicar las cinco características esenciales de NIST SP 800-145 para decidir si un servicio es realmente cloud o solo hosting renombrado.
3. Distinguir IaaS, PaaS y SaaS como fronteras de responsabilidad operativa, no como niveles de comodidad.
4. Calcular un presupuesto de latencia extremo a extremo y detectar qué componente lo consume.
5. Explicar por qué la elasticidad cambia la unidad económica de la infraestructura: de activo amortizado a gasto por consumo.

## Conceptos centrales

| Concepto           | Comprensión verificable                                                                                                                                                                                                |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| latencia           | Tiempo entre emitir una petición y recibir la primera respuesta. Está acotada por la velocidad de la luz en fibra (200.000 km/s), así que ninguna optimización de software reduce el mínimo físico entre dos ciudades. |
| ancho de banda     | Volumen transferido por unidad de tiempo. Se compra; la latencia no. Añadir ancho de banda no acelera una petición pequeña, solo permite más peticiones simultáneas.                                                   |
| elasticidad        | Capacidad de aprovisionar y liberar recursos en minutos siguiendo la demanda. Es lo que convierte capacidad ociosa en coste evitado; sin ella, la nube es un centro de datos con factura mensual.                      |
| multi-tenencia     | Varios clientes compartiendo el mismo hardware con aislamiento lógico. Explica a la vez el precio (se amortiza entre muchos) y la clase de riesgo (vecino ruidoso, escapes del hipervisor).                            |
| modelo de servicio | Dónde se traza la línea entre lo que administra el proveedor y lo que sigue siendo tuyo. IaaS te deja el sistema operativo; PaaS, el runtime; SaaS, solo los datos y la configuración.                                 |

## Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart TB
    subgraph fisico["Lo que existe de verdad"]
        CPU["CPU · registros y caché<br/>1-4 ns"] --> RAM["Memoria principal<br/>~100 ns"]
        RAM --> SSD["Almacenamiento NVMe<br/>~16 µs"]
        SSD --> RED["Red del centro de datos<br/>~0,5 ms ida y vuelta"]
        RED --> WAN["Enlace intercontinental<br/>~150 ms ida y vuelta"]
    end
    subgraph alquiler["Lo que se alquila"]
        IAAS["IaaS · máquina y disco"] --> PAAS["PaaS · runtime gestionado"]
        PAAS --> SAAS["SaaS · solo datos y configuración"]
    end
    fisico -->|"virtualización y<br/>multi-tenencia"| alquiler
```

## Desarrollo

### 1. La jerarquía de memoria manda sobre el resto del diseño

Un procesador moderno ejecuta del orden de 10<sup>10</sup> instrucciones por segundo y por núcleo, pero pasa la mayor parte del tiempo esperando datos. Los órdenes de magnitud que hay que tener memorizados, popularizados por Jeff Dean en Latency Numbers Every Programmer Should Know:

|                                         |         |                        |
|-----------------------------------------|---------|------------------------|
| referencia a caché L1                   | ~1 ns   |                        |
| referencia a caché L2                   | ~4 ns   |                        |
| referencia a memoria principal          | ~100 ns | (100x L1)              |
| lectura aleatoria en SSD NVMe           | ~16 µs  | (160x RAM)             |
| ida y vuelta dentro del centro de datos | ~0,5 ms | (31x SSD)              |
| búsqueda en disco mecánico              | ~2 ms   |                        |
| ida y vuelta California - Países Bajos  | ~150 ms | (300x centro de datos) |

Entre el primer y el último renglón hay un factor de 150 millones. Esto tiene una consecuencia de diseño que se repetirá en las 288 clases: mover el cómputo hacia los datos casi siempre gana a mover los datos hacia el cómputo. Cuando en la parte 16 se estudien CDN y edge, y en la parte 12 la colocación de réplicas, la razón será siempre esta tabla.

### 2. La latencia tiene un suelo físico y el ancho de banda no

La luz viaja a 299.792 km/s en el vacío, pero en fibra óptica el índice de refracción la frena a unos 200.000 km/s. Madrid–Nueva York son 5.760 km en línea recta:

|                           |                                     |
|---------------------------|-------------------------------------|
| mínimo teórico ida        | = 5.760 km / 200.000 km/s = 28,8 ms |
| mínimo teórico ida+vuelta | = 57,6 ms                           |
| medido en la práctica     | ~ 85-100 ms                         |

La diferencia entre 57,6 y 90 ms son enrutamiento no geodésico, colas en routers y conmutación. Puedes optimizar esa parte; los 57,6 ms no. Ninguna inversión hace que una petición Madrid–Nueva York baje de 58 ms. Por eso la respuesta a un problema de latencia rara vez es un servidor más rápido: es una réplica más cerca, una caché, o menos idas y vueltas.

El ancho de banda se comporta al revés: es una mercancía que se compra. Duplicar el caudal no acorta una petición de 2 KB, pero permite atender el doble de ellas.

### 3. Qué define a la nube según NIST, y qué no

La definición operativa la fija el NIST SP 800-145 (Mell y Grance, 2011) con cinco características esenciales. Un servicio que no cumple las cinco es hosting, por mucho que se anuncie como cloud:

| Característica            | Prueba concreta                                                 |
|---------------------------|-----------------------------------------------------------------|
| Autoservicio bajo demanda | ¿Puedes aprovisionar sin abrir un ticket ni hablar con nadie?   |
| Acceso amplio por red     | ¿Se consume por protocolos estándar desde cualquier cliente?    |
| Agrupación de recursos    | ¿El proveedor reasigna capacidad entre clientes de forma opaca? |
| Elasticidad rápida        | ¿Escala y libera en minutos, no en semanas?                     |
| Servicio medido           | ¿Se factura por consumo observable y auditable?                 |



La quinta es la que más cuesta interiorizar y la que más aparece en la factura: si el recurso se mide, cada decisión de arquitectura es también una decisión de coste. Un bucle mal escrito en un centro de datos propio es capacidad desperdiciada; el mismo bucle en cloud es una línea en la factura del mes siguiente.

#### 4. Los modelos de servicio son fronteras de responsabilidad

IaaS, PaaS y SaaS no son «niveles de facilidad». Son dónde se corta la responsabilidad operativa, y esa línea determina a quién despiertan de madrugada:

| Capa                                   | On-premise | IaaS      | PaaS      | SaaS      |
|----------------------------------------|------------|-----------|-----------|-----------|
| Datos y acceso                         | tú         | tú        | tú        | tú        |
| Aplicación                             | tú         | tú        | tú        | proveedor |
| Runtime y middleware                   | tú         | tú        | proveedor | proveedor |
| Sistema operativo                      | tú         | tú        | proveedor | proveedor |
| Virtualización, servidores, red física | tú         | proveedor | proveedor | proveedor |

Observa la primera fila: los datos y el control de acceso nunca cambian de dueño. Ese es el núcleo del modelo de responsabilidad compartida que se estudia en la clase 010, y el origen de la mayoría de incidentes públicos atribuidos a «fallos del proveedor» que en realidad fueron configuraciones del cliente.

#### 5. Elasticidad: el cambio económico, no el técnico

La virtualización existe desde los años sesenta (CP-40 de IBM, 1967). Lo que la nube añadió no fue la técnica sino el modelo económico: pasar de CAPEX amortizado a OPEX medido.

Supón un servicio con pico de 100 servidores durante 3 horas al día y 20 el resto:

Compra:  $100 \text{ servidores} \times 24 \text{ h} \times 30 \text{ días} = 72.000 \text{ servidor-hora contratadas}$   
 Uso real:  $(100 \times 3 + 20 \times 21) \times 30 = 21.600 \text{ servidor-hora útiles}$   
 utilización =  $21.600 / 72.000 = 30 \%$

En compra pagas el 100 % y usas el 30 %. Con elasticidad pagas cerca del 30 %, a un precio unitario mayor. El punto de equilibrio depende de esa relación, no de la moda: si tu carga es plana y predecible, comprar suele salir más barato. Multi-cloud y elasticidad se justifican por requisitos medibles, nunca por defecto.

### Ejemplo trabajado

CloudShop necesita responder una página de producto en menos de 300 ms al percentil 95. El equipo desglosa el presupuesto de latencia para un usuario en Santiago de Chile y un despliegue en la región us-east-1 (Virginia), a unos 7.400 km:

|                                             |        |                   |
|---------------------------------------------|--------|-------------------|
| resolución DNS (con caché fría)             | 40 ms  |                   |
| handshake TLS 1.3 (1 RTT sobre ~120 ms)     | 120 ms |                   |
| petición HTTP ida y vuelta                  | 120 ms |                   |
| consulta a base de datos en la misma región | 8 ms   |                   |
| renderizado en servidor                     | 25 ms  |                   |
| -----                                       |        |                   |
| total                                       | 313 ms | ✗ incumple el SLO |

El componente dominante no es la aplicación —33 ms entre consulta y render— sino los 240 ms de red. Optimizar el código no salva el SLO: aunque el render bajara a 0 ms, quedarían 288 ms.

Dos intervenciones sobre la red:

|                                          |        |          |
|------------------------------------------|--------|----------|
| CDN en Santiago para el HTML (RTT 10 ms) |        |          |
| DNS cacheado                             | 2 ms   |          |
| TLS al borde (1 RTT sobre 10)            | 10 ms  |          |
| HTTP al borde                            | 10 ms  |          |
| origen solo para datos (1 RTT)           | 120 ms |          |
| consulta + render                        | 33 ms  |          |
| -----                                    |        |          |
| total                                    | 175 ms | ✓ cumple |

La decisión no fue «usar un CDN» sino reconocer que 240 de los 313 ms eran físicos y solo se atacan acercando el punto de terminación. El coste añadido del borde se justifica contra los 138 ms recuperados; si el SLO hubiera sido 500

ms, el gasto no tendría defensa.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/001-computacion-digital-y-modelo-mental-de-la-nube/lab.py
```

El laboratorio selecciona el motor de práctica foundation y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa mapa-de-componentes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es un mapa con fronteras, entradas, salidas y supuestos. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye mapa-de-componentes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■■ Errores frecuentes

| Síntoma                                                                 | Causa probable                                                                              | Corrección                                                                                                                                |
|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| El sistema es lento y se responde comprando instancias más grandes      | Se confundió latencia con capacidad de cómputo                                              | Desglosa el presupuesto de latencia por componente antes de escalar; si domina la red, más CPU no cambia nada.                            |
| «Migramos a la nube» pero la factura supera al centro de datos anterior | Se replicó una carga plana sin usar elasticidad                                             | Calcula la utilización real; si es estable y alta, la nube solo compensa por otros motivos (alcance, servicios gestionados, continuidad). |
| Se asume que el proveedor protege los datos                             | Se leyó el modelo de servicio como nivel de comodidad y no como frontera de responsabilidad | Sitúa cada capa en la tabla de responsabilidad; datos y acceso siguen siendo tuyos en IaaS, PaaS y SaaS.                                  |
| Una prueba local va rápida y en producción no                           | En local todo cabe en RAM y la red es de 0 ms                                               | Mide con las latencias reales entre zonas y regiones; la jerarquía de memoria del entorno de pruebas no es la de producción.              |
| Se llama cloud a un servidor alquilado por meses                        | No se aplicaron las cinco características de NIST                                           | Comprueba autoservicio, elasticidad y medición; si faltan, las decisiones de coste y escala no aplicarán.                                 |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Cuántos órdenes de magnitud separan una referencia a caché L1 de una ida y vuelta intercontinental, y qué decisión de arquitectura se deriva de esa diferencia?
2. Un servicio tarda 400 ms y el equipo propone duplicar la CPU. ¿Qué medición pedirías antes de aprobar el gasto?
3. ¿Cuál de las cinco características de NIST es la que convierte cada decisión técnica en una decisión de coste, y por qué?
4. En un despliegue PaaS, ¿qué sigue siendo responsabilidad tuya y qué pasa a ser del proveedor?
5. Una carga usa el 85 % de su capacidad las 24 horas. ¿Qué argumento económico queda a favor de la nube, si es que queda alguno?

## Referencias

- Mell, P. y Grance, T. (2011). The NIST Definition of Cloud Computing, SP 800-145. National Institute of Standards and Technology. <<https://doi.org/10.6028/NIST.SP.800-145>>
- Dean, J. (2009). Latency Numbers Every Programmer Should Know — cifras de referencia de la jerarquía de memoria. <<https://static.googleusercontent.com/media/research.google.com/en//people/jeff/stanford-295-talk.pdf>>
- Kurose, J. y Ross, K. (2021). Computer Networking: A Top-Down Approach, 8.ª ed., cap. 1.4 — retardo, pérdida y throughput.
- Barroso, L., Hölzle, U. y Ranganathan, P. (2018). The Datacenter as a Computer, 3.ª ed. — economía y utilización del centro de datos. <<https://doi.org/10.2200/S00874ED3V01Y201809CAC046>>
- Gray, J. (2008). Distributed Computing Economics. ACM Queue 6(3) — por qué conviene mover el cómputo hacia los datos. <<https://doi.org/10.1145/1394127.1394131>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 001 Computación digital y modelo mental de la nube

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega mapa-de-componentes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.

- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

## Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 002 — Terminal, sistema de archivos, procesos y variables de entorno

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: shell · Estado: EXECUTABLECORE

## Propósito

Dominar la terminal como interfaz primaria de operación cloud: qué es realmente un proceso, cómo el kernel le entrega su entorno, y por qué los descriptores de fichero y las variables de entorno explican la mitad de los incidentes de despliegue. Toda la automatización posterior —contenedores, CI/CD, runbooks— es este modelo repetido a escala.

## Resultados de aprendizaje

Al finalizar podrás:

1. Describir el ciclo `fork` → `exec` → `wait` y qué hereda un proceso hijo de su padre.
2. Predecir qué variables de entorno ve un proceso según cómo se lanzó, y por qué un servicio `systemd` no ve tu `.bashrc`.
3. Redirigir `stdout` y `stderr` por separado y explicar por qué `2>&1 >f` no equivale a `>f 2>&1`.
4. Interpretar un código de salida y una señal de terminación para diagnosticar sin adivinar.
5. Componer una tubería que resuelva un problema real de operación sin scripts intermedios.

## Conceptos centrales

| Concepto              | Comprensión verificable                                                                                                                                                                                                                           |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| proceso               | Instancia en ejecución de un programa, con su propio espacio de direcciones, tabla de descriptores y entorno. El kernel lo identifica por PID y lo relaciona con su padre por PPID.                                                               |
| descriptor de fichero | Entero que indexa la tabla de ficheros abiertos del proceso. Por convención 0 es <code>stdin</code> , 1 <code>stdout</code> y 2 <code>stderr</code> ; el resto se asignan por orden. Redirigir es duplicar entradas de esa tabla, no mover datos. |
| variable de entorno   | Par clave-valor copiado del padre al hijo en el momento del <code>exec</code> . La copia es unidireccional: un hijo no puede modificar el entorno del padre, lo que explica por qué <code>cd</code> debe ser un builtin del shell.                |
| código de salida      | Entero de 0 a 255 que devuelve un proceso al terminar. 0 significa éxito; 1-125 son errores del programa; 126 y 127 indican no ejecutable y no encontrado; 128+N significa terminado por la señal N.                                              |
| señal                 | Notificación asíncrona del kernel a un proceso. <code>SIGTERM</code> (15) pide terminar y puede atraparse; <code>SIGKILL</code> (9) no es atrapable. Esa diferencia es la base del apagado ordenado en contenedores.                              |

## Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    S["shell (PID 4210)"] -->|fork()| H["hijo: copia del padre<br/>mismo entorno y descriptores"]
    H -->|exec()| P["nuevo programa<br/>conserva PID, descriptores y entorno"]
    P --> R["termina"]
    R -->|exit(0)| OK["código 0 · éxito"]
    R -->|exit(n)| ERR["código n · error del programa"]
    R -->|señal N| SIG["código 128+N"]
    OK --> W["shell recoge con wait()"]
    ERR --> W
    SIG --> W
```

## Desarrollo

### 1. Un proceso nace por copia y se transforma por reemplazo

En UNIX no existe «lanzar un programa» como operación única. Son dos llamadas:

1. `fork()` duplica el proceso actual. El hijo recibe una copia del espacio de direcciones, la tabla de descriptores y el entorno. Devuelve 0 en el hijo y el PID del hijo en el padre; ese es el único modo que tiene cada uno de saber quién es.
2. `exec()` reemplaza la imagen del proceso por otro programa conservando el PID, los descriptores abiertos y el entorno.

```
$ echo $$          # PID del shell actual
4210
$ bash -c 'echo $$; echo $PPID'
4877              # el hijo tiene PID nuevo
4210              # y recuerda a su padre
```

La consecuencia práctica aparece en cuanto se automatiza: un proceso solo puede heredar hacia abajo. Por eso `cd` no puede ser un binario —cambiaría el directorio de un hijo que muere inmediatamente— y por eso exportar una variable en un script no la deja disponible en la terminal que lo invocó.

### 2. El entorno se copia en el `exec`, no se consulta en vivo

El entorno es un vector de cadenas `CLAVE=valor` que se pasa al `exec`. Se copia una vez, en ese instante. Un proceso que lleva tres días corriendo tiene el entorno del momento en que arrancó, aunque el fichero de configuración haya cambiado.

```
$ VAR=solo_este_comando env | grep VAR    # prefijo: solo para esa invocación
VAR=solo_este_comando
$ VAR=local; env | grep VAR               # sin export: no llega al hijo
$ export VAR=heredable; env | grep VAR    # con export: sí llega
VAR=heredable
```

Esto explica el incidente más repetido en despliegues: un servicio gestionado por `systemd` no lee `./bashrc` ni `./profile`. Esos ficheros los interpreta el shell al iniciar sesión de forma interactiva, y `systemd` no arranca un shell de login. El comando funciona a mano y falla como servicio, con la misma imagen y el mismo binario. La variable hay que declararla en la unidad (`Environment=` o `EnvironmentFile=`), y en contenedores en el `ENV` del `Dockerfile` o el manifiesto.

### 3. Redirección: se duplican descriptores, y el orden importa

> no «envía» la salida a ningún sitio: hace que el descriptor 1 apunte al mismo fichero abierto que otro. Como es una asignación secuencial, el orden cambia el resultado:

```
# stderr se copia al destino ACTUAL de stdout (la terminal) y luego stdout va al fichero
$ comando 2>&1 > salida.log          # errores a la terminal, salida al fichero

# stdout va al fichero y DESPUÉS stderr se copia a ese mismo destino
$ comando > salida.log 2>&1          # ambos al fichero
```

Es la causa clásica de un log que «pierde» los errores. En Bash moderno, `&>` fichero hace lo correcto sin ambigüedad.

La separación `stdout/stderr` no es cosmética: es un contrato. `stdout` lleva el resultado destinado a la siguiente etapa de la tubería; `stderr` lleva diagnóstico destinado al operador. Un programa que escribe avisos en `stdout` rompe cualquier tubería que lo consuma.

## 4. Códigos de salida y señales: el lenguaje del diagnóstico

El código de salida es el único canal estructurado que un proceso tiene para decir qué pasó. Sus rangos están convenidos:

| Código | Significado                   | Aparece cuando                                |
|--------|-------------------------------|-----------------------------------------------|
| 0      | Éxito                         | Todo bien                                     |
| 1-125  | Error del programa            | Fallo de lógica o validación                  |
| 126    | Encontrado pero no ejecutable | Falta el bit de ejecución                     |
| 127    | Orden no encontrada           | PATH incorrecto — típico en contenedores      |
| 128+N  | Terminado por la señal N      | 137 = 128+9 (SIGKILL), 143 = 128+15 (SIGTERM) |

El 137 merece memorizarse: en Kubernetes casi siempre significa que el contenedor excedió su límite de memoria y el OOM killer lo mató. En la parte 06 aparecerá con ese nombre; aquí ya sabes de dónde sale el número.

La diferencia entre SIGTERM y SIGKILL define el apagado ordenado: el orquestador envía SIGTERM, espera un plazo de gracia (30 s por defecto en Kubernetes) y solo entonces envía SIGKILL. Un proceso que ignora SIGTERM pierde las conexiones en vuelo.

## 5. Tuberías: procesos concurrentes, no secuenciales

`a | b` no ejecuta `a` y después `b`. Arranca ambos a la vez y conecta el descriptor 1 de `a` con el 0 de `b` mediante un búfer del kernel de 64 KB. Cuando el búfer se llena, el escritor se bloquea; cuando se vacía, el lector se bloquea. Es control de flujo gratuito, y permite procesar ficheros mayores que la memoria disponible.

```
# Los cinco procesos que más memoria residente consumen
$ ps -eo pid,rss,comm --sort=-rss | head -6

# Las 10 IP con más peticiones en un log de acceso
$ awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10
```

El código de salida de una tubería es el del último comando, lo que oculta fallos intermedios. En scripts de operación esto es una trampa seria:

```
$ false | true; echo $?
0                                # el fallo desaparece
$ set -o pipefail
$ false | true; echo $?
1                                # ahora se propaga
```

`set -euo pipefail` al principio de cada script de despliegue evita que un fallo silencioso se declare éxito.

## Ejemplo trabajado

Un despliegue de CloudShop funciona a mano y falla como servicio. El binario es el mismo y la imagen también. El operador ejecuta el diagnóstico en orden:

```
$ ./cloudshop-api
escuchando en :8080                                # a mano funciona

$ sudo systemctl start cloudshop
$ systemctl show cloudshop -p ExecMainStatus
ExecMainStatus=127
```

127 = orden no encontrada. No es un fallo de la aplicación: es que algo del PATH no está. Se compara el entorno de ambos contextos:

```
$ echo $PATH
/home/ops/.local/bin:/usr/local/bin:/usr/bin:/bin

$ sudo systemctl show cloudshop -p Environment
Environment=
$ sudo systemd-run --quiet --pipe /usr/bin/env | grep ^PATH
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

La diferencia es /home/ops/.local/bin, que el shell interactivo añade desde /.profile y systemd no. El binario auxiliar que invoca el servicio vive ahí.

Se corrige declarando la dependencia en la unidad, no exportando en un perfil:

```
[Service]
Environment="PATH=/usr/local/bin:/usr/bin:/bin"
ExecStart=/opt/cloudshop/bin/cloudshop-api

$ sudo systemctl restart cloudshop; systemctl show cloudshop -p ExecMainStatus
ExecMainStatus=0
```

La lección no es el arreglo sino el método: el código 127 acotó el problema a resolución de ruta antes de leer una sola línea de la aplicación. Un diagnóstico sin ese dato habría empezado por los logs de la aplicación, donde no había nada que ver.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/002-terminal-sistema-de-archivos-
procesos-y-variables-de-entorno/lab.py
```

El laboratorio selecciona el motor de práctica shell y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa bitacora-de-comandos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una bitácora reproducible de comandos y resultados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye bitacora-de-comandos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.



## ■ Errores frecuentes

| Síntoma                                                                     | Causa probable                                                                   | Corrección                                                                                             |
|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| El comando funciona en la terminal y falla como servicio o en el contenedor | El entorno interactivo carga perfiles que systemd y Docker no leen               | Declara las variables en la unidad (Environment=) o en el ENV de la imagen; nunca dependas de .bashrc. |
| El fichero de log no contiene los errores                                   | Se escribió 2>&1 > f, que copia stderr al destino anterior de stdout             | Usa > f 2>&1 o &> f; el orden de la redirección es una asignación secuencial.                          |
| Un script de despliegue termina en éxito pese a que un paso falló           | El código de salida de una tubería es solo el del último comando                 | Empieza los scripts con set -euo pipefail.                                                             |
| El contenedor muere con código 137 y no hay error en la aplicación          | 128+9: fue SIGKILL, casi siempre el OOM killer por exceder el límite de memoria  | Revisa el límite de memoria y el consumo real; el problema es de recursos, no de código.               |
| El servicio pierde peticiones en vuelo en cada despliegue                   | El proceso no atrapa SIGTERM y se le aplica SIGKILL al agotar el plazo de gracia | Atrapa SIGTERM, deja de aceptar conexiones nuevas y drena las abiertas antes de salir.                 |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Por qué cd tiene que ser un builtin del shell y no puede ser un binario en /usr/bin?
2. Un proceso lleva tres días ejecutándose y alguien cambia una variable en /etc/environment. ¿La ve el proceso? ¿Por qué?
3. ¿Qué diferencia práctica hay entre comando 2>&1 > f y comando > f 2>&1?
4. Un contenedor termina con código 143 y otro con 137. ¿Cuál se apagó ordenadamente y cuál no?
5. ¿Qué añade set -o pipefail que set -e por sí solo no cubre?

## Referencias

- The Open Group (2018). POSIX.1-2017, System Interfaces: fork, execve, wait.  
<<https://pubs.opengroup.org/onlinepubs/9699919799/functions/fork.html>>
- Kerrisk, M. signal(7) — Linux manual pages: catálogo de señales y semántica de SIGTERM y SIGKILL.  
<<https://man7.org/linux/man-pages/man7/signal.7.html>>
- Kerrisk, M. (2010). The Linux Programming Interface, caps. 24-27 — creación de procesos y ejecución de programas.
- systemd (2024). systemd.exec(5) — cómo se construye el entorno de un servicio.  
<<https://www.freedesktop.org/software/systemd/man/systemd.exec.html>>
- Ward, B. (2021). How Linux Works, 3.<sup>a</sup> ed., caps. 1-2 — procesos, dispositivos y arranque.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 002 Terminal, sistema de archivos, procesos y variables de entorno

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega bitacora-de-comandos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 003 — Git, GitHub y trabajo reproducible

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: git · Estado: EXECUTABLECORE

### Propósito

Entender Git como un grafo dirigido acíclico de instantáneas inmutables direccionadas por contenido, no como una lista de parches. Esa diferencia explica por qué rebase, revert y reset hacen lo que hacen, y es la base de todo lo que viene después: GitOps, entrega continua, trazabilidad de auditoría y reproducibilidad de un despliegue.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cómo el hash SHA-1/SHA-256 de un commit se deriva de árbol, padres, autor y mensaje, y qué implica para la integridad del historial.
2. Distinguir reset --soft, --mixed y --hard por lo que hacen sobre HEAD, índice y árbol de trabajo.
3. Elegir entre merge, rebase y revert según si la rama es compartida y si el historial debe ser auditable.
4. Recuperar trabajo aparentemente perdido usando reflog, entendiendo por qué existe la ventana de recuperación.
5. Diseñar un flujo de ramas trazable que permita responder «qué código exacto había en producción el martes a las 14:00».

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                                                                                                                              |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| objeto     | Unidad inmutable de almacenamiento en Git, nombrada por el hash de su contenido. Hay cuatro tipos: blob (contenido de fichero), tree (directorio), commit (instantánea con metadatos) y tag anotado. |
| commit     | Objeto que apunta a un árbol completo, a cero o más padres, y lleva autor, fecha y mensaje. No guarda un diff: guarda el estado entero. Los diffs se calculan al vuelo comparando árboles.           |
| índice     | Área intermedia entre el árbol de trabajo y el repositorio, también llamada staging. Es un fichero binario que lista qué versión de cada ruta entrará en el próximo commit.                          |
| referencia | Puntero mutable a un commit. Las ramas y HEAD son referencias; por eso crear una rama cuesta 41 bytes y es una operación instantánea sea cual sea el tamaño del repositorio.                         |
| reflog     | Registro local de todos los valores por los que ha pasado cada referencia. Permite recuperar commits que ya no alcanza ninguna rama, durante 90 días por defecto.                                    |

### Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart RL
    subgraph objetos["Base de objetos · inmutable"]
        C3["commit c3f9a1<br/>árbol + padre + autor"] --> C2["commit 8b2e40"]
        C2 --> C1["commit 1a7dd3"]
        C3 --> T["tree raíz"]
        T --> B1["blob README"]
        T --> B2["blob main.py"]
    end
    subgraph refs["Referencias · mutables"]
        MAIN["main"] --> C3
        HEAD["HEAD"] --> MAIN
        TAG["v2.0.0"] --> C2
    end
end
```

## Desarrollo

### 1. Git guarda instantáneas, no diferencias

La intuición equivocada más extendida es que un commit almacena «lo que cambió». Almacena el árbol completo del proyecto en ese instante. Los ficheros que no cambiaron no se duplican: el árbol nuevo reutiliza los mismos objetos blob, porque están nombrados por el hash de su contenido.

```
$ git cat-file -p HEAD
tree a4f2c81e9b3d5a7c2f8e1b6d4a9c3e7f2b8d5a1c
parent 8b2e40d7c1a5f9e3b6d2a8c4f7e1b9d3a5c8e2f4
author Vladimir Acuña <...> 1785546717 -0300
committer Vladimir Acuña <...> 1785546717 -0300
```

Optimiza el README principal

El hash del commit se calcula sobre exactamente ese texto. Cambiar una coma del mensaje, la fecha o el padre produce un hash distinto. De ahí se sigue la propiedad que sostiene toda auditoría: no se puede alterar historia pasada sin cambiar los identificadores de todo lo que vino después. Si alguien reescribe un commit de hace un mes, los 200 commits posteriores cambian de hash y cualquier clon lo detecta.

### 2. Tres zonas y el comando que las mueve

Todo el modelo operativo de Git cabe en tres zonas y una tabla:

```
árbol de trabajo → índice (staging) → repositorio (objetos)
git add ■■■■■■■■      git commit ■■■■■■■■
```

git reset es un único comando que actúa sobre las tres según el modificador, y confundirlos es la causa habitual de trabajo perdido:

| Comando           | HEAD  | Índice   | Árbol de trabajo | Uso típico                              |
|-------------------|-------|----------|------------------|-----------------------------------------|
| reset --soft <c>  | mueve | intacto  | intacto          | Rehacer el mensaje o agrupar commits    |
| reset --mixed <c> | mueve | reinicia | intacto          | Deshacer un add (por defecto)           |
| reset --hard <c>  | mueve | reinicia | descarta         | Descartar todo; es el único destructivo |

Solo --hard toca el árbol de trabajo. Los cambios no confirmados que destruye no están en ningún objeto, así que el reflog no los recupera: nunca llegaron a existir para Git.

### 3. Merge, rebase y revert: tres respuestas a preguntas distintas

No son alternativas de estilo. Responden a preguntas diferentes:

- merge crea un commit con dos padres. Preserva la historia tal como ocurrió y no cambia hashes existentes. Es la única opción segura sobre una rama que otros ya tienen.
- rebase reescribe: toma tus commits y los vuelve a aplicar sobre otra base, creando commits nuevos con hashes nuevos. Produce historia lineal y legible, pero si la rama es compartida, todo el que la tenga verá divergencia.
- revert crea un commit nuevo que deshace los efectos de otro. No borra nada. Es lo único aceptable para deshacer algo que ya está en producción o en una rama protegida.

La regla operativa, conocida como golden rule of rebasing: no reescribas historia que otros puedan tener. En la práctica: rebase libremente en tu rama local antes del primer push; después, merge o revert.

Para un repositorio con requisitos de auditoría —los de la parte 11— revert es obligatorio: deja constancia de que hubo un error y de cuándo se corrigió. Un reset --hard seguido de push --force borra esa evidencia.

#### 4. El reflog: por qué casi nada se pierde de verdad

Cuando una rama deja de apuntar a un commit, ese commit queda huérfano: sigue en la base de objetos pero no lo alcanza ninguna referencia. El reflog registra cada movimiento de cada referencia, así que aún se puede llegar a él.

```
$ git reset --hard HEAD~3          # "perdí" tres commits
$ git reflog
c3f9a1 HEAD@{0}: reset: moving to HEAD~3
7d2b8e HEAD@{1}: commit: añade validación de contratos
$ git reset --hard HEAD@{1}       # recuperados
```

Los objetos huérfanos sobreviven hasta que git gc los recoge: 90 días para los alcanzables desde el reflog y 30 para el resto, según gc.reflogExpire. Dos matices que importan en operación:

1. El reflog es estrictamente local. No se clona ni se envía. Un push --force que destruye commits en el servidor no deja reflog para quien clone después.
2. Solo registra lo que llegó a ser un commit. Cambios en el árbol de trabajo que nunca se confirmaron no aparecen.

#### 5. Trazabilidad: del commit al artefacto desplegado

La pregunta que toda auditoría acaba haciendo es: ¿qué código exacto estaba corriendo cuando ocurrió el incidente? Responderla exige una cadena sin huecos:

```
commit (sha) → build (id) → artefacto (digest) → despliegue (revisión) → instante
```

Cada eslabón debe ser inmutable y verificable. Por eso en las partes 05 y 08 se insistirá en referenciar imágenes por digest (sha256:...) y no por etiqueta: una etiqueta como :latest o :v2.0 es una referencia mutable —igual que una rama de Git— y puede apuntar a contenido distinto mañana.

El equivalente en Git es el tag anotado, que sí es un objeto con su propio hash, autor y mensaje, frente al tag ligero que es solo un puntero:

```
$ git tag -a v2.0.0 -m "Release 2.0.0"    # objeto, firmable y con metadatos
$ git tag v2.0.0                          # solo una referencia más
```

Para releases, el tag anotado y firmado (-s) es el que sostiene la afirmación «esto es lo que publicamos».

### Ejemplo trabajado

Un despliegue del martes rompió el checkout de CloudShop y hay que responder tres preguntas: qué se desplegó, quién lo aprobó y cómo se revierte sin perder la evidencia.

Paso 1 — identificar el commit exacto que estaba en producción:

```
$ git log --format='%h %ad %an %s' --date=iso -3 v2.4.1
7d2b8e 2026-07-28 14:02:11 -0300 Ana Ruiz   Cambia el cálculo de impuestos
1a7dd3 2026-07-28 11:40:02 -0300 Luis Vega Añade índice a pedidos
8b2e40 2026-07-27 17:15:44 -0300 Ana Ruiz   Actualiza dependencias
```

Paso 2 — confirmar que el artefacto desplegado corresponde a ese commit. La etiqueta no basta; se compara el digest:

```
$ git rev-parse v2.4.1
7d2b8e91c4a2f7d5b3e8a1c6f9d2b4e7a5c8f1d3
$ kubectl get deploy cloudshop -o jsonpath='{...image}'
registry/cloudshop@sha256:4f2a9c...
$ crane config registry/cloudshop@sha256:4f2a9c... | jq -r '.config.Labels."org.opencontainers.image.revision"'
7d2b8e91c4a2f7d5b3e8a1c6f9d2b4e7a5c8f1d3      # coincide
```

Paso 3 — aislar el cambio culpable con búsqueda binaria sobre el historial:

```
$ git bisect start v2.4.1 v2.4.0
$ git bisect run ./tests/checkout_smoke.sh
7d2b8e91 is the first bad commit
```

Con 3 commits bastan 2 pruebas; con 1.000 bastarían 10, porque bisect es logarítmico:  $\lceil \log_2(1000) \rceil = 10$ .

Paso 4 — revertir sin borrar:

```
$ git revert 7d2b8e -m "Revierte el cálculo de impuestos: rompe el checkout"
[main 9e4c2a] Revert "Cambia el cálculo de impuestos"
```

El historial conserva ahora el error y su corrección. Un `reset --hard 1a7dd3 && push --force` habría dejado producción igual de sana y la auditoría sin nada que leer: no habría constancia de que el fallo existió, ni de cuánto tardó en detectarse.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/003-git-github-y-trabajo-reproducible/
lab.py
```

El laboratorio selecciona el motor de práctica git y produce `labresult.json`. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee `exercise.steps` y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de `negativetest` y explica la señal observada.
4. Materializa repositorio-reproducible en `evidence/` usando la plantilla indicada.
5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

### Evidencia esperada

El artefacto principal es un historial pequeño, legible y verificable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye repositorio-reproducible para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                                                                          | Causa probable                                                                               | Corrección                                                                                               |
|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Tras un rebase sobre una rama compartida, el equipo ve commits duplicados y conflictos repetidos | Se reescribieron hashes que otros ya tenían                                                  | Rebase solo antes del primer push; sobre ramas compartidas usa merge.                                    |
| Se perdió trabajo con <code>reset --hard</code> y el reflog no lo recupera                       | Los cambios nunca se confirmaron, así que no existe objeto que recuperar                     | Confirma o usa <code>git stash</code> antes de cualquier operación destructiva.                          |
| Nadie puede decir qué código había en producción durante el incidente                            | El artefacto se referenció por etiqueta mutable en vez de por digest                         | Referencia imágenes por sha256: y graba el commit en una etiqueta OCI del artefacto.                     |
| El historial de la rama principal no permite auditar una corrección urgente                      | Se usó <code>reset --hard</code> y <code>push --force</code> en lugar de <code>revert</code> | Deshaz siempre con <code>revert</code> en ramas protegidas; deja el error visible junto a su corrección. |
| Un <code>git tag</code> no lleva autor ni fecha y no se puede firmar                             | Se creó un tag ligero, que es solo un puntero                                                | Usa <code>git tag -a</code> (o <code>-s</code> para firmarlo) en cualquier release.                      |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. Si un commit guarda el árbol completo, ¿por qué un repositorio con 10.000 commits no ocupa 10.000 veces el tamaño del proyecto?
2. ¿Cuál de los tres modos de `reset` puede destruir trabajo que el reflog no recuperará, y por qué precisamente ese?
3. Un compañero ya hizo pull de tu rama. ¿Qué te impide hacer rebase sobre ella y qué usarías en su lugar?
4. ¿Por qué desplegar `imagen:v2.0` no permite responder qué código corría el martes, y qué referencia lo permitiría?
5. Con 1.024 commits entre una versión buena y una mala, ¿cuántas pruebas necesita `git bisect` en el peor caso?

## Referencias

- Chacon, S. y Straub, B. (2014). Pro Git, 2.<sup>a</sup> ed., cap. 10 «Git Internals» — objetos, referencias y modelo de almacenamiento. <<https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>>
- Git (2024). `git-reset(1)` — tabla oficial del efecto de cada modo sobre HEAD, índice y árbol de trabajo. <<https://git-scm.com/docs/git-reset#resetpathspect>>
- Git (2024). `git-bisect(1)` — búsqueda binaria automatizada sobre el historial. <<https://git-scm.com/docs/git-bisect>>
- Open Container Initiative (2024). Image Specification — anotaciones estándar, incluida `org.opencontainers.image.revision`. <<https://github.com/opencontainers/image-spec/blob/main/annotations.md>>
- Git (2024). `gitrevisions(7)` — sintaxis de `HEAD@{n}`, y `^` para navegar el grafo. <<https://git-scm.com/docs/gitrevisions>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 003 Git, GitHub y trabajo reproducible

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.

2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

## Reto verificable

Entrega repositorio-reproducible con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

## Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

## Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |



## 004 — Python, JSON y automatización mínima

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: automation · Estado: EXECUTABLECORE

### Propósito

Usar Python y JSON como el pegamento mínimo de la operación cloud: leer una respuesta de API, validar su forma antes de confiar en ella y producir salida estructurada que otro programa pueda consumir. Casi todos los lab.py del programa emiten JSON, y casi toda automatización real consiste en encadenar contratos de este tipo.

### Resultados de aprendizaje

Al finalizar podrás:

1. Distinguir los tipos de JSON de los de Python y anticipar las conversiones que rompen datos (enteros grandes, None, claves no textuales).
2. Explicar por qué los números en coma flotante de JSON no representan dinero y qué usar en su lugar.
3. Validar una estructura recibida antes de usarla, en vez de confiar en que la API cumple su documentación.
4. Producir salida determinista: mismas entradas y misma semilla producen bytes idénticos.
5. Diferenciar un fallo esperado, que se maneja, de uno inesperado, que debe propagarse con contexto.

### Conceptos centrales

| Concepto          | Comprensión verificable                                                                                                                                                                                     |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| serialización     | Convertir estructuras en memoria a una secuencia de bytes transportable. JSON solo admite seis tipos: objeto, array, cadena, número, booleano y null; todo lo demás debe traducirse explícitamente.         |
| contrato de datos | Acuerdo explícito sobre qué campos existen, de qué tipo son y cuáles son obligatorios. Sin él, un cambio en el productor rompe al consumidor en producción y no en pruebas.                                 |
| determinismo      | Propiedad por la que las mismas entradas producen exactamente la misma salida. Exige orden estable de claves, semilla fija en cualquier aleatoriedad y ausencia de marcas de tiempo en la salida comparada. |
| idempotencia      | Propiedad por la que repetir una operación no cambia el resultado respecto de ejecutarla una vez. Es lo que hace seguro reintentar tras un error de red.                                                    |
| excepción         | Mecanismo para señalar que una operación no puede completarse. Capturar Exception de forma genérica convierte un fallo diagnosticable en uno silencioso.                                                    |

### Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    API["Respuesta HTTP<br/>bytes"] --> D["json.loads()"]
    D --> V{"¿cumple el contrato?"}
    V -->|"no"| E["error con contexto:<br/>campo, tipo esperado, recibido"]
    V -->|"sí"| L["lógica de negocio"]
    L --> S["json.dumps(sort_keys=True)"]
    S --> O["salida determinista<br/>comparable byte a byte"]
```

## Desarrollo

### 1. JSON y Python no comparten sistema de tipos

El mapeo parece directo hasta que deja de serlo:

| JSON       | Python al leer | Trampa                                                   |
|------------|----------------|----------------------------------------------------------|
| object     | dict           | Las claves siempre acaban siendo str                     |
| array      | list           | Las tuplas se serializan como array y vuelven como lista |
| number     | int o float    | No hay distinción en JSON: la decide el punto decimal    |
| true/false | bool           | —                                                        |
| null       | None           | —                                                        |

La conversión no es reversible:

```
>>> import json
>>> json.loads(json.dumps({1: "a", (2, 3): "b"}))
{'1': 'a', '2, 3': 'b'}      # las claves se volvieron cadenas
>>> json.loads(json.dumps((1, 2)))
[1, 2]                      # la tupla ya no es tupla
```

En una automatización que lee un estado, lo modifica y lo vuelve a escribir, esto significa que el ciclo no es una identidad. Si el consumidor compara claves numéricas, el segundo ciclo falla y el primero no.

### 2. Los números de JSON no sirven para dinero

JSON no define precisión: la mayoría de implementaciones usan IEEE 754 de doble precisión, que es binario. Los decimales que no son suma de potencias de dos no tienen representación exacta:

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

Aplicado a una factura cloud con 43.200 líneas de consumo horario, el error se acumula. La corrección es no usar coma flotante para valores exactos:

```
>>> from decimal import Decimal
>>> Decimal("0.1") + Decimal("0.2") == Decimal("0.3")
True
```

Hay un segundo límite, y es de interoperabilidad: JavaScript representa todos los números como double, así que solo garantiza enteros exactos hasta  $2^{53}-1 = 9.007.199.254.740.991$ . Un identificador de 64 bits enviado como número JSON pierde precisión al pasar por cualquier consumidor JavaScript. Por eso las APIs serias envían los identificadores grandes como cadena; en la parte 09 se verá la misma decisión en los sistemas de mensajería.

### 3. Validar antes de confiar

El error de automatización más caro es asumir que la respuesta tiene la forma documentada. Una API puede devolver 200 con un cuerpo distinto del esperado, y el fallo aparece tres capas más abajo, sin contexto:

```
# frágil: revienta en otro sitio, con un KeyError sin pistas
costo = respuesta["data"]["billing"]["total"]

# explícito: falla donde está el problema y dice cuál es
def leer_costo(respuesta: dict) -> Decimal:
    for clave in ("data", "billing"):
        if clave not in respuesta:
            raise ValueError(f"falta '{clave}' en la respuesta; claves: {sorted(respuesta)}")
        respuesta = respuesta[clave]
    bruto = respuesta.get("total")
    if not isinstance(bruto, str):
        raise TypeError(f"'total' debía ser cadena decimal, llegó {type(bruto).__name__}")
    return Decimal(bruto)
```

La diferencia práctica: el primero produce `KeyError: 'billing'` a las 3 de la madrugada; el segundo dice qué campo falta y qué llegó en su lugar. En un runbook —parte 21— esa distinción es la diferencia entre cinco minutos y dos horas.

## 4. Determinismo: la propiedad que hace verificable un laboratorio

Los 288 laboratorios de este programa emiten un contrato JSON que debe ser byte a byte idéntico con la misma semilla. Tres condiciones lo garantizan:

```
import json, random

random.seed(42) # 1. aleatoriedad reproducible
resultado = {"decision": "...", "evidencia": [...]}
print(json.dumps(resultado,
                  sort_keys=True, # 2. orden estable de claves
                  ensure_ascii=False)) # 3. codificación estable
```

Sin sortkeys, Python conserva el orden de inserción y dos ejecuciones que construyen el diccionario por caminos distintos producen bytes distintos con el mismo contenido. Sin semilla, no hay repetición posible.

Lo que nunca debe entrar en una salida comparada es la hora actual: convierte cualquier comparación en un falso negativo. Si hace falta registrar el instante, va en un fichero aparte o en un campo excluido de la comparación.

## 5. Fallos esperados frente a fallos inesperados

Un error de red al llamar a una API es esperado: ocurre, y el código debe reintentar. Un TypeError porque una función recibió una lista donde esperaba un diccionario es inesperado: es un defecto, y ocultarlo lo hace indetectable.

```
# destruye el diagnóstico: cualquier fallo se vuelve None
try:
    datos = pedir(url)
except Exception:
    datos = None

# maneja lo esperado y deja propagar lo demás
for intento in range(5):
    try:
        datos = pedir(url)
        break
    except (TimeoutError, ConnectionError) as e:
        espera = 2 ** intento + random.uniform(0, 1) # 1, 2, 4, 8, 16 s + jitter
        log.warning("intento %d falló (%s); reintento en %.1f s", intento + 1, e, espera)
        time.sleep(espera)
else:
    raise RuntimeError(f"{url} no respondió tras 5 intentos")
```

El retroceso exponencial con jitter no es un detalle: sin el término aleatorio, mil clientes que fallan a la vez reintentan a la vez y mantienen caído el servicio que intentan usar. Es el thundering herd, y reaparece en las partes 10 y 21.

## Ejemplo trabajado

Un script de FinOps suma el consumo horario de CloudShop y su total no cuadra con la factura del proveedor. La diferencia es de 0,37 USD sobre 8.412,55 — pequeña, pero impide cerrar el mes.

El script original:

```
total = 0.0
for linea in json.loads(respuesta)["lineItems"]:
    total += linea["cost"] # cost llega como número JSON
print(f"{total:.2f}")
```

Reproducción del error con las 720 líneas de un mes:

```
>>> valores = [0.0117] * 720 # coste horario de una instancia pequeña
>>> sum(valores)
8.423999999999999
>>> float(Decimal("0.0117") * 720)
8.424
```

Sobre una línea el error es de 10<sup>-4</sup>; sobre 43.200 líneas de consumo mensual —60 recursos × 720 horas— el error acumulado alcanza el orden de los céntimos. No es un fallo del proveedor: es que la suma se hizo en binario.

Corrección, sumando en decimal y validando el tipo de entrada:

```
from decimal import Decimal, ROUND_HALF_UP

total = Decimal("0")
```

```

for n, linea in enumerate(json.loads(respuesta)["lineItems"]):
    bruto = linea["cost"]
    if isinstance(bruto, float):
        raise TypeError(f"línea {n}: 'cost' llegó como float; exige cadena decimal")
    total += Decimal(str(bruto))
print(total.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))

8412.55          # cuadra con la factura

```

La comprobación de tipo es deliberada: convierte un error silencioso de céntimos en un fallo ruidoso en la primera línea. Si el proveedor cambia el formato y empieza a enviar cost como número, el script se para en vez de producir un total plausible pero incorrecto.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/004-python-json-y-automatizacion-minima/lab.py
```

El laboratorio selecciona el motor de práctica automation y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa script-de-diagnostico en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un script idempotente con salida estructurada. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye script-de-diagnostico para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                                                                  | Causa probable                                                                            | Corrección                                                                        |
|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Un total monetario difiere en céntimos de la factura oficial                             | Se sumó en coma flotante IEEE 754, que no representa decimales exactos                    | Usa Decimal construido desde cadena y redondea solo al final.                     |
| Un identificador de 64 bits llega alterado al frontend                                   | JavaScript solo garantiza enteros exactos hasta $2^{53}-1$                                | Transporta los identificadores grandes como cadena JSON.                          |
| Dos ejecuciones con la misma semilla producen ficheros distintos                         | El orden de claves depende del orden de inserción, o hay una marca de tiempo en la salida | Serializa con <code>sortkeys=True</code> y saca la hora de la salida comparada.   |
| El script falla con <code>KeyError</code> en un punto que no tiene relación con la causa | Se accedió a campos anidados sin validar la forma recibida                                | Valida presencia y tipo en el borde, y falla con un mensaje que diga qué faltaba. |
| Tras un incidente, mil clientes reintentan a la vez y el servicio no levanta             | Retroceso sin jitter: todos reintentan sincronizados                                      | Añade un término aleatorio al retroceso exponencial.                              |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Por qué `json.loads(json.dumps(d))` puede no devolver `d`? Da dos casos concretos.
2. Un endpoint devuelve `"id": 9007199254740993`. ¿Qué le ocurre a ese valor en un consumidor JavaScript y cómo se evita?
3. ¿Qué tres condiciones debe cumplir un laboratorio para que su salida sea comparable byte a byte?
4. ¿Cuándo es correcto capturar una excepción y cuándo hacerlo oculta un defecto?
5. Sin jitter, ¿qué le ocurre a un servicio que se recupera si mil clientes usan el mismo retroceso exponencial?

## Referencias

- Bray, T., ed. (2017). RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format — sección 6 sobre los límites de precisión numérica. <<https://www.rfc-editor.org/rfc/rfc8259#section-6>>
- IEEE (2019). Standard for Floating-Point Arithmetic (IEEE 754-2019). <<https://doi.org/10.1109/IEEESTD.2019.8766229>>
- Goldberg, D. (1991). What Every Computer Scientist Should Know About Floating-Point Arithmetic. ACM Computing Surveys 23(1). <<https://doi.org/10.1145/103162.103163>>
- Python Software Foundation (2024). decimal — Decimal fixed-point and floating-point arithmetic. <<https://docs.python.org/3/library/decimal.html>>
- Brooker, M. (2015). Exponential Backoff and Jitter. AWS Architecture Blog — datos comparados de las variantes de jitter. <<https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 004 Python, JSON y automatización mínima

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.

2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

## Reto verificable

Entrega script-de-diagnostico con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

## Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

## Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 005 — Redes por capas, TCP/IP, puertos y sockets

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Entender por qué una red se organiza en capas y qué garantiza cada una, para poder diagnosticar sin adivinar. Cuando en la parte 16 aparezcan VPC, tablas de rutas, NAT y balanceadores, todo será este modelo aplicado; y cuando un servicio «no responda», sabrás en qué capa mirar antes de tocar nada.

## Resultados de aprendizaje

Al finalizar podrás:

1. Situar un síntoma en la capa correcta y elegir la herramienta de diagnóstico que corresponde a esa capa.
2. Explicar qué garantiza TCP que UDP no, y qué cuesta esa garantía en tiempo de establecimiento.
3. Identificar una conexión por su cuádrupla y deducir cuántas conexiones simultáneas admite un cliente contra un mismo destino.
4. Interpretar los estados de un socket —LISTEN, SYNSENT, TIMEWAIT— para distinguir un puerto cerrado de un filtrado.
5. Calcular el throughput máximo de una conexión TCP a partir de su ventana y su latencia.

## Conceptos centrales

| Concepto             | Comprensión verificable                                                                                                                                                                                                                    |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| encapsulación        | Cada capa envuelve los datos de la superior con su propia cabecera y no interpreta el contenido. Es lo que permite cambiar de medio físico sin tocar la aplicación.                                                                        |
| socket               | Extremo de una comunicación, identificado por protocolo, IP y puerto. El sistema operativo lo expone como descriptor de fichero, de ahí que read y write funcionen sobre él igual que sobre un archivo.                                    |
| cuádrupla            | IP origen, puerto origen, IP destino y puerto destino. Identifica unívocamente una conexión TCP; dos conexiones pueden compartir tres de los cuatro valores pero no los cuatro.                                                            |
| MTU                  | Tamaño máximo de trama que un enlace transporta sin fragmentar. 1500 bytes en Ethernet estándar; los túneles restan espacio para su propia cabecera y son la causa habitual de conexiones que se establecen pero se cuelgan al transferir. |
| ventana de recepción | Bytes que el receptor declara poder aceptar sin confirmar. Junto con la latencia determina el techo de velocidad de una conexión TCP, independientemente del ancho de banda contratado.                                                    |

## Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

```
flowchart TB
    A["Aplicación · HTTP, DNS<br/>diagnóstico: curl, dig"] --> T["Transporte · TCP, UDP<br/>puertos, fiabilidad<br/>diagnóstico: ss, nc"]
    T --> R["Red · IP<br/>direccionamiento y rutas<br/>diagnóstico: ip route, traceroute"]
    R --> E["Enlace · Ethernet, ARP<br/>MTU y trama"]
    E --> F["Físico · fibra, radio"]
    F --> A
```

F --> A

## Desarrollo

### 1. Cada capa garantiza una cosa y solo una

El valor del modelo por capas no es taxonómico: es que acota dónde buscar. Cada capa promete algo concreto y no se hace cargo de lo demás:

| Capa             | Garantiza                           | No garantiza                   | Herramienta            |
|------------------|-------------------------------------|--------------------------------|------------------------|
| Aplicación       | Semántica del mensaje               | Que llegue                     | curl -v, dig           |
| Transporte (TCP) | Orden, integridad, entrega          | Tiempo de llegada              | ss -tnp, nc -vz        |
| Transporte (UDP) | Nada más que multiplexar por puerto | Orden ni entrega               | nc -u                  |
| Red (IP)         | Mejor esfuerzo hacia un destino     | Orden, entrega ni ruta estable | ip route, traceroute   |
| Enlace           | Entrega dentro de un mismo segmento | Nada fuera de él               | ip neigh, ping con MTU |

La regla de diagnóstico es de abajo hacia arriba: si la capa 3 no alcanza el destino, leer logs de la aplicación es tiempo perdido. En la parte 21, los runbooks de triaje seguirán exactamente este orden.

### 2. TCP paga con tiempo lo que ofrece en garantías

TCP establece la conexión antes de enviar un solo byte útil, con el saludo de tres vías:

```
cliente ■■■SYN■■■■■■■■■■■ servidor
cliente ■■■SYN+ACK■■■■■■■■■ servidor
cliente ■■■ACK■■■■■■■■■■■ servidor      ya se pueden enviar datos
```

Eso es un RTT completo antes del primer byte. Con Madrid–Virginia a 90 ms de ida y vuelta:

```
TCP                      90 ms
TLS 1.2 (2 RTT)          180 ms
-----
total antes del primer byte 270 ms

TCP                      90 ms
TLS 1.3 (1 RTT)          90 ms
-----
total                    180 ms
```

Por eso TLS 1.3 no es una mejora cosmética: elimina un RTT completo de cada conexión nueva. Y por eso la reutilización de conexiones (keep-alive, pooling) importa tanto: amortiza ese coste entre muchas peticiones. Un cliente que abre y cierra conexión por petición paga 180 ms de peaje cada vez.

UDP no establece nada: el primer datagrama ya lleva datos. A cambio, la aplicación se hace cargo del orden, la pérdida y la congestión. Es la elección correcta para DNS, telemetría y voz, donde llegar tarde es peor que no llegar.

### 3. Puertos, cuádruplas y el límite real de conexiones

El puerto no identifica una conexión: identifica un extremo. La conexión es la cuádrupla completa, y por eso mil clientes pueden hablar con el puerto 443 del mismo servidor sin ambigüedad: difieren en IP y puerto de origen.

El límite aparece al revés, cuando un cliente abre muchas conexiones al mismo destino. Solo puede variar su puerto de origen, y el rango efímero de Linux es:

```
$ cat /proc/sys/net/ipv4/ip_local_port_range
32768■■60999          # 28.232 puertos disponibles
```

Un proxy o un pod que llama a un mismo backend tiene un techo de 28.232 conexiones simultáneas, y en la práctica bastante menos porque cada cierre deja el puerto en TIMEWAIT durante 2×MSL (60 s en Linux):

conexiones/s sostenibles  $\approx 28.232 / 60 \text{ s} \approx 470$  por segundo

Superado ese ritmo aparece EADDRNOTAVAIL y el síntoma es «la aplicación falla bajo carga sin error de la aplicación». La solución no es más CPU: es reutilizar conexiones. Este cálculo reaparecerá en las partes 06 y 16 al dimensionar NAT gateways, que tienen exactamente el mismo límite por IP.



## 4. El estado del socket dice qué falla

ss muestra en qué punto del protocolo se atascó la conexión, y cada estado apunta a una causa distinta:

```
$ ss -tnp state all '( dport = :443 )'
State      Recv-Q Send-Q Local Address:Port Peer Address:Port
ESTAB      0      0      10.0.1.5:51234      93.184.216.34:443
SYN-SENT   0      1      10.0.1.5:51240      10.0.9.9:443
TIME-WAIT  0      0      10.0.1.5:51201      93.184.216.34:443
```

| Síntoma                           | Estado observado           | Causa probable                                     |
|-----------------------------------|----------------------------|----------------------------------------------------|
| Rechazo inmediato                 | Nada, ECONNREFUSED         | Puerto cerrado: nadie escucha, pero la IP responde |
| Cuelgue hasta timeout             | SYN-SENT persistente       | Filtrado: un firewall descarta en silencio         |
| Conecta y se cuelga al transferir | ESTAB con Send-Q creciendo | MTU: el saludo cabe, los datos no                  |
| Falla solo con carga              | Muchos TIME-WAIT           | Agotamiento de puertos efímeros                    |

La distinción entre rechazado y filtrado es la más útil de todas: rechazado significa que llegaste al host y el servicio no está; filtrado significa que no llegaste. Son dos equipos distintos los que arreglan cada cosa.

## 5. El producto ancho de banda × retardo pone el techo

Una conexión TCP no puede ir más rápido que lo que permite su ventana dividida por el tiempo de ida y vuelta. El emisor manda una ventana y espera confirmación antes de continuar:

$$\text{throughput}_{\text{máximo}} = \text{ventana} / \text{RTT}$$

Con la ventana clásica de 64 KB y un enlace transatlántico de 90 ms:

$$65.536 \text{ bytes} / 0,090 \text{ s} = 728.178 \text{ B/s} \approx 5,8 \text{ Mbit/s}$$

Da igual que el enlace sea de 10 Gbit/s: una sola conexión no pasará de 5,8 Mbit/s. Para llenar 1 Gbit/s con ese RTT hace falta:

$$\text{ventana} = 1.000.000.000 \text{ bit/s} \times 0,090 \text{ s} / 8 = 11,25 \text{ MB}$$

De ahí el window scaling de RFC 7323, que permite ventanas de hasta 1 GB, y de ahí que las transferencias intercontinentales usen varias conexiones en paralelo. Cuando alguien diga «contratamos más ancho de banda y no mejoró», la respuesta suele estar en esta fórmula.

## Ejemplo trabajado

El servicio de pagos de CloudShop falla de forma intermitente contra el proveedor externo. Los logs de la aplicación solo dicen «timeout». El operador diagnostica por capas, de abajo hacia arriba.

Capa 3 — ¿se alcanza el destino?

```
$ ip route get 203.0.113.40
203.0.113.40 via 10.0.0.1 dev eth0 src 10.0.1.5
$ traceroute -n 203.0.113.40 | tail -2
7  198.51.100.9  12.4 ms
8  203.0.113.40  14.1 ms      # alcanzable
```

Capa 4 — ¿se establece la conexión?

```
$ nc -vz 203.0.113.40 443
Connection to 203.0.113.40 443 port [tcp/https] succeeded!
```

Conecta. Así que no es ruta ni firewall. El fallo es intermitente y bajo carga, así que se mira el estado de los sockets durante el pico:

```
$ ss -tan state time-wait | wc -l
27891
$ ss -tan state all '( dport = :443 )' | grep -c ESTAB
412
$ dmesg | tail -1
TCP: request_sock_TCP: Possible SYN flooding...
$ cat /proc/sys/net/ipv4/ip_local_port_range
32768 60999
```

27.891 puertos en TIMEWAIT sobre 28.232 disponibles. El servicio agota el rango efímero:

```

disponibles          = 60.999 - 32.768 = 28.232
en TIME_WAIT         = 27.891  (98,8 %)
libres               =    341
ritmo sostenible     = 28.232 / 60 s ≈ 470 conexiones/s
ritmo observado en pico      ≈ 610 conexiones/s

```

El servicio abre una conexión TCP nueva por cada pago y la cierra. A 610 pagos por segundo pide 140 conexiones/s más de las que el sistema puede reciclar, y los connect() empiezan a devolver EADDRNOTAVAIL, que el cliente HTTP reporta como «timeout».

La corrección es reutilizar conexiones, no tocar el sistema operativo:

```

sesion = requests.Session()          # pool persistente con keep-alive
adapter = HTTPAdapter(pool_maxsize=64)
sesion.mount("https://", adapter)

$ ss -tan state time-wait | wc -l
743                                # de 27.891 a 743

```

64 conexiones reutilizadas sustituyen a 610 por segundo. Subir net.ipv4.tcp\_twreuse habría escondido el síntoma un tiempo; el problema era de diseño del cliente, y estaba a dos capas de donde los logs decían.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/005-redes-por-capas-tcp-ip-puertos-y-sockets/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa captura-y-mapa-de-red en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye captura-y-mapa-de-red para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                                                    | Causa probable                                                                   | Corrección                                                                                     |
|----------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| El servicio falla solo bajo carga y sin error propio de la aplicación      | Agotamiento de puertos efímeros por abrir una conexión nueva por petición        | Usa un pool con keep-alive; el techo es 28.000 puertos y 60 s de TIMEWAIT por cierre.          |
| La conexión se establece pero se cuelga al transferir datos                | MTU insuficiente en un túnel: el saludo cabe en paquetes pequeños y los datos no | Prueba con ping -M do -s para hallar la MTU real y ajusta MSS clamping.                        |
| Se contrató más ancho de banda y la transferencia no mejoró                | El techo lo pone ventana/RTT, no el caudal del enlace                            | Habilita window scaling o paraleliza conexiones; calcula el producto ancho de banda × retardo. |
| Se depuran logs de la aplicación durante una hora y el problema era de red | Se diagnosticó de arriba hacia abajo                                             | Verifica ruta y establecimiento de conexión antes de leer la aplicación.                       |
| No se distingue si el puerto está cerrado o filtrado                       | Se interpretó cualquier fallo de conexión como lo mismo                          | Rechazo inmediato es puerto cerrado; cuelgue en SYN-SENT es filtrado. Son equipos distintos.   |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Cuántos RTT median entre el primer paquete y el primer byte útil con TCP + TLS 1.3, y cuántos con TLS 1.2?
2. Un pod abre conexiones a un solo backend. ¿Cuál es su techo teórico de conexiones simultáneas y de dónde sale ese número?
3. Con ventana de 64 KB y RTT de 200 ms, ¿cuál es el throughput máximo de una conexión, y cambia si contratas 10 Gbit/s?
4. ¿Qué diferencia operativa hay entre un ECONNREFUSED inmediato y un cuelgue en SYN-SENT?
5. Una conexión se establece y se cuelga al enviar un cuerpo grande. ¿Qué capa sospechas y con qué comando lo confirmas?

## Referencias

- Kurose, J. y Ross, K. (2021). Computer Networking: A Top-Down Approach, 8.<sup>a</sup> ed., caps. 3-4 — transporte y capa de red.
- Postel, J., ed. (1981). RFC 793: Transmission Control Protocol — saludo de tres vías y máquina de estados. <<https://www.rfc-editor.org/rfc/rfc793>>
- Borman, D. et al. (2014). RFC 7323: TCP Extensions for High Performance — window scaling y el producto ancho de banda × retardo. <<https://www.rfc-editor.org/rfc/rfc7323>>
- Rescorla, E. (2018). RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3 — saludo de 1-RTT. <<https://www.rfc-editor.org/rfc/rfc8446>>
- Kerrisk, M. ss(8) y tcp(7) — estados de socket y parámetros del núcleo de Linux. <<https://man7.org/linux/man-pages/man7/tcp.7.html>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 005 Redes por capas, TCP/IP, puertos y sockets

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega captura-y-mapa-de-red con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 006 — DNS, HTTP, HTTPS y TLS de extremo a extremo

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Seguir una petición HTTPS desde el nombre hasta el byte: quién resuelve el nombre y con qué caché, qué negocia TLS y qué demuestra un certificado, y qué significan realmente los códigos y cabeceras de HTTP. Es el camino que recorren todas las peticiones del programa, y donde se originan la mayoría de incidentes de disponibilidad percibida.

## Resultados de aprendizaje

Al finalizar podrás:

1. Trazar una resolución DNS completa e identificar en qué caché se queda un cambio que «no se propaga».
2. Explicar qué demuestra un certificado TLS y qué no, y por qué el cifrado no implica confianza en el otro extremo.
3. Elegir un TTL de DNS en función de la ventana de conmutación que necesita tu plan de continuidad.
4. Distinguir un 502 de un 503 y de un 504 para saber a qué componente apunta cada uno.
5. Diseñar una política de caché HTTP que permita invalidar sin esperar a que expire.

## Conceptos centrales

| Concepto            | Comprensión verificable                                                                                                                                                                                                           |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| resolutor recursivo | Servidor que hace el trabajo de consultar la cadena de autoridades en nombre del cliente y guarda el resultado en caché. Es quien realmente decide durante cuánto tiempo verás una respuesta antigua.                             |
| TTL                 | Segundos que un registro DNS puede permanecer en caché. Fija el suelo del tiempo de conmutación: con TTL de 3600, un cambio de IP tarda hasta una hora en verse, sin importar lo rápido que lo publiques.                         |
| SNI                 | Extensión de TLS que envía el nombre del host solicitado en claro dentro del saludo, para que un servidor con muchos certificados sepa cuál presentar. Al ir en claro, revela qué sitio visitas aunque el contenido vaya cifrado. |
| cadena de confianza | Secuencia de firmas desde el certificado del servidor hasta una raíz que el cliente ya considera fiable. Un certificado válido demuestra control del nombre, no honestidad del operador.                                          |
| caché condicional   | Mecanismo por el que el cliente pregunta «¿ha cambiado desde esta versión?» con If-None-Match. Si no cambió, el servidor responde 304 sin cuerpo, ahorrando la transferencia pero no el viaje.                                    |

## Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    C["cliente"] -->|1 ¿A de shop.ejemplo?"| R["resolutor recursivo"]
    R -->|2| ROOT["raíz · NS de .cl"]
    R -->|3| TLD["TLD · NS de ejemplo.cl"]
    R -->|4| AUT["autoritativo · A = 203.0.113.7"]
    R -->|5 respuesta + TTL"| C
    C -->|6 TCP + TLS: SNI, certificado"| S["servidor 203.0.113.7"]
    S -->|7 HTTP: método, cabeceras, código"| C
    R -.->|caché: sirve sin pasos 2-4 hasta que expira el TTL"| C
```

## Desarrollo

### 1. DNS: la caché que decide cuánto dura tu error

Una resolución completa recorre la jerarquía, pero casi ninguna lo hace: el resolutor responde desde caché. Esa caché es la que gobierna la realidad operativa.

```
$ dig +trace shop.ejemplo.cl A | tail -4
ejemplo.cl.      172800  IN  NS   ns1.ejemplo.cl.
shop.ejemplo.cl. 300    IN  A    203.0.113.7      # TTL de 300 s

$ dig shop.ejemplo.cl A +noall +answer
shop.ejemplo.cl. 187    IN  A    203.0.113.7      # quedan 187 s en caché
```

El TTL es una decisión de arquitectura de continuidad, no un parámetro técnico:

| TTL     | Coste en consultas | Ventana de conmutación | Cuándo                             |
|---------|--------------------|------------------------|------------------------------------|
| 60 s    | Alto               | ≤ 1 min                | Antes de una migración planificada |
| 300 s   | Medio              | ≤ 5 min                | Servicios con failover DNS         |
| 3600 s  | Bajo               | ≤ 1 h                  | Registros estables                 |
| 86400 s | Mínimo             | ≤ 24 h                 | NS, MX, registros de verificación  |

La regla operativa: baja el TTL antes de necesitarlo. Bajarlo a 60 s en mitad de un incidente no sirve, porque los resolutores siguen sirviendo la respuesta anterior con su TTL original. Hay que bajarlo con al menos un TTL antiguo de antelación.

Y hay un límite que no controlas: muchos resolutores públicos y sistemas operativos imponen mínimos propios e ignoran TTL muy bajos. DNS es control de tráfico con granularidad de minutos, nunca de segundos; por eso en la parte 13 el failover regional no se apoya solo en DNS.

### 2. Qué demuestra un certificado y qué no

TLS resuelve tres problemas distintos y conviene no confundirlos:

1. Confidencialidad: nadie en el camino lee el contenido.
2. Integridad: nadie lo modifica sin que se detecte.
3. Autenticación del servidor: hablas con quien dice el nombre.

El certificado solo aporta el tercero, y de forma acotada: prueba que alguien demostró control sobre ese nombre de dominio ante una autoridad en la que tu cliente confía. No dice nada sobre la honestidad del operador ni la seguridad de su código. Un sitio de phishing con certificado válido es exactamente eso: cifrado, íntegro y fraudulento.

```
$ openssl s_client -connect shop.ejemplo.cl:443 -servername shop.ejemplo.cl </dev/null 2>/dev/null \
| openssl x509 -noout -subject -issuer -dates
subject=CN=shop.ejemplo.cl
issuer=C=US, O=Let's Encrypt, CN=R11
notBefore=Jul  2 00:00:00 2026 GMT
notAfter=Sep 30 23:59:59 2026 GMT
```

El -servername es SNI y es obligatorio: sin él, un servidor con varios certificados presenta el que tenga por defecto y el diagnóstico induce a error. Como SNI viaja en claro, un observador de red sabe qué sitio visitas aunque no vea el contenido; ECH (Encrypted Client Hello) es la respuesta en curso a esa fuga.

### 3. Los códigos HTTP dicen quién falló

Los códigos 5xx no son intercambiables: cada uno apunta a un componente distinto de la cadena, y confundirlos alarga los incidentes.

| Código | Significado                 | Quién falló                                          |
|--------|-----------------------------|------------------------------------------------------|
| 500    | Error interno               | La aplicación de origen: excepción no controlada     |
| 502    | Puerta de enlace incorrecta | El proxy habló con el origen y recibió basura o nada |

| Código | Significado                 | Quién falló                                       |
|--------|-----------------------------|---------------------------------------------------|
| 503    | Servicio no disponible      | El origen se declara saturado o en mantenimiento  |
| 504    | Tiempo agotado en la puerta | El proxy esperó y el origen no respondió a tiempo |

La distinción práctica entre 502 y 504: en el 502 el origen respondió algo inválido o cerró la conexión, así que hay que mirar sus logs; en el 504 no respondió nada dentro del plazo, así que hay que mirar su latencia y saturación. Buscar excepciones en el origen ante un 504 no lleva a ningún sitio, porque probablemente la petición sigue procesándose.

El 503 tiene un matiz que casi nadie usa: admite la cabecera `Retry-After`, que le dice al cliente cuándo volver. Sin ella, los clientes reintentan de inmediato y agravan la saturación que causó el 503 — el mismo thundering herd de la clase 004.

#### 4. Caché HTTP: el viaje que no se hace

La petición más rápida es la que no ocurre. HTTP ofrece dos niveles, y elegir mal el primero condena a soportar contenido obsoleto:

```
Cache-Control: public, max-age=31536000, immutable
```

Para recursos con huella en el nombre (`app.4f2a9c.js`): un año, sin revalidar. Como el nombre cambia con el contenido, invalidar es publicar un nombre nuevo; nunca hay que purgar nada.

```
Cache-Control: no-cache
ETag: "7d2b8e91"
```

Para documentos que cambian (`index.html`): `no-cache` no significa «no almacenar», significa «almacena pero revalida antes de usar». El cliente pregunta con `If-None-Match: "7d2b8e91"` y recibe 304 Not Modified sin cuerpo si no cambió: se ahorra la transferencia, no el viaje.

Para prohibir el almacenamiento de verdad hace falta `no-store`, y solo tiene sentido en respuestas con datos personales o tokens.

El error clásico es servir `index.html` con `max-age` largo: entonces una corrección urgente no llega hasta que expire, y no hay forma de forzarlo desde el servidor. Por eso el patrón que se repetirá en las partes 16 y 17 es siempre el mismo: HTML corto y revalidable, activos largos e inmutables con huella en el nombre.

#### 5. El coste completo de una petición en frío

Sumando lo anterior, la primera petición a un origen lejano paga varios peajes secuenciales:

|                             |                             |
|-----------------------------|-----------------------------|
| resolución DNS (caché fría) | 1 RTT al resolutor + cadena |
| establecimiento TCP         | 1 RTT                       |
| saludo TLS 1.3              | 1 RTT                       |
| petición y respuesta HTTP   | 1 RTT                       |
| -----                       |                             |
| mínimo                      | ≈ 4 RTT                     |

Con un RTT de 90 ms son 360 ms antes de mostrar nada, y ninguna optimización del servidor los reduce. Las palancas son todas de red:

- Reutilizar la conexión elimina TCP y TLS de las peticiones siguientes.
- Acercar la terminación (CDN, edge) reduce el RTT que multiplica a todo lo demás.
- 0-RTT de TLS 1.3 permite enviar datos en el primer paquete al reconectar, con la contrapartida de que esos datos son vulnerables a repetición: solo vale para peticiones idempotentes.

Es el mismo cálculo de la clase 001, ahora con los protocolos concretos que lo producen.

### Ejemplo trabajado

CloudShop migra su frontend a una IP nueva. El equipo publica el cambio a las 10:00 y a las 11:30 sigue habiendo usuarios llegando al servidor viejo. Se acusa al proveedor de DNS.

Primero se comprueba qué publica la autoridad, sin pasar por caché:

```
$ dig @ns1.ejemplo.cl shop.ejemplo.cl A +noall +answer
shop.ejemplo.cl. 3600 IN A 203.0.113.44      # correcto, ya es la IP nueva
```

La autoridad está bien. Ahora qué ven los resolutores públicos:

```
$ dig @8.8.8.8 shop.ejemplo.cl A +noall +answer
shop.ejemplo.cl. 1187 IN A 203.0.113.7      # IP VIEJA, 1187 s restantes
$ dig @1.1.1.1 shop.ejemplo.cl A +noall +answer
shop.ejemplo.cl. 942 IN A 203.0.113.7
```

El TTL era 3600 s. Cuando se publicó el cambio, los resolutores ya tenían la respuesta anterior cacheada, cada uno con su reloj propio:

|                                  |       |                                                             |
|----------------------------------|-------|-------------------------------------------------------------|
| publicación del cambio           | 10:00 |                                                             |
| peor caso: un resolutor cacheó a | 09:59 | → expira a 10:59                                            |
| ventana teórica de convergencia  |       | → hasta 11:00                                               |
| observado a 11:30                |       | → hay resolutores con TTL de 3600 s que refrescaron a 10:30 |

La aritmética explica el residual: cada resolutor que refresca justo antes del cambio arrastra una hora más. El tiempo de convergencia no es el TTL: es hasta dos veces el TTL si el cambio coincide con el peor momento de refresco.

Lo que debió hacerse, con un día de antelación:

|           |                                           |                            |
|-----------|-------------------------------------------|----------------------------|
| D-1 09:00 | bajar TTL de 3600 a 60                    | (empieza a propagarse)     |
| D-1 10:00 | todos los resolutores tienen ya TTL de 60 |                            |
| D 10:00   | cambiar la IP                             | (converge en $\leq 2$ min) |
| D 12:00   | restaurar TTL a 3600                      |                            |

En el incidente real, la mitigación no fue DNS sino red: se mantuvo el servidor viejo respondiendo con un 308 Permanent Redirect hacia el nuevo hasta que la caché expiró.

La lección: DNS no es un conmutador. Es una caché distribuida cuyo tiempo de reacción se decide con un TTL, y ese TTL hay que bajarlo antes de necesitarlo. Por eso los planes de continuidad de la parte 13 no dependen de DNS para conmutaciones de segundos.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/006-dns-http-https-y-tls-de-extremo-a-extremo/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa traza-de-solicitud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye traza-de-solicitud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.



- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                                                     | Causa probable                                                                         | Corrección                                                                                                 |
|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Se cambia una IP y horas después sigue llegando tráfico al servidor antiguo | El TTL alto estaba cacheado en los resolutores antes del cambio                        | Baja el TTL con al menos un TTL de antelación; cuenta hasta dos veces el TTL como ventana de convergencia. |
| openssl sclient muestra un certificado que no corresponde al sitio          | Se omitió -servername, así que el servidor presentó su certificado por defecto         | Incluye siempre SNI con -servername al diagnosticar hosts virtuales.                                       |
| Se buscan excepciones en el origen ante un 504 y no aparece ninguna         | El 504 significa que el origen no respondió a tiempo, no que fallara                   | Ante 504 mira latencia y saturación del origen; ante 502, sus logs de error.                               |
| Una corrección urgente del HTML no llega a los usuarios                     | Se sirvió el documento con max-age largo y no hay forma de invalidar desde el servidor | HTML con no-cache y ETag; activos inmutables con huella en el nombre.                                      |
| Un 503 provoca una avalancha de reintentos que empeora la saturación        | La respuesta no incluyó Retry-After y los clientes reintentaron de inmediato           | Devuelve Retry-After y aplica retroceso con jitter en el cliente.                                          |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. Con un TTL de 3600 s, ¿cuál es el peor caso de convergencia tras cambiar una IP, y por qué no es una hora?
2. ¿Qué demuestra exactamente un certificado TLS válido, y qué afirmación sobre el sitio no respalda?
3. Un proxy devuelve 502 y otro 504. ¿En cuál mirarías los logs del origen y en cuál sus métricas de latencia?
4. ¿Por qué no-cache no impide almacenar, y qué cabecera sí lo impide?
5. ¿Cuántos RTT tiene una petición HTTPS en frío y cuáles se eliminan reutilizando la conexión?

## Referencias

- Mockapetris, P. (1987). RFC 1035: Domain Names — Implementation and Specification.  
<<https://www.rfc-editor.org/rfc/rfc1035>>
- Rescorla, E. (2018). RFC 8446: TLS 1.3 — saludo de 1-RTT, 0-RTT y sus riesgos de repetición.  
<<https://www.rfc-editor.org/rfc/rfc8446>>
- Fielding, R. y Reschke, J., eds. (2022). RFC 9110: HTTP Semantics — semántica de códigos de estado.  
<<https://www.rfc-editor.org/rfc/rfc9110>>
- Fielding, R. et al., eds. (2022). RFC 9111: HTTP Caching — Cache-Control, validadores y respuestas 304.  
<<https://www.rfc-editor.org/rfc/rfc9111>>
- Grigorik, I. (2013). High Performance Browser Networking, caps. 1-4 — coste en RTT de DNS, TCP y TLS.  
<<https://hpbnp.co/>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 006 DNS, HTTP, HTTPS y TLS de extremo a extremo

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega traza-de-solicitud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 007 — Linux: usuarios, permisos, servicios y logs

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: linux · Estado: EXECUTABLECORE

## Propósito

Entender el modelo de identidad y privilegio de Linux, y el ciclo de vida de un servicio bajo systemd, porque son la base literal de todo lo que viene: los contenedores de la parte 05 son procesos Linux con espacios de nombres, y el mínimo privilegio de IAM en la parte 11 es esta misma idea trasladada a la nube.

## Resultados de aprendizaje

Al finalizar podrás:

1. Descomponer un modo de permisos octal y explicar por qué el bit de ejecución significa cosas distintas en fichero y en directorio.
2. Sustituir el uso de root por capacidades concretas, justificando cuál se necesita y por qué.
3. Definir una unidad de systemd con reinicio, límites y usuario propio, y explicar cada directiva.
4. Consultar logs con journalctl filtrando por unidad, prioridad y ventana temporal.
5. Diagnosticar un servicio que no arranca distinguiendo permisos, dependencias y configuración.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                                                                                                      |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UID efectivo | Identificador con el que el núcleo evalúa cada comprobación de permiso. Puede diferir del UID real tras un setuid, y es el que importa para decidir si una operación se permite.             |
| capacidad    | Fragmento del poder histórico de root, otorgable por separado. CAPNETBINDSERVICE permite abrir puertos bajo 1024 sin conceder las otras 40 capacidades que root arrastra.                    |
| unidad       | Objeto que systemd gestiona: servicio, socket, temporizador, punto de montaje o destino. Se declara de forma descriptiva y systemd deduce el orden de arranque a partir de las dependencias. |
| journal      | Registro binario, indexado y estructurado de systemd. Cada entrada lleva metadatos —unidad, PID, UID, prioridad— consultables como campos, no como texto.                                    |
| umask        | Máscara que se resta de los permisos por defecto al crear ficheros. Con umask 022, un fichero nace 644 y un directorio 755; explica por qué lo que creas no es de escritura para el grupo.   |

## Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

```
flowchart TB
    K["núcleo: ¿permitir esta operación?"] --> U1["¿UID efectivo = propietario?"]
    U1 -->|sí| PU1["aplica los 3 bits de usuario"]
    U1 -->|no| G1["¿pertenece al grupo?"]
    G1 -->|sí| PG1["aplica los 3 bits de grupo"]
    G1 -->|no| PO1["aplica los 3 bits de otros"]
    PU1 --> D1["decisión"]
    PG1 --> D1
    PO1 --> D1
    D1 -->|si UID efectivo = 0<br/>se omite y se miran capacidades| C["CAP_NET_BIND_SERVICE,<br/>CAP_CHOWN, CAP_SYS_ADMIN..."]
```

## Desarrollo

### 1. Permisos: tres tríos y una regla que sorprende

Cada objeto del sistema de ficheros tiene propietario, grupo y tres tríos de bits. El núcleo evalúa el primer trío que aplica y se detiene ahí:

```
$ ls -l /opt/cloudshop/config.yaml
-rw-r----- 1 cloudshop cloudshop 412 Aug  1 09:12 config.yaml
# ↑↑↑↑↑↑↑↑
#  rw-  usuario cloudshop: leer y escribir
#    r--  grupo cloudshop: solo leer
#      ---  otros: nada
```

En octal, 640. La consecuencia contraintuitiva: si eres el propietario y el trío de usuario deniega, no te salva pertenecer al grupo. Un fichero 0640 propiedad de ana y grupo ana es ilegible para ana si el primer trío fuera ---, aunque su grupo pueda leerlo.

El bit de ejecución significa cosas distintas según el objeto:

| Bit | En fichero             | En directorio                         |
|-----|------------------------|---------------------------------------|
| r   | Leer el contenido      | Listar los nombres                    |
| w   | Modificar el contenido | Crear y borrar entradas               |
| x   | Ejecutarlo             | Atravesarlo para llegar a su interior |

De ahí dos efectos que confunden a diario:

- Un directorio r-- deja ver los nombres pero no consultar los metadatos de lo que contiene: ls funciona y ls -l da «permiso denegado».
- Para borrar un fichero no hace falta permiso sobre el fichero, sino w sobre el directorio que lo contiene. Por eso /tmp lleva el sticky bit (1777): permite a todos crear, pero solo al propietario borrar lo suyo.

### 2. root no es un usuario: es la ausencia de comprobaciones

Cuando el UID efectivo es 0, el núcleo omite la comprobación de permisos. Eso no es un privilegio más: es no tener ninguno que aplicar. Linux fragmentó ese poder en unas 40 capacidades otorgables por separado:

| Capacidad         | Permite                        | Uso legítimo             |
|-------------------|--------------------------------|--------------------------|
| CAPNETBINDSERVICE | Abrir puertos < 1024           | Un servidor web en el 80 |
| CAPCHOWN          | Cambiar el propietario         | Gestores de paquetes     |
| CAPSYSADMIN       | Montar, espacios de nombres... | Casi equivale a root     |

El caso que se repite: un servicio quiere escuchar en el 443 y alguien lo ejecuta como root «porque si no, no puede». La alternativa correcta cede solo lo necesario:

```
[Service]
User=cloudshop
AmbientCapabilities=CAP_NET_BIND_SERVICE
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
```

Si ese proceso resulta vulnerable, el atacante obtiene la capacidad de abrir puertos bajos y nada más: no puede leer /etc/shadow, ni cargar módulos, ni cambiar propietarios. CAPSYSADMIN es la excepción: concederla equivale prácticamente a conceder root, y verla en un manifiesto debe activar la misma alarma que privileged: true en la parte 06.

### 3. systemd declara el estado deseado, no los pasos

Una unidad no describe cómo arrancar: describe qué debe cumplirse. systemd deduce el orden a partir de las dependencias y paraleliza el resto.

```
[Unit]
Description=API de CloudShop
After=network-online.target           # orden: después de que haya red
```

```
Wants=network-online.target          # dependencia débil: si falla, seguimos

[Service]
Type=notify                          # el proceso avisa cuando está listo
User=cloudshop
ExecStart=/opt/cloudshop/bin/api
Restart=on-failure                   # no reiniciar si salió con 0
RestartSec=5
StartLimitBurst=5                    # 5 intentos...
StartLimitIntervalSec=300            # ...en 5 minutos, o se rinde

NoNewPrivileges=true                 # ningún hijo escala privilegios
ProtectSystem=strict                 # todo el sistema en solo lectura
ProtectHome=true
PrivateTmp=true                      # /tmp propio, aislado
ReadWritePaths=/var/lib/cloudshop    # única excepción de escritura
MemoryMax=512M

[Install]
WantedBy=multi-user.target
```

Dos distinciones que evitan incidentes:

- Requires frente a Wants: Requires arrastra la caída del dependiente si la dependencia falla; Wants no. Usar Requires a la ligera propaga fallos en cascada.
- After no implica dependencia: solo fija orden. Un servicio con After=X pero sin Wants=X arranca aunque X no exista.

StartLimitBurst merece atención: sin él, un servicio que falla al arrancar entra en bucle de reinicio consumiendo CPU. Con él, systemd se rinde y deja el fallo visible en vez de esconderlo tras reintentos infinitos.

#### 4. El journal es estructurado: consúltalo como tal

journalctl no busca texto: filtra campos indexados. Tratarlo con grep desperdicia su ventaja y produce falsos negativos.

```
# Errores y peores de una unidad en la última hora
$ journalctl -u cloudshop -p err --since "1 hour ago"

# Solo el arranque actual, siguiendo en vivo
$ journalctl -u cloudshop -b -f

# Salida estructurada para automatizar
$ journalctl -u cloudshop -o json --since today | jq -r 'select(.PRIORITY<="3") | .MESSAGE'

# Qué escribió un PID concreto, aunque ya no exista
$ journalctl _PID=4877
```

Las prioridades siguen la escala syslog, de 0 (emergencia) a 7 (depuración); -p err incluye 0 a 3. Un servicio que escribe todo como info hace inútil este filtro: la prioridad es parte del contrato del log, igual que stdout y stderr lo eran en la clase 002.

Dos límites operativos que importan: el journal es local y rotatorio, con tamaño acotado por SystemMaxUse. Si el disco se llena, se descartan las entradas más antiguas. Por eso en la parte 10 los logs se envían fuera del host: un log que solo existe en la máquina que falló no sirve para el postmortem.

#### 5. Diagnóstico de un servicio que no arranca

El orden ahorra horas, porque cada paso descarta una familia de causas:

```
$ systemctl status cloudshop          # 1. ¿qué dice systemd?
$ journalctl -u cloudshop -n 50 -p warning # 2. ¿qué dijo el proceso?
$ systemd-analyze verify cloudshop.service # 3. ¿la unidad es válida?
$ sudo -u cloudshop /opt/cloudshop/bin/api # 4. ¿arranca a mano con ese usuario?
$ systemctl show cloudshop -p ExecMainStatus -p ExecMainCode
```

El paso 4 es el que más separa: si a mano funciona con el mismo usuario, el problema está en el entorno de la unidad —PATH, variables, directorio de trabajo, endurecimiento— y no en la aplicación. Es exactamente el fallo de la clase 002.

Los códigos más frecuentes y su lectura inmediata:

| ExecMainStatus | Causa habitual                                         |
|----------------|--------------------------------------------------------|
| 203            | ExecStart no existe o no es ejecutable                 |
| 200            | El directorio de trabajo no existe                     |
| 208            | User= no existe                                        |
| 226            | El endurecimiento bloqueó una ruta necesaria           |
| 1-125          | Fallo propio de la aplicación: ahora sí, mira sus logs |

Los códigos 200-241 son de systemd, no de tu programa: significan que el proceso nunca llegó a ejecutarse.

## Ejemplo trabajado

La API de CloudShop arranca en desarrollo y falla en el servidor nuevo. El log de la aplicación está vacío.

```
$ systemctl status cloudshop
● cloudshop.service - API de CloudShop
   Active: failed (Result: exit-code)
   Process: 8812 ExecStart=/opt/cloudshop/bin/api (code=exited, status=226/NAMESPACE)
```

226/NAMESPACE: no es la aplicación. systemd impidió el arranque por el endurecimiento antes de ejecutar una sola línea. El log vacío ahora se explica: el proceso nunca corrió.

```
$ journalctl -u cloudshop -n 5 -p err
cloudshop.service: Failed to set up mount namespaces:
/var/log/cloudshop: Read-only file system
```

La unidad declara ProtectSystem=strict, que monta todo el sistema en solo lectura, y la aplicación escribe en /var/log/cloudshop, que no está en las excepciones:

```
$ systemctl show cloudshop -p ProtectSystem -p ReadWritePaths
ProtectSystem=strict
ReadWritePaths=/var/lib/cloudshop
```

Hay dos correcciones posibles y no son equivalentes:

```
# A) Ampliar la excepción: mantiene el endurecimiento
ReadWritePaths=/var/lib/cloudshop /var/log/cloudshop

# B) Escribir al journal por stdout: elimina la necesidad
# (se quita la ruta de log de la aplicación; systemd captura stdout)
```

Se elige B. Razón operativa, no estética: un fichero de log local se pierde cuando la instancia se recicla, no rota solo, y llena el disco. Escribir a stdout deja que systemd lo estructure y que el recolector de la parte 10 lo envíe fuera del host. Es además lo que exigirá el contenedor de la parte 05, donde escribir logs a fichero es directamente un antipatrón.

Comprobación final del privilegio real que conserva el servicio:

```
$ systemctl restart cloudshop && systemctl show cloudshop -p ExecMainStatus
ExecMainStatus=0
$ grep CapEff /proc/$(systemctl show -p MainPID --value cloudshop)/status
CapEff:█00000000000000400          # solo CAP_NET_BIND_SERVICE
```

0x400 es el bit 10, exactamente CAPNETBINDSERVICE. Si esa API resultara vulnerable, el atacante hereda el poder de abrir puertos bajos y nada más. Ese es el mismo razonamiento que en la parte 11 se aplicará a un rol de IAM: no «qué necesita para funcionar», sino qué obtiene quien lo comprometa.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/007-linux-usuarios-permisos-servicios-y-logs/lab.py
```

El laboratorio selecciona el motor de práctica linux y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.

2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa servicio-auditado en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un servicio con identidad, permisos y logs inspeccionados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye servicio-auditado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                                                  | Causa probable                                                             | Corrección                                                                                    |
|--------------------------------------------------------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Un servicio se ejecuta como root solo para escuchar en el puerto 443     | Se concedió todo el poder para obtener una capacidad concreta              | Usa User= propio más AmbientCapabilities=CAPNETBINDSERVICE.                                   |
| El servicio falla con 226/NAMESPACE y el log de la aplicación está vacío | El endurecimiento de systemd bloqueó una ruta antes de ejecutar el proceso | Los códigos 200-241 son de systemd: revisa ReadWritePaths y ProtectSystem, no el código.      |
| Un servicio que falla al arrancar consume CPU en bucle de reinicio       | Restart=always sin límite de intentos                                      | Usa Restart=on-failure con StartLimitBurst y StartLimitIntervalSec.                           |
| Tras un incidente no hay logs de la máquina afectada                     | El journal es local y rotatorio; se perdió con la instancia                | Envía los logs fuera del host; el journal sirve para diagnóstico en vivo, no para postmortem. |
| Un usuario no puede borrar un fichero del que es propietario             | Borrar exige w sobre el directorio contenedor, no sobre el fichero         | Revisa los permisos del directorio; el sticky bit en /tmp explota justamente esta regla.      |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. Un fichero es 0640, propiedad de ana, grupo ana. ¿Puede ana escribirlo? ¿Y si el trío de usuario fuera --- y el de grupo rw-?
2. ¿Por qué ls funciona y ls -l falla en un directorio con permisos r--?
3. ¿Qué obtiene un atacante que compromete un proceso con CAPNETBINDSERVICE frente a uno que corre como root?

- ¿Qué diferencia práctica hay entre Requires= y Wants=, y cuál propaga fallos en cascada?
- Un servicio devuelve ExecMainStatus=226. ¿Tiene sentido revisar el código de la aplicación? ¿Por qué?

## Referencias

- Kerrisk, M. capabilities(7) — catálogo completo de capacidades de Linux y sus implicaciones. <<https://man7.org/linux/man-pages/man7/capabilities.7.html>>
- systemd (2024). systemd.exec(5) — directivas de endurecimiento: ProtectSystem, ReadWritePaths, NoNewPrivileges. <<https://www.freedesktop.org/software/systemd/man/systemd.exec.html>>
- systemd (2024). systemd.service(5) — códigos de salida 200-241 y política de reinicio. <<https://www.freedesktop.org/software/systemd/man/systemd.service.html>>
- systemd (2024). journalctl(1) — filtrado por campos, prioridades y arranque. <<https://www.freedesktop.org/software/systemd/man/journalctl.html>>
- Ward, B. (2021). How Linux Works, 3.<sup>a</sup> ed., caps. 6-7 — arranque, systemd y configuración del sistema.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 007 Linux: usuarios, permisos, servicios y logs

### Preguntas

- Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
- Identifica una frontera de responsabilidad y su propietario.
- Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
- ¿Qué demuestra el laboratorio y qué no puede demostrar?
- Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega servicio-auditado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |



## 008 — Virtualización, hipervisores e imágenes

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: virtualization · Estado: EXECUTABLECORE

### Propósito

Distinguir los tres mecanismos que la industria llama «virtualización» —máquinas virtuales, contenedores y microVM— por lo que realmente aíslan y lo que realmente cuestan. Sin esa distinción, la elección entre EC2, Fargate y Lambda en la parte 17 se hace por moda; con ella, se hace por requisito de aislamiento y perfil de arranque.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar qué aísla un hipervisor que un espacio de nombres no, y qué superficie de ataque queda en cada caso.
2. Comparar tiempo de arranque, sobrecarga y densidad de VM, contenedor y microVM con órdenes de magnitud.
3. Justificar el uso de microVM cuando se ejecuta código de terceros no confiable.
4. Describir cómo el almacenamiento por capas hace que 50 contenedores de la misma imagen ocupen poco más que uno.
5. Reconocer que una imagen es una lista ordenada de capas más un manifiesto, y por qué el digest es su única referencia inmutable.

### Conceptos centrales

| Concepto           | Comprensión verificable                                                                                                                                                                                     |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| hipervisor         | Capa que presenta hardware virtual a varios sistemas operativos huésped. De tipo 1 corre sobre el hardware desnudo; de tipo 2, sobre un sistema anfitrión. El aislamiento lo impone la CPU, no el software. |
| espacio de nombres | Mecanismo del núcleo Linux que da a un grupo de procesos su propia vista de un recurso: PID, red, montajes, usuarios. Aísla lo que se ve, no lo que se ejecuta: el núcleo sigue siendo compartido.          |
| cgroup             | Subsistema que limita y contabiliza CPU, memoria y E/S de un grupo de procesos. Complementa a los espacios de nombres: estos aíslan la vista y aquel reparte el recurso.                                    |
| microVM            | Máquina virtual reducida al mínimo dispositivo emulado para arrancar en decenas de milisegundos. Combina el aislamiento de hardware del hipervisor con un perfil de arranque cercano al del contenedor.     |
| capa               | Conjunto de diferencias del sistema de ficheros, inmutable y direccionado por su digest. Las imágenes que comparten una capa base la almacenan y descargan una sola vez.                                    |

### Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart TB
    subgraph VM["Máquina virtual"]
        HW1["hardware"] --> HYP["hipervisor"]
        HYP --> K1["núcleo huésped A"] --> P1["procesos"]
        HYP --> K2["núcleo huésped B"] --> P2["procesos"]
    end
    end
    subgraph CT["Contenedores"]
        HW2["hardware"] --> KS["núcleo COMPARTIDO"]
        KS --> NS1["ns + cgroup A"] --> C1["procesos"]
        KS --> NS2["ns + cgroup B"] --> C2["procesos"]
    end
    end
    KS -->|"un fallo aquí<br/>alcanza a todos"| CT
```

## Desarrollo

### 1. Dos aislamientos que no son comparables

La diferencia decisiva cabe en una frase: una máquina virtual tiene su propio núcleo; un contenedor comparte el del anfitrión.

En una VM, el huésped ejecuta su núcleo y el hipervisor le presenta hardware virtual. El aislamiento lo hace cumplir la propia CPU mediante extensiones de virtualización (Intel VT-x, AMD-V): para escapar hay que romper el hipervisor, que expone una interfaz reducida.

En un contenedor no hay huésped. Son procesos normales del anfitrión con una vista restringida por espacios de nombres y un reparto impuesto por cgroups. La superficie de ataque es toda la interfaz de llamadas al sistema del núcleo Linux: más de 300 llamadas, algunas con historial de vulnerabilidades de escalada.

```
$ docker run --rm alpine uname -r
6.8.0-45-generic          # el núcleo del anfitrión, no de la imagen
$ uname -r
6.8.0-45-generic          # idéntico
```

La imagen aporta bibliotecas y binarios; el núcleo siempre es el del anfitrión. De ahí tres consecuencias que reaparecerán en la parte 05: no se pueden ejecutar contenedores Windows sobre un núcleo Linux; un contenedor no puede cargar módulos; y una vulnerabilidad del núcleo afecta simultáneamente a todos los contenedores de esa máquina.

### 2. El coste del aislamiento, en números

Los órdenes de magnitud, no las cifras exactas, son lo que hay que retener:

|                       | Máquina virtual | Contenedor        | microVM (Firecracker) |
|-----------------------|-----------------|-------------------|-----------------------|
| Arranque              | 30-60 s         | 50-200 ms         | 125 ms                |
| Sobrecarga de memoria | 512 MB-2 GB     | 1-10 MB           | < 5 MB                |
| Densidad por host     | decenas         | miles             | miles                 |
| Aislamiento           | hardware        | núcleo compartido | hardware              |
| Núcleo                | propio          | del anfitrión     | propio, reducido      |

Una VM arranca lento porque hace lo mismo que una máquina física: firmware, cargador, núcleo, init, servicios. Un contenedor solo ejecuta un proceso más con una vista distinta.

Firecracker rompe la disyuntiva: elimina casi todos los dispositivos emulados —sin BIOS, sin USB, sin VGA— y deja solo lo mínimo. Arranca en unos 125 ms con menos de 5 MB de sobrecarga, y mantiene el aislamiento por hardware. Por eso AWS lo usa bajo Lambda y Fargate: ejecutan código de clientes distintos en el mismo host y el aislamiento por núcleo compartido no sería aceptable.

La regla de decisión: si ejecutas tu código, el contenedor basta. Si ejecutas código de terceros que no controlas, necesitas frontera de hardware.

### 3. Espacios de nombres y cgroups: qué aísla cada cosa

Un contenedor no es una primitiva del núcleo. Es la combinación de varios mecanismos independientes que se pueden usar por separado:

| Espacio de nombres | Aísla                                                               |
|--------------------|---------------------------------------------------------------------|
| pid                | La tabla de procesos: dentro, tu proceso es el PID 1                |
| net                | Interfaces, rutas, reglas de firewall y puertos                     |
| mnt                | El árbol de montajes visible                                        |
| uts                | Nombre de host y de dominio                                         |
| ipc                | Colas de mensajes y memoria compartida                              |
| user               | El mapeo de UID: root dentro puede ser un UID sin privilegios fuera |
| cgroup             | La vista de la propia jerarquía de cgroups                          |

Se ven directamente:

```
$ docker run --rm -d --name demo alpine sleep 300
$ pid=$(docker inspect -f '{{.State.Pid}}' demo)
$ sudo ls -l /proc/$pid/ns/
lrwxrwxrwx net -> 'net:[4026532501]'      # espacio propio
lrwxrwxrwx pid -> 'pid:[4026532503]'
lrwxrwxrwx user -> 'user:[4026531837]'    # el MISMO del anfitrión
```

Ese último renglón es el hallazgo importante: por defecto el espacio de usuarios no está aislado, así que el UID 0 dentro del contenedor es el UID 0 del anfitrión. Si el proceso escapa, escapa como root. Activar user remapea root a un UID sin privilegios y es la mitigación más rentable frente a una fuga.

Los cgroups son la otra mitad: sin un límite de memoria, un contenedor consume la del anfitrión y el OOM killer mata al que más memoria use, que puede ser otro contenedor inocente. Ese es el origen del código 137 de la clase 002.

### 4. Capas: por qué 50 contenedores caben en el disco de uno

Una imagen es una lista ordenada de capas inmutables, cada una nombrada por el digest de su contenido, más un manifiesto que las enumera. Al arrancar, el runtime apila esas capas en solo lectura y añade encima una capa de escritura propia del contenedor, mediante copy-on-write con OverlayFS.

```
$ docker image inspect cloudshop:2.4 -f '{{range .RootFS.Layers}}{{println .}}{{end}}'
sha256:8e012198eba1...      # base: alpine 3.20
sha256:4f2a9c73b8d5...      # dependencias
sha256:7d2b8e91c4a2...      # aplicación
```

Si 50 contenedores usan esta imagen, las tres capas base se almacenan una vez. Cada contenedor solo añade lo que escribe:

```
imagen                      180 MB   (compartida por los 50)
capa de escritura por contenedor ~2 MB
-----
50 contenedores              180 + 50×2 = 280 MB
50 VM con el mismo software  50 × 180 = 9.000 MB
```

De aquí sale una regla de construcción que dominará la parte 05: ordena las capas de menos a más volátil. Si copias el código antes de instalar dependencias, cualquier cambio de una línea invalida la capa de dependencias y obliga a reconstruirla y redescargarla entera.

Y de aquí sale también por qué el digest importa: una etiqueta como :2.4 es mutable —puede reapuntarse mañana— mientras que sha256:... identifica exactamente ese contenido. Es la misma distinción entre rama y commit de la clase 003.

### 5. Qué elegir, y con qué criterio

La decisión se toma con dos preguntas, en este orden:

1. ¿De quién es el código que se ejecuta?

- Propio o auditado → contenedor.
- De terceros, sin auditar, o multi-cliente → frontera de hardware: VM o microVM.

## 2. ¿Qué perfil de arranque exige la carga?

- Larga vida, estado en disco, núcleo específico → VM.
- Efímera, escalado por ráfagas, sin estado → contenedor o microVM.

| Situación                                   | Elección   | Por qué                                          |
|---------------------------------------------|------------|--------------------------------------------------|
| API propia con escalado horizontal          | Contenedor | Arranque rápido, densidad alta, código confiable |
| Ejecutar plugins de clientes                | microVM    | Frontera de hardware entre inquilinos            |
| Base de datos con disco y ajuste de núcleo  | VM         | Control del núcleo y del almacenamiento          |
| Trabajo por lotes de minutos                | Contenedor | Sobrecarga de arranque despreciable              |
| Función invocada miles de veces por segundo | microVM    | Aislamiento sin pagar 30 s de arranque           |

Lo que no es criterio válido: «los contenedores son más modernos». Un contenedor y una VM resuelven problemas de aislamiento distintos, y el coste de equivocarse no aparece en las pruebas: aparece cuando alguien escapa del núcleo compartido.

## Ejemplo trabajado

CloudShop quiere ofrecer reglas de descuento programables por sus comercios: cada uno sube un fragmento de código que se ejecuta en cada compra. El equipo propone contenedores «porque ya usamos Kubernetes». Se evalúa con las dos preguntas.

Pregunta 1 — ¿de quién es el código? De terceros y sin auditar. El aislamiento por núcleo compartido pone en juego toda la interfaz de llamadas al sistema:

```
$ docker run --rm alpine sh -c 'grep -c . /proc/kallsyms; ls /proc/sys/kernel | wc -l'
# la superficie visible es la del núcleo del anfitrión
```

Se cuantifica el riesgo con datos, no con opinión. Escapes históricos por CVE de contenedores conocidos: CVE-2019-5736 (reescritura de runc desde el contenedor), CVE-2022-0492 (escalada por cgroups v1), CVE-2024-21626 (fuga de descriptor en runc). Todos permitieron ejecución en el anfitrión desde dentro.

Pregunta 2 — ¿qué perfil de arranque? Una regla se ejecuta por compra: hasta 600 invocaciones por segundo en pico, de unos 20 ms cada una. Una VM clásica queda descartada por aritmética:

```
arranque de VM      ≈ 40 s
trabajo útil        ≈ 0,02 s
sobrecarga          = 40 / 0,02 = 2.000x    inviable
```

Y el modelo de preaprovisionamiento tampoco cierra:

```
600 inv/s × 0,02 s = 12 ejecuciones concurrentes en media
VM preaprovisionadas para pico (×3)      = 36 VM permanentes
memoria: 36 × 1 GB de sobrecarga          = 36 GB solo de hipervisor
```

Con microVM:

```
arranque de microVM ≈ 0,125 s
sobrecarga           = 0,125 / 0,02 = 6,25x    asumible con reutilización
memoria: 36 × 5 MB   = 180 MB
```

180 MB frente a 36 GB, con el mismo aislamiento de hardware. Ese factor de 200 es lo que hace viable el producto.

Decisión registrada: microVM por comercio, reutilizada entre invocaciones del mismo comercio y destruida al cambiar de comercio. El contenedor se descarta no por rendimiento —sería mejor— sino porque el requisito es ejecutar código no confiable de inquilinos distintos, y ahí el núcleo compartido no es una frontera aceptable.

Límite explícito de la decisión: si en el futuro el código dejara de ser de terceros —por ejemplo, si se pasara a un lenguaje de reglas restringido que no permite ejecución arbitraria— la pregunta 1 cambiaría de respuesta y el contenedor volvería a ser correcto.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/008-virtualizacion-hipervisores-e-  
imagenes/lab.py
```

El laboratorio selecciona el motor de práctica virtualization y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa comparativa-de-aislamiento en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es una comparación medida de aislamiento y consumo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye comparativa-de-aislamiento para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                                                   | Causa probable                                                                           | Corrección                                                                       |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Se ejecuta código de terceros en contenedores compartiendo núcleo         | Se confundió aislamiento de vista con aislamiento de seguridad                           | Para código no confiable usa frontera de hardware: microVM o VM dedicada.        |
| Un contenedor comprometido da acceso root al anfitrión                    | El espacio de nombres de usuario no estaba activado, así que UID 0 dentro es UID 0 fuera | Activa el remapeo de usuarios y ejecuta el proceso con un UID sin privilegios.   |
| Un contenedor consume toda la memoria y el núcleo mata a otro distinto    | No había límite de cgroup: el OOM killer elige por consumo, no por culpa                 | Define límites de memoria en todos los contenedores, no solo en los sospechosos. |
| Cada cambio de una línea de código reconstruye y redescarga cientos de MB | El código se copió antes de instalar dependencias e invalida la capa de estas            | Ordena las capas de menos a más volátil: dependencias primero, código al final.  |
| Un despliegue reproducible deja de serlo sin que nadie cambie nada        | Se referenció la imagen por etiqueta mutable en vez de por digest                        | Fija imagen@sha256:... en todo lo que deba ser reproducible.                     |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué aísla un hipervisor que un espacio de nombres no, y qué superficie de ataque queda en cada caso?
2. ¿Por qué no se pueden ejecutar contenedores Windows sobre un núcleo Linux, si la imagen trae todas sus bibliotecas?
3. Una carga se invoca 600 veces por segundo y dura 20 ms. ¿Por qué una VM clásica es inviable y una microVM no?
4. Por defecto, ¿qué UID del anfitrión corresponde a root dentro de un contenedor, y qué mitigación lo cambia?
5. ¿Por qué 50 contenedores de la misma imagen no ocupan 50 veces su tamaño en disco?

## Referencias

- Agache, A. et al. (2020). Firecracker: Lightweight Virtualization for Serverless Applications. USENIX NSDI — cifras de arranque y sobrecarga. <<https://www.usenix.org/conference/nsdi20/presentation/agache>>
- Kerrisk, M. namespaces(7) — los siete espacios de nombres de Linux y su semántica. <<https://man7.org/linux/man-pages/man7/namespaces.7.html>>
- Kerrisk, M. cgroups(7) — límites, contabilidad y jerarquía de control de recursos. <<https://man7.org/linux/man-pages/man7/cgroups.7.html>>
- Open Container Initiative (2024). Image Specification — manifiesto, capas y direccionamiento por digest. <<https://github.com/opencontainers/image-spec/blob/main/spec.md>>
- Rice, L. (2020). Container Security, caps. 4-8 — espacios de nombres, capacidades y vectores de escape.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 008 Virtualización, hipervisores e imágenes

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega comparativa-de-aislamiento con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.

- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

## Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 009 — APIs REST, autenticación y contratos

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: api · Estado: EXECUTABLECORE

## Propósito

Diseñar y consumir APIs HTTP tratando el contrato como la unidad que de verdad se despliega. Toda la nube se opera por API —la consola es solo un cliente más— y las decisiones de esta clase sobre idempotencia, paginación, autenticación y versionado son las que hacen que una automatización sobreviva a un reintento, a un fallo parcial y a un cambio del proveedor.

## Resultados de aprendizaje

Al finalizar podrás:

1. Elegir el método HTTP correcto a partir de sus propiedades de seguridad e idempotencia, no de la costumbre.
2. Hacer idempotente una operación que por naturaleza no lo es, mediante clave de idempotencia.
3. Distinguir autenticación de autorización y explicar qué prueba un token y qué no.
4. Paginar un conjunto grande sin perder ni duplicar elementos cuando la colección cambia durante el recorrido.
5. Evolucionar un contrato sin romper a los consumidores existentes, sabiendo qué cambio es compatible y cuál no.

## Conceptos centrales

| Concepto              | Comprensión verificable                                                                                                                                                                                    |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| método seguro         | El que no altera el estado del servidor: GET, HEAD, OPTIONS. Permite que proxies y navegadores lo repitan o lo precarguen sin consecuencias, lo que a su vez prohíbe usar GET para operaciones con efecto. |
| idempotencia          | Propiedad por la que N ejecuciones idénticas dejan el mismo estado que una. PUT y DELETE lo son por definición; POST no, y por eso necesita una clave explícita para poder reintentarse.                   |
| clave de idempotencia | Identificador único que el cliente genera y envía para que el servidor reconozca un reintento y devuelva el resultado original en vez de repetir el efecto.                                                |
| token portador        | Credencial que autoriza a quien la presente, sin más prueba. Quien lo roba lo puede usar: de ahí que su vida deba ser corta y su transporte siempre cifrado.                                               |
| cursor                | Marca opaca de posición en una colección, estable frente a inserciones y borrados. Sustituye al desplazamiento numérico, que salta o repite elementos cuando la colección cambia entre páginas.            |

## Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

```
flowchart LR
  C["cliente"] -->|"POST /pagos<br/>Idempotency-Key: 7d2b8e"| A["API"]
  A --> K{"¿clave ya vista?"}
  K -->|"no"| E["ejecuta y guarda<br/>clave → resultado"]
  K -->|"sí"| R["devuelve el resultado<br/>guardado, sin repetir"]
  E --> O1["201 Created"]
  R --> O2["200 con el mismo cuerpo"]
  O1 -->|"timeout de red:<br/>el cliente reintenta"| C
```



## Desarrollo

### 1. Los métodos tienen propiedades, no solo nombres

Elegir el método no es cuestión de estilo: sus propiedades determinan qué pueden hacer con la petición los proxies, los navegadores y las bibliotecas de reintento.

| Método | Seguro | Idempotente    | Consecuencia práctica                                |
|--------|--------|----------------|------------------------------------------------------|
| GET    | Sí     | Sí             | Cacheable; un proxy puede repetirlo sin avisar       |
| HEAD   | Sí     | Sí             | Como GET sin cuerpo; útil para comprobar existencia  |
| PUT    | No     | Sí             | Reintentable sin riesgo: sustituye el recurso entero |
| DELETE | No     | Sí             | Reintentable: borrar dos veces deja el mismo estado  |
| POST   | No     | No             | No reintentable sin protección adicional             |
| PATCH  | No     | No garantizada | Depende del formato del parche                       |

La fila de GET explica un incidente clásico: exponer GET /pedidos/42/cancelar parece cómodo hasta que un precargador del navegador, un rastreador o un proxy lo visitan y cancelan pedidos que nadie pidió cancelar. Si tiene efecto, no puede ser GET.

Y la fila de POST explica el problema central de esta clase: es el único método común que no se puede reintentar con seguridad, y es justo el que se usa para crear pagos, pedidos y recursos.

### 2. Idempotencia: la propiedad que hace segura una red poco fiable

Un cliente envía POST /pagos, la red se corta antes de la respuesta y el cliente no sabe si el pago se ejecutó. Solo tiene dos opciones malas: reintentar y arriesgarse a cobrar dos veces, o no reintentar y arriesgarse a no cobrar.

La solución es que el cliente genere un identificador único por intención y lo repita en cada reintento:

```
POST /pagos HTTP/1.1
Idempotency-Key: <uuid v4 que genera el cliente>
Content-Type: application/json

{"pedido": "A-1042", "importe": "149.90", "moneda": "CLP"}
```

El servidor guarda la clave junto al resultado. Al recibirla de nuevo:

1. Si no la ha visto, ejecuta y persiste clave → resultado en la misma transacción que el efecto.
2. Si ya la vio y terminó, devuelve el resultado original sin repetir el cobro.
3. Si ya la vio y sigue en curso, responde 409 Conflict.

El punto 1 es donde suele fallar la implementación: si el registro de la clave y el cobro no son atómicos, existe una ventana en la que el cobro ocurrió y la clave no se guardó, y el reintento cobra otra vez. La clave debe persistirse en la misma transacción que el efecto que protege.

Es el mismo mecanismo que en la parte 09 aparecerá como deduplicación en sistemas de mensajería, y por la misma razón: la entrega «exactamente una vez» no existe a nivel de red; se construye con idempotencia en el receptor.

### 3. Autenticación no es autorización

Son dos preguntas distintas y se resuelven en momentos distintos:

- Autenticación: ¿quién eres? Se resuelve una vez y produce una credencial.
- Autorización: ¿puedes hacer esto sobre este recurso? Se resuelve en cada petición.

Saltarse la segunda produce la vulnerabilidad más común de las APIs, catalogada como BOLA (Broken Object Level Authorization, primer puesto del OWASP API Security Top 10):

```
# vulnerable: autentica pero no autoriza sobre el objeto
@app.get("/pedidos/{id}")
def ver(id: str, usuario = Depends(authenticar)):
    return db.pedidos.get(id)          # cualquiera autenticado ve CUALQUIER pedido
```

```
# correcto: la autorización es por objeto
@app.get("/pedidos/{id}")
def ver(id: str, usuario = Depends(authenticar)):
    pedido = db.pedidos.get(id)
    if pedido is None or pedido.cliente_id != usuario.cliente_id:
        raise HTTPException(404) # 404, no 403: no revelar existencia
    return pedido
```

Devolver 404 en vez de 403 es deliberado: un 403 confirma que el recurso existe y permite enumerar identificadores ajenos. Es la misma lógica por la que un formulario de acceso no debe decir «esa contraseña es incorrecta» frente a «ese usuario no existe».

Los códigos correctos: 401 significa «no sé quién eres» y admite reintento con credenciales; 403 significa «sé quién eres y no puedes», y reintentar no ayuda.

#### 4. Paginación: por qué el desplazamiento numérico pierde datos

GET /pedidos?offset=100&limit=50 parece razonable y falla en cuanto la colección cambia durante el recorrido. Con orden descendente por fecha:

```
t0 el cliente lee offset=0..49 (elementos 1-50)
t1 se insertan 3 pedidos nuevos al principio
t2 el cliente lee offset=50..99
  → los elementos 48, 49 y 50 se han desplazado y se REPITEN
  → si en vez de insertar se hubieran borrado 3, se PERDERÍAN
```

Y hay un segundo problema, de coste: OFFSET 100000 obliga a la base de datos a leer y descartar cien mil filas antes de devolver la página. El coste crece linealmente con la profundidad.

La paginación por cursor usa una marca estable —la clave del último elemento— en vez de una posición:

```
GET /pedidos?limit=50
{"datos": [...], "siguiente": "eyJ0IjoimjAyNi0wOC0wMVQwOToxMiJ9"}

GET /pedidos?limit=50&cursor=eyJ0IjoimjAyNi0wOC0wMVQwOToxMiJ9
```

La consulta pasa a ser WHERE (creado, id) < (:t, :id) ORDER BY creado DESC, id DESC LIMIT 50, que usa índice y cuesta lo mismo en la página 1 que en la 10.000. El cursor debe ser opaco para el cliente: si se documenta su estructura, se convierte en parte del contrato y ya no se puede cambiar.

El orden debe incluir un desempate único (id): sin él, dos filas con la misma marca de tiempo pueden repetirse o perderse en el corte de página.

#### 5. Evolucionar sin romper: qué cambio es compatible

Un contrato desplegado tiene consumidores que no controlas. La distinción práctica:

| Cambio                                  | ¿Compatible? | Por qué                                    |
|-----------------------------------------|--------------|--------------------------------------------|
| Añadir un campo opcional a la respuesta | Sí           | Los clientes ignoran lo que no conocen     |
| Añadir un parámetro opcional            | Sí           | Su ausencia mantiene el comportamiento     |
| Añadir un valor nuevo a un enumerado    | No           | Los clientes con switch exhaustivo fallan  |
| Renombrar o eliminar un campo           | No           | Rotura directa                             |
| Cambiar un tipo (número → cadena)       | No           | Rotura de deserialización                  |
| Hacer obligatorio un campo opcional     | No           | Rotura para quien no lo enviaba            |
| Endurecer una validación                | No           | Peticiones antes válidas empiezan a fallar |

La fila del enumerado es la que más sorprende: parece aditiva, pero un consumidor que hace exhaustivo sobre los valores conocidos falla al recibir uno nuevo. Por eso los contratos maduros documentan desde el principio que los enumerados pueden crecer y exigen un caso por defecto.

Cuando el cambio es incompatible, versionar. Y la regla que importa no es dónde poner la versión —ruta, cabecera o tipo de medio— sino cuánto tiempo mantienes la anterior viva y cómo avisas: una versión sin fecha de retirada publicada no es una versión, es deuda.

El campo Deprecation y Sunset (RFC 8594) permiten anunciarlo en la propia respuesta, para que el cliente se entere antes de romperse.

## Ejemplo trabajado

Durante una promoción, 1 de cada 400 clientes de CloudShop aparece cobrado dos veces. El importe duplicado siempre coincide, y los dos cargos distan menos de 30 segundos.

La hipótesis es reintento sobre POST no idempotente. Se confirma en los logs:

```
$ jq -r 'select(.ruta=="/pagos") | [.trace, .pedido, .estado, .ms] | @tsv' pagos.jsonl | sort | head -4
4f2a9c A-1042 timeout 30021
7d2b8e A-1042 201 412
```

Dos trazas distintas para el mismo pedido: la primera agotó el plazo del cliente a los 30 s, pero el servidor la completó—el cargo existe— y el cliente reintentó con una petición nueva.

La aritmética del incidente:

|                                      |                                               |
|--------------------------------------|-----------------------------------------------|
| peticiones de pago en la promoción   | 48.000                                        |
| timeout del cliente                  | 30 s                                          |
| p99,8 de latencia del proveedor      | 31 s                                          |
| peticiones que superan el timeout    | $\approx 0,25\% \rightarrow 120$              |
| de ellas, completadas en el servidor | $\approx 100\% \rightarrow 120$ cobros dobles |
| observado                            | 118                                           |

Cuadra: el problema no es la tasa de error del proveedor, es que el timeout del cliente es menor que la cola de latencia del servidor. Subir el timeout reduce la frecuencia pero no elimina la clase de fallo: siempre habrá una petición que se corte después de tener efecto.

Corrección con clave de idempotencia, generada por el cliente una vez por intención y reutilizada en los reintentos:

```
clave = str(uuid.uuid4()) # fuera del bucle: la MISMA en cada reintento
for intento in range(3):
    try:
        r = sesion.post("/pagos", json=cuerpo,
                        headers={"Idempotency-Key": clave}, timeout=35)
        break
    except (TimeoutError, ConnectionError):
        time.sleep(2 ** intento + random.uniform(0, 1))
```

Y en el servidor, con la clave persistida en la misma transacción que el cargo:

```
BEGIN;
INSERT INTO idempotencia (clave, estado) VALUES ($1, 'en_curso')
ON CONFLICT (clave) DO NOTHING;
-- si no insertó ninguna fila, es un reintento: devolver lo guardado
INSERT INTO cargos (pedido, importe) VALUES ($2, $3);
UPDATE idempotencia SET estado='hecho', respuesta=$4 WHERE clave=$1;
COMMIT;
```

Resultado tras el despliegue: 0 cobros dobles en 52.000 pagos de la promoción siguiente, con 96 reintentos que devolvieron el resultado original en vez de repetir el cargo.

El detalle que decide todo es dónde se genera la clave: fuera del bucle de reintento. Generarla dentro produce una clave distinta por intento y deja el sistema exactamente igual de roto, con la ilusión de estar protegido.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/009-apis-rest-autenticacion-y-contratos/lab.py
```

El laboratorio selecciona el motor de práctica api y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.

3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cliente-api-verificado en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un contrato versionado con pruebas positivas y negativas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye cliente-api-verificado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                                                            | Causa probable                                                                     | Corrección                                                                                                  |
|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Se ejecutan acciones que nadie solicitó                                            | Una operación con efecto se expuso como GET y la repitió un precargador o un proxy | GET debe ser seguro; usa POST, PUT o DELETE para cualquier cosa con efecto.                                 |
| Un reintento tras timeout duplica un cobro                                         | POST no es idempotente y no había clave de idempotencia                            | Genera la clave una vez por intención, fuera del bucle, y persístela en la misma transacción que el efecto. |
| Un usuario autenticado accede a datos de otro cambiando el identificador de la URL | Se autenticó pero no se autorizó por objeto (BOLA)                                 | Comprueba la propiedad del recurso en cada petición y responde 404, no 403.                                 |
| Un recorrido paginado repite o pierde elementos                                    | Paginación por desplazamiento sobre una colección que cambia                       | Usa cursor opaco sobre una clave estable con desempate único.                                               |
| Añadir un valor a un enumerado rompe consumidores en producción                    | Se asumió que todo cambio aditivo es compatible                                    | Documenta desde el principio que los enumerados crecen y exige un caso por defecto en el cliente.           |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Por qué PUT y DELETE se pueden reintentar sin riesgo y POST no?
2. ¿En qué punto exacto debe persistirse la clave de idempotencia para que no quede una ventana de doble efecto?
3. Un usuario autenticado pide un recurso ajeno. ¿Por qué 404 es mejor respuesta que 403?
4. Explica con un ejemplo cómo la paginación por desplazamiento pierde un elemento cuando la colección cambia.

5. ¿Cuáles de estos cambios rompen a un consumidor: añadir un campo opcional, añadir un valor a un enumerado, hacer obligatorio un campo opcional?

## Referencias

- Fielding, R. y Reschke, J., eds. (2022). RFC 9110: HTTP Semantics, secs. 9.2.1-9.2.2 — métodos seguros e idempotentes. <<https://www.rfc-editor.org/rfc/rfc9110#section-9.2>>
- Jena, J. et al. (2024). The Idempotency-Key HTTP Header Field, IETF draft — semántica y manejo de reintentos. <<https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/>>
- OWASP (2023). API Security Top 10, API1:2023 Broken Object Level Authorization. <<https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>>
- Wilde, E. (2019). RFC 8594: The Sunset HTTP Header Field — anunciar la retirada de un contrato. <<https://www.rfc-editor.org/rfc/rfc8594>>
- Nottingham, M. (2022). RFC 9205: Building Protocols with HTTP — buenas prácticas de diseño sobre HTTP. <<https://www.rfc-editor.org/rfc/rfc9205>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 009 APIs REST, autenticación y contratos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega cliente-api-verificado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 010 — Responsabilidad compartida y pensamiento de riesgo

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

## Propósito

Trazar con precisión dónde termina la responsabilidad del proveedor y empieza la tuya, y convertir esa línea en un método de análisis de riesgo aplicable antes de escribir infraestructura. Es la clase que explica por qué casi ningún incidente público «de la nube» fue del proveedor, y la que fija el criterio con el que se juzgarán todas las decisiones de las 278 clases restantes.

## Resultados de aprendizaje

Al finalizar podrás:

1. Situar cualquier control concreto del lado correcto de la línea según el modelo de servicio contratado.
2. Distinguir disponibilidad del servicio, durabilidad del dato y confidencialidad, y qué garantiza el proveedor de cada una.
3. Leer un SLA identificando qué mide, qué excluye y cuál es la compensación real.
4. Aplicar un método de análisis de riesgo que produzca controles verificables en vez de una lista de miedos.
5. Calcular el riesgo residual de una decisión y declararlo explícitamente en vez de dejarlo implícito.

## Conceptos centrales

| Concepto                   | Comprensión verificable                                                                                                                                                                                              |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| responsabilidad compartida | Reparto contractual de controles entre proveedor y cliente. El proveedor responde de la seguridad de la nube; el cliente, de la seguridad en la nube. La línea se mueve con el modelo de servicio, nunca desaparece. |
| SLA                        | Compromiso contractual con una métrica, un umbral y una compensación. Casi siempre mide disponibilidad del plano de servicio y excluye lo que más duele: tus datos, tu configuración y tu lucro cesante.             |
| durabilidad                | Probabilidad de que un objeto almacenado siga existiendo tras un periodo. Es independiente de la disponibilidad: un dato puede ser durable y estar inaccesible, o estar disponible y haber sido borrado por ti.      |
| riesgo residual            | Lo que queda después de aplicar los controles. No es un fallo del análisis: es el resultado esperado. Un diseño honesto lo nombra y dice quién lo acepta.                                                            |
| radio de impacto           | Alcance de lo que se rompe cuando algo falla. Reducirlo —por celdas, cuentas o regiones— suele ser más barato y más eficaz que intentar evitar el fallo.                                                             |

## Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart TB
    subgraph tuyo["Siempre tuyo · en los tres modelos"]
        D["Datos y su clasificación"]
        I["Identidades y permisos"]
        CFG["Configuración que eliges"]
    end
    subgraph movil["La línea se mueve con el modelo"]
        AP["Aplicación"] --> RT["Runtime"] --> SO["Sistema operativo"]
    end
    subgraph prov["Siempre del proveedor"]
        HW["Hardware, red física, energía"]
        DC["Seguridad física del centro de datos"]
    end
    tuyo -.->|IaaS: S0 tuyo<br/>PaaS: runtime del proveedor<br/>SaaS: solo datos y config| movil
    movil -.-> prov
```

## Desarrollo

### 1. Lo que nunca cambia de lado

El modelo de servicio mueve la línea, pero tres cosas permanecen siempre del lado del cliente, en IaaS, en PaaS y en SaaS:

1. Los datos: qué guardas, cómo los clasificas y cuánto tiempo los conservas.
2. Las identidades y sus permisos: quién puede hacer qué.
3. La configuración que eliges: si un bucket es público, lo hiciste público tú.

Esto explica un hecho incómodo: la inmensa mayoría de las brechas atribuidas públicamente a «la nube» fueron configuraciones del cliente, no fallos del proveedor. Almacenamiento expuesto sin autenticación, claves de acceso subidas a un repositorio público, permisos comodín concedidos «temporalmente» hace dos años.

Gartner lleva años sosteniendo la misma proyección: hasta 2025, en torno al 99 % de los fallos de seguridad en la nube serán culpa del cliente. El dato importa menos que su consecuencia de diseño: el trabajo de seguridad no está en elegir proveedor, está en el lado que te toca.

El error simétrico también existe: asumir que nada hace el proveedor. El cifrado del disco físico, la destrucción segura de medios, el control de acceso al centro de datos y el parcheo del hipervisor no son tuyos y no deberías replicarlos.

### 2. Disponibilidad, durabilidad y confidencialidad son tres cosas distintas

Se confunden a diario y cada una tiene garantías, riesgos y controles diferentes:

| Propiedad        | Pregunta                | Garantía típica                  | Qué la rompe                                  |
|------------------|-------------------------|----------------------------------|-----------------------------------------------|
| Disponibilidad   | ¿Puedo acceder ahora?   | 99,9 %-99,99 % en SLA            | Caída de zona, saturación, error de red       |
| Durabilidad      | ¿Seguirá existiendo?    | 99,999999999 % anual (11 nueves) | Un borrado tuyo, no un fallo de disco         |
| Confidencialidad | ¿Solo lo ve quien debe? | No hay SLA                       | Configuración, permisos, fuga de credenciales |

Los once nueves de durabilidad de un almacenamiento de objetos significan que la probabilidad anual de perder un objeto dado por fallo de infraestructura es  $10^{-11}$ . Con 10 millones de objetos:

$$\begin{aligned} \text{pérdida esperada} &= 10.000.000 \times 10^{-11} = 0,0001 \text{ objetos al año} \\ &\approx 1 \text{ objeto cada } 10.000 \text{ años} \end{aligned}$$

Y sin embargo se pierden datos constantemente. La razón es que la durabilidad no protege del borrado autorizado: un DELETE con credenciales válidas, un ciclo de vida mal configurado o un ransomware con tus permisos destruyen el objeto sin que la infraestructura falle en absoluto.

De ahí que la copia de seguridad siga siendo tuya, con dos propiedades que la durabilidad no da: versionado —para volver atrás— e inmutabilidad —para que ni tus propias credenciales puedan borrarla—. Se retoma en la parte 21.

### 3. Leer un SLA por lo que excluye

Un SLA tiene tres partes y la tercera casi nunca es la que se recuerda:

1. Métrica y umbral: por ejemplo, 99,99 % mensual de disponibilidad del servicio.
2. Compensación: un porcentaje de crédito sobre lo facturado de ese servicio.
3. Exclusiones: lo que no cuenta como indisponibilidad.

La aritmética del umbral, en un mes de 30 días:

99,9 % → 43,2 min de caída permitida al mes  
99,95 % → 21,6 min  
99,99 % → 4,3 min  
99,999 % → 26 s

El salto de 99,9 a 99,99 no es «un poco mejor»: exige reducir la ventana de fallo diez veces, lo que en la práctica obliga a redundancia entre zonas y a automatizar la conmutación, porque ninguna intervención humana cabe en 4,3 minutos.

Las exclusiones habituales, que conviene buscar antes de firmar:

- Indisponibilidad por tu configuración o tu código.
- Ventanas de mantenimiento anunciadas.
- Fuerza mayor y fallos de tu proveedor de red.
- Servicios en vista previa o beta.

Y la exclusión que más importa: la compensación es un crédito de servicio, no una indemnización. Si una caída de 4 horas te cuesta 80.000 USD en ventas, el SLA te devuelve un porcentaje de lo que pague por ese servicio ese mes —quizá 200 USD—. El SLA no transfiere tu riesgo de negocio: acota el del proveedor. Tu continuidad la pagas tú, con arquitectura.

### 4. Un método de riesgo que produce controles, no miedos

Una lista de amenazas sin estructura no sirve para decidir. Cinco preguntas, en orden, sí:

1. ¿Qué protegemos? El activo, con su clasificación. «La base de datos» no es un activo; «los datos personales de clientes, categoría alta» sí.
2. ¿De quién y de qué? Actor y capacidad: un atacante externo sin credenciales, un empleado con acceso legítimo, un error de operación, un fallo de infraestructura.
3. ¿Por dónde? El vector concreto: credencial filtrada, permiso excesivo, dependencia comprometida, error de configuración.
4. ¿Qué control lo corta? Preventivo, detectivo o correctivo, y verificable.
5. ¿Qué queda? El riesgo residual, nombrado y aceptado por alguien con autoridad para aceptarlo.

La columna que separa un análisis útil de un documento decorativo es la cuarta: cada control debe tener una prueba. «Cifrado en tránsito» no es un control; «TLS 1.2 mínimo, verificado por una prueba automática que falla el despliegue si se acepta TLS 1.0» sí lo es.

Y los tres tipos no son intercambiables. Un control preventivo que falle en silencio es peor que uno detectivo que avise: el primero da confianza falsa. Por eso todo control preventivo necesita una prueba negativa —comprobar que efectivamente deniega— que es exactamente lo que hacen los lab.py de este programa.

### 5. Radio de impacto: contener sale más barato que evitar

Los fallos ocurren. La pregunta de diseño no es cómo evitarlos todos, sino qué se rompe cuando ocurren.

Una cuenta, todo junto: 1 credencial comprometida → todo el negocio  
Cuentas separadas por entorno: 1 credencial comprometida → un entorno  
Cuentas por dominio + entorno: 1 credencial comprometida → un dominio de un entorno

La separación no impide la brecha; acota su alcance, y eso suele ser mucho más barato que intentar hacer la brecha imposible. Es el mismo razonamiento de la clase 007 sobre capacidades: no «qué necesita para funcionar», sino «qué obtiene quien lo comprometa».



Tres preguntas que hay que poder responder de cualquier diseño:

- ¿Qué se rompe si esto falla? Si la respuesta es «todo», el diseño tiene un punto único de fallo aunque el componente sea redundante.
- ¿Cuánto tarda en detectarse? Un fallo silencioso de días es peor que una caída ruidosa de minutos.
- ¿Se puede revertir? Un cambio irreversible exige un nivel de control distinto al de uno reversible.

Estas tres preguntas son el guion de los ADR de la parte 12 y de los game days de la parte 21.

## Ejemplo trabajado

Un informe de exposición avisa a CloudShop de que un bucket con facturas de clientes es accesible sin autenticación. El equipo quiere responder «el proveedor garantiza cifrado y once nuevos». Se aplica el método.

1. ¿Qué protegemos? 412.000 facturas en PDF con nombre, RUT, dirección y detalle de compra. Clasificación: datos personales, categoría alta. Retención legal: 6 años.
2. ¿De quién? Actor externo sin credenciales. No hace falta más: el objeto es legible por cualquiera con la URL.
3. ¿Por dónde? Se reconstruye el vector:

```
$ aws s3api get-bucket-policy --bucket cloudshop-facturas | jq -r '.Policy' | jq '.Statement[0]'
{"Effect": "Allow", "Principal": "*", "Action": "s3:GetObject", ...}
$ aws cloudtrail lookup-events --lookup-attributes \
  AttributeKey=EventName,AttributeValue=PutBucketPolicy --max-items 1 \
  | jq -r '.Events[0] | "\(.EventTime) \(.Username)'"
2024-11-14T16:22:08Z ci-deploy-role
```

La política la puso un despliegue hace 21 meses. El cifrado en reposo estaba activo todo el tiempo y no sirvió de nada: cifra frente a quien acceda al disco físico, no frente a quien hace una petición autorizada por la política. La durabilidad de once nuevos tampoco: garantiza que el objeto no se pierda, no que no se lea.

Cuantificación de la exposición:

```
objetos legibles          412.000
ventana                   21 meses ≈ 638 días
peticiones GET anónimas en los logs  1.847 (de 12 IP distintas)
```

4. ¿Qué controles, y cómo se verifican?

| Tipo       | Control                                            | Prueba                                                   |
|------------|----------------------------------------------------|----------------------------------------------------------|
| Preventivo | Bloqueo de acceso público a nivel de cuenta        | Prueba negativa: intentar publicar y comprobar que falla |
| Preventivo | Política de organización que prohíbe Principal: "" | Despliegue de prueba que debe ser rechazado              |
| Detectivo  | Alerta sobre cualquier PutBucketPolicy             | Ejecutarla y verificar que notifica en < 5 min           |
| Correctivo | Versionado + retención inmutable                   | Restaurar un objeto borrado en un simulacro              |

Cada fila tiene una prueba ejecutable. La segunda columna sin la tercera es una intención, no un control.

5. ¿Qué riesgo queda? Los datos estuvieron expuestos 638 días y eso no se puede revertir: hay que asumir que fueron copiados y actuar en consecuencia —notificación a los afectados y a la autoridad, según la normativa aplicable—. Además, un operador con permiso legítimo sigue pudiendo exportarlos; ese riesgo se acota con registro de acceso y revisión periódica, no se elimina.

La conclusión que importa: ni el cifrado ni la durabilidad protegían de esto, porque ninguno de los dos estaba del lado del problema. El control que faltaba era de configuración, y la configuración es siempre del cliente en los tres modelos de servicio.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/010-responsabilidad-compartida-y-pensamiento-de-riesgo/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-de-responsabilidad en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye matriz-de-responsabilidad para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                                                       | Causa probable                                                                           | Corrección                                                                                                         |
|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Se asume que el cifrado del proveedor protege los datos de un acceso indebido | El cifrado en reposo protege frente al disco físico, no frente a una petición autorizada | Separa los controles por propiedad: confidencialidad se protege con permisos y política, no con cifrado en reposo. |
| Se confía en los once nueves de durabilidad como estrategia de respaldo       | La durabilidad no protege del borrado autorizado, que es la causa real de pérdida        | Añade versionado y retención inmutable; son tuyos, no del proveedor.                                               |
| Tras una caída larga, el SLA compensa una fracción mínima de la pérdida       | La compensación es un crédito sobre lo facturado de ese servicio, no una indemnización   | Dimensiona la continuidad con arquitectura; el SLA acota el riesgo del proveedor, no el tuyo.                      |
| El análisis de riesgo es una lista de amenazas que nadie usa para decidir     | No llegó a controles verificables ni nombró el riesgo residual                           | Exige a cada control una prueba ejecutable y un responsable que acepte lo que queda.                               |
| Una credencial comprometida da acceso a todo el negocio                       | No hay separación de cuentas ni entornos: el radio de impacto es total                   | Separa por entorno y dominio; contener el alcance es más barato que evitar la brecha.                              |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué tres responsabilidades siguen siendo del cliente en IaaS, PaaS y SaaS por igual?
2. Un almacenamiento con once años de durabilidad pierde datos de un cliente. ¿Cómo es posible sin que falle la infraestructura?
3. ¿Cuántos minutos de caída mensual permite un SLA de 99,99 %, y por qué eso descarta la intervención humana?
4. Un control dice «cifrado en tránsito». ¿Qué le falta para ser verificable?
5. ¿Por qué un control preventivo que falla en silencio es peor que uno detectivo?

## Referencias

- AWS (2024). Shared Responsibility Model — reparto de controles por modelo de servicio. <<https://aws.amazon.com/compliance/shared-responsibility-model/>>
- Microsoft (2024). Shared responsibility in the cloud — tabla comparada IaaS/PaaS/SaaS. <<https://learn.microsoft.com/en-us/azure/security/fundamentals/shared-responsibility>>
- Cloud Security Alliance (2024). Top Threats to Cloud Computing — taxonomía de amenazas con incidentes documentados. <<https://cloudsecurityalliance.org/research/topics/top-threats>>
- Shostack, A. (2014). Threat Modeling: Designing for Security — las cuatro preguntas y el uso de STRIDE.
- NIST (2018). Framework for Improving Critical Infrastructure Cybersecurity v1.1 — funciones identificar, proteger, detectar, responder y recuperar. <<https://doi.org/10.6028/NIST.CSWP.04162018>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 010 Responsabilidad compartida y pensamiento de riesgo

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega matriz-de-responsabilidad con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

## Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 011 — Costo, energía, capacidad y medición básica

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: finops · Estado: EXECUTABLECORE

## Propósito

Convertir el coste en una magnitud de ingeniería con la que se decide, no en una factura que se revisa al final del mes. Aquí se establece la unidad económica que el programa usará en las 277 clases restantes: coste por transacción útil. Sin ella, «optimizar costes» es apagar cosas al azar y esperar que nada se rompa.

## Resultados de aprendizaje

Al finalizar podrás:

1. Definir una unidad de coste ligada a algo que el negocio entiende, y calcularla a partir de la factura.
2. Aplicar la ley de Little para dimensionar capacidad a partir de tasa de llegada y tiempo de servicio.
3. Explicar por qué la latencia se dispara antes de llegar al 100 % de utilización, y a partir de qué punto.
4. Comparar modelos de precio —bajo demanda, compromiso, capacidad excedente— con un umbral de utilización calculado.
5. Estimar el coste energético de una carga y situarlo frente a su coste monetario.

## Conceptos centrales

| Concepto         | Comprensión verificable                                                                                                                                                                                                    |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| unidad de coste  | Denominador que convierte la factura en una métrica decidible: coste por pedido, por usuario activo, por GB procesado. Sin denominador, un aumento de factura no distingue crecimiento de derroche.                        |
| utilización      | Fracción del tiempo que un recurso está ocupado, entre 0 y 1. Por encima de 0,7 el tiempo de espera crece de forma no lineal, así que perseguir el 100 % es perseguir una cola infinita.                                   |
| ley de Little    | En un sistema estable, $L = \lambda \cdot W$ : el número medio de elementos en el sistema es la tasa de llegada por el tiempo medio de permanencia. No supone ninguna distribución, lo que la hace aplicable casi siempre. |
| coste amortizado | Coste efectivo de un compromiso repartido en su plazo. Permite comparar un descuento por reserva de 1 o 3 años con el precio bajo demanda sobre la misma base.                                                             |
| PUE              | Power Usage Effectiveness: energía total del centro de datos dividida por la que llega al equipamiento informático. 1,0 sería perfecto; los hiperescalares operan cerca de 1,1 y una sala tradicional ronda 1,8.           |

## Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

flowchart LR

```
F["Factura mensual<br/>USD"] --> U["dividir por<br/>unidad de negocio"]
T["Transacciones útiles<br/>pedidos, usuarios, GB"] --> U
U --> C["Coste unitario<br/>USD / pedido"]
C --> D["¿sube o baja<br/>mes a mes?"]
D -->| "sube con volumen plano" | W["derroche: investigar"]
D -->| "baja con volumen creciente" | E["economía de escala: sano"]
D -->| "plano con volumen creciente" | L["lineal: sin escala, revisar diseño"]
```

## Desarrollo

### 1. Sin denominador, la factura no dice nada

«La factura subió un 40 %» es una observación inútil hasta que se divide por algo. Con denominador, la misma cifra puede significar tres cosas opuestas:

mes 1: 12.400 USD / 620.000 pedidos = 0,0200 USD/pedido  
mes 2: 17.360 USD / 980.000 pedidos = 0,0177 USD/pedido ← +40 % factura, -11 % unitario  
mes 3: 17.360 USD / 610.000 pedidos = 0,0285 USD/pedido ← +43 % unitario: hay derroche

El mes 2 es crecimiento sano: se factura más porque se vende más, y cada pedido cuesta menos. El mes 3 con la misma factura es una alarma: el volumen cayó y el coste no.

La unidad debe cumplir tres condiciones para servir:

1. La entiende el negocio: pedidos, usuarios activos, GB procesados. No «horas de instancia».
2. Se puede atribuir: existe un mecanismo de etiquetado que asigna cada recurso a un producto o equipo.
3. Se mide igual todos los meses: cambiar el denominador rompe la serie histórica.

La condición 2 es la que exige trabajo previo: sin etiquetado consistente, el 30-40 % de la factura acaba en un cubo de «sin asignar» que nadie reclama y nadie optimiza. Por eso el etiquetado es la primera tarea de FinOps, antes que cualquier optimización.

### 2. La ley de Little: dimensionar sin adivinar

Para cualquier sistema estable, con independencia de la distribución de llegadas o servicios:

$$L = \lambda \cdot W$$

L = elementos en el sistema (conurrencia)  
 $\lambda$  = tasa de llegada (peticiones por segundo)  
W = tiempo medio en el sistema (segundos)

Sirve para dimensionar antes de desplegar. Con 800 peticiones por segundo y 250 ms de latencia media:

$$L = 800 \times 0,250 = 200 \text{ peticiones concurrentes}$$

Si cada trabajador atiende una petición a la vez, hacen falta 200 trabajadores solo para el estado estacionario. Con 40 trabajadores por instancia:

$$\text{instancias} = 200 / 40 = 5 \quad (\text{mínimo teórico, utilización } 1,0)$$

Y ese «mínimo teórico» es exactamente el número que no hay que desplegar, por lo que viene a continuación.

La ley también funciona al revés, para detectar incoherencias: si mides 200 conexiones concurrentes y 800 peticiones por segundo, la latencia media tiene que ser 250 ms. Si tu panel dice 80 ms, alguna de las tres medidas está mal —normalmente porque se está midiendo la media de una distribución con cola larga—.

### 3. Por qué el 100 % de utilización es una trampa

En un sistema con llegadas aleatorias, el tiempo de espera no crece de forma lineal con la utilización. Para una cola M/M/1:

$$W = W_{\text{servicio}} / (1 - \rho) \quad \rho = \text{utilización}$$

Con un servicio de 100 ms:

| $\rho$ | Tiempo total | Multiplificador |
|--------|--------------|-----------------|
| 0,50   | 200 ms       | 2×              |
| 0,70   | 333 ms       | 3,3×            |
| 0,80   | 500 ms       | 5×              |
| 0,90   | 1.000 ms     | 10×             |
| 0,95   | 2.000 ms     | 20×             |
| 0,99   | 10.000 ms    | 100×            |

Entre 0,70 y 0,90 la utilización sube 20 puntos y la latencia se triplica. Ese es el motivo de que el objetivo operativo habitual sea 0,60-0,70 y no 0,95: el 30 % de capacidad «ociosa» no es derroche, es lo que absorbe la varianza de las llegadas.

Aplicado al cálculo anterior:

mínimo teórico ( $p = 1,0$ )                      5 instancias  
 objetivo  $p = 0,70$                                $5 / 0,70 = 7,1 \rightarrow 8$  instancias  
 tolerancia a la caída de una zona ( $n+1$ )                       $\rightarrow 9$  instancias

Las 4 instancias adicionales sobre el mínimo teórico no son exceso: son el precio de un percentil 95 estable y de sobrevivir a la pérdida de una zona. Optimizar costes recortando ahí es cambiar dinero por incidentes, y el intercambio hay que hacerlo explícito.

#### 4. Modelos de precio: el umbral que decide

Los tres modelos comunes, con el mismo recurso como referencia:

| Modelo              | Precio relativo | Compromiso      | Riesgo                          |
|---------------------|-----------------|-----------------|---------------------------------|
| Bajo demanda        | 1,00            | Ninguno         | Ninguno                         |
| Compromiso 1 año    | 0,60            | Pagas uses o no | Sobredimensionar                |
| Compromiso 3 años   | 0,40            | Ídem, más largo | Cambio tecnológico              |
| Capacidad excedente | 0,10-0,30       | Ninguno         | Interrupción con preaviso corto |

El umbral de utilización a partir del cual compensa comprometerse sale de igualar ambos costes:

coste bajo demanda =  $u \times 730 \text{ h} \times P$   
 coste comprometido =  $730 \text{ h} \times 0,60 P$

umbral:  $u \times P = 0,60 P \rightarrow u = 0,60$

Con más del 60 % de uso sostenido, el compromiso a un año sale a cuenta. Por debajo, no. Ese único número evita la mayoría de las discusiones sobre reservas.

La capacidad excedente merece su propio criterio: cuesta entre un 70 % y un 90 % menos, pero puede retirarse con un preaviso de dos minutos. Es correcta para trabajos por lotes reanudables, colas asíncronas y entornos de pruebas. Nunca para un componente cuya caída rompa una petición de usuario, salvo que la arquitectura absorba la pérdida sin degradación visible.

#### 5. La energía es el coste que no aparece en la factura

El coste monetario ya incorpora la energía, pero la huella no, y cada vez más organizaciones tienen que declararla.

$\text{energía\_total} = \text{potencia\_TI} \times \text{horas} \times \text{PUE}$

Para 9 instancias de 80 W durante un mes:

potencia TI                      =  $9 \times 80 \text{ W} = 720 \text{ W} = 0,72 \text{ kW}$   
 energía TI                      =  $0,72 \text{ kW} \times 730 \text{ h} = 525,6 \text{ kWh}$   
 con PUE 1,12                      =  $525,6 \times 1,12 = 588,7 \text{ kWh/mes}$

Y su traducción a emisiones depende por completo de dónde se ejecuta:

región con 50 g CO<sub>2</sub>e/kWh:  $588,7 \times 0,050 = 29,4 \text{ kg CO}_2\text{e/mes}$   
 región con 400 g CO<sub>2</sub>e/kWh:  $588,7 \times 0,400 = 235,5 \text{ kg CO}_2\text{e/mes}$

Un factor de 8 por la misma carga, con la misma factura. La elección de región, que en la clase 001 se justificaba por latencia y en la 010 por soberanía del dato, tiene aquí un tercer eje.

Dos matices honestos: los factores de emisión varían por hora según el mix de generación, y el PUE no incluye el coste de fabricación del hardware. Cualquier cifra de este tipo es una estimación con orden de magnitud útil, no una medición.

### Ejemplo trabajado

La factura de CloudShop pasa de 12.400 a 17.360 USD en un mes y dirección pide «bajar costes un 30 %». El equipo empieza calculando el denominador antes de tocar nada.

mes anterior 12.400 USD / 620.000 pedidos = 0,0200 USD/pedido  
mes actual 17.360 USD / 980.000 pedidos = 0,0177 USD/pedido

El coste unitario bajó un 11 %. El aumento es crecimiento, no derroche, y ese dato cambia la conversación: recortar un 30 % lineal significaría recortar capacidad mientras el volumen sube un 58 %.

Se desglosa por unidad para encontrar dónde sí hay margen:

|                         |       |        |      |
|-------------------------|-------|--------|------|
| cómputo (9 instancias)  | 6.480 | 0,0066 | 37 % |
| base de datos           | 4.900 | 0,0050 | 28 % |
| transferencia de salida | 3.100 | 0,0032 | 18 % |
| almacenamiento          | 1.480 | 0,0015 | 9 %  |
| observabilidad          | 1.400 | 0,0014 | 8 %  |

Se revisan las dos partidas mayores con los criterios de la clase.

Cómputo. Utilización medida sobre 30 días: media 0,58, percentil 95 diario 0,71. Por la ley de Little con  $\lambda=800/s$  y  $W=250$  ms hacen falta 200 concurrentes; a  $p=0,70$  son 8 instancias y con tolerancia  $n+1$ , nueve. Están correctamente dimensionadas: no hay recorte sin degradar. Pero la utilización sostenida del 58 % está cerca del umbral de compromiso:

umbral de compromiso a 1 año:  $u > 0,60$   
medido: 0,58 en media, 0,71 en p95  
→ comprometer 6 de las 9 instancias (base estable), dejar 3 bajo demanda  
ahorro:  $6 \times 720 \text{ USD} \times 0,40 = 1.728 \text{ USD/mes}$  (−27 % del cómputo)

Transferencia de salida. 3.100 USD equivalen a unos 34 TB. Se revisa la tasa de acierto del CDN:

|                     |       |
|---------------------|-------|
| aciertos de caché   | 61 %  |
| salida desde origen | 34 TB |

Subir el acierto al 90 % ajustando Cache-Control en los activos con huella —lo de la clase 006— reduce la salida desde origen:

salida estimada  $34 \text{ TB} \times (1 - 0,90)/(1 - 0,61) = 8,7 \text{ TB}$   
ahorro  $\approx 2.300 \text{ USD/mes}$  (−74 % de esa partida)

Resultado combinado:

|                      |                                                |         |
|----------------------|------------------------------------------------|---------|
| ahorro total         | $1.728 + 2.300 = 4.028 \text{ USD/mes}$        | (−23 %) |
| coste unitario nuevo | $13.332 / 980.000 = 0,0136 \text{ USD/pedido}$ | (−32 %) |

No se llegó al 30 % de la factura, pero sí se superó el 30 % en coste unitario, que es la métrica que el negocio quería sin saber nombrarla. Y ningún ahorro salió de recortar capacidad: salieron de comprometer la base estable y de dejar de pagar transferencia evitable.

Riesgo residual declarado: el compromiso a un año sobre 6 instancias deja de ser rentable si el volumen cae más de un 40 %. Se acepta con revisión trimestral.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/011-costo-energia-capacidad-y-medicion-basica/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa estimacion-de-costo en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.



## Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye estimacion-de-costo para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                                                      | Causa probable                                                          | Corrección                                                                                                      |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Se pide recortar un porcentaje de la factura sin más contexto                | No hay unidad de coste, así que no se distingue crecimiento de derroche | Divide la factura por una unidad de negocio antes de decidir; el unitario puede bajar mientras la factura sube. |
| La latencia se dispara al subir la utilización del 70 % al 90 %              | El tiempo de espera crece como $1/(1-p)$ , no linealmente               | Fija el objetivo de utilización en 0,60-0,70; la capacidad libre absorbe la varianza.                           |
| Un compromiso de 3 años se convierte en gasto muerto                         | Se comprometió capacidad con utilización por debajo del umbral          | Compromete solo la base estable, con $u > 0,60$ ; deja el pico bajo demanda.                                    |
| Una carga en capacidad excedente rompe peticiones de usuario al ser retirada | Se usó capacidad interrumpible para un componente del camino crítico    | Resérvala para trabajos reanudables; el preaviso puede ser de dos minutos.                                      |
| El 35 % de la factura está en «sin asignar» y nadie lo optimiza              | Falta etiquetado consistente, así que no hay atribución posible         | Impón etiquetas obligatorias en el aprovisionamiento antes de intentar cualquier optimización.                  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. La factura sube un 40 % y el coste unitario baja un 11 %. ¿Hay un problema de costes? Justifica.
2. Con 500 peticiones por segundo y 400 ms de latencia media, ¿cuántas peticiones concurrentes hay en el sistema?
3. ¿Por qué desplegar al 95 % de utilización sale más caro que al 70 %, aunque use menos instancias?
4. ¿A partir de qué utilización sostenida compensa un compromiso de un año al 60 % del precio, y de dónde sale ese umbral?
5. Dos regiones tienen el mismo precio. ¿Qué otro criterio, con qué orden de magnitud, puede decidir entre ellas?

## Referencias

- Little, J. D. C. (1961). A Proof for the Queuing Formula:  $L = \lambda W$ . Operations Research 9(3), 383-387. <<https://doi.org/10.1287/opre.9.3.383>>
- Storment, J. R. y Fuller, M. (2023). Cloud FinOps, 2.ª ed., caps. 6-9 — unidad de coste, etiquetado y modelos de compromiso.
- Gunther, N. (2007). Guerrilla Capacity Planning — leyes de escalabilidad y el efecto de la utilización sobre la latencia.
- Barroso, L., Hölzle, U. y Ranganathan, P. (2018). The Datacenter as a Computer, 3.ª ed., cap. 6 — PUE, eficiencia y coste total. <<https://doi.org/10.2200/S00874ED3V01Y201809CAC046>>
- FinOps Foundation (2024). FinOps Framework — capacidades de asignación, previsión y optimización de tasa. <<https://www.finops.org/framework/>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 011 Costo, energía, capacidad y medición básica

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega estimacion-de-costo con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 012 — Proyecto: servicio local reproducible y observable

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Integrar las once clases anteriores en un artefacto único: un servicio local que arranca con un comando, se observa, falla de forma controlada y se recupera. Es el punto de partida de CloudShop, el sistema que evolucionará durante las 276 clases restantes hasta operar en tres nubes. Lo que se construye aquí no es un ejercicio: es la línea base contra la que se medirá cada cambio posterior.

## Resultados de aprendizaje

Al finalizar podrás:

1. Ensamblar un servicio con comprobaciones de vida y de disponibilidad distinguiendo correctamente sus semánticas.
2. Instrumentar las cuatro señales doradas y justificar por qué la media es insuficiente para la latencia.
3. Provocar un fallo de dependencia y demostrar que el servicio degrada en vez de caer.
4. Establecer una línea base de rendimiento reproducible que sirva de referencia para comparar cambios futuros.
5. Documentar el servicio de modo que otra persona lo levante, lo observe y lo recupere sin conocimiento tácito.

## Conceptos centrales

| Concepto             | Comprensión verificable                                                                                                                                                                                                                      |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| liveness             | Comprobación de si el proceso debe reiniciarse. Debe fallar solo ante un estado irrecuperable: si comprueba dependencias externas, una caída de la base de datos reinicia todas las réplicas y convierte una degradación en una caída total. |
| readiness            | Comprobación de si el proceso puede recibir tráfico ahora. Sí debe mirar dependencias: al fallar, el balanceador retira la instancia sin matarla, y vuelve a incluirla cuando se recupera.                                                   |
| señales doradas      | Latencia, tráfico, errores y saturación. Cuatro métricas que, según el SRE Book, bastan para detectar la mayoría de los problemas de un servicio orientado a peticiones.                                                                     |
| degradación elegante | Responder con funcionalidad reducida en vez de fallar por completo cuando una dependencia no esencial no está disponible.                                                                                                                    |
| línea base           | Medición reproducible del comportamiento actual. Sin ella, cualquier afirmación posterior sobre mejora o regresión es una opinión.                                                                                                           |

## Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart TB
    CL["cliente"] --> LB["balanceador"]
    LB -->|"/health/ready"| API["servicio CloudShop"]
    API --> DB["base de datos<br/>esencial"]
    API -->|"/opcional"| CA["caché"]
    API -->|"/opcional"| REC["recomendaciones"]
    API --> M["/metrics<br/>latencia, tráfico,<br/>errores, saturación"]
    DB -->|"/cae → ready falla<br/>→ se retira del balanceador"| LB
    REC -->|"/cae → responde sin<br/>recomendaciones: 200"| CL
```

## Desarrollo

### 1. Liveness y readiness responden a preguntas opuestas

Confundirlas es el error que convierte una degradación en una caída total, y ocurre porque ambas «comprueban si el servicio está bien».

|                            | Liveness             | Readiness               |
|----------------------------|----------------------|-------------------------|
| Pregunta                   | ¿Hay que reiniciar?  | ¿Puede recibir tráfico? |
| Acción al fallar           | Matar y reiniciar    | Retirar del balanceador |
| ¿Mira dependencias?        | Nunca                | Sí                      |
| Coste de un falso positivo | Reinicio innecesario | Capacidad reducida      |

```
@app.get("/health/live")
def live():
    return {"status": "ok"} # solo: ¿responde el bucle de eventos?

@app.get("/health/ready")
def ready():
    fallos = []
    if not db.ping(timeout=1.0):
        fallos.append("db") # esencial
    if fallos:
        raise HTTPException(503, {"no_listo": fallos})
    return {"status": "ok", "degradado": estado_opcionales()}
```

El escenario que justifica la regla: la base de datos se cae 40 segundos. Con liveness mirando la base de datos, las 9 réplicas fallan la comprobación a la vez y el orquestador las mata todas; cuando la base vuelve, no hay servicio porque todas están arrancando en frío. Con la separación correcta, las 9 salen del balanceador, siguen vivas, y vuelven a entrar en cuanto la dependencia responde.

Hay una tercera comprobación que conviene desde el principio: startup, para procesos con arranque lento. Sin ella hay que relajar el plazo de liveness para todo el ciclo de vida, lo que retrasa la detección de bloqueos reales.

### 2. Las cuatro señales doradas, y por qué la media miente

El SRE Book reduce la instrumentación mínima a cuatro señales:

| Señal      | Qué mide                                   | Cómo se instrumenta  |
|------------|--------------------------------------------|----------------------|
| Latencia   | Cuánto tarda, separando éxitos de errores  | Histograma, no media |
| Tráfico    | Demanda: peticiones/s                      | Contador             |
| Errores    | Tasa de fallo, explícito e implícito       | Contador por código  |
| Saturación | Cuán lleno está el recurso más restrictivo | Medidor              |

La separación de latencia entre éxitos y errores no es un detalle: los errores suelen ser rápidos, así que mezclarlos baja artificialmente la media justo cuando el servicio está peor.

Y la media es insuficiente por sí misma. Con 1.000 peticiones:

```
950 peticiones a 50 ms
50 peticiones a 2.000 ms
```

media =  $(950 \times 50 + 50 \times 2000) / 1000 = 147,5$  ms ← parece aceptable  
p95 = 50 ms  
p99 = 2.000 ms ← 1 de cada 100 usuarios espera 2 s

La media de 147 ms no la experimenta nadie: unos ven 50 ms y otros 2 segundos. Por eso los SLO se fijan sobre percentiles, y por eso las métricas se exportan como histograma —que permite calcular percentiles agregando entre instancias— y no como media previamente calculada, que es matemáticamente imposible de agregar.

La saturación es la más olvidada y la más predictiva: es la única que avisa antes de que la latencia y los errores se degraden. Con lo visto en la clase 011, la saturación por encima de 0,7 anticipa el problema que las otras tres señales aún no muestran.

### 3. Degradar en vez de caer

No todas las dependencias son iguales, y tratarlas igual convierte cualquier fallo en una caída completa. Clasificarlas es una decisión de producto, no técnica:

| Dependencia              | Clase    | Si no está                       |
|--------------------------|----------|----------------------------------|
| Base de datos de pedidos | Esencial | 503: no se puede servir          |
| Caché                    | Opcional | Responder más lento desde origen |
| Recomendaciones          | Opcional | Responder sin esa sección        |
| Telemetría               | Opcional | Registrar localmente y continuar |

```
async def pagina_producto(pid: str):
    producto = await db.producto(pid)                # esencial: si falla, propaga
    recomendados, degradado = [], []
    try:
        recomendados = await asyncio.wait_for(rec.para(pid), timeout=0.15)
    except (asyncio.TimeoutError, ConnectionError):
        degradado.append("recomendaciones")           # opcional: se anota y sigue
    return {"producto": producto, "recomendados": recomendados, "degradado": degradado}
```

Tres detalles que hacen que esto funcione de verdad:

1. El plazo es obligatorio. Sin timeout, una dependencia lenta bloquea al llamante y propaga la saturación hacia arriba. Una dependencia opcional sin plazo es una dependencia esencial disfrazada.
2. La degradación se declara en la respuesta. El campo degradado permite que el cliente y el panel sepan que el 200 no es completo.
3. El plazo debe ser menor que el del llamante. Si el usuario espera 300 ms, una dependencia opcional con plazo de 500 ms nunca llega a tiempo y solo añade latencia.

### 4. Una línea base que se pueda repetir

Sin medición previa, «esto mejoró el rendimiento» es una creencia. La línea base debe fijar carga, duración, entorno y percentiles, y guardarse junto al código:

```
$ hey -z 60s -c 50 -q 20 http://localhost:8080/api/productos/A-1042
```

```
Summary:
  Total:          60.0031 secs
  Requests/sec:  987.4210
```

```
Latency distribution:
  50% in 0.0384 secs
  95% in 0.0912 secs
  99% in 0.1847 secs
```

```
Status code distribution:
  [200] 59248 responses
```

Se registra como contrato comparable:

```
{"escenario": "productos-lectura", "concurrency": 50, "duracion_s": 60,
  "rps": 987.4, "p50_ms": 38.4, "p95_ms": 91.2, "p99_ms": 184.7, "errores": 0}
```

Dos verificaciones con lo aprendido en la clase 011. Ley de Little:

$L = \lambda \cdot W = 987 \times 0,0384 = 37,9$  concurrentes

Coherente con los 50 solicitados: el sistema no está saturado, hay holgura. Si L hubiera dado 50 exactos, la concurrencia sería el cuello de botella y la medida estaría limitada por el generador de carga, no por el servicio.

Relación p99/p50 = 184,7/38,4 = 4,8×. Es la métrica que hay que vigilar entre versiones: si sube, la cola se está alargando aunque la media no se mueva.

## 5. Reproducible significa sin conocimiento tácito

El criterio de aceptación del proyecto no es que funcione en tu máquina: es que otra persona lo levante sin preguntarte nada. Eso exige que el repositorio contenga:

|                     |                                                          |
|---------------------|----------------------------------------------------------|
| projects/cloudshop/ |                                                          |
| ■■■■ README.md      | qué es, cómo se levanta, cómo se comprueba, cómo se para |
| ■■■■ Dockerfile     | entorno de ejecución fijado                              |
| ■■■■ compose.yaml   | el servicio y sus dependencias                           |
| ■■■■ app.py         | el servicio                                              |
| ■■■■ smoke.sh       | comprobación de extremo a extremo con código de salida   |
| ■■■■ baseline.json  | la línea base medida, con su escenario                   |

Y que el recorrido completo quepa en cuatro comandos con salida verificable:

```
$ docker compose up -d
$ ./smoke.sh # sale 0 si todo responde; distinto de 0 si no
$ curl -s localhost:8080/metrics | grep -c '^http_request_duration'
$ docker compose down -v
```

El smoke.sh con código de salida no es cosmético: es lo que permite que este mismo recorrido se ejecute en CI en la parte 08 sin cambiar nada. Un script que imprime «OK» pero siempre sale 0 no verifica nada.

Lo que no debe estar: credenciales reales, rutas absolutas de tu máquina, dependencias instaladas globalmente y no declaradas, y pasos que solo tú conoces. Cada uno de esos es conocimiento tácito, y el conocimiento tácito es lo que hace que un servicio sea irrecuperable cuando la persona que lo sabe está de vacaciones.

## Ejemplo trabajado

Se ensambla CloudShop v0 y se comprueba con un fallo provocado que degrada en vez de caer.

Línea base con todas las dependencias sanas:

```
$ docker compose up -d && ./smoke.sh
ready: ok . degradado: []
$ hey -z 30s -c 50 http://localhost:8080/api/productos/A-1042 | grep -E '95%|99%|Requests/sec'
Requests/sec: 987.4210
95% in 0.0912 secs
99% in 0.1847 secs
```

Prueba negativa 1 — cae una dependencia opcional. Se detiene el servicio de recomendaciones:

```
$ docker compose stop recomendaciones
$ curl -s -o /dev/null -w '%{http_code}\n' localhost:8080/api/productos/A-1042
200
$ curl -s localhost:8080/api/productos/A-1042 | jq -c '.degradado'
["recomendaciones"]
$ curl -s localhost:8080/health/ready | jq -c '.'
{"status":"ok","degradado":["recomendaciones"]}
```

Sigue respondiendo 200 y declara la degradación. El impacto en latencia se mide, no se supone:

|                         |         |                                        |
|-------------------------|---------|----------------------------------------|
| p95 con recomendaciones | 91,2 ms |                                        |
| p95 sin recomendaciones | 73,8 ms | ← más rápido: ya no espera esa llamada |

Prueba negativa 2 — la dependencia opcional no cae, se vuelve lenta. Este es el caso que de verdad rompe sistemas, porque nada falla:

```
$ docker compose exec recomendaciones tc qdisc add dev eth0 root netem delay 800ms
$ hey -z 30s -c 50 http://localhost:8080/api/productos/A-1042 | grep '95%'
95% in 0.0947 secs
```

El p95 apenas se mueve: 94,7 ms frente a 91,2. El plazo de 150 ms corta la llamada lenta y el resultado es equivalente a que estuviera caída. Sin ese plazo, el p95 habría subido a más de 800 ms y la saturación se habría propagado hacia arriba: cada petición mantendría un trabajador ocupado 800 ms, y por la ley de Little la concurrencia necesaria se

multiplicaría por 8.

Prueba negativa 3 — cae la dependencia esencial.

```
$ docker compose stop db
$ curl -s -o /dev/null -w '%{http_code}\n' localhost:8080/health/ready
503
$ curl -s -o /dev/null -w '%{http_code}\n' localhost:8080/health/live
200                                # sigue VIVO: no debe reiniciarse
$ docker compose start db && sleep 3
$ curl -s localhost:8080/health/ready | jq -r .status
ok                                # vuelve solo, sin reiniciar
```

Ese 200 en liveness durante la caída es el resultado que valida el diseño: el orquestador retira la instancia del balanceador pero no la mata, así que cuando la base vuelve hay capacidad caliente inmediata. Si liveness hubiera consultado la base, las nueve réplicas habrían muerto a la vez y la recuperación habría añadido el arranque en frío al tiempo de indisponibilidad.

Evidencia registrada en evidence/: baseline.json, las tres pruebas negativas con su salida, y una limitación explícita — la línea base se midió en una sola máquina sin latencia de red entre componentes; los números no son extrapolables a un despliegue distribuido, solo comparables contra sí mismos.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/012-proyecto-servicio-local-reproducible-y-observable/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa servicio-local en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye servicio-local para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                                                    | Causa probable                                                                   | Corrección                                                                             |
|----------------------------------------------------------------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Cae la base de datos y el orquestador reinicia todas las réplicas a la vez | La comprobación de liveness consulta dependencias externas                       | Liveness solo mira el propio proceso; las dependencias van en readiness.               |
| El panel muestra latencia media aceptable mientras los usuarios se quejan  | La media oculta la cola y mezcla errores rápidos con éxitos lentos               | Exporta histogramas, separa éxitos de errores y fija los SLO sobre p95 y p99.          |
| Una dependencia opcional se vuelve lenta y satura todo el servicio         | La llamada no tenía plazo, así que la dependencia opcional era esencial de hecho | Toda llamada opcional necesita timeout menor que el presupuesto del llamante.          |
| Nadie puede afirmar si un cambio mejoró o empeoró el rendimiento           | No existe una línea base reproducible con escenario y percentiles                | Registra carga, duración, entorno y p50/p95/p99 en un fichero versionado.              |
| Solo una persona sabe levantar el servicio                                 | El recorrido depende de conocimiento tácito no escrito                           | Exige que otra persona lo levante siguiendo solo el README, y corrige lo que le falte. |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Por qué liveness no debe consultar la base de datos, y qué ocurre exactamente si lo hace durante una caída de 40 s?
2. Con 950 peticiones a 50 ms y 50 a 2.000 ms, ¿cuál es la media y cuál el p99? ¿Cuál describe mejor la experiencia?
3. Una dependencia opcional se vuelve lenta pero no falla. ¿Qué la convierte de hecho en esencial y cómo se evita?
4. ¿Por qué no se pueden agregar medias de latencia entre instancias, y qué formato de métrica sí lo permite?
5. Tu línea base da 987 rps y p50 de 38 ms con concurrencia 50. Por la ley de Little, ¿estaba saturado el sistema?

## Referencias

- Beyer, B. et al., eds. (2016). Site Reliability Engineering, cap. 6 «Monitoring Distributed Systems» — las cuatro señales doradas. <<https://sre.google/sre-book/monitoring-distributed-systems/>>
- Kubernetes (2024). Configure Liveness, Readiness and Startup Probes — semántica y efectos de cada sonda. <<https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>>
- Nygard, M. (2018). Release It!, 2.<sup>a</sup> ed., caps. 4-5 — patrones de estabilidad: timeouts, bulkheads y circuit breaker.
- Prometheus (2024). Histograms and summaries — por qué los histogramas son agregables y los cuantiles precalculados no. <<https://prometheus.io/docs/practices/histograms/>>
- Dean, J. y Barroso, L. (2013). The Tail at Scale. Communications of the ACM 56(2), 74-80. <<https://doi.org/10.1145/2408776.2408794>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 012 Proyecto: servicio local reproducible y observable

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.



2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

## Reto verificable

Entrega servicio-local con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

## Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

## Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 01 — Principios, estrategia y adopción cloud

← Parte 00 · Índice completo · Parte 02 →

Nivel: inicial-intermedio · Clases: 12 · Duración sugerida: 6–8 semanas

Decidir cuándo, por qué y cómo adoptar nube con criterios técnicos y económicos.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

## Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                         | Laboratorio  | Horas |
|-----|---------------------------------------------------------------|--------------|-------|
| 013 | Definición NIST y características esenciales de cloud         | foundation   | 4     |
| 014 | Regiones, zonas de disponibilidad, puntos de presencia y edge | architecture | 4     |
| 015 | IaaS, PaaS, SaaS, CaaS y FaaS                                 | decision     | 4     |
| 016 | Elasticidad, escalabilidad, disponibilidad y resiliencia      | reliability  | 4     |
| 017 | Tenancy, cuentas, suscripciones, proyectos y jerarquías       | governance   | 4     |
| 018 | Identidad, roles, políticas y federación                      | iam          | 4     |
| 019 | Modelo de responsabilidad compartida por servicio             | security     | 4     |
| 020 | TCO, costos variables, unit economics y FinOps                | finops       | 4     |
| 021 | Well-Architected y atributos de calidad                       | architecture | 4     |
| 022 | Cloud Adoption Framework y modelo operativo                   | governance   | 4     |
| 023 | Descubrimiento y clasificación de workloads                   | migration    | 4     |
| 024 | Proyecto: decisión de migración sustentada con ADR            | capstone     | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## Pauta bibliográfica

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.

# 013 — Definición NIST y características esenciales de cloud

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: foundation · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar definición nist y características esenciales de cloud dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar definición nist y características esenciales de cloud con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-nist con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto        | Comprensión verificable                                                                                        |
|-----------------|----------------------------------------------------------------------------------------------------------------|
| definición      | Define su papel en definición nist y características esenciales de cloud y cómo observarlo en un sistema real. |
| nist            | Define su papel en definición nist y características esenciales de cloud y cómo observarlo en un sistema real. |
| características | Define su papel en definición nist y características esenciales de cloud y cómo observarlo en un sistema real. |
| esenciales      | Define su papel en definición nist y características esenciales de cloud y cómo observarlo en un sistema real. |
| cloud           | Define su papel en definición nist y características esenciales de cloud y cómo observarlo en un sistema real. |

## Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Definición NIST y características esenciales de cloud"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar definición nist y características esenciales de cloud. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/013-definicion-nist-y-caracteristicas-esenciales-de-cloud/lab.py
```

El laboratorio selecciona el motor de práctica foundation y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa matriz-nist en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es un mapa con fronteras, entradas, salidas y supuestos. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye matriz-nist para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 013 Definición NIST y características esenciales de cloud

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega matriz-nist con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 014 — Regiones, zonas de disponibilidad, puntos de presencia y edge

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar regiones, zonas de disponibilidad, puntos de presencia y edge dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar regiones, zonas de disponibilidad, puntos de presencia y edge con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar mapa-de-topologia con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                                                |
|----------------|------------------------------------------------------------------------------------------------------------------------|
| regiones       | Define su papel en regiones, zonas de disponibilidad, puntos de presencia y edge y cómo observarlo en un sistema real. |
| zonas          | Define su papel en regiones, zonas de disponibilidad, puntos de presencia y edge y cómo observarlo en un sistema real. |
| disponibilidad | Define su papel en regiones, zonas de disponibilidad, puntos de presencia y edge y cómo observarlo en un sistema real. |
| puntos         | Define su papel en regiones, zonas de disponibilidad, puntos de presencia y edge y cómo observarlo en un sistema real. |
| presencia      | Define su papel en regiones, zonas de disponibilidad, puntos de presencia y edge y cómo observarlo en un sistema real. |
| edge           | Define su papel en regiones, zonas de disponibilidad, puntos de presencia y edge y cómo observarlo en un sistema real. |

## Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.



## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Regiones, zonas de disponibilidad, puntos de presencia y edge"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar regiones, zonas de disponibilidad, puntos de presencia y edge. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/014-regiones-zonas-de-disponibilidad-puntos-
```

de-presencia-y-edge/lab.py

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa mapa-de-topologia en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye mapa-de-topologia para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 014 Regiones, zonas de disponibilidad, puntos de presencia y edge

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega mapa-de-topología con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 015 — IaaS, PaaS, SaaS, CaaS y FaaS

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: decision · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar IaaS, PaaS, SaaS, CaaS y FaaS dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar IaaS, PaaS, SaaS, CaaS y FaaS con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-de-servicios con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                |
|----------|----------------------------------------------------------------------------------------|
| IaaS     | Define su papel en IaaS, PaaS, SaaS, CaaS y FaaS y cómo observarlo en un sistema real. |
| PaaS     | Define su papel en IaaS, PaaS, SaaS, CaaS y FaaS y cómo observarlo en un sistema real. |
| SaaS     | Define su papel en IaaS, PaaS, SaaS, CaaS y FaaS y cómo observarlo en un sistema real. |
| CaaS     | Define su papel en IaaS, PaaS, SaaS, CaaS y FaaS y cómo observarlo en un sistema real. |
| FaaS     | Define su papel en IaaS, PaaS, SaaS, CaaS y FaaS y cómo observarlo en un sistema real. |

## Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: IaaS, PaaS, SaaS, CaaS y FaaS"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar iaas, paas, saas, caas y faas. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/015-iaas-paas-saas-caas-y-faas/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-de-servicios en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye matriz-de-servicios para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 015 IaaS, PaaS, SaaS, CaaS y FaaS

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega matriz-de-servicios con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 016 — Elasticidad, escalabilidad, disponibilidad y resiliencia

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: reliability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar elasticidad, escalabilidad, disponibilidad y resiliencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar elasticidad, escalabilidad, disponibilidad y resiliencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-de-capacidad con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                                           |
|----------------|-------------------------------------------------------------------------------------------------------------------|
| elasticidad    | Define su papel en elasticidad, escalabilidad, disponibilidad y resiliencia y cómo observarlo en un sistema real. |
| escalabilidad  | Define su papel en elasticidad, escalabilidad, disponibilidad y resiliencia y cómo observarlo en un sistema real. |
| disponibilidad | Define su papel en elasticidad, escalabilidad, disponibilidad y resiliencia y cómo observarlo en un sistema real. |
| resiliencia    | Define su papel en elasticidad, escalabilidad, disponibilidad y resiliencia y cómo observarlo en un sistema real. |

## Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Elasticidad, escalabilidad, disponibilidad y resiliencia"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar elasticidad, escalabilidad, disponibilidad y resiliencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/016-elasticidad-escalabilidad-disponibilidad-y-resiliencia/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modelo-de-capacidad en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye modelo-de-capacidad para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 016 Elasticidad, escalabilidad, disponibilidad y resiliencia

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega modelo-de-capacidad con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 017 — Tenancy, cuentas, suscripciones, proyectos y jerarquías

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar tenancy, cuentas, suscripciones, proyectos y jerarquías dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar tenancy, cuentas, suscripciones, proyectos y jerarquías con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar arbol-de-recursos con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                                          |
|---------------|------------------------------------------------------------------------------------------------------------------|
| tenancy       | Define su papel en tenancy, cuentas, suscripciones, proyectos y jerarquías y cómo observarlo en un sistema real. |
| cuentas       | Define su papel en tenancy, cuentas, suscripciones, proyectos y jerarquías y cómo observarlo en un sistema real. |
| suscripciones | Define su papel en tenancy, cuentas, suscripciones, proyectos y jerarquías y cómo observarlo en un sistema real. |
| proyectos     | Define su papel en tenancy, cuentas, suscripciones, proyectos y jerarquías y cómo observarlo en un sistema real. |
| jerarquías    | Define su papel en tenancy, cuentas, suscripciones, proyectos y jerarquías y cómo observarlo en un sistema real. |

## Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Tenancy, cuentas, suscripciones, proyectos y jerarquías"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar tenancy, cuentas, suscripciones, proyectos y jerarquías. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/017-tenancy-cuentas-suscripciones-proyectos-y-jerarquias/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa arbol-de-recursos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye arbol-de-recursos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 017 Tenancy, cuentas, suscripciones, proyectos y jerarquías

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega árbol-de-recursos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 018 — Identidad, roles, políticas y federación

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar identidad, roles, políticas y federación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar identidad, roles, políticas y federación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-rbac con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                           |
|------------|---------------------------------------------------------------------------------------------------|
| identidad  | Define su papel en identidad, roles, políticas y federación y cómo observarlo en un sistema real. |
| roles      | Define su papel en identidad, roles, políticas y federación y cómo observarlo en un sistema real. |
| políticas  | Define su papel en identidad, roles, políticas y federación y cómo observarlo en un sistema real. |
| federación | Define su papel en identidad, roles, políticas y federación y cómo observarlo en un sistema real. |

## Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Identidad, roles, políticas y federación"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar identidad, roles, políticas y federación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/018-identidad-roles-politicas-y-federacion/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-rbac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye matriz-rbac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 018 Identidad, roles, políticas y federación

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega matriz-rbac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 019 — Modelo de responsabilidad compartida por servicio

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar modelo de responsabilidad compartida por servicio dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar modelo de responsabilidad compartida por servicio con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar raci-de-controles con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto        | Comprensión verificable                                                                                    |
|-----------------|------------------------------------------------------------------------------------------------------------|
| modelo          | Define su papel en modelo de responsabilidad compartida por servicio y cómo observarlo en un sistema real. |
| responsabilidad | Define su papel en modelo de responsabilidad compartida por servicio y cómo observarlo en un sistema real. |
| compartida      | Define su papel en modelo de responsabilidad compartida por servicio y cómo observarlo en un sistema real. |
| servicio        | Define su papel en modelo de responsabilidad compartida por servicio y cómo observarlo en un sistema real. |

## Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Modelo de responsabilidad compartida por servicio"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar modelo de responsabilidad compartida por servicio. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/019-modelo-de-responsabilidad-compartida-por-servicio/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa raci-de-controles en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye raci-de-controles para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 019 Modelo de responsabilidad compartida por servicio

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega raci-de-contrôles con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 020 — TCO, costos variables, unit economics y FinOps

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: finops · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar tco, costos variables, unit economics y finops dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar tco, costos variables, unit economics y finops con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar caso-tco con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                                 |
|-----------|---------------------------------------------------------------------------------------------------------|
| tco       | Define su papel en tco, costos variables, unit economics y finops y cómo observarlo en un sistema real. |
| costos    | Define su papel en tco, costos variables, unit economics y finops y cómo observarlo en un sistema real. |
| variables | Define su papel en tco, costos variables, unit economics y finops y cómo observarlo en un sistema real. |
| unit      | Define su papel en tco, costos variables, unit economics y finops y cómo observarlo en un sistema real. |
| economics | Define su papel en tco, costos variables, unit economics y finops y cómo observarlo en un sistema real. |
| finops    | Define su papel en tco, costos variables, unit economics y finops y cómo observarlo en un sistema real. |

## Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: TCO, costos variables, unit economics y FinOps"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar tco, costos variables, unit economics y finops. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/020-tco-costos-variables-unit-economics-y-finops/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa caso-tco en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye caso-tco para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 020 TCO, costos variables, unit economics y FinOps

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega caso-tco con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 021 — Well-Architected y atributos de calidad

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar well-architected y atributos de calidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar well-architected y atributos de calidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar revision-well-architected con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                          |
|-------------|--------------------------------------------------------------------------------------------------|
| well        | Define su papel en well-architected y atributos de calidad y cómo observarlo en un sistema real. |
| architected | Define su papel en well-architected y atributos de calidad y cómo observarlo en un sistema real. |
| atributos   | Define su papel en well-architected y atributos de calidad y cómo observarlo en un sistema real. |
| calidad     | Define su papel en well-architected y atributos de calidad y cómo observarlo en un sistema real. |

### Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Well-Architected y atributos de calidad"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar well-architected y atributos de calidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/021-well-architected-y-atributos-de-calidad/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa revision-well-architected en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye revision-well-architected para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 021 Well-Architected y atributos de calidad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega revision-well-architected con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 022 — Cloud Adoption Framework y modelo operativo

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar cloud adoption framework y modelo operativo dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud adoption framework y modelo operativo con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar backlog-de-adopcion con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                              |
|-----------|------------------------------------------------------------------------------------------------------|
| cloud     | Define su papel en cloud adoption framework y modelo operativo y cómo observarlo en un sistema real. |
| adoption  | Define su papel en cloud adoption framework y modelo operativo y cómo observarlo en un sistema real. |
| framework | Define su papel en cloud adoption framework y modelo operativo y cómo observarlo en un sistema real. |
| modelo    | Define su papel en cloud adoption framework y modelo operativo y cómo observarlo en un sistema real. |
| operativo | Define su papel en cloud adoption framework y modelo operativo y cómo observarlo en un sistema real. |

### Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Cloud Adoption Framework y modelo operativo"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud adoption framework y modelo operativo. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/022-cloud-adoption-framework-y-modelo-operativo/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa backlog-de-adopcion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye backlog-de-adopcion para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 022 Cloud Adoption Framework y modelo operativo

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega backlog-de-adopcion con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 023 — Descubrimiento y clasificación de workloads

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: migration · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar descubrimiento y clasificación de workloads dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar descubrimiento y clasificación de workloads con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar inventario-de-workloads con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto       | Comprensión verificable                                                                              |
|----------------|------------------------------------------------------------------------------------------------------|
| descubrimiento | Define su papel en descubrimiento y clasificación de workloads y cómo observarlo en un sistema real. |
| clasificación  | Define su papel en descubrimiento y clasificación de workloads y cómo observarlo en un sistema real. |
| workloads      | Define su papel en descubrimiento y clasificación de workloads y cómo observarlo en un sistema real. |

### Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Descubrimiento y clasificación de workloads"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar descubrimiento y clasificación de workloads. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/023-descubrimiento-y-clasificacion-de-workloads/lab.py
```

El laboratorio selecciona el motor de práctica migration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa inventario-de-workloads en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un inventario, dependencias, riesgo y oleadas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye inventario-de-workloads para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 023 Descubrimiento y clasificación de workloads

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega inventario-de-workloads con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 024 — Proyecto: decisión de migración sustentada con ADR

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar proyecto: decisión de migración sustentada con adr dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: decisión de migración sustentada con adr con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar adr-de-migracion con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                     |
|------------|-------------------------------------------------------------------------------------------------------------|
| proyecto   | Define su papel en proyecto: decisión de migración sustentada con adr y cómo observarlo en un sistema real. |
| decisión   | Define su papel en proyecto: decisión de migración sustentada con adr y cómo observarlo en un sistema real. |
| migración  | Define su papel en proyecto: decisión de migración sustentada con adr y cómo observarlo en un sistema real. |
| sustentada | Define su papel en proyecto: decisión de migración sustentada con adr y cómo observarlo en un sistema real. |
| adr        | Define su papel en proyecto: decisión de migración sustentada con adr y cómo observarlo en un sistema real. |

## Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Proyecto: decisión de migración sustentada con ADR"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: decisión de migración sustentada con adr. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/024-proyecto-decision-de-migracion-sustentada-con-adr/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa adr-de-migracion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye adr-de-migracion para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 024 Proyecto: decisión de migración sustentada con ADR

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega adr-de-migracion con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 02 — AWS: plataforma esencial

← Parte 01 · Índice completo · Parte 03 →

Nivel: intermedio · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar y operar una carga completa en AWS con controles explícitos.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

## Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                              | Laboratorio   | Horas |
|-----|----------------------------------------------------|---------------|-------|
| 025 | Organizations, cuentas, OU, SCP y landing zone     | governance    | 4     |
| 026 | IAM, roles, políticas, STS y federación            | iam           | 4     |
| 027 | VPC, subredes, rutas, NAT, endpoints y seguridad   | network       | 4     |
| 028 | EC2, AMI, EBS y selección de capacidad             | compute       | 4     |
| 029 | Elastic Load Balancing y Auto Scaling              | reliability   | 4     |
| 030 | S3: objetos, versionado, lifecycle y replicación   | storage       | 4     |
| 031 | RDS, DynamoDB y ElastiCache: decisión de datos     | data          | 4     |
| 032 | Lambda, API Gateway y Step Functions               | serverless    | 4     |
| 033 | SQS, SNS y EventBridge                             | messaging     | 4     |
| 034 | CloudWatch, CloudTrail, Config y Systems Manager   | observability | 4     |
| 035 | KMS, Secrets Manager, WAF y controles de seguridad | security      | 4     |
| 036 | Proyecto: aplicación de tres capas en AWS          | capstone      | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## Pauta bibliográfica

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.

## 025 — Organizations, cuentas, OU, SCP y landing zone

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar organizations, cuentas, ou, scp y landing zone dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar organizations, cuentas, ou, scp y landing zone con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar landing-zone-aws con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto      | Comprensión verificable                                                                                 |
|---------------|---------------------------------------------------------------------------------------------------------|
| organizations | Define su papel en organizations, cuentas, ou, scp y landing zone y cómo observarlo en un sistema real. |
| cuentas       | Define su papel en organizations, cuentas, ou, scp y landing zone y cómo observarlo en un sistema real. |
| ou            | Define su papel en organizations, cuentas, ou, scp y landing zone y cómo observarlo en un sistema real. |
| scp           | Define su papel en organizations, cuentas, ou, scp y landing zone y cómo observarlo en un sistema real. |
| landing       | Define su papel en organizations, cuentas, ou, scp y landing zone y cómo observarlo en un sistema real. |
| zone          | Define su papel en organizations, cuentas, ou, scp y landing zone y cómo observarlo en un sistema real. |

### Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Organizations, cuentas, OU, SCP y landing zone"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar organizations, cuentas, ou, scp y landing zone. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/025-organizations-cuentas-ou-scp-y-landing-zone/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa landing-zone-aws en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `landing-zone-aws` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.



- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 025 Organizations, cuentas, OU, SCP y landing zone

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega landing-zone-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 026 — IAM, roles, políticas, STS y federación

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar iam, roles, políticas, sts y federación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar iam, roles, políticas, sts y federación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar politica-iam-aws con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                          |
|------------|--------------------------------------------------------------------------------------------------|
| iam        | Define su papel en iam, roles, políticas, sts y federación y cómo observarlo en un sistema real. |
| roles      | Define su papel en iam, roles, políticas, sts y federación y cómo observarlo en un sistema real. |
| políticas  | Define su papel en iam, roles, políticas, sts y federación y cómo observarlo en un sistema real. |
| sts        | Define su papel en iam, roles, políticas, sts y federación y cómo observarlo en un sistema real. |
| federación | Define su papel en iam, roles, políticas, sts y federación y cómo observarlo en un sistema real. |

### Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: IAM, roles, políticas, STS y federación"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar iam, roles, políticas, sts y federación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/026-iam-roles-politicas-sts-y-federacion/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa politica-iam-aws en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `politica-iam-aws` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.

- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 026 IAM, roles, políticas, STS y federación

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega politica-iam-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 027 — VPC, subredes, rutas, NAT, endpoints y seguridad

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar vpc, subredes, rutas, nat, endpoints y seguridad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar vpc, subredes, rutas, nat, endpoints y seguridad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar vpc-aws con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                                   |
|-----------|-----------------------------------------------------------------------------------------------------------|
| vpc       | Define su papel en vpc, subredes, rutas, nat, endpoints y seguridad y cómo observarlo en un sistema real. |
| subredes  | Define su papel en vpc, subredes, rutas, nat, endpoints y seguridad y cómo observarlo en un sistema real. |
| rutas     | Define su papel en vpc, subredes, rutas, nat, endpoints y seguridad y cómo observarlo en un sistema real. |
| nat       | Define su papel en vpc, subredes, rutas, nat, endpoints y seguridad y cómo observarlo en un sistema real. |
| endpoints | Define su papel en vpc, subredes, rutas, nat, endpoints y seguridad y cómo observarlo en un sistema real. |
| seguridad | Define su papel en vpc, subredes, rutas, nat, endpoints y seguridad y cómo observarlo en un sistema real. |

## Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: VPC, subredes, rutas, NAT, endpoints y seguridad"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar vpc, subredes, rutas, nat, endpoints y seguridad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/027-vpc-subredes-rutas-nat-endpoints-y-seguridad/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa vpc-aws en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `vpc-aws` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.



- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 027 VPC, subredes, rutas, NAT, endpoints y seguridad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega vpc-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 028 — EC2, AMI, EBS y selección de capacidad

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: compute · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar ec2, ami, ebs y selección de capacidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ec2, ami, ebs y selección de capacidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar instancia-aws con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                         |
|-----------|-------------------------------------------------------------------------------------------------|
| ec2       | Define su papel en ec2, ami, ebs y selección de capacidad y cómo observarlo en un sistema real. |
| ami       | Define su papel en ec2, ami, ebs y selección de capacidad y cómo observarlo en un sistema real. |
| ebs       | Define su papel en ec2, ami, ebs y selección de capacidad y cómo observarlo en un sistema real. |
| selección | Define su papel en ec2, ami, ebs y selección de capacidad y cómo observarlo en un sistema real. |
| capacidad | Define su papel en ec2, ami, ebs y selección de capacidad y cómo observarlo en un sistema real. |

### Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: EC2, AMI, EBS y selección de capacidad"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ec2, ami, ebs y selección de capacidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/028-ec2-ami-ebs-y-seleccion-de-capacidad/lab.py
```

El laboratorio selecciona el motor de práctica compute y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa instancia-aws en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una selección de capacidad justificada y observable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye instancia-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.

- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 028 EC2, AMI, EBS y selección de capacidad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega instancia-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 029 — Elastic Load Balancing y Auto Scaling

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: reliability · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar elastic load balancing y auto scaling dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar elastic load balancing y auto scaling con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar servicio-elastico-aws con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                        |
|-----------|------------------------------------------------------------------------------------------------|
| elastic   | Define su papel en elastic load balancing y auto scaling y cómo observarlo en un sistema real. |
| load      | Define su papel en elastic load balancing y auto scaling y cómo observarlo en un sistema real. |
| balancing | Define su papel en elastic load balancing y auto scaling y cómo observarlo en un sistema real. |
| auto      | Define su papel en elastic load balancing y auto scaling y cómo observarlo en un sistema real. |
| scaling   | Define su papel en elastic load balancing y auto scaling y cómo observarlo en un sistema real. |

### Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Elastic Load Balancing y Auto Scaling"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar elastic load balancing y auto scaling. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/029-elastic-load-balancing-y-auto-scaling/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa servicio-elastico-aws en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `servicio-elastico-aws` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.



- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 029 Elastic Load Balancing y Auto Scaling

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega servicio-elastico-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 030 — S3: objetos, versionado, lifecycle y replicación

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: storage · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar s3: objetos, versionado, lifecycle y replicación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar s3: objetos, versionado, lifecycle y replicación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar bucket-gobernado-aws con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                                   |
|-------------|-----------------------------------------------------------------------------------------------------------|
| s3          | Define su papel en s3: objetos, versionado, lifecycle y replicación y cómo observarlo en un sistema real. |
| objetos     | Define su papel en s3: objetos, versionado, lifecycle y replicación y cómo observarlo en un sistema real. |
| versionado  | Define su papel en s3: objetos, versionado, lifecycle y replicación y cómo observarlo en un sistema real. |
| lifecycle   | Define su papel en s3: objetos, versionado, lifecycle y replicación y cómo observarlo en un sistema real. |
| replicación | Define su papel en s3: objetos, versionado, lifecycle y replicación y cómo observarlo en un sistema real. |

### Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: S3: objetos, versionado, lifecycle y replicación"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar s3: objetos, versionado, lifecycle y replicación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/030-s3-objetos-versionado-lifecycle-y-replicacion/lab.py
```

El laboratorio selecciona el motor de práctica storage y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa bucket-gobernado-aws en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una política de durabilidad, acceso, retención y costo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `bucket-gobernado-aws` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.

- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 030 S3: objetos, versionado, lifecycle y replicación

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega bucket-gobernado-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 031 — RDS, DynamoDB y ElastiCache: decisión de datos

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar rds, dynamodb y elasticsearch: decisión de datos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar rds, dynamodb y elasticsearch: decisión de datos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-datos-aws con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                                   |
|---------------|-----------------------------------------------------------------------------------------------------------|
| rds           | Define su papel en rds, dynamodb y elasticsearch: decisión de datos y cómo observarlo en un sistema real. |
| dynamodb      | Define su papel en rds, dynamodb y elasticsearch: decisión de datos y cómo observarlo en un sistema real. |
| elasticsearch | Define su papel en rds, dynamodb y elasticsearch: decisión de datos y cómo observarlo en un sistema real. |
| decisión      | Define su papel en rds, dynamodb y elasticsearch: decisión de datos y cómo observarlo en un sistema real. |
| datos         | Define su papel en rds, dynamodb y elasticsearch: decisión de datos y cómo observarlo en un sistema real. |

## Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: RDS, DynamoDB y ElastiCache: decisión de datos"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar rds, dynamodb y elasticsearch: decisión de datos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/031-rds-dynamodb-y-elasticsearch-decision-de-datos/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-datos-aws en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `matriz-datos-aws` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.



- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 031 RDS, DynamoDB y ElastiCache: decisión de datos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega matriz-datos-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 032 — Lambda, API Gateway y Step Functions

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: serverless · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar lambda, api gateway y step functions dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar lambda, api gateway y step functions con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar api-serverless-aws con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                       |
|-----------|-----------------------------------------------------------------------------------------------|
| lambda    | Define su papel en lambda, api gateway y step functions y cómo observarlo en un sistema real. |
| api       | Define su papel en lambda, api gateway y step functions y cómo observarlo en un sistema real. |
| gateway   | Define su papel en lambda, api gateway y step functions y cómo observarlo en un sistema real. |
| step      | Define su papel en lambda, api gateway y step functions y cómo observarlo en un sistema real. |
| functions | Define su papel en lambda, api gateway y step functions y cómo observarlo en un sistema real. |

### Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Lambda, API Gateway y Step Functions"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar lambda, api gateway y step functions. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/032-lambda-api-gateway-y-step-functions/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa api-serverless-aws en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `api-serverless-aws` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.

- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 032 Lambda, API Gateway y Step Functions

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega api-serverless-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 033 — SQS, SNS y EventBridge

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: messaging · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar sqs, sns y eventbridge dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sqs, sns y eventbridge con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar flujo-eventos-aws con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                         |
|-------------|---------------------------------------------------------------------------------|
| sqs         | Define su papel en sqs, sns y eventbridge y cómo observarlo en un sistema real. |
| sns         | Define su papel en sqs, sns y eventbridge y cómo observarlo en un sistema real. |
| eventbridge | Define su papel en sqs, sns y eventbridge y cómo observarlo en un sistema real. |

### Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: SQS, SNS y EventBridge"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar sqs, sns y eventbridge. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/033-sqs-sns-y-eventbridge/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa flujo-eventos-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye flujo-eventos-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 033 SQS, SNS y EventBridge

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega flujo-eventos-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 034 — CloudWatch, CloudTrail, Config y Systems Manager

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar cloudwatch, cloudtrail, config y systems manager dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloudwatch, cloudtrail, config y systems manager con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar evidencia-operativa-aws con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                                   |
|------------|-----------------------------------------------------------------------------------------------------------|
| cloudwatch | Define su papel en cloudwatch, cloudtrail, config y systems manager y cómo observarlo en un sistema real. |
| cloudtrail | Define su papel en cloudwatch, cloudtrail, config y systems manager y cómo observarlo en un sistema real. |
| config     | Define su papel en cloudwatch, cloudtrail, config y systems manager y cómo observarlo en un sistema real. |
| systems    | Define su papel en cloudwatch, cloudtrail, config y systems manager y cómo observarlo en un sistema real. |
| manager    | Define su papel en cloudwatch, cloudtrail, config y systems manager y cómo observarlo en un sistema real. |

### Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: CloudWatch, CloudTrail, Config y Systems Manager"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloudwatch, cloudtrail, config y systems manager. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/034-cloudwatch-cloudtrail-config-y-systems-manager/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa evidencia-operativa-aws en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye evidencia-operativa-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.

- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 034 CloudWatch, CloudTrail, Config y Systems Manager

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega evidencia-operativa-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 035 — KMS, Secrets Manager, WAF y controles de seguridad

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar kms, secrets manager, waf y controles de seguridad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar kms, secrets manager, waf y controles de seguridad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar controles-aws con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                                     |
|-----------|-------------------------------------------------------------------------------------------------------------|
| kms       | Define su papel en kms, secrets manager, waf y controles de seguridad y cómo observarlo en un sistema real. |
| secrets   | Define su papel en kms, secrets manager, waf y controles de seguridad y cómo observarlo en un sistema real. |
| manager   | Define su papel en kms, secrets manager, waf y controles de seguridad y cómo observarlo en un sistema real. |
| waf       | Define su papel en kms, secrets manager, waf y controles de seguridad y cómo observarlo en un sistema real. |
| controles | Define su papel en kms, secrets manager, waf y controles de seguridad y cómo observarlo en un sistema real. |
| seguridad | Define su papel en kms, secrets manager, waf y controles de seguridad y cómo observarlo en un sistema real. |

## Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: KMS, Secrets Manager, WAF y controles de seguridad"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar kms, secrets manager, waf y controles de seguridad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/035-kms-secrets-manager-waf-y-controles-de-seguridad/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa controles-aws en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye controles-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.



- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 035 KMS, Secrets Manager, WAF y controles de seguridad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega controles-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 036 — Proyecto: aplicación de tres capas en AWS

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar proyecto: aplicación de tres capas en aws dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: aplicación de tres capas en aws con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-aws con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                            |
|------------|----------------------------------------------------------------------------------------------------|
| proyecto   | Define su papel en proyecto: aplicación de tres capas en aws y cómo observarlo en un sistema real. |
| aplicación | Define su papel en proyecto: aplicación de tres capas en aws y cómo observarlo en un sistema real. |
| tres       | Define su papel en proyecto: aplicación de tres capas en aws y cómo observarlo en un sistema real. |
| capas      | Define su papel en proyecto: aplicación de tres capas en aws y cómo observarlo en un sistema real. |
| aws        | Define su papel en proyecto: aplicación de tres capas en aws y cómo observarlo en un sistema real. |

### Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Proyecto: aplicación de tres capas en AWS"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: aplicación de tres capas en aws. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/036-proyecto-aplicacion-de-tres-capas-en-aws/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-aws en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `plataforma-aws` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.

- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 036 Proyecto: aplicación de tres capas en AWS

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plataforma-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## Parte 03 — Azure: plataforma esencial

← Parte 02 · Índice completo · Parte 04 →

Nivel: intermedio · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar y operar una carga empresarial en Azure desde su jerarquía de gobierno.

### Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

### Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

### Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

### Clases

| ID  | Clase                                                       | Laboratorio   | Horas |
|-----|-------------------------------------------------------------|---------------|-------|
| 037 | Tenant, management groups, suscripciones y resource groups  | governance    | 4     |
| 038 | Microsoft Entra ID, RBAC, managed identities y PIM          | iam           | 4     |
| 039 | Virtual Network, subredes, NSG, UDR, peering y Private Link | network       | 4     |
| 040 | Virtual Machines, Scale Sets y Load Balancer                | compute       | 4     |
| 041 | Blob Storage, Files, redundancia y lifecycle                | storage       | 4     |
| 042 | Azure SQL, Cosmos DB y Azure Cache for Redis                | data          | 4     |
| 043 | App Service, Functions y Container Apps                     | serverless    | 4     |
| 044 | Service Bus, Event Grid y Event Hubs                        | messaging     | 4     |
| 045 | Azure Monitor, Log Analytics y Application Insights         | observability | 4     |
| 046 | Key Vault, Defender for Cloud y Azure Policy                | security      | 4     |
| 047 | Bicep, plantillas y despliegues por alcance                 | iac           | 4     |
| 048 | Proyecto: aplicación de tres capas en Azure                 | capstone      | 8     |

### Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## Pauta bibliográfica

- Azure Architecture Center — Microsoft.
- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.

# 037 — Tenant, management groups, suscripciones y resource groups

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar tenant, management groups, suscripciones y resource groups dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar tenant, management groups, suscripciones y resource groups con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar jerarquía-azure con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                                             |
|---------------|---------------------------------------------------------------------------------------------------------------------|
| tenant        | Define su papel en tenant, management groups, suscripciones y resource groups y cómo observarlo en un sistema real. |
| management    | Define su papel en tenant, management groups, suscripciones y resource groups y cómo observarlo en un sistema real. |
| groups        | Define su papel en tenant, management groups, suscripciones y resource groups y cómo observarlo en un sistema real. |
| suscripciones | Define su papel en tenant, management groups, suscripciones y resource groups y cómo observarlo en un sistema real. |
| resource      | Define su papel en tenant, management groups, suscripciones y resource groups y cómo observarlo en un sistema real. |
| groups        | Define su papel en tenant, management groups, suscripciones y resource groups y cómo observarlo en un sistema real. |

## Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.



## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Tenant, management groups, suscripciones y resource groups"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar tenant, management groups, suscripciones y resource groups. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/037-tenant-management-groups-suscripciones-y-resource-
```

groups/lab.py

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa jerarquia-azure en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye jerarquia-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.
- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 037 Tenant, management groups, suscripciones y resource groups

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega jerarquía-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 038 — Microsoft Entra ID, RBAC, managed identities y PIM

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar microsoft entra id, rbac, managed identities y pim dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar microsoft entra id, rbac, managed identities y pim con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar identidad-azure con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                     |
|------------|-------------------------------------------------------------------------------------------------------------|
| microsoft  | Define su papel en microsoft entra id, rbac, managed identities y pim y cómo observarlo en un sistema real. |
| entra      | Define su papel en microsoft entra id, rbac, managed identities y pim y cómo observarlo en un sistema real. |
| id         | Define su papel en microsoft entra id, rbac, managed identities y pim y cómo observarlo en un sistema real. |
| rbac       | Define su papel en microsoft entra id, rbac, managed identities y pim y cómo observarlo en un sistema real. |
| managed    | Define su papel en microsoft entra id, rbac, managed identities y pim y cómo observarlo en un sistema real. |
| identities | Define su papel en microsoft entra id, rbac, managed identities y pim y cómo observarlo en un sistema real. |

## Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Microsoft Entra ID, RBAC, managed identities y PIM"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar microsoft entra id, rbac, managed identities y pim. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/038-microsoft-entra-id-rbac-managed-identities-y-pim/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa identidad-azure en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye identidad-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.

- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 038 Microsoft Entra ID, RBAC, managed identities y PIM

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega identidad-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 039 — Virtual Network, subredes, NSG, UDR, peering y Private Link

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar virtual network, subredes, nsg, udr, peering y private link dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar virtual network, subredes, nsg, udr, peering y private link con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar vnet-azure con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                                              |
|----------|----------------------------------------------------------------------------------------------------------------------|
| virtual  | Define su papel en virtual network, subredes, nsg, udr, peering y private link y cómo observarlo en un sistema real. |
| network  | Define su papel en virtual network, subredes, nsg, udr, peering y private link y cómo observarlo en un sistema real. |
| subredes | Define su papel en virtual network, subredes, nsg, udr, peering y private link y cómo observarlo en un sistema real. |
| nsg      | Define su papel en virtual network, subredes, nsg, udr, peering y private link y cómo observarlo en un sistema real. |
| udr      | Define su papel en virtual network, subredes, nsg, udr, peering y private link y cómo observarlo en un sistema real. |
| peering  | Define su papel en virtual network, subredes, nsg, udr, peering y private link y cómo observarlo en un sistema real. |

## Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Virtual Network, subredes, NSG, UDR, peering y Private Link"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar virtual network, subredes, nsg, udr, peering y private link. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/039-virtual-network-subredes-nsg-udr-peering-y-private-link/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa vnet-azure en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye vnet-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.
- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 039 Virtual Network, subredes, NSG, UDR, peering y Private Link

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega vnet-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 040 — Virtual Machines, Scale Sets y Load Balancer

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: compute · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar virtual machines, scale sets y load balancer dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar virtual machines, scale sets y load balancer con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar servicio-elastico-azure con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                               |
|----------|-------------------------------------------------------------------------------------------------------|
| virtual  | Define su papel en virtual machines, scale sets y load balancer y cómo observarlo en un sistema real. |
| machines | Define su papel en virtual machines, scale sets y load balancer y cómo observarlo en un sistema real. |
| scale    | Define su papel en virtual machines, scale sets y load balancer y cómo observarlo en un sistema real. |
| sets     | Define su papel en virtual machines, scale sets y load balancer y cómo observarlo en un sistema real. |
| load     | Define su papel en virtual machines, scale sets y load balancer y cómo observarlo en un sistema real. |
| balancer | Define su papel en virtual machines, scale sets y load balancer y cómo observarlo en un sistema real. |

### Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Virtual Machines, Scale Sets y Load Balancer"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar virtual machines, scale sets y load balancer. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/040-virtual-machines-scale-sets-y-load-balancer/lab.py
```

El laboratorio selecciona el motor de práctica compute y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa servicio-elastico-azure en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una selección de capacidad justificada y observable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye servicio-elastico-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.

- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 040 Virtual Machines, Scale Sets y Load Balancer

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega servicio-elastico-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 041 — Blob Storage, Files, redundancia y lifecycle

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: storage · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar blob storage, files, redundancia y lifecycle dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar blob storage, files, redundancia y lifecycle con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar storage-gobernado-azure con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                               |
|-------------|-------------------------------------------------------------------------------------------------------|
| blob        | Define su papel en blob storage, files, redundancia y lifecycle y cómo observarlo en un sistema real. |
| storage     | Define su papel en blob storage, files, redundancia y lifecycle y cómo observarlo en un sistema real. |
| files       | Define su papel en blob storage, files, redundancia y lifecycle y cómo observarlo en un sistema real. |
| redundancia | Define su papel en blob storage, files, redundancia y lifecycle y cómo observarlo en un sistema real. |
| lifecycle   | Define su papel en blob storage, files, redundancia y lifecycle y cómo observarlo en un sistema real. |

## Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Blob Storage, Files, redundancia y lifecycle"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar blob storage, files, redundancia y lifecycle. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/041-blob-storage-files-redundancia-y-lifecycle/lab.py
```

El laboratorio selecciona el motor de práctica storage y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa storage-gobernado-azure en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una política de durabilidad, acceso, retención y costo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `storage-gobernado-azure` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.

- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 041 Blob Storage, Files, redundancia y lifecycle

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega storage-gobernado-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 042 — Azure SQL, Cosmos DB y Azure Cache for Redis

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar azure sql, cosmos db y azure cache for redis dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar azure sql, cosmos db y azure cache for redis con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-datos-azure con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                               |
|----------|-------------------------------------------------------------------------------------------------------|
| azure    | Define su papel en azure sql, cosmos db y azure cache for redis y cómo observarlo en un sistema real. |
| sql      | Define su papel en azure sql, cosmos db y azure cache for redis y cómo observarlo en un sistema real. |
| cosmos   | Define su papel en azure sql, cosmos db y azure cache for redis y cómo observarlo en un sistema real. |
| db       | Define su papel en azure sql, cosmos db y azure cache for redis y cómo observarlo en un sistema real. |
| azure    | Define su papel en azure sql, cosmos db y azure cache for redis y cómo observarlo en un sistema real. |
| cache    | Define su papel en azure sql, cosmos db y azure cache for redis y cómo observarlo en un sistema real. |

## Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Azure SQL, Cosmos DB y Azure Cache for Redis"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar azure sql, cosmos db y azure cache for redis. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/042-azure-sql-cosmos-db-y-azure-cache-for-redis/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-datos-azure en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye matriz-datos-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.

- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 042 Azure SQL, Cosmos DB y Azure Cache for Redis

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega matriz-datos-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 043 — App Service, Functions y Container Apps

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: serverless · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar app service, functions y container apps dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar app service, functions y container apps con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar api-plataforma-azure con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                          |
|-----------|--------------------------------------------------------------------------------------------------|
| app       | Define su papel en app service, functions y container apps y cómo observarlo en un sistema real. |
| service   | Define su papel en app service, functions y container apps y cómo observarlo en un sistema real. |
| functions | Define su papel en app service, functions y container apps y cómo observarlo en un sistema real. |
| container | Define su papel en app service, functions y container apps y cómo observarlo en un sistema real. |
| apps      | Define su papel en app service, functions y container apps y cómo observarlo en un sistema real. |

## Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: App Service, Functions y Container Apps"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar app service, functions y container apps. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/043-app-service-functions-y-container-apps/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa api-plataforma-azure en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `api-plataforma-azure` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.

- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 043 App Service, Functions y Container Apps

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega api-plataforma-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 044 — Service Bus, Event Grid y Event Hubs

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: messaging · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar service bus, event grid y event hubs dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar service bus, event grid y event hubs con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar flujo-eventos-azure con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                       |
|----------|-----------------------------------------------------------------------------------------------|
| service  | Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real. |
| bus      | Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real. |
| event    | Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real. |
| grid     | Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real. |
| event    | Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real. |
| hubs     | Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real. |

### Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Service Bus, Event Grid y Event Hubs"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar service bus, event grid y event hubs. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/044-service-bus-event-grid-y-event-hubs/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa flujo-eventos-azure en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye flujo-eventos-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.

- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 044 Service Bus, Event Grid y Event Hubs

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega flujo-eventos-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 045 — Azure Monitor, Log Analytics y Application Insights

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar azure monitor, log analytics y application insights dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que expicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar azure monitor, log analytics y application insights con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar evidencia-operativa-azure con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                      |
|-------------|--------------------------------------------------------------------------------------------------------------|
| azure       | Define su papel en azure monitor, log analytics y application insights y cómo observarlo en un sistema real. |
| monitor     | Define su papel en azure monitor, log analytics y application insights y cómo observarlo en un sistema real. |
| log         | Define su papel en azure monitor, log analytics y application insights y cómo observarlo en un sistema real. |
| analytics   | Define su papel en azure monitor, log analytics y application insights y cómo observarlo en un sistema real. |
| application | Define su papel en azure monitor, log analytics y application insights y cómo observarlo en un sistema real. |
| insights    | Define su papel en azure monitor, log analytics y application insights y cómo observarlo en un sistema real. |

## Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Azure Monitor, Log Analytics y Application Insights"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar azure monitor, log analytics y application insights. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/045-azure-monitor-log-analytics-y-application-insights/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa evidencia-operativa-azure en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye evidencia-operativa-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.
- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 045 Azure Monitor, Log Analytics y Application Insights

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega evidencia-operativa-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 046 — Key Vault, Defender for Cloud y Azure Policy

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar key vault, defender for cloud y azure policy dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar key vault, defender for cloud y azure policy con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar controles-azure con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                               |
|----------|-------------------------------------------------------------------------------------------------------|
| key      | Define su papel en key vault, defender for cloud y azure policy y cómo observarlo en un sistema real. |
| vault    | Define su papel en key vault, defender for cloud y azure policy y cómo observarlo en un sistema real. |
| defender | Define su papel en key vault, defender for cloud y azure policy y cómo observarlo en un sistema real. |
| for      | Define su papel en key vault, defender for cloud y azure policy y cómo observarlo en un sistema real. |
| cloud    | Define su papel en key vault, defender for cloud y azure policy y cómo observarlo en un sistema real. |
| azure    | Define su papel en key vault, defender for cloud y azure policy y cómo observarlo en un sistema real. |

### Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Key Vault, Defender for Cloud y Azure Policy"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar key vault, defender for cloud y azure policy. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/046-key-vault-defender-for-cloud-y-azure-policy/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa controles-azure en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye controles-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.

- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 046 Key Vault, Defender for Cloud y Azure Policy

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega controles-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 047 — Bicep, plantillas y despliegues por alcance

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar bicep, plantillas y despliegues por alcance dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar bicep, plantillas y despliegues por alcance con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar infraestructura-bicep con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                              |
|-------------|------------------------------------------------------------------------------------------------------|
| bicep       | Define su papel en bicep, plantillas y despliegues por alcance y cómo observarlo en un sistema real. |
| plantillas  | Define su papel en bicep, plantillas y despliegues por alcance y cómo observarlo en un sistema real. |
| despliegues | Define su papel en bicep, plantillas y despliegues por alcance y cómo observarlo en un sistema real. |
| alcance     | Define su papel en bicep, plantillas y despliegues por alcance y cómo observarlo en un sistema real. |

## Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Bicep, plantillas y despliegues por alcance"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar bicep, plantillas y despliegues por alcance. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/047-bicep-plantillas-y-despliegues-por-alcance/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa infraestructura-bicep en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye infraestructura-bicep para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.
- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 047 Bicep, plantillas y despliegues por alcance

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega infraestructura-bicep con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 048 — Proyecto: aplicación de tres capas en Azure

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar proyecto: aplicación de tres capas en azure dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: aplicación de tres capas en azure con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-azure con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                              |
|------------|------------------------------------------------------------------------------------------------------|
| proyecto   | Define su papel en proyecto: aplicación de tres capas en azure y cómo observarlo en un sistema real. |
| aplicación | Define su papel en proyecto: aplicación de tres capas en azure y cómo observarlo en un sistema real. |
| tres       | Define su papel en proyecto: aplicación de tres capas en azure y cómo observarlo en un sistema real. |
| capas      | Define su papel en proyecto: aplicación de tres capas en azure y cómo observarlo en un sistema real. |
| azure      | Define su papel en proyecto: aplicación de tres capas en azure y cómo observarlo en un sistema real. |

## Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Proyecto: aplicación de tres capas en Azure"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: aplicación de tres capas en azure. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/048-proyecto-aplicacion-de-tres-capas-en-azure/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-azure en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye plataforma-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.

- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 048 Proyecto: aplicación de tres capas en Azure

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plataforma-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 04 — Google Cloud: plataforma esencial

← Parte 03 · Índice completo · Parte 05 →

Nivel: intermedio · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar y operar una carga en Google Cloud con proyectos, identidades y datos bien delimitados.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

## Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                    | Laboratorio   | Horas |
|-----|----------------------------------------------------------|---------------|-------|
| 049 | Organización, folders, proyectos, billing y cuotas       | governance    | 4     |
| 050 | IAM, service accounts y Workload Identity Federation     | iam           | 4     |
| 051 | VPC global, subredes regionales, firewall y Cloud NAT    | network       | 4     |
| 052 | Compute Engine, managed instance groups y load balancing | compute       | 4     |
| 053 | Cloud Storage, clases, lifecycle y replicación           | storage       | 4     |
| 054 | Cloud SQL, Spanner, Firestore y Memorystore              | data          | 4     |
| 055 | Cloud Run, Cloud Functions y API Gateway                 | serverless    | 4     |
| 056 | Pub/Sub, Cloud Tasks y Workflows                         | messaging     | 4     |
| 057 | Cloud Logging, Monitoring, Trace y Audit Logs            | observability | 4     |
| 058 | Cloud KMS, Secret Manager y Security Command Center      | security      | 4     |
| 059 | Terraform y despliegues reproducibles en GCP             | iac           | 4     |
| 060 | Proyecto: aplicación de tres capas en Google Cloud       | capstone      | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.



## Pauta bibliográfica

- Google Cloud Architecture Framework — Google.
- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.

## 049 — Organización, folders, proyectos, billing y cuotas

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar organización, folders, proyectos, billing y cuotas dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar organización, folders, proyectos, billing y cuotas con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar jerarquía-gcp con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                                                     |
|--------------|-------------------------------------------------------------------------------------------------------------|
| organización | Define su papel en organización, folders, proyectos, billing y cuotas y cómo observarlo en un sistema real. |
| folders      | Define su papel en organización, folders, proyectos, billing y cuotas y cómo observarlo en un sistema real. |
| proyectos    | Define su papel en organización, folders, proyectos, billing y cuotas y cómo observarlo en un sistema real. |
| billing      | Define su papel en organización, folders, proyectos, billing y cuotas y cómo observarlo en un sistema real. |
| cuotas       | Define su papel en organización, folders, proyectos, billing y cuotas y cómo observarlo en un sistema real. |

### Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Organización, folders, proyectos, billing y cuotas"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar organización, folders, proyectos, billing y cuotas. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/049-organizacion-folders-proyectos-billing-y-cuotas/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa jerarquia-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `jerarquia-gcp` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.

- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 049 Organización, folders, proyectos, billing y cuotas

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega jerarquía-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 050 — IAM, service accounts y Workload Identity Federation

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar iam, service accounts y workload identity federation dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar iam, service accounts y workload identity federation con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar identidad-gcp con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                       |
|------------|---------------------------------------------------------------------------------------------------------------|
| iam        | Define su papel en iam, service accounts y workload identity federation y cómo observarlo en un sistema real. |
| service    | Define su papel en iam, service accounts y workload identity federation y cómo observarlo en un sistema real. |
| accounts   | Define su papel en iam, service accounts y workload identity federation y cómo observarlo en un sistema real. |
| workload   | Define su papel en iam, service accounts y workload identity federation y cómo observarlo en un sistema real. |
| identity   | Define su papel en iam, service accounts y workload identity federation y cómo observarlo en un sistema real. |
| federation | Define su papel en iam, service accounts y workload identity federation y cómo observarlo en un sistema real. |

## Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: IAM, service accounts y Workload Identity Federation"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar iam, service accounts y workload identity federation. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/050-iam-service-accounts-y-workload-identity-federation/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa identidad-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye identidad-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Google Cloud Architecture Framework — Google.
- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 050 IAM, service accounts y Workload Identity Federation

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega identidad-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 051 — VPC global, subredes regionales, firewall y Cloud NAT

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar vpc global, subredes regionales, firewall y cloud nat dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar vpc global, subredes regionales, firewall y cloud nat con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar vpc-gcp con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                        |
|------------|----------------------------------------------------------------------------------------------------------------|
| vpc        | Define su papel en vpc global, subredes regionales, firewall y cloud nat y cómo observarlo en un sistema real. |
| global     | Define su papel en vpc global, subredes regionales, firewall y cloud nat y cómo observarlo en un sistema real. |
| subredes   | Define su papel en vpc global, subredes regionales, firewall y cloud nat y cómo observarlo en un sistema real. |
| regionales | Define su papel en vpc global, subredes regionales, firewall y cloud nat y cómo observarlo en un sistema real. |
| firewall   | Define su papel en vpc global, subredes regionales, firewall y cloud nat y cómo observarlo en un sistema real. |
| cloud      | Define su papel en vpc global, subredes regionales, firewall y cloud nat y cómo observarlo en un sistema real. |

## Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: VPC global, subredes regionales, firewall y Cloud NAT"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar vpc global, subredes regionales, firewall y cloud nat. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/051-vpc-global-subredes-regionales-firewall-y-cloud-nat/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa vpc-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye vpc-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 051 VPC global, subredes regionales, firewall y Cloud NAT

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega vpc-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 052 — Compute Engine, managed instance groups y load balancing

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: compute · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar compute engine, managed instance groups y load balancing dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar compute engine, managed instance groups y load balancing con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar servicio-elastico-gcp con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                                           |
|----------|-------------------------------------------------------------------------------------------------------------------|
| compute  | Define su papel en compute engine, managed instance groups y load balancing y cómo observarlo en un sistema real. |
| engine   | Define su papel en compute engine, managed instance groups y load balancing y cómo observarlo en un sistema real. |
| managed  | Define su papel en compute engine, managed instance groups y load balancing y cómo observarlo en un sistema real. |
| instance | Define su papel en compute engine, managed instance groups y load balancing y cómo observarlo en un sistema real. |
| groups   | Define su papel en compute engine, managed instance groups y load balancing y cómo observarlo en un sistema real. |
| load     | Define su papel en compute engine, managed instance groups y load balancing y cómo observarlo en un sistema real. |

## Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Compute Engine, managed instance groups y load balancing"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar compute engine, managed instance groups y load balancing. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/052-compute-engine-managed-instance-groups-y-load-balancing/
```

lab.py

El laboratorio selecciona el motor de práctica compute y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa servicio-elastico-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una selección de capacidad justificada y observable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye servicio-elastico-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.



## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 052 Compute Engine, managed instance groups y load balancing

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega servicio-elastico-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 053 — Cloud Storage, clases, lifecycle y replicación

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: storage · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar cloud storage, clases, lifecycle y replicación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud storage, clases, lifecycle y replicación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar bucket-gobernado-gcp con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                                 |
|-------------|---------------------------------------------------------------------------------------------------------|
| cloud       | Define su papel en cloud storage, clases, lifecycle y replicación y cómo observarlo en un sistema real. |
| storage     | Define su papel en cloud storage, clases, lifecycle y replicación y cómo observarlo en un sistema real. |
| clases      | Define su papel en cloud storage, clases, lifecycle y replicación y cómo observarlo en un sistema real. |
| lifecycle   | Define su papel en cloud storage, clases, lifecycle y replicación y cómo observarlo en un sistema real. |
| replicación | Define su papel en cloud storage, clases, lifecycle y replicación y cómo observarlo en un sistema real. |

### Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Cloud Storage, clases, lifecycle y replicación"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud storage, clases, lifecycle y replicación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/053-cloud-storage-clases-lifecycle-y-replicacion/lab.py
```

El laboratorio selecciona el motor de práctica storage y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa bucket-gobernado-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una política de durabilidad, acceso, retención y costo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `bucket-gobernado-gcp` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.

- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 053 Cloud Storage, clases, lifecycle y replicación

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega bucket-gobernado-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 054 — Cloud SQL, Spanner, Firestore y Memorystore

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar cloud sql, spanner, firestore y memorystore dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud sql, spanner, firestore y memorystore con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-datos-gcp con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                              |
|-------------|------------------------------------------------------------------------------------------------------|
| cloud       | Define su papel en cloud sql, spanner, firestore y memorystore y cómo observarlo en un sistema real. |
| sql         | Define su papel en cloud sql, spanner, firestore y memorystore y cómo observarlo en un sistema real. |
| spanner     | Define su papel en cloud sql, spanner, firestore y memorystore y cómo observarlo en un sistema real. |
| firestore   | Define su papel en cloud sql, spanner, firestore y memorystore y cómo observarlo en un sistema real. |
| memorystore | Define su papel en cloud sql, spanner, firestore y memorystore y cómo observarlo en un sistema real. |

## Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Cloud SQL, Spanner, Firestore y Memorystore"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud sql, spanner, firestore y memorystore. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/054-cloud-sql-spanner-firestore-y-memorystore/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-datos-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `matriz-datos-gcp` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.



- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 054 Cloud SQL, Spanner, Firestore y Memorystore

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega matriz-datos-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 055 — Cloud Run, Cloud Functions y API Gateway

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: serverless · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar cloud run, cloud functions y api gateway dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud run, cloud functions y api gateway con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar api-serverless-gcp con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                           |
|-----------|---------------------------------------------------------------------------------------------------|
| cloud     | Define su papel en cloud run, cloud functions y api gateway y cómo observarlo en un sistema real. |
| run       | Define su papel en cloud run, cloud functions y api gateway y cómo observarlo en un sistema real. |
| cloud     | Define su papel en cloud run, cloud functions y api gateway y cómo observarlo en un sistema real. |
| functions | Define su papel en cloud run, cloud functions y api gateway y cómo observarlo en un sistema real. |
| api       | Define su papel en cloud run, cloud functions y api gateway y cómo observarlo en un sistema real. |
| gateway   | Define su papel en cloud run, cloud functions y api gateway y cómo observarlo en un sistema real. |

## Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Cloud Run, Cloud Functions y API Gateway"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud run, cloud functions y api gateway. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/055-cloud-run-cloud-functions-y-api-gateway/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa api-serverless-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `api-serverless-gcp` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.

- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 055 Cloud Run, Cloud Functions y API Gateway

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega api-serverless-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 056 — Pub/Sub, Cloud Tasks y Workflows

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: messaging · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar pub/sub, cloud tasks y workflows dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar pub/sub, cloud tasks y workflows con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar flujo-eventos-gcp con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                   |
|-----------|-------------------------------------------------------------------------------------------|
| pub       | Define su papel en pub/sub, cloud tasks y workflows y cómo observarlo en un sistema real. |
| sub       | Define su papel en pub/sub, cloud tasks y workflows y cómo observarlo en un sistema real. |
| cloud     | Define su papel en pub/sub, cloud tasks y workflows y cómo observarlo en un sistema real. |
| tasks     | Define su papel en pub/sub, cloud tasks y workflows y cómo observarlo en un sistema real. |
| workflows | Define su papel en pub/sub, cloud tasks y workflows y cómo observarlo en un sistema real. |

### Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Pub/Sub, Cloud Tasks y Workflows"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E["¿Cumple seguridad, SLO y costo?"]
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar pub/sub, cloud tasks y workflows. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/056-pub-sub-cloud-tasks-y-workflows/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa flujo-eventos-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye flujo-eventos-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 056 Pub/Sub, Cloud Tasks y Workflows

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega flujo-eventos-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 057 — Cloud Logging, Monitoring, Trace y Audit Logs

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar cloud logging, monitoring, trace y audit logs dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud logging, monitoring, trace y audit logs con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar evidencia-operativa-gcp con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                |
|------------|--------------------------------------------------------------------------------------------------------|
| cloud      | Define su papel en cloud logging, monitoring, trace y audit logs y cómo observarlo en un sistema real. |
| logging    | Define su papel en cloud logging, monitoring, trace y audit logs y cómo observarlo en un sistema real. |
| monitoring | Define su papel en cloud logging, monitoring, trace y audit logs y cómo observarlo en un sistema real. |
| trace      | Define su papel en cloud logging, monitoring, trace y audit logs y cómo observarlo en un sistema real. |
| audit      | Define su papel en cloud logging, monitoring, trace y audit logs y cómo observarlo en un sistema real. |
| logs       | Define su papel en cloud logging, monitoring, trace y audit logs y cómo observarlo en un sistema real. |

## Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Cloud Logging, Monitoring, Trace y Audit Logs"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud logging, monitoring, trace y audit logs. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/057-cloud-logging-monitoring-trace-y-audit-logs/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa evidencia-operativa-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye evidencia-operativa-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.

- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 057 Cloud Logging, Monitoring, Trace y Audit Logs

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega evidencia-operativa-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 058 — Cloud KMS, Secret Manager y Security Command Center

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar cloud kms, secret manager y security command center dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud kms, secret manager y security command center con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar controles-gcp con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                                      |
|----------|--------------------------------------------------------------------------------------------------------------|
| cloud    | Define su papel en cloud kms, secret manager y security command center y cómo observarlo en un sistema real. |
| kms      | Define su papel en cloud kms, secret manager y security command center y cómo observarlo en un sistema real. |
| secret   | Define su papel en cloud kms, secret manager y security command center y cómo observarlo en un sistema real. |
| manager  | Define su papel en cloud kms, secret manager y security command center y cómo observarlo en un sistema real. |
| security | Define su papel en cloud kms, secret manager y security command center y cómo observarlo en un sistema real. |
| command  | Define su papel en cloud kms, secret manager y security command center y cómo observarlo en un sistema real. |

## Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Cloud KMS, Secret Manager y Security Command Center"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud kms, secret manager y security command center. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/058-cloud-kms-secret-manager-y-security-command-center/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa controles-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye controles-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.



- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 058 Cloud KMS, Secret Manager y Security Command Center

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega controles-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 059 — Terraform y despliegues reproducibles en GCP

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar terraform y despliegues reproducibles en gcp dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar terraform y despliegues reproducibles en gcp con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar infraestructura-gcp con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                               |
|---------------|-------------------------------------------------------------------------------------------------------|
| terraform     | Define su papel en terraform y despliegues reproducibles en gcp y cómo observarlo en un sistema real. |
| despliegues   | Define su papel en terraform y despliegues reproducibles en gcp y cómo observarlo en un sistema real. |
| reproducibles | Define su papel en terraform y despliegues reproducibles en gcp y cómo observarlo en un sistema real. |
| gcp           | Define su papel en terraform y despliegues reproducibles en gcp y cómo observarlo en un sistema real. |

## Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Terraform y despliegues reproducibles en GCP"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar terraform y despliegues reproducibles en gcp. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/059-terraform-y-despliegues-reproducibles-en-gcp/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa infraestructura-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye infraestructura-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 059 Terraform y despliegues reproducibles en GCP

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega infraestructura-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 060 — Proyecto: aplicación de tres capas en Google Cloud

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar proyecto: aplicación de tres capas en google cloud dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: aplicación de tres capas en google cloud con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-gcp con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                     |
|------------|-------------------------------------------------------------------------------------------------------------|
| proyecto   | Define su papel en proyecto: aplicación de tres capas en google cloud y cómo observarlo en un sistema real. |
| aplicación | Define su papel en proyecto: aplicación de tres capas en google cloud y cómo observarlo en un sistema real. |
| tres       | Define su papel en proyecto: aplicación de tres capas en google cloud y cómo observarlo en un sistema real. |
| capas      | Define su papel en proyecto: aplicación de tres capas en google cloud y cómo observarlo en un sistema real. |
| google     | Define su papel en proyecto: aplicación de tres capas en google cloud y cómo observarlo en un sistema real. |
| cloud      | Define su papel en proyecto: aplicación de tres capas en google cloud y cómo observarlo en un sistema real. |

## Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Proyecto: aplicación de tres capas en Google Cloud"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: aplicación de tres capas en google cloud. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/060-proyecto-aplicacion-de-tres-capas-en-google-cloud/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `plataforma-gcp` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.



- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 060 Proyecto: aplicación de tres capas en Google Cloud

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plataforma-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 05 — Contenedores, Docker y OCI

← Parte 04 · Índice completo · Parte 06 →

Nivel: intermedio · Clases: 12 · Duración sugerida: 6–8 semanas

Empaquetar servicios portables, observables y endurecidos.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

## Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                              | Laboratorio   | Horas |
|-----|----------------------------------------------------|---------------|-------|
| 061 | Imágenes, capas, registros y estándar OCI          | container     | 4     |
| 062 | Dockerfile reproducible y builds multi-stage       | container     | 4     |
| 063 | Namespaces, cgroups y runtime de contenedores      | container     | 4     |
| 064 | Volúmenes, bind mounts y persistencia              | storage       | 4     |
| 065 | Redes bridge, DNS interno y publicación de puertos | network       | 4     |
| 066 | Docker Compose y aplicaciones multiservicio        | orchestration | 4     |
| 067 | Registros, SBOM, firma y procedencia de imágenes   | supply-chain  | 4     |
| 068 | Límites, health checks y apagado ordenado          | reliability   | 4     |
| 069 | Rootless, capabilities, seccomp y secretos         | security      | 4     |
| 070 | Diagnóstico de CPU, memoria, red y filesystem      | observability | 4     |
| 071 | Migración de una aplicación legacy a contenedores  | migration     | 4     |
| 072 | Proyecto: stack OCI endurecido y observable        | capstone      | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## Pauta bibliográfica

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.

# 061 — Imágenes, capas, registros y estándar OCI

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: container · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar imágenes, capas, registros y estándar oci dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar imágenes, capas, registros y estándar oci con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar inspeccion-imagen con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                            |
|-----------|----------------------------------------------------------------------------------------------------|
| imágenes  | Define su papel en imágenes, capas, registros y estándar oci y cómo observarlo en un sistema real. |
| capas     | Define su papel en imágenes, capas, registros y estándar oci y cómo observarlo en un sistema real. |
| registros | Define su papel en imágenes, capas, registros y estándar oci y cómo observarlo en un sistema real. |
| estándar  | Define su papel en imágenes, capas, registros y estándar oci y cómo observarlo en un sistema real. |
| oci       | Define su papel en imágenes, capas, registros y estándar oci y cómo observarlo en un sistema real. |

## Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Imágenes, capas, registros y estándar OCI"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar imágenes, capas, registros y estándar oci. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/061-imagenes-capas-registros-y-estandar-oci/lab.py
```

El laboratorio selecciona el motor de práctica container y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa inspeccion-imagen en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una imagen mínima, escaneada y ejecutada sin privilegios. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye inspeccion-imagen para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Docker Deep Dive — Nigel Poulton.

- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 061 Imágenes, capas, registros y estándar OCI

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega inspeccion-imagen con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 062 — Dockerfile reproducible y builds multi-stage

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: container · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar dockerfile reproducible y builds multi-stage dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar dockerfile reproducible y builds multi-stage con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar imagen-minima con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                                               |
|--------------|-------------------------------------------------------------------------------------------------------|
| dockerfile   | Define su papel en dockerfile reproducible y builds multi-stage y cómo observarlo en un sistema real. |
| reproducible | Define su papel en dockerfile reproducible y builds multi-stage y cómo observarlo en un sistema real. |
| builds       | Define su papel en dockerfile reproducible y builds multi-stage y cómo observarlo en un sistema real. |
| multi        | Define su papel en dockerfile reproducible y builds multi-stage y cómo observarlo en un sistema real. |
| stage        | Define su papel en dockerfile reproducible y builds multi-stage y cómo observarlo en un sistema real. |

### Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Dockerfile reproducible y builds multi-stage"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar dockerfile reproducible y builds multi-stage. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/062-dockerfile-reproducible-y-builds-multi-stage/lab.py
```

El laboratorio selecciona el motor de práctica container y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa imagen-minima en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una imagen mínima, escaneada y ejecutada sin privilegios. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye imagen-minima para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Docker Deep Dive — Nigel Poulton.

- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 062 Dockerfile reproducible y builds multi-stage

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega imagen-minima con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 063 — Namespaces, cgroups y runtime de contenedores

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: container · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar namespaces, cgroups y runtime de contenedores dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar namespaces, cgroups y runtime de contenedores con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar mapa-aislamiento con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                                                |
|--------------|--------------------------------------------------------------------------------------------------------|
| namespaces   | Define su papel en namespaces, cgroups y runtime de contenedores y cómo observarlo en un sistema real. |
| cgroups      | Define su papel en namespaces, cgroups y runtime de contenedores y cómo observarlo en un sistema real. |
| runtime      | Define su papel en namespaces, cgroups y runtime de contenedores y cómo observarlo en un sistema real. |
| contenedores | Define su papel en namespaces, cgroups y runtime de contenedores y cómo observarlo en un sistema real. |

### Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Namespaces, cgroups y runtime de contenedores"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar namespaces, cgroups y runtime de contenedores. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/063-namespaces-cgroups-y-runtime-de-contenedores/lab.py
```

El laboratorio selecciona el motor de práctica container y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa mapa-aislamiento en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una imagen mínima, escaneada y ejecutada sin privilegios. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye mapa-aislamiento para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 063 Namespaces, cgroups y runtime de contenedores

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega mapa-aislamiento con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 064 — Volúmenes, bind mounts y persistencia

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: storage · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar volúmenes, bind mounts y persistencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar volúmenes, bind mounts y persistencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar persistencia-contenedor con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                                        |
|--------------|------------------------------------------------------------------------------------------------|
| volúmenes    | Define su papel en volúmenes, bind mounts y persistencia y cómo observarlo en un sistema real. |
| bind         | Define su papel en volúmenes, bind mounts y persistencia y cómo observarlo en un sistema real. |
| mounts       | Define su papel en volúmenes, bind mounts y persistencia y cómo observarlo en un sistema real. |
| persistencia | Define su papel en volúmenes, bind mounts y persistencia y cómo observarlo en un sistema real. |

### Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Volúmenes, bind mounts y persistencia"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar volúmenes, bind mounts y persistencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/064-volumenes-bind-mounts-y-persistencia/lab.py
```

El laboratorio selecciona el motor de práctica storage y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa persistencia-contenedor en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una política de durabilidad, acceso, retención y costo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye persistencia-contenedor para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 064 Volúmenes, bind mounts y persistencia

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega persistencia-contenedor con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 065 — Redes bridge, DNS interno y publicación de puertos

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar redes bridge, dns interno y publicación de puertos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar redes bridge, dns interno y publicación de puertos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar red-contenedores con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                     |
|-------------|-------------------------------------------------------------------------------------------------------------|
| redes       | Define su papel en redes bridge, dns interno y publicación de puertos y cómo observarlo en un sistema real. |
| bridge      | Define su papel en redes bridge, dns interno y publicación de puertos y cómo observarlo en un sistema real. |
| dns         | Define su papel en redes bridge, dns interno y publicación de puertos y cómo observarlo en un sistema real. |
| interno     | Define su papel en redes bridge, dns interno y publicación de puertos y cómo observarlo en un sistema real. |
| publicación | Define su papel en redes bridge, dns interno y publicación de puertos y cómo observarlo en un sistema real. |
| puertos     | Define su papel en redes bridge, dns interno y publicación de puertos y cómo observarlo en un sistema real. |

## Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Redes bridge, DNS interno y publicación de puertos"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar redes bridge, dns interno y publicación de puertos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/065-redes-bridge-dns-interno-y-publicacion-de-puertos/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa red-contenedores en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye red-contenedores para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 065 Redes bridge, DNS interno y publicación de puertos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega red-contenedores con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 066 — Docker Compose y aplicaciones multiservicio

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: orchestration · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar docker compose y aplicaciones multiservicio dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar docker compose y aplicaciones multiservicio con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar stack-compose con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                              |
|---------------|------------------------------------------------------------------------------------------------------|
| docker        | Define su papel en docker compose y aplicaciones multiservicio y cómo observarlo en un sistema real. |
| compose       | Define su papel en docker compose y aplicaciones multiservicio y cómo observarlo en un sistema real. |
| aplicaciones  | Define su papel en docker compose y aplicaciones multiservicio y cómo observarlo en un sistema real. |
| multiservicio | Define su papel en docker compose y aplicaciones multiservicio y cómo observarlo en un sistema real. |

## Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Docker Compose y aplicaciones multiservicio"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar docker compose y aplicaciones multiservicio. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/066-docker-compose-y-aplicaciones-multiservicio/lab.py
```

El laboratorio selecciona el motor de práctica orchestration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa stack-compose en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es servicios coordinados con health checks y apagado limpio. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye stack-compose para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 066 Docker Compose y aplicaciones multiservicio

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega stack-compose con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 067 — Registros, SBOM, firma y procedencia de imágenes

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: supply-chain · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar registros, sbom, firma y procedencia de imágenes dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar registros, sbom, firma y procedencia de imágenes con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cadena-suministro-imagen con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                   |
|-------------|-----------------------------------------------------------------------------------------------------------|
| registros   | Define su papel en registros, sbom, firma y procedencia de imágenes y cómo observarlo en un sistema real. |
| sbom        | Define su papel en registros, sbom, firma y procedencia de imágenes y cómo observarlo en un sistema real. |
| firma       | Define su papel en registros, sbom, firma y procedencia de imágenes y cómo observarlo en un sistema real. |
| procedencia | Define su papel en registros, sbom, firma y procedencia de imágenes y cómo observarlo en un sistema real. |
| imágenes    | Define su papel en registros, sbom, firma y procedencia de imágenes y cómo observarlo en un sistema real. |

## Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Registros, SBOM, firma y procedencia de imágenes"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar registros, sbom, firma y procedencia de imágenes. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/067-registros-sbom-firma-y-procedencia-de-imagenes/lab.py
```

El laboratorio selecciona el motor de práctica supply-chain y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cadena-suministro-imagen en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es procedencia, inventario y verificación del artefacto. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye cadena-suministro-imagen para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Docker Deep Dive — Nigel Poulton.

- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 067 Registros, SBOM, firma y procedencia de imágenes

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega cadena-suministro-imagen con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 068 — Límites, health checks y apagado ordenado

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: reliability · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar límites, health checks y apagado ordenado dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar límites, health checks y apagado ordenado con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar contenedor-operable con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                            |
|----------|----------------------------------------------------------------------------------------------------|
| límites  | Define su papel en límites, health checks y apagado ordenado y cómo observarlo en un sistema real. |
| health   | Define su papel en límites, health checks y apagado ordenado y cómo observarlo en un sistema real. |
| checks   | Define su papel en límites, health checks y apagado ordenado y cómo observarlo en un sistema real. |
| apagado  | Define su papel en límites, health checks y apagado ordenado y cómo observarlo en un sistema real. |
| ordenado | Define su papel en límites, health checks y apagado ordenado y cómo observarlo en un sistema real. |

### Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Límites, health checks y apagado ordenado"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar límites, health checks y apagado ordenado. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/068-limites-health-checks-y-apagado-ordenado/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa contenedor-operable en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye contenedor-operable para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Docker Deep Dive — Nigel Poulton.

- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 068 Límites, health checks y apagado ordenado

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega contenedor-operable con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 069 — Rootless, capabilities, seccomp y secretos

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar rootless, capabilities, seccomp y secretos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar rootless, capabilities, seccomp y secretos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar contenedor-endurecido con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                                             |
|--------------|-----------------------------------------------------------------------------------------------------|
| rootless     | Define su papel en rootless, capabilities, seccomp y secretos y cómo observarlo en un sistema real. |
| capabilities | Define su papel en rootless, capabilities, seccomp y secretos y cómo observarlo en un sistema real. |
| seccomp      | Define su papel en rootless, capabilities, seccomp y secretos y cómo observarlo en un sistema real. |
| secretos     | Define su papel en rootless, capabilities, seccomp y secretos y cómo observarlo en un sistema real. |

### Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Rootless, capabilities, seccomp y secretos"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar rootless, capabilities, seccomp y secretos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/069-rootless-capabilities-seccomp-y-secretos/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa contenedor-endurecido en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye contenedor-endurecido para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 069 Rootless, capabilities, seccomp y secretos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega contenedor-endurecido con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 070 — Diagnóstico de CPU, memoria, red y filesystem

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar diagnóstico de cpu, memoria, red y filesystem dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar diagnóstico de cpu, memoria, red y filesystem con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar informe-diagnostico con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                |
|-------------|--------------------------------------------------------------------------------------------------------|
| diagnóstico | Define su papel en diagnóstico de cpu, memoria, red y filesystem y cómo observarlo en un sistema real. |
| cpu         | Define su papel en diagnóstico de cpu, memoria, red y filesystem y cómo observarlo en un sistema real. |
| memoria     | Define su papel en diagnóstico de cpu, memoria, red y filesystem y cómo observarlo en un sistema real. |
| red         | Define su papel en diagnóstico de cpu, memoria, red y filesystem y cómo observarlo en un sistema real. |
| filesystem  | Define su papel en diagnóstico de cpu, memoria, red y filesystem y cómo observarlo en un sistema real. |

## Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Diagnóstico de CPU, memoria, red y filesystem"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar diagnóstico de cpu, memoria, red y filesystem. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/070-diagnostico-de-cpu-memoria-red-y-filesystem/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa informe-diagnostico en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye informe-diagnostico para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Docker Deep Dive — Nigel Poulton.

- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 070 Diagnóstico de CPU, memoria, red y filesystem

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega informe-diagnostico con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 071 — Migración de una aplicación legacy a contenedores

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: migration · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar migración de una aplicación legacy a contenedores dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar migración de una aplicación legacy a contenedores con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plan-modernización con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                    |
|--------------|------------------------------------------------------------------------------------------------------------|
| migración    | Define su papel en migración de una aplicación legacy a contenedores y cómo observarlo en un sistema real. |
| aplicación   | Define su papel en migración de una aplicación legacy a contenedores y cómo observarlo en un sistema real. |
| legacy       | Define su papel en migración de una aplicación legacy a contenedores y cómo observarlo en un sistema real. |
| contenedores | Define su papel en migración de una aplicación legacy a contenedores y cómo observarlo en un sistema real. |

## Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Migración de una aplicación legacy a contenedores"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar migración de una aplicación legacy a contenedores. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/071-migracion-de-una-aplicacion-legacy-a-contenedores/lab.py
```

El laboratorio selecciona el motor de práctica migration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plan-modernizacion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un inventario, dependencias, riesgo y oleadas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye plan-modernización para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 071 Migración de una aplicación legacy a contenedores

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plan-modernización con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 072 — Proyecto: stack OCI endurecido y observable

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar proyecto: stack oci endurecido y observable dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: stack oci endurecido y observable con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-contenedores con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                              |
|------------|------------------------------------------------------------------------------------------------------|
| proyecto   | Define su papel en proyecto: stack oci endurecido y observable y cómo observarlo en un sistema real. |
| stack      | Define su papel en proyecto: stack oci endurecido y observable y cómo observarlo en un sistema real. |
| oci        | Define su papel en proyecto: stack oci endurecido y observable y cómo observarlo en un sistema real. |
| endurecido | Define su papel en proyecto: stack oci endurecido y observable y cómo observarlo en un sistema real. |
| observable | Define su papel en proyecto: stack oci endurecido y observable y cómo observarlo en un sistema real. |

## Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Proyecto: stack OCI endurecido y observable"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: stack oci endurecido y observable. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/072-proyecto-stack-oci-endurecido-y-observable/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-contenedores en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye plataforma-contenedores para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Docker Deep Dive — Nigel Poulton.

- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 072 Proyecto: stack OCI endurecido y observable

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plataforma-contenedores con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 06 — Kubernetes y plataformas administradas

← Parte 05 · Índice completo · Parte 07 →

Nivel: intermedio-avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Operar cargas declarativas y portables sobre EKS, AKS o GKE.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

## Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                              | Laboratorio   | Horas |
|-----|----------------------------------------------------|---------------|-------|
| 073 | API server, etcd, scheduler, controllers y kubelet | kubernetes    | 4     |
| 074 | Pods, ReplicaSets, Deployments y Jobs              | kubernetes    | 4     |
| 075 | Services, DNS, Ingress y Gateway API               | network       | 4     |
| 076 | ConfigMaps, Secrets y configuración externa        | configuration | 4     |
| 077 | Volumes, PersistentVolumes, CSI y StatefulSets     | storage       | 4     |
| 078 | Requests, limits, scheduling y autoscaling         | capacity      | 4     |
| 079 | Probes, rollouts, rollback y PodDisruptionBudget   | reliability   | 4     |
| 080 | Namespaces, RBAC, NetworkPolicy y admission        | security      | 4     |
| 081 | Helm, Kustomize y gestión de paquetes              | configuration | 4     |
| 082 | Logs, métricas, eventos y depuración               | observability | 4     |
| 083 | EKS, AKS y GKE: similitudes y diferencias          | decision      | 4     |
| 084 | Proyecto: plataforma Kubernetes portable           | capstone      | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..

# 073 — API server, etcd, scheduler, controllers y kubelet

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: kubernetes · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar api server, etcd, scheduler, controllers y kubelet dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar api server, etcd, scheduler, controllers y kubelet con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar mapa-control-plane con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                     |
|-------------|-------------------------------------------------------------------------------------------------------------|
| api         | Define su papel en api server, etcd, scheduler, controllers y kubelet y cómo observarlo en un sistema real. |
| server      | Define su papel en api server, etcd, scheduler, controllers y kubelet y cómo observarlo en un sistema real. |
| etcd        | Define su papel en api server, etcd, scheduler, controllers y kubelet y cómo observarlo en un sistema real. |
| scheduler   | Define su papel en api server, etcd, scheduler, controllers y kubelet y cómo observarlo en un sistema real. |
| controllers | Define su papel en api server, etcd, scheduler, controllers y kubelet y cómo observarlo en un sistema real. |
| kubelet     | Define su papel en api server, etcd, scheduler, controllers y kubelet y cómo observarlo en un sistema real. |

## Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: API server, etcd, scheduler, controllers y kubelet"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar api server, etcd, scheduler, controllers y kubelet. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/073-api-server-etcd-scheduler-controllers-y-kubelet/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa mapa-control-plane en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye mapa-control-plane para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Kubernetes: Up and Running — Burns, Benda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 073 API server, etcd, scheduler, controllers y kubelet

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega mapa-control-plane con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 074 — Pods, ReplicaSets, Deployments y Jobs

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: kubernetes · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar pods, replicasets, deployments y jobs dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar pods, replicasets, deployments y jobs con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar workload-kubernetes con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                        |
|-------------|------------------------------------------------------------------------------------------------|
| pods        | Define su papel en pods, replicasets, deployments y jobs y cómo observarlo en un sistema real. |
| replicasets | Define su papel en pods, replicasets, deployments y jobs y cómo observarlo en un sistema real. |
| deployments | Define su papel en pods, replicasets, deployments y jobs y cómo observarlo en un sistema real. |
| jobs        | Define su papel en pods, replicasets, deployments y jobs y cómo observarlo en un sistema real. |

## Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Pods, ReplicaSets, Deployments y Jobs"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar pods, replicasets, deployments y jobs. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/074-pods-replicasets-deployments-y-jobs/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa workload-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye workload-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 074 Pods, ReplicaSets, Deployments y Jobs

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega workload-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 075 — Services, DNS, Ingress y Gateway API

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar services, dns, ingress y gateway api dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar services, dns, ingress y gateway api con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar exposicion-kubernetes con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                       |
|----------|-----------------------------------------------------------------------------------------------|
| services | Define su papel en services, dns, ingress y gateway api y cómo observarlo en un sistema real. |
| dns      | Define su papel en services, dns, ingress y gateway api y cómo observarlo en un sistema real. |
| ingress  | Define su papel en services, dns, ingress y gateway api y cómo observarlo en un sistema real. |
| gateway  | Define su papel en services, dns, ingress y gateway api y cómo observarlo en un sistema real. |
| api      | Define su papel en services, dns, ingress y gateway api y cómo observarlo en un sistema real. |

## Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Services, DNS, Ingress y Gateway API"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar services, dns, ingress y gateway api. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/075-services-dns-ingress-y-gateway-api/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa exposicion-kubernetes en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `exposicion-kubernetes` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.



- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 075 Services, DNS, Ingress y Gateway API

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega exposicion-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 076 — ConfigMaps, Secrets y configuración externa

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: configuration · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar configmaps, secrets y configuración externa dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar configmaps, secrets y configuración externa con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar configuracion-kubernetes con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                              |
|---------------|------------------------------------------------------------------------------------------------------|
| configmaps    | Define su papel en configmaps, secrets y configuración externa y cómo observarlo en un sistema real. |
| secrets       | Define su papel en configmaps, secrets y configuración externa y cómo observarlo en un sistema real. |
| configuración | Define su papel en configmaps, secrets y configuración externa y cómo observarlo en un sistema real. |
| externa       | Define su papel en configmaps, secrets y configuración externa y cómo observarlo en un sistema real. |

## Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: ConfigMaps, Secrets y configuración externa"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar configmaps, secrets y configuración externa. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/076-configmaps-secrets-y-configuracion-externa/lab.py
```

El laboratorio selecciona el motor de práctica configuration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa configuracion-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es configuración separada, validada y promovible. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye configuración-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 076 ConfigMaps, Secrets y configuración externa

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega configuracion-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 077 — Volumes, PersistentVolumes, CSI y StatefulSets

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: storage · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar volumes, persistentvolumes, csi y statefulsets dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar volumes, persistentvolumes, csi y statefulsets con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar estado-kubernetes con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto          | Comprensión verificable                                                                                 |
|-------------------|---------------------------------------------------------------------------------------------------------|
| volumes           | Define su papel en volumes, persistentvolumes, csi y statefulsets y cómo observarlo en un sistema real. |
| persistentvolumes | Define su papel en volumes, persistentvolumes, csi y statefulsets y cómo observarlo en un sistema real. |
| csi               | Define su papel en volumes, persistentvolumes, csi y statefulsets y cómo observarlo en un sistema real. |
| statefulsets      | Define su papel en volumes, persistentvolumes, csi y statefulsets y cómo observarlo en un sistema real. |

## Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Volumes, PersistentVolumes, CSI y StatefulSets"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar volumes, persistentvolumes, csi y statefulsets. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/077-volumes-persistentvolumes-csi-y-statefulsets/lab.py
```

El laboratorio selecciona el motor de práctica storage y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa estado-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una política de durabilidad, acceso, retención y costo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye estado-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 077 Volumes, PersistentVolumes, CSI y StatefulSets

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega estado-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 078 — Requests, limits, scheduling y autoscaling

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: capacity · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar requests, limits, scheduling y autoscaling dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar requests, limits, scheduling y autoscaling con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar capacidad-kubernetes con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                             |
|-------------|-----------------------------------------------------------------------------------------------------|
| requests    | Define su papel en requests, limits, scheduling y autoscaling y cómo observarlo en un sistema real. |
| limits      | Define su papel en requests, limits, scheduling y autoscaling y cómo observarlo en un sistema real. |
| scheduling  | Define su papel en requests, limits, scheduling y autoscaling y cómo observarlo en un sistema real. |
| autoscaling | Define su papel en requests, limits, scheduling y autoscaling y cómo observarlo en un sistema real. |

### Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Requests, limits, scheduling y autoscaling"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar requests, limits, scheduling y autoscaling. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/078-requests-limits-scheduling-y-autoscaling/lab.py
```

El laboratorio selecciona el motor de práctica capacity y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa capacidad-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es requests, límites y una decisión de escalado medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye capacidad-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 078 Requests, limits, scheduling y autoscaling

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega capacidad-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 079 — Probes, rollouts, rollback y PodDisruptionBudget

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: reliability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar probes, rollouts, rollback y poddisruptionbudget dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar probes, rollouts, rollback y poddisruptionbudget con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar despliegue-resiliente con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto            | Comprensión verificable                                                                                   |
|---------------------|-----------------------------------------------------------------------------------------------------------|
| probes              | Define su papel en probes, rollouts, rollback y poddisruptionbudget y cómo observarlo en un sistema real. |
| rollouts            | Define su papel en probes, rollouts, rollback y poddisruptionbudget y cómo observarlo en un sistema real. |
| rollback            | Define su papel en probes, rollouts, rollback y poddisruptionbudget y cómo observarlo en un sistema real. |
| poddisruptionbudget | Define su papel en probes, rollouts, rollback y poddisruptionbudget y cómo observarlo en un sistema real. |

## Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Probes, rollouts, rollback y PodDisruptionBudget"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar probes, rollouts, rollback y poddisruptionbudget. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/079-probes-rollouts-rollback-y-poddisruptionbudget/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa despliegue-resiliente en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye despliegue-resiliente para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Kubernetes: Up and Running — Burns, Benda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 079 Probes, rollouts, rollback y PodDisruptionBudget

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega despliegue-resiliente con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 080 — Namespaces, RBAC, NetworkPolicy y admission

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar namespaces, rbac, networkpolicy y admission dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar namespaces, rbac, networkpolicy y admission con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar tenant-kubernetes con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto      | Comprensión verificable                                                                              |
|---------------|------------------------------------------------------------------------------------------------------|
| namespaces    | Define su papel en namespaces, rbac, networkpolicy y admission y cómo observarlo en un sistema real. |
| rbac          | Define su papel en namespaces, rbac, networkpolicy y admission y cómo observarlo en un sistema real. |
| networkpolicy | Define su papel en namespaces, rbac, networkpolicy y admission y cómo observarlo en un sistema real. |
| admission     | Define su papel en namespaces, rbac, networkpolicy y admission y cómo observarlo en un sistema real. |

### Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Namespaces, RBAC, NetworkPolicy y admission"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar namespaces, rbac, networkpolicy y admission. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/080-namespaces-rbac-networkpolicy-y-admission/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa tenant-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye tenant-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 080 Namespaces, RBAC, NetworkPolicy y admission

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega tenant-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 081 — Helm, Kustomize y gestión de paquetes

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: configuration · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar helm, kustomize y gestión de paquetes dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar helm, kustomize y gestión de paquetes con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar paquete-kubernetes con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                        |
|-----------|------------------------------------------------------------------------------------------------|
| helm      | Define su papel en helm, kustomize y gestión de paquetes y cómo observarlo en un sistema real. |
| kustomize | Define su papel en helm, kustomize y gestión de paquetes y cómo observarlo en un sistema real. |
| gestión   | Define su papel en helm, kustomize y gestión de paquetes y cómo observarlo en un sistema real. |
| paquetes  | Define su papel en helm, kustomize y gestión de paquetes y cómo observarlo en un sistema real. |

## Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Helm, Kustomize y gestión de paquetes"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar helm, kustomize y gestión de paquetes. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/081-helm-kustomize-y-gestion-de-paquetes/lab.py
```

El laboratorio selecciona el motor de práctica configuration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa paquete-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es configuración separada, validada y promovible. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye paquete-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 081 Helm, Kustomize y gestión de paquetes

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega paquete-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 082 — Logs, métricas, eventos y depuración

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar logs, métricas, eventos y depuración dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar logs, métricas, eventos y depuración con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar diagnostico-kubernetes con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                       |
|------------|-----------------------------------------------------------------------------------------------|
| logs       | Define su papel en logs, métricas, eventos y depuración y cómo observarlo en un sistema real. |
| métricas   | Define su papel en logs, métricas, eventos y depuración y cómo observarlo en un sistema real. |
| eventos    | Define su papel en logs, métricas, eventos y depuración y cómo observarlo en un sistema real. |
| depuración | Define su papel en logs, métricas, eventos y depuración y cómo observarlo en un sistema real. |

### Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Logs, métricas, eventos y depuración"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar logs, métricas, eventos y depuración. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/082-logs-metricas-eventos-y-depuracion/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa diagnostico-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye diagnostico-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 082 Logs, métricas, eventos y depuración

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega diagnóstico-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 083 — EKS, AKS y GKE: similitudes y diferencias

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: decision · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar eks, aks y gke: similitudes y diferencias dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar eks, aks y gke: similitudes y diferencias con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-kubernetes-administrado con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                            |
|-------------|----------------------------------------------------------------------------------------------------|
| eks         | Define su papel en eks, aks y gke: similitudes y diferencias y cómo observarlo en un sistema real. |
| aks         | Define su papel en eks, aks y gke: similitudes y diferencias y cómo observarlo en un sistema real. |
| gke         | Define su papel en eks, aks y gke: similitudes y diferencias y cómo observarlo en un sistema real. |
| similitudes | Define su papel en eks, aks y gke: similitudes y diferencias y cómo observarlo en un sistema real. |
| diferencias | Define su papel en eks, aks y gke: similitudes y diferencias y cómo observarlo en un sistema real. |

### Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: EKS, AKS y GKE: similitudes y diferencias"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar eks, aks y gke: similitudes y diferencias. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/083-eks-aks-y-gke-similitudes-y-diferencias/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-kubernetes-administrado en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye matriz-kubernetes-administrado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.



- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 083 EKS, AKS y GKE: similitudes y diferencias

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega matriz-kubernetes-administrado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 084 — Proyecto: plataforma Kubernetes portable

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar proyecto: plataforma kubernetes portable dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: plataforma kubernetes portable con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-kubernetes con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                           |
|------------|---------------------------------------------------------------------------------------------------|
| proyecto   | Define su papel en proyecto: plataforma kubernetes portable y cómo observarlo en un sistema real. |
| plataforma | Define su papel en proyecto: plataforma kubernetes portable y cómo observarlo en un sistema real. |
| kubernetes | Define su papel en proyecto: plataforma kubernetes portable y cómo observarlo en un sistema real. |
| portable   | Define su papel en proyecto: plataforma kubernetes portable y cómo observarlo en un sistema real. |

### Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: plataforma Kubernetes portable"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: plataforma kubernetes portable. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/084-proyecto-plataforma-kubernetes-portable/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye plataforma-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 084 Proyecto: plataforma Kubernetes portable

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plataforma-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 07 — Infraestructura como código y configuración

← Parte 06 · Índice completo · Parte 08 →

Nivel: intermedio-avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Convertir infraestructura y políticas en cambios revisables, probables y repetibles.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

## Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                | Laboratorio   | Horas |
|-----|------------------------------------------------------|---------------|-------|
| 085 | Declarativo, imperativo, idempotencia y convergencia | iac           | 4     |
| 086 | Terraform: HCL, providers, resources y grafo         | iac           | 4     |
| 087 | Estado remoto, locking, cifrado y recuperación       | iac           | 4     |
| 088 | Módulos, contratos, versiones y composición          | iac           | 4     |
| 089 | Variables, outputs, locals y data sources            | iac           | 4     |
| 090 | Plan, apply, drift, import y refactor con moved      | iac           | 4     |
| 091 | Validación, lint, pruebas y policy as code           | testing       | 4     |
| 092 | Secretos y datos sensibles en IaC                    | security      | 4     |
| 093 | CloudFormation, Bicep, Pulumi y Terraform            | decision      | 4     |
| 094 | Ansible e imagen dorada para configuración           | configuration | 4     |
| 095 | Plantillas, golden paths y catálogo interno          | platform      | 4     |
| 096 | Proyecto: infraestructura multiambiente promovible   | capstone      | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.

# 085 — Declarativo, imperativo, idempotencia y convergencia

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar declarativo, imperativo, idempotencia y convergencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar declarativo, imperativo, idempotencia y convergencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar comparativa-iac con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                       |
|--------------|---------------------------------------------------------------------------------------------------------------|
| declarativo  | Define su papel en declarativo, imperativo, idempotencia y convergencia y cómo observarlo en un sistema real. |
| imperativo   | Define su papel en declarativo, imperativo, idempotencia y convergencia y cómo observarlo en un sistema real. |
| idempotencia | Define su papel en declarativo, imperativo, idempotencia y convergencia y cómo observarlo en un sistema real. |
| convergencia | Define su papel en declarativo, imperativo, idempotencia y convergencia y cómo observarlo en un sistema real. |

## Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Declarativo, imperativo, idempotencia y convergencia"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar declarativo, imperativo, idempotencia y convergencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infraestructure-as-code-configuration/085-declarativo-imperativo-idempotencia-y-convergencia/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa comparativa-iac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye comparativa-iac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 085 Declarativo, imperativo, idempotencia y convergencia

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega comparativa-iac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 086 — Terraform: HCL, providers, resources y grafo

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar terraform: hcl, providers, resources y grafo dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar terraform: hcl, providers, resources y grafo con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar stack-terraform con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                               |
|-----------|-------------------------------------------------------------------------------------------------------|
| terraform | Define su papel en terraform: hcl, providers, resources y grafo y cómo observarlo en un sistema real. |
| hcl       | Define su papel en terraform: hcl, providers, resources y grafo y cómo observarlo en un sistema real. |
| providers | Define su papel en terraform: hcl, providers, resources y grafo y cómo observarlo en un sistema real. |
| resources | Define su papel en terraform: hcl, providers, resources y grafo y cómo observarlo en un sistema real. |
| grafo     | Define su papel en terraform: hcl, providers, resources y grafo y cómo observarlo en un sistema real. |

### Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Terraform: HCL, providers, resources y grafo"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar terraform: hcl, providers, resources y grafo. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infraestructure-as-code-configuration/086-terraform-hcl-providers-resources-y-grafo/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa stack-terraform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye stack-terraform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 086 Terraform: HCL, providers, resources y grafo

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega stack-terraform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 087 — Estado remoto, locking, cifrado y recuperación

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar estado remoto, locking, cifrado y recuperación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar estado remoto, locking, cifrado y recuperación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar backend-terraform con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                 |
|--------------|---------------------------------------------------------------------------------------------------------|
| estado       | Define su papel en estado remoto, locking, cifrado y recuperación y cómo observarlo en un sistema real. |
| remoto       | Define su papel en estado remoto, locking, cifrado y recuperación y cómo observarlo en un sistema real. |
| locking      | Define su papel en estado remoto, locking, cifrado y recuperación y cómo observarlo en un sistema real. |
| cifrado      | Define su papel en estado remoto, locking, cifrado y recuperación y cómo observarlo en un sistema real. |
| recuperación | Define su papel en estado remoto, locking, cifrado y recuperación y cómo observarlo en un sistema real. |

## Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Estado remoto, locking, cifrado y recuperación"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar estado remoto, locking, cifrado y recuperación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infraestructure-as-code-configuration/087-estado-remoto-locking-cifrado-y-recuperacion/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa backend-terraform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye backend-terraform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 087 Estado remoto, locking, cifrado y recuperación

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega backend-terraform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 088 — Módulos, contratos, versiones y composición

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar módulos, contratos, versiones y composición dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar módulos, contratos, versiones y composición con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modulo-terraform con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                              |
|-------------|------------------------------------------------------------------------------------------------------|
| módulos     | Define su papel en módulos, contratos, versiones y composición y cómo observarlo en un sistema real. |
| contratos   | Define su papel en módulos, contratos, versiones y composición y cómo observarlo en un sistema real. |
| versiones   | Define su papel en módulos, contratos, versiones y composición y cómo observarlo en un sistema real. |
| composición | Define su papel en módulos, contratos, versiones y composición y cómo observarlo en un sistema real. |

### Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Módulos, contratos, versiones y composición"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar módulos, contratos, versiones y composición. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/088-modulos-contratos-versiones-y-composicion/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modulo-terraform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye modulo-terraform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 088 Módulos, contratos, versiones y composición

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega modulo-terraform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 089 — Variables, outputs, locals y data sources

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar variables, outputs, locals y data sources dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar variables, outputs, locals y data sources con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar interfaz-infraestructura con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                            |
|-----------|----------------------------------------------------------------------------------------------------|
| variables | Define su papel en variables, outputs, locals y data sources y cómo observarlo en un sistema real. |
| outputs   | Define su papel en variables, outputs, locals y data sources y cómo observarlo en un sistema real. |
| locals    | Define su papel en variables, outputs, locals y data sources y cómo observarlo en un sistema real. |
| data      | Define su papel en variables, outputs, locals y data sources y cómo observarlo en un sistema real. |
| sources   | Define su papel en variables, outputs, locals y data sources y cómo observarlo en un sistema real. |

### Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Variables, outputs, locals y data sources"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar variables, outputs, locals y data sources. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/089-variables-outputs-locals-y-data-sources/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa interfaz-infraestructura en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye interfaz-infraestructura para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 089 Variables, outputs, locals y data sources

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega interfaz-infraestructura con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 090 — Plan, apply, drift, import y refactor con moved

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar plan, apply, drift, import y refactor con moved dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar plan, apply, drift, import y refactor con moved con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar ciclo-cambio-iac con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                                  |
|----------|----------------------------------------------------------------------------------------------------------|
| plan     | Define su papel en plan, apply, drift, import y refactor con moved y cómo observarlo en un sistema real. |
| apply    | Define su papel en plan, apply, drift, import y refactor con moved y cómo observarlo en un sistema real. |
| drift    | Define su papel en plan, apply, drift, import y refactor con moved y cómo observarlo en un sistema real. |
| import   | Define su papel en plan, apply, drift, import y refactor con moved y cómo observarlo en un sistema real. |
| refactor | Define su papel en plan, apply, drift, import y refactor con moved y cómo observarlo en un sistema real. |
| moved    | Define su papel en plan, apply, drift, import y refactor con moved y cómo observarlo en un sistema real. |

## Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Plan, apply, drift, import y refactor con moved"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar plan, apply, drift, import y refactor con moved. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/090-plan-apply-drift-import-y-refactor-con-moved/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa ciclo-cambio-iac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye ciclo-cambio-iac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 090 Plan, apply, drift, import y refactor con moved

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega ciclo-cambio-iac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 091 — Validación, lint, pruebas y policy as code

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: testing · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar validación, lint, pruebas y policy as code dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar validación, lint, pruebas y policy as code con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar pipeline-iac con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                             |
|------------|-----------------------------------------------------------------------------------------------------|
| validación | Define su papel en validación, lint, pruebas y policy as code y cómo observarlo en un sistema real. |
| lint       | Define su papel en validación, lint, pruebas y policy as code y cómo observarlo en un sistema real. |
| pruebas    | Define su papel en validación, lint, pruebas y policy as code y cómo observarlo en un sistema real. |
| policy     | Define su papel en validación, lint, pruebas y policy as code y cómo observarlo en un sistema real. |
| as         | Define su papel en validación, lint, pruebas y policy as code y cómo observarlo en un sistema real. |
| code       | Define su papel en validación, lint, pruebas y policy as code y cómo observarlo en un sistema real. |

## Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Validación, lint, pruebas y policy as code"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar validación, lint, pruebas y policy as code. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/091-validacion-lint-pruebas-y-policy-as-code/lab.py
```

El laboratorio selecciona el motor de práctica testing y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa pipeline-iac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es pruebas automatizadas con fallos diagnósticos. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye pipeline-iac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 091 Validación, lint, pruebas y policy as code

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega pipeline-iac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 092 — Secretos y datos sensibles en IaC

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar secretos y datos sensibles en iac dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar secretos y datos sensibles en iac con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-secretos-iac con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                    |
|-----------|--------------------------------------------------------------------------------------------|
| secretos  | Define su papel en secretos y datos sensibles en iac y cómo observarlo en un sistema real. |
| datos     | Define su papel en secretos y datos sensibles en iac y cómo observarlo en un sistema real. |
| sensibles | Define su papel en secretos y datos sensibles en iac y cómo observarlo en un sistema real. |
| iac       | Define su papel en secretos y datos sensibles en iac y cómo observarlo en un sistema real. |

### Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Secretos y datos sensibles en IaC"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar secretos y datos sensibles en iac. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/092-secretos-y-datos-sensibles-en-iac/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modelo-secretos-iac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye modelo-secretos-iac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 092 Secretos y datos sensibles en IaC

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega modelo-secretos-iac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 093 — CloudFormation, Bicep, Pulumi y Terraform

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: decision · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar cloudformation, bicep, pulumi y terraform dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloudformation, bicep, pulumi y terraform con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar adr-herramienta-iac con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto       | Comprensión verificable                                                                            |
|----------------|----------------------------------------------------------------------------------------------------|
| cloudformation | Define su papel en cloudformation, bicep, pulumi y terraform y cómo observarlo en un sistema real. |
| bicep          | Define su papel en cloudformation, bicep, pulumi y terraform y cómo observarlo en un sistema real. |
| pulumi         | Define su papel en cloudformation, bicep, pulumi y terraform y cómo observarlo en un sistema real. |
| terraform      | Define su papel en cloudformation, bicep, pulumi y terraform y cómo observarlo en un sistema real. |

### Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: CloudFormation, Bicep, Pulumi y Terraform"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloudformation, bicep, pulumi y terraform. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/093-cloudformation-bicep-pulumi-y-terraform/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa adr-herramienta-iac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye adr-herramienta-iac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 093 CloudFormation, Bicep, Pulumi y Terraform

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega adr-herramienta-iac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 094 — Ansible e imagen dorada para configuración

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: configuration · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar ansible e imagen dorada para configuración dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ansible e imagen dorada para configuración con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar configuracion-servidor con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto      | Comprensión verificable                                                                             |
|---------------|-----------------------------------------------------------------------------------------------------|
| ansible       | Define su papel en ansible e imagen dorada para configuración y cómo observarlo en un sistema real. |
| e             | Define su papel en ansible e imagen dorada para configuración y cómo observarlo en un sistema real. |
| imagen        | Define su papel en ansible e imagen dorada para configuración y cómo observarlo en un sistema real. |
| dorada        | Define su papel en ansible e imagen dorada para configuración y cómo observarlo en un sistema real. |
| configuración | Define su papel en ansible e imagen dorada para configuración y cómo observarlo en un sistema real. |

### Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Ansible e imagen dorada para configuración"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ansible e imagen dorada para configuración. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/094-ansible-e-imagen-dorada-para-configuracion/lab.py
```

El laboratorio selecciona el motor de práctica configuration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa configuracion-servidor en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es configuración separada, validada y promovible. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye configuracion-servidor para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 094 Ansible e imagen dorada para configuración

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega configuracion-servidor con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 095 — Plantillas, golden paths y catálogo interno

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: platform · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar plantillas, golden paths y catálogo interno dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar plantillas, golden paths y catálogo interno con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plantilla-plataforma con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                              |
|------------|------------------------------------------------------------------------------------------------------|
| plantillas | Define su papel en plantillas, golden paths y catálogo interno y cómo observarlo en un sistema real. |
| golden     | Define su papel en plantillas, golden paths y catálogo interno y cómo observarlo en un sistema real. |
| paths      | Define su papel en plantillas, golden paths y catálogo interno y cómo observarlo en un sistema real. |
| catálogo   | Define su papel en plantillas, golden paths y catálogo interno y cómo observarlo en un sistema real. |
| interno    | Define su papel en plantillas, golden paths y catálogo interno y cómo observarlo en un sistema real. |

### Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Plantillas, golden paths y catálogo interno"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar plantillas, golden paths y catálogo interno. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/095-plantillas-golden-paths-y-catalogo-interno/lab.py
```

El laboratorio selecciona el motor de práctica platform y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa plantilla-plataforma en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una capacidad autoservicio con contrato y golden path. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye plantilla-plataforma para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 095 Plantillas, golden paths y catálogo interno

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plantilla-plataforma con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 096 — Proyecto: infraestructura multiambiente promovible

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar proyecto: infraestructura multiambiente promovible dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: infraestructura multiambiente promovible con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-iac con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto        | Comprensión verificable                                                                                     |
|-----------------|-------------------------------------------------------------------------------------------------------------|
| proyecto        | Define su papel en proyecto: infraestructura multiambiente promovible y cómo observarlo en un sistema real. |
| infraestructura | Define su papel en proyecto: infraestructura multiambiente promovible y cómo observarlo en un sistema real. |
| multiambiente   | Define su papel en proyecto: infraestructura multiambiente promovible y cómo observarlo en un sistema real. |
| promovible      | Define su papel en proyecto: infraestructura multiambiente promovible y cómo observarlo en un sistema real. |

## Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: infraestructura multiambiente promovible"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: infraestructura multiambiente promovible. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/096-proyecto-infraestructura-multiambiente-promovible/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-iac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye plataforma-iac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 096 Proyecto: infraestructura multiambiente promovible

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plataforma-iac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 08 — Entrega continua y platform engineering

← Parte 07 · Índice completo · Parte 09 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar un flujo de entrega rápido, seguro y observable para equipos de producto.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

## Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                    | Laboratorio  | Horas |
|-----|----------------------------------------------------------|--------------|-------|
| 097 | Integración continua, trunk-based development y feedback | delivery     | 4     |
| 098 | GitHub Actions: workflows, runners, permisos y caché     | delivery     | 4     |
| 099 | Artefactos inmutables, semver y promoción                | supply-chain | 4     |
| 100 | Pruebas, calidad y puertas de cambio                     | testing      | 4     |
| 101 | SAST, SCA, secretos, SBOM y firma en pipeline            | security     | 4     |
| 102 | Rolling, blue-green, canary y rollback                   | delivery     | 4     |
| 103 | GitOps con Argo CD o Flux                                | gitops       | 4     |
| 104 | Ambientes efímeros y promoción entre entornos            | delivery     | 4     |
| 105 | Feature flags y separación deploy-release                | delivery     | 4     |
| 106 | Platform engineering e Internal Developer Platform       | platform     | 4     |
| 107 | Developer experience, DORA y carga cognitiva             | metrics      | 4     |
| 108 | Proyecto: fábrica de software multi-cloud                | capstone     | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.



Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.

# 097 — Integración continua, trunk-based development y feedback

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: delivery · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar integración continua, trunk-based development y feedback dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar integración continua, trunk-based development y feedback con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar flujo-ci con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                           |
|-------------|-------------------------------------------------------------------------------------------------------------------|
| integración | Define su papel en integración continua, trunk-based development y feedback y cómo observarlo en un sistema real. |
| continua    | Define su papel en integración continua, trunk-based development y feedback y cómo observarlo en un sistema real. |
| trunk       | Define su papel en integración continua, trunk-based development y feedback y cómo observarlo en un sistema real. |
| based       | Define su papel en integración continua, trunk-based development y feedback y cómo observarlo en un sistema real. |
| development | Define su papel en integración continua, trunk-based development y feedback y cómo observarlo en un sistema real. |
| feedback    | Define su papel en integración continua, trunk-based development y feedback y cómo observarlo en un sistema real. |

## Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Integración continua, trunk-based development y feedback"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar integración continua, trunk-based development y feedback. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/097-integracion-continua-trunk-based-
```

development-y-feedback/lab.py

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa flujo-ci en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye flujo-ci para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 097 Integración continua, trunk-based development y feedback

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega flujo-ci con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 098 — GitHub Actions: workflows, runners, permisos y caché

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: delivery · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar github actions: workflows, runners, permisos y caché dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar github actions: workflows, runners, permisos y caché con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar pipeline-github-actions con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                                       |
|-----------|---------------------------------------------------------------------------------------------------------------|
| github    | Define su papel en github actions: workflows, runners, permisos y caché y cómo observarlo en un sistema real. |
| actions   | Define su papel en github actions: workflows, runners, permisos y caché y cómo observarlo en un sistema real. |
| workflows | Define su papel en github actions: workflows, runners, permisos y caché y cómo observarlo en un sistema real. |
| runners   | Define su papel en github actions: workflows, runners, permisos y caché y cómo observarlo en un sistema real. |
| permisos  | Define su papel en github actions: workflows, runners, permisos y caché y cómo observarlo en un sistema real. |
| caché     | Define su papel en github actions: workflows, runners, permisos y caché y cómo observarlo en un sistema real. |

## Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: GitHub Actions: workflows, runners, permisos y caché"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar github actions: workflows, runners, permisos y caché. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/098-github-actions-workflows-runners-permisos-y-cache/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa pipeline-github-actions en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye pipeline-github-actions para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 098 GitHub Actions: workflows, runners, permisos y caché

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega pipeline-github-actions con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 099 — Artefactos inmutables, semver y promoción

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: supply-chain · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar artefactos inmutables, semver y promoción dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar artefactos inmutables, semver y promoción con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar registro-artefactos con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                            |
|------------|----------------------------------------------------------------------------------------------------|
| artefactos | Define su papel en artefactos inmutables, semver y promoción y cómo observarlo en un sistema real. |
| inmutables | Define su papel en artefactos inmutables, semver y promoción y cómo observarlo en un sistema real. |
| semver     | Define su papel en artefactos inmutables, semver y promoción y cómo observarlo en un sistema real. |
| promoción  | Define su papel en artefactos inmutables, semver y promoción y cómo observarlo en un sistema real. |

### Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Artefactos inmutables, semver y promoción"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar artefactos inmutables, semver y promoción. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/099-artefactos-inmutables-semver-y-promocion/lab.py
```

El laboratorio selecciona el motor de práctica supply-chain y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa registro-artefactos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es procedencia, inventario y verificación del artefacto. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye registro-artefactos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 099 Artefactos inmutables, semver y promoción

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega registro-artefactos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 100 — Pruebas, calidad y puertas de cambio

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: testing · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar pruebas, calidad y puertas de cambio dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar pruebas, calidad y puertas de cambio con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar quality-gates con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                       |
|----------|-----------------------------------------------------------------------------------------------|
| pruebas  | Define su papel en pruebas, calidad y puertas de cambio y cómo observarlo en un sistema real. |
| calidad  | Define su papel en pruebas, calidad y puertas de cambio y cómo observarlo en un sistema real. |
| puertas  | Define su papel en pruebas, calidad y puertas de cambio y cómo observarlo en un sistema real. |
| cambio   | Define su papel en pruebas, calidad y puertas de cambio y cómo observarlo en un sistema real. |

## Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Pruebas, calidad y puertas de cambio"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar pruebas, calidad y puertas de cambio. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/100-pruebas-calidad-y-puertas-de-cambio/lab.py
```

El laboratorio selecciona el motor de práctica testing y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa quality-gates en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es pruebas automatizadas con fallos diagnósticos. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye quality-gates para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 100 Pruebas, calidad y puertas de cambio

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega quality-gates con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 101 — SAST, SCA, secretos, SBOM y firma en pipeline

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar sast, sca, secretos, sbom y firma en pipeline dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sast, sca, secretos, sbom y firma en pipeline con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar pipeline-devsecops con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                                |
|----------|--------------------------------------------------------------------------------------------------------|
| sast     | Define su papel en sast, sca, secretos, sbom y firma en pipeline y cómo observarlo en un sistema real. |
| sca      | Define su papel en sast, sca, secretos, sbom y firma en pipeline y cómo observarlo en un sistema real. |
| secretos | Define su papel en sast, sca, secretos, sbom y firma en pipeline y cómo observarlo en un sistema real. |
| sbom     | Define su papel en sast, sca, secretos, sbom y firma en pipeline y cómo observarlo en un sistema real. |
| firma    | Define su papel en sast, sca, secretos, sbom y firma en pipeline y cómo observarlo en un sistema real. |
| pipeline | Define su papel en sast, sca, secretos, sbom y firma en pipeline y cómo observarlo en un sistema real. |

## Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: SAST, SCA, secretos, SBOM y firma en pipeline"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar sast, sca, secretos, sbom y firma en pipeline. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/101-sast-sca-secretos-sbom-y-firma-en-pipeline/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa pipeline-devsecops en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye pipeline-devsecops para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 101 SAST, SCA, secretos, SBOM y firma en pipeline

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega pipeline-devsecops con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 102 — Rolling, blue-green, canary y rollback

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: delivery · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar rolling, blue-green, canary y rollback dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar rolling, blue-green, canary y rollback con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar estrategia-despliegue con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                         |
|----------|-------------------------------------------------------------------------------------------------|
| rolling  | Define su papel en rolling, blue-green, canary y rollback y cómo observarlo en un sistema real. |
| blue     | Define su papel en rolling, blue-green, canary y rollback y cómo observarlo en un sistema real. |
| green    | Define su papel en rolling, blue-green, canary y rollback y cómo observarlo en un sistema real. |
| canary   | Define su papel en rolling, blue-green, canary y rollback y cómo observarlo en un sistema real. |
| rollback | Define su papel en rolling, blue-green, canary y rollback y cómo observarlo en un sistema real. |

## Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Rolling, blue-green, canary y rollback"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar rolling, blue-green, canary y rollback. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/102-rolling-blue-green-canary-y-rollback/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa estrategia-despliegue en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye estrategia-despliegue para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 102 Rolling, blue-green, canary y rollback

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega estrategia-despliegue con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 103 — GitOps con Argo CD o Flux

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: gitops · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar gitops con argo cd o flux dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar gitops con argo cd o flux con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar reconciliacion-gitops con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                            |
|----------|------------------------------------------------------------------------------------|
| gitops   | Define su papel en gitops con argo cd o flux y cómo observarlo en un sistema real. |
| argo     | Define su papel en gitops con argo cd o flux y cómo observarlo en un sistema real. |
| cd       | Define su papel en gitops con argo cd o flux y cómo observarlo en un sistema real. |
| o        | Define su papel en gitops con argo cd o flux y cómo observarlo en un sistema real. |
| flux     | Define su papel en gitops con argo cd o flux y cómo observarlo en un sistema real. |

### Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: GitOps con Argo CD o Flux"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar gitops con argo cd o flux. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/103-gitops-con-argo-cd-o-flux/lab.py
```

El laboratorio selecciona el motor de práctica gitops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa reconciliacion-gitops en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es reconciliación declarativa y evidencia de drift. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye reconciliación-gitops para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 103 GitOps con Argo CD o Flux

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega reconciliación-gitops con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 104 — Ambientes efímeros y promoción entre entornos

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: delivery · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar ambientes efímeros y promoción entre entornos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ambientes efímeros y promoción entre entornos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar flujo-ambientes con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                                |
|-----------|--------------------------------------------------------------------------------------------------------|
| ambientes | Define su papel en ambientes efímeros y promoción entre entornos y cómo observarlo en un sistema real. |
| efímeros  | Define su papel en ambientes efímeros y promoción entre entornos y cómo observarlo en un sistema real. |
| promoción | Define su papel en ambientes efímeros y promoción entre entornos y cómo observarlo en un sistema real. |
| entre     | Define su papel en ambientes efímeros y promoción entre entornos y cómo observarlo en un sistema real. |
| entornos  | Define su papel en ambientes efímeros y promoción entre entornos y cómo observarlo en un sistema real. |

## Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Ambientes efímeros y promoción entre entornos"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ambientes efímeros y promoción entre entornos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/104-ambientes-efimeros-y-promocion-entre-entornos/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

- 4. Materializa flujo-ambientes en evidence/ usando la plantilla indicada.
- 5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye flujo-ambientes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 104 Ambientes efímeros y promoción entre entornos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega flujo-ambientes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 105 — Feature flags y separación deploy-release

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: delivery · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar feature flags y separación deploy-release dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar feature flags y separación deploy-release con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plan-feature-flags con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                            |
|------------|----------------------------------------------------------------------------------------------------|
| feature    | Define su papel en feature flags y separación deploy-release y cómo observarlo en un sistema real. |
| flags      | Define su papel en feature flags y separación deploy-release y cómo observarlo en un sistema real. |
| separación | Define su papel en feature flags y separación deploy-release y cómo observarlo en un sistema real. |
| deploy     | Define su papel en feature flags y separación deploy-release y cómo observarlo en un sistema real. |
| release    | Define su papel en feature flags y separación deploy-release y cómo observarlo en un sistema real. |

## Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Feature flags y separación deploy-release"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar feature flags y separación deploy-release. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/105-feature-flags-y-separacion-deploy-release/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa plan-feature-flags en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye plan-feature-flags para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 105 Feature flags y separación deploy-release

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plan-feature-flags con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 106 — Platform engineering e Internal Developer Platform

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: platform · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar platform engineering e internal developer platform dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar platform engineering e internal developer platform con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar mapa-capacidades-plataforma con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                     |
|-------------|-------------------------------------------------------------------------------------------------------------|
| platform    | Define su papel en platform engineering e internal developer platform y cómo observarlo en un sistema real. |
| engineering | Define su papel en platform engineering e internal developer platform y cómo observarlo en un sistema real. |
| e           | Define su papel en platform engineering e internal developer platform y cómo observarlo en un sistema real. |
| internal    | Define su papel en platform engineering e internal developer platform y cómo observarlo en un sistema real. |
| developer   | Define su papel en platform engineering e internal developer platform y cómo observarlo en un sistema real. |
| platform    | Define su papel en platform engineering e internal developer platform y cómo observarlo en un sistema real. |

## Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Platform engineering e Internal Developer Platform"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar platform engineering e internal developer platform. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/106-platform-engineering-e-internal-developer-platform/lab.py
```

El laboratorio selecciona el motor de práctica platform y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa mapa-capacidades-plataforma en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es una capacidad autoservicio con contrato y golden path. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye mapa-capacidades-plataforma para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 106 Platform engineering e Internal Developer Platform

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega mapa-capacidades-plataforma con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 107 — Developer experience, DORA y carga cognitiva

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: metrics · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar developer experience, dora y carga cognitiva dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar developer experience, dora y carga cognitiva con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar tablero-dora con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                               |
|------------|-------------------------------------------------------------------------------------------------------|
| developer  | Define su papel en developer experience, dora y carga cognitiva y cómo observarlo en un sistema real. |
| experience | Define su papel en developer experience, dora y carga cognitiva y cómo observarlo en un sistema real. |
| dora       | Define su papel en developer experience, dora y carga cognitiva y cómo observarlo en un sistema real. |
| carga      | Define su papel en developer experience, dora y carga cognitiva y cómo observarlo en un sistema real. |
| cognitiva  | Define su papel en developer experience, dora y carga cognitiva y cómo observarlo en un sistema real. |

## Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Developer experience, DORA y carga cognitiva"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar developer experience, dora y carga cognitiva. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/107-developer-experience-dora-y-carga-cognitiva/lab.py
```

El laboratorio selecciona el motor de práctica metrics y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa tablero-dora en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es métricas definidas, consultables y vinculadas a una decisión. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye tablero-dora para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 107 Developer experience, DORA y carga cognitiva

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega tablero-dora con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 108 — Proyecto: fábrica de software multi-cloud

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar proyecto: fábrica de software multi-cloud dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: fábrica de software multi-cloud con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-entrega con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                            |
|----------|----------------------------------------------------------------------------------------------------|
| proyecto | Define su papel en proyecto: fábrica de software multi-cloud y cómo observarlo en un sistema real. |
| fábrica  | Define su papel en proyecto: fábrica de software multi-cloud y cómo observarlo en un sistema real. |
| software | Define su papel en proyecto: fábrica de software multi-cloud y cómo observarlo en un sistema real. |
| multi    | Define su papel en proyecto: fábrica de software multi-cloud y cómo observarlo en un sistema real. |
| cloud    | Define su papel en proyecto: fábrica de software multi-cloud y cómo observarlo en un sistema real. |

## Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Proyecto: fábrica de software multi-cloud"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: fábrica de software multi-cloud. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/108-proyecto-fabrica-de-software-multi-cloud/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa plataforma-entrega en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye plataforma-entrega para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 108 Proyecto: fábrica de software multi-cloud

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plataforma-entrega con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 09 — Datos, mensajería, serverless e integración

← Parte 08 · Índice completo · Parte 10 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Elegir servicios de datos e integración por sus garantías y patrones de acceso.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

## Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                             | Laboratorio   | Horas |
|-----|---------------------------------------------------|---------------|-------|
| 109 | Bases relacionales administradas y pooling        | data          | 4     |
| 110 | NoSQL: clave-valor, documento, columna y grafo    | data          | 4     |
| 111 | Caché, invalidación, TTL y consistencia           | data          | 4     |
| 112 | Object storage, data lake y formatos columnares   | data          | 4     |
| 113 | Colas, entrega, reintentos y dead-letter queues   | messaging     | 4     |
| 114 | Pub/sub, streams, particiones y orden             | messaging     | 4     |
| 115 | Arquitectura dirigida por eventos y contratos     | architecture  | 4     |
| 116 | Sagas, outbox, idempotencia y deduplicación       | distributed   | 4     |
| 117 | Serverless: límites, cold starts y concurrencia   | serverless    | 4     |
| 118 | API management, cuotas, versiones y monetización  | api           | 4     |
| 119 | Workflows y orquestación durable                  | orchestration | 4     |
| 120 | Proyecto: pipeline de pedidos orientado a eventos | capstone      | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.

# 109 — Bases relacionales administradas y pooling

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar bases relacionales administradas y pooling dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar bases relacionales administradas y pooling con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-relacional con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                             |
|---------------|-----------------------------------------------------------------------------------------------------|
| bases         | Define su papel en bases relacionales administradas y pooling y cómo observarlo en un sistema real. |
| relacionales  | Define su papel en bases relacionales administradas y pooling y cómo observarlo en un sistema real. |
| administradas | Define su papel en bases relacionales administradas y pooling y cómo observarlo en un sistema real. |
| pooling       | Define su papel en bases relacionales administradas y pooling y cómo observarlo en un sistema real. |

## Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Bases relacionales administradas y pooling"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar bases relacionales administradas y pooling. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/109-bases-relacionales-administradas-y-pooling/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modelo-relacional en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye modelo-relacional para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 109 Bases relacionales administradas y pooling

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega modelo-relacional con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 110 — NoSQL: clave-valor, documento, columna y grafo

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar nosql: clave-valor, documento, columna y grafo dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar nosql: clave-valor, documento, columna y grafo con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-nosql con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                                 |
|-----------|---------------------------------------------------------------------------------------------------------|
| nosql     | Define su papel en nosql: clave-valor, documento, columna y grafo y cómo observarlo en un sistema real. |
| clave     | Define su papel en nosql: clave-valor, documento, columna y grafo y cómo observarlo en un sistema real. |
| valor     | Define su papel en nosql: clave-valor, documento, columna y grafo y cómo observarlo en un sistema real. |
| documento | Define su papel en nosql: clave-valor, documento, columna y grafo y cómo observarlo en un sistema real. |
| columna   | Define su papel en nosql: clave-valor, documento, columna y grafo y cómo observarlo en un sistema real. |
| grafo     | Define su papel en nosql: clave-valor, documento, columna y grafo y cómo observarlo en un sistema real. |

## Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: NoSQL: clave-valor, documento, columna y grafo"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar nosql: clave-valor, documento, columna y grafo. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/110-nosql-clave-valor-documento-columna-y-grafo/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa matriz-nosql en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye matriz-nosql para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 110 NoSQL: clave-valor, documento, columna y grafo

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega matriz-nosql con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 111 — Caché, invalidación, TTL y consistencia

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar caché, invalidación, ttl y consistencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar caché, invalidación, ttl y consistencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar estrategia-cache con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                          |
|--------------|--------------------------------------------------------------------------------------------------|
| caché        | Define su papel en caché, invalidación, ttl y consistencia y cómo observarlo en un sistema real. |
| invalidación | Define su papel en caché, invalidación, ttl y consistencia y cómo observarlo en un sistema real. |
| ttl          | Define su papel en caché, invalidación, ttl y consistencia y cómo observarlo en un sistema real. |
| consistencia | Define su papel en caché, invalidación, ttl y consistencia y cómo observarlo en un sistema real. |

## Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Caché, invalidación, TTL y consistencia"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar caché, invalidación, ttl y consistencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/111-cache-invalidacion-ttl-y-consistencia/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa estrategia-cache en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye estrategia-cache para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 111 Caché, invalidación, TTL y consistencia

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega estrategia-cache con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 112 — Object storage, data lake y formatos columnares

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar object storage, data lake y formatos columnares dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar object storage, data lake y formatos columnares con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar diseno-data-lake con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                  |
|------------|----------------------------------------------------------------------------------------------------------|
| object     | Define su papel en object storage, data lake y formatos columnares y cómo observarlo en un sistema real. |
| storage    | Define su papel en object storage, data lake y formatos columnares y cómo observarlo en un sistema real. |
| data       | Define su papel en object storage, data lake y formatos columnares y cómo observarlo en un sistema real. |
| lake       | Define su papel en object storage, data lake y formatos columnares y cómo observarlo en un sistema real. |
| formatos   | Define su papel en object storage, data lake y formatos columnares y cómo observarlo en un sistema real. |
| columnares | Define su papel en object storage, data lake y formatos columnares y cómo observarlo en un sistema real. |

## Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Object storage, data lake y formatos columnares"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar object storage, data lake y formatos columnares. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/112-object-storage-data-lake-y-formatos-columnares/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa diseno-data-lake en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye diseno-data-lake para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 112 Object storage, data lake y formatos columnares

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega diseño-data-lake con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 113 — Colas, entrega, reintentos y dead-letter queues

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: messaging · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar colas, entrega, reintentos y dead-letter queues dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar colas, entrega, reintentos y dead-letter queues con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cola-resiliente con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                  |
|------------|----------------------------------------------------------------------------------------------------------|
| colas      | Define su papel en colas, entrega, reintentos y dead-letter queues y cómo observarlo en un sistema real. |
| entrega    | Define su papel en colas, entrega, reintentos y dead-letter queues y cómo observarlo en un sistema real. |
| reintentos | Define su papel en colas, entrega, reintentos y dead-letter queues y cómo observarlo en un sistema real. |
| dead       | Define su papel en colas, entrega, reintentos y dead-letter queues y cómo observarlo en un sistema real. |
| letter     | Define su papel en colas, entrega, reintentos y dead-letter queues y cómo observarlo en un sistema real. |
| queues     | Define su papel en colas, entrega, reintentos y dead-letter queues y cómo observarlo en un sistema real. |

## Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Colas, entrega, reintentos y dead-letter queues"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar colas, entrega, reintentos y dead-letter queues. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/113-colas-entrega-reintentos-y-dead-letter-queues/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa cola-resiliente en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye cola-resiliente para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 113 Colas, entrega, reintentos y dead-letter queues

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega cola-resiliente con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 114 — Pub/sub, streams, particiones y orden

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: messaging · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar pub/sub, streams, particiones y orden dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar pub/sub, streams, particiones y orden con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar stream-particionado con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                        |
|-------------|------------------------------------------------------------------------------------------------|
| pub         | Define su papel en pub/sub, streams, particiones y orden y cómo observarlo en un sistema real. |
| sub         | Define su papel en pub/sub, streams, particiones y orden y cómo observarlo en un sistema real. |
| streams     | Define su papel en pub/sub, streams, particiones y orden y cómo observarlo en un sistema real. |
| particiones | Define su papel en pub/sub, streams, particiones y orden y cómo observarlo en un sistema real. |
| orden       | Define su papel en pub/sub, streams, particiones y orden y cómo observarlo en un sistema real. |

## Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Pub/sub, streams, particiones y orden"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar pub/sub, streams, particiones y orden. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/114-pub-sub-streams-particiones-y-orden/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa stream-particionado en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye stream-particionado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 114 Pub/sub, streams, particiones y orden

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega stream-particionado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 115 — Arquitectura dirigida por eventos y contratos

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar arquitectura dirigida por eventos y contratos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar arquitectura dirigida por eventos y contratos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar catalogo-eventos con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                |
|--------------|--------------------------------------------------------------------------------------------------------|
| arquitectura | Define su papel en arquitectura dirigida por eventos y contratos y cómo observarlo en un sistema real. |
| dirigida     | Define su papel en arquitectura dirigida por eventos y contratos y cómo observarlo en un sistema real. |
| eventos      | Define su papel en arquitectura dirigida por eventos y contratos y cómo observarlo en un sistema real. |
| contratos    | Define su papel en arquitectura dirigida por eventos y contratos y cómo observarlo en un sistema real. |

## Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Arquitectura dirigida por eventos y contratos"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar arquitectura dirigida por eventos y contratos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/115-arquitectura-dirigida-por-eventos-y-contratos/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa catalogo-eventos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye catalogo-eventos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 115 Arquitectura dirigida por eventos y contratos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega catalogo-eventos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 116 — Sagas, outbox, idempotencia y deduplicación

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: distributed · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar sagas, outbox, idempotencia y deduplicación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sagas, outbox, idempotencia y deduplicación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar transaccion-distribuida con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                              |
|---------------|------------------------------------------------------------------------------------------------------|
| sagas         | Define su papel en sagas, outbox, idempotencia y deduplicación y cómo observarlo en un sistema real. |
| outbox        | Define su papel en sagas, outbox, idempotencia y deduplicación y cómo observarlo en un sistema real. |
| idempotencia  | Define su papel en sagas, outbox, idempotencia y deduplicación y cómo observarlo en un sistema real. |
| deduplicación | Define su papel en sagas, outbox, idempotencia y deduplicación y cómo observarlo en un sistema real. |

## Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Sagas, outbox, idempotencia y deduplicación"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar sagas, outbox, idempotencia y deduplicación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/116-sagas-outbox-idempotencia-y-deduplicacion/lab.py
```

El laboratorio selecciona el motor de práctica distributed y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa transaccion-distribuida en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una traza de consistencia, reintento o fallo parcial. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye transaccion-distribuida para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 116 Sagas, outbox, idempotencia y deduplicación

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega transaccion-distribuida con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 117 — Serverless: límites, cold starts y concurrencia

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: serverless · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar serverless: límites, cold starts y concurrencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar serverless: límites, cold starts y concurrencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar funcion-operable con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                  |
|--------------|----------------------------------------------------------------------------------------------------------|
| serverless   | Define su papel en serverless: límites, cold starts y concurrencia y cómo observarlo en un sistema real. |
| límites      | Define su papel en serverless: límites, cold starts y concurrencia y cómo observarlo en un sistema real. |
| cold         | Define su papel en serverless: límites, cold starts y concurrencia y cómo observarlo en un sistema real. |
| starts       | Define su papel en serverless: límites, cold starts y concurrencia y cómo observarlo en un sistema real. |
| concurrencia | Define su papel en serverless: límites, cold starts y concurrencia y cómo observarlo en un sistema real. |

## Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Serverless: límites, cold starts y concurrencia"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar serverless: límites, cold starts y concurrencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/117-serverless-limit-cold-starts-y-concurrencia/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa funcion-operable en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye funcion-operable para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 117 Serverless: límites, cold starts y concurrencia

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega funcion-operable con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 118 — API management, cuotas, versiones y monetización

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: api · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar api management, cuotas, versiones y monetización dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar api management, cuotas, versiones y monetización con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar producto-api con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                   |
|--------------|-----------------------------------------------------------------------------------------------------------|
| api          | Define su papel en api management, cuotas, versiones y monetización y cómo observarlo en un sistema real. |
| management   | Define su papel en api management, cuotas, versiones y monetización y cómo observarlo en un sistema real. |
| cuotas       | Define su papel en api management, cuotas, versiones y monetización y cómo observarlo en un sistema real. |
| versiones    | Define su papel en api management, cuotas, versiones y monetización y cómo observarlo en un sistema real. |
| monetización | Define su papel en api management, cuotas, versiones y monetización y cómo observarlo en un sistema real. |

## Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: API management, cuotas, versiones y monetización"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar api management, cuotas, versiones y monetización. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/118-api-management-cuotas-versiones-y-monetizacion/lab.py
```

El laboratorio selecciona el motor de práctica api y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

- 4. Materializa producto-api en evidence/ usando la plantilla indicada.
- 5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es un contrato versionado con pruebas positivas y negativas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye producto-api para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 118 API management, cuotas, versiones y monetización

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega producto-api con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 119 — Workflows y orquestación durable

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: orchestration · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar workflows y orquestación durable dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar workflows y orquestación durable con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar workflow-durable con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                   |
|--------------|-------------------------------------------------------------------------------------------|
| workflows    | Define su papel en workflows y orquestación durable y cómo observarlo en un sistema real. |
| orquestación | Define su papel en workflows y orquestación durable y cómo observarlo en un sistema real. |
| durable      | Define su papel en workflows y orquestación durable y cómo observarlo en un sistema real. |

## Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Workflows y orquestación durable"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar workflows y orquestación durable. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/119-workflows-y-orquestacion-durable/lab.py
```

El laboratorio selecciona el motor de práctica orchestration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa workflow-durable en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es servicios coordinados con health checks y apagado limpio. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye workflow-durable para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 119 Workflows y orquestación durable

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega workflow-durable con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 120 — Proyecto: pipeline de pedidos orientado a eventos

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar proyecto: pipeline de pedidos orientado a eventos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: pipeline de pedidos orientado a eventos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-eventos con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                                    |
|-----------|------------------------------------------------------------------------------------------------------------|
| proyecto  | Define su papel en proyecto: pipeline de pedidos orientado a eventos y cómo observarlo en un sistema real. |
| pipeline  | Define su papel en proyecto: pipeline de pedidos orientado a eventos y cómo observarlo en un sistema real. |
| pedidos   | Define su papel en proyecto: pipeline de pedidos orientado a eventos y cómo observarlo en un sistema real. |
| orientado | Define su papel en proyecto: pipeline de pedidos orientado a eventos y cómo observarlo en un sistema real. |
| eventos   | Define su papel en proyecto: pipeline de pedidos orientado a eventos y cómo observarlo en un sistema real. |

## Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Proyecto: pipeline de pedidos orientado a eventos"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: pipeline de pedidos orientado a eventos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/120-proyecto-pipeline-de-pedidos-orientado-a-eventos/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa plataforma-eventos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye plataforma-eventos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 120 Proyecto: pipeline de pedidos orientado a eventos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plataforma-eventos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 10 — Observabilidad, SRE y confiabilidad

← Parte 09 · Índice completo · Parte 11 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Operar sistemas mediante señales, objetivos y aprendizaje de incidentes.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

## Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                  | Laboratorio   | Horas |
|-----|--------------------------------------------------------|---------------|-------|
| 121 | Logs, métricas, trazas y eventos como señales          | observability | 4     |
| 122 | Logging estructurado, correlación y retención          | observability | 4     |
| 123 | Métricas, cardinalidad y modelos RED y USE             | metrics       | 4     |
| 124 | Tracing distribuido y OpenTelemetry                    | observability | 4     |
| 125 | Dashboards, alertas accionables y fatiga               | observability | 4     |
| 126 | SLI, SLO, SLA y presupuesto de error                   | sre           | 4     |
| 127 | Incidentes, severidad, comando y comunicación          | incident      | 4     |
| 128 | Runbooks, playbooks y automatización operativa         | operations    | 4     |
| 129 | Capacidad, rendimiento y pruebas de carga              | performance   | 4     |
| 130 | Timeouts, retries, backoff, circuit breaker y bulkhead | reliability   | 4     |
| 131 | Chaos engineering y game days                          | chaos         | 4     |
| 132 | Proyecto: operación SRE de CloudShop                   | capstone      | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## Pauta bibliográfica

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.

# 121 — Logs, métricas, trazas y eventos como señales

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar logs, métricas, trazas y eventos como señales dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar logs, métricas, trazas y eventos como señales con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar mapa-telemetria con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                                |
|----------|--------------------------------------------------------------------------------------------------------|
| logs     | Define su papel en logs, métricas, trazas y eventos como señales y cómo observarlo en un sistema real. |
| métricas | Define su papel en logs, métricas, trazas y eventos como señales y cómo observarlo en un sistema real. |
| trazas   | Define su papel en logs, métricas, trazas y eventos como señales y cómo observarlo en un sistema real. |
| eventos  | Define su papel en logs, métricas, trazas y eventos como señales y cómo observarlo en un sistema real. |
| señales  | Define su papel en logs, métricas, trazas y eventos como señales y cómo observarlo en un sistema real. |

## Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Logs, métricas, trazas y eventos como señales"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar logs, métricas, trazas y eventos como señales. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/121-logs-metricas-trazas-y-eventos-como-senales/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa mapa-telemetría en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye mapa-telemetría para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 121 Logs, métricas, trazas y eventos como señales

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega mapa-telemetría con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 122 — Logging estructurado, correlación y retención

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar logging estructurado, correlación y retención dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar logging estructurado, correlación y retención con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar contrato-logs con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                                                |
|--------------|--------------------------------------------------------------------------------------------------------|
| logging      | Define su papel en logging estructurado, correlación y retención y cómo observarlo en un sistema real. |
| estructurado | Define su papel en logging estructurado, correlación y retención y cómo observarlo en un sistema real. |
| correlación  | Define su papel en logging estructurado, correlación y retención y cómo observarlo en un sistema real. |
| retención    | Define su papel en logging estructurado, correlación y retención y cómo observarlo en un sistema real. |

### Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Logging estructurado, correlación y retención"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar logging estructurado, correlación y retención. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/122-logging-estructurado-correlacion-y-retencion/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa contrato-logs en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye contrato-logs para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 122 Logging estructurado, correlación y retención

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega contrato-logs con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 123 — Métricas, cardinalidad y modelos RED y USE

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: metrics · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar métricas, cardinalidad y modelos red y use dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar métricas, cardinalidad y modelos red y use con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar catalogo-metricas con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                             |
|--------------|-----------------------------------------------------------------------------------------------------|
| métricas     | Define su papel en métricas, cardinalidad y modelos red y use y cómo observarlo en un sistema real. |
| cardinalidad | Define su papel en métricas, cardinalidad y modelos red y use y cómo observarlo en un sistema real. |
| modelos      | Define su papel en métricas, cardinalidad y modelos red y use y cómo observarlo en un sistema real. |
| red          | Define su papel en métricas, cardinalidad y modelos red y use y cómo observarlo en un sistema real. |
| use          | Define su papel en métricas, cardinalidad y modelos red y use y cómo observarlo en un sistema real. |

## Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Métricas, cardinalidad y modelos RED y USE"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar métricas, cardinalidad y modelos red y use. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/123-metricas-cardinalidad-y-modelos-red-y-use/lab.py
```

El laboratorio selecciona el motor de práctica metrics y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa catalogo-metricas en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es métricas definidas, consultables y vinculadas a una decisión. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye catalogo-metricas para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 123 Métricas, cardinalidad y modelos RED y USE

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega catalogo-metricas con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 124 — Tracing distribuido y OpenTelemetry

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar tracing distribuido y opentelemetry dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar tracing distribuido y opentelemetry con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar traza-distribuida con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                      |
|---------------|----------------------------------------------------------------------------------------------|
| tracing       | Define su papel en tracing distribuido y opentelemetry y cómo observarlo en un sistema real. |
| distribuido   | Define su papel en tracing distribuido y opentelemetry y cómo observarlo en un sistema real. |
| opentelemetry | Define su papel en tracing distribuido y opentelemetry y cómo observarlo en un sistema real. |

## Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Tracing distribuido y OpenTelemetry"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar tracing distribuido y opentelemetry. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/124-tracing-distribuido-y-opentelemetry/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa traza-distribuida en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye traza-distribuida para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 124 Tracing distribuido y OpenTelemetry

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega traza-distribuida con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 125 — Dashboards, alertas accionables y fatiga

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar dashboards, alertas accionables y fatiga dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar dashboards, alertas accionables y fatiga con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar tablero-operativo con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                           |
|-------------|---------------------------------------------------------------------------------------------------|
| dashboards  | Define su papel en dashboards, alertas accionables y fatiga y cómo observarlo en un sistema real. |
| alertas     | Define su papel en dashboards, alertas accionables y fatiga y cómo observarlo en un sistema real. |
| accionables | Define su papel en dashboards, alertas accionables y fatiga y cómo observarlo en un sistema real. |
| fatiga      | Define su papel en dashboards, alertas accionables y fatiga y cómo observarlo en un sistema real. |

## Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Dashboards, alertas accionables y fatiga"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar dashboards, alertas accionables y fatiga. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/125-dashboards-alertas-accionables-y-fatiga/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa tablero-operativo en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye tablero-operativo para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 125 Dashboards, alertas accionables y fatiga

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega tablero-operativo con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 126 — SLI, SLO, SLA y presupuesto de error

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: sre · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar sli, slo, sla y presupuesto de error dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sli, slo, sla y presupuesto de error con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar slo-servicio con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                       |
|-------------|-----------------------------------------------------------------------------------------------|
| sli         | Define su papel en sli, slo, sla y presupuesto de error y cómo observarlo en un sistema real. |
| slo         | Define su papel en sli, slo, sla y presupuesto de error y cómo observarlo en un sistema real. |
| sla         | Define su papel en sli, slo, sla y presupuesto de error y cómo observarlo en un sistema real. |
| presupuesto | Define su papel en sli, slo, sla y presupuesto de error y cómo observarlo en un sistema real. |
| error       | Define su papel en sli, slo, sla y presupuesto de error y cómo observarlo en un sistema real. |

### Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: SLI, SLO, SLA y presupuesto de error"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar sli, slo, sla y presupuesto de error. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/126-sli-slo-sla-y-presupuesto-de-error/lab.py
```

El laboratorio selecciona el motor de práctica sre y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa slo-servicio en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un SLO con presupuesto de error y política de acción. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye slo-servicio para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 126 SLI, SLO, SLA y presupuesto de error

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega slo-servicio con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 127 — Incidentes, severidad, comando y comunicación

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: incident · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar incidentes, severidad, comando y comunicación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar incidentes, severidad, comando y comunicación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plan-incidente con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                |
|--------------|--------------------------------------------------------------------------------------------------------|
| incidentes   | Define su papel en incidentes, severidad, comando y comunicación y cómo observarlo en un sistema real. |
| severidad    | Define su papel en incidentes, severidad, comando y comunicación y cómo observarlo en un sistema real. |
| comando      | Define su papel en incidentes, severidad, comando y comunicación y cómo observarlo en un sistema real. |
| comunicación | Define su papel en incidentes, severidad, comando y comunicación y cómo observarlo en un sistema real. |

## Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Incidentes, severidad, comando y comunicación"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar incidentes, severidad, comando y comunicación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/127-incidentes-severidad-comando-y-comunicacion/lab.py
```

El laboratorio selecciona el motor de práctica incident y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plan-incidente en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una cronología, roles, comunicación y aprendizaje. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye plan-incidente para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 127 Incidentes, severidad, comando y comunicación

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plan-incidente con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 128 — Runbooks, playbooks y automatización operativa

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: operations · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar runbooks, playbooks y automatización operativa dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar runbooks, playbooks y automatización operativa con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar runbook-verificado con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                                 |
|----------------|---------------------------------------------------------------------------------------------------------|
| runbooks       | Define su papel en runbooks, playbooks y automatización operativa y cómo observarlo en un sistema real. |
| playbooks      | Define su papel en runbooks, playbooks y automatización operativa y cómo observarlo en un sistema real. |
| automatización | Define su papel en runbooks, playbooks y automatización operativa y cómo observarlo en un sistema real. |
| operativa      | Define su papel en runbooks, playbooks y automatización operativa y cómo observarlo en un sistema real. |

## Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Runbooks, playbooks y automatización operativa"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar runbooks, playbooks y automatización operativa. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/128-runbooks-playbooks-y-automatizacion-operativa/lab.py
```

El laboratorio selecciona el motor de práctica operations y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa runbook-verificado en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un runbook probado por otra persona. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye runbook-verificado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 128 Runbooks, playbooks y automatización operativa

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega runbook-verificado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 129 — Capacidad, rendimiento y pruebas de carga

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: performance · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar capacidad, rendimiento y pruebas de carga dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capacidad, rendimiento y pruebas de carga con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar informe-carga con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                            |
|-------------|----------------------------------------------------------------------------------------------------|
| capacidad   | Define su papel en capacidad, rendimiento y pruebas de carga y cómo observarlo en un sistema real. |
| rendimiento | Define su papel en capacidad, rendimiento y pruebas de carga y cómo observarlo en un sistema real. |
| pruebas     | Define su papel en capacidad, rendimiento y pruebas de carga y cómo observarlo en un sistema real. |
| carga       | Define su papel en capacidad, rendimiento y pruebas de carga y cómo observarlo en un sistema real. |

## Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capacidad, rendimiento y pruebas de carga"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capacidad, rendimiento y pruebas de carga. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/129-capacidad-rendimiento-y-pruebas-de-carga/lab.py
```

El laboratorio selecciona el motor de práctica performance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa informe-carga en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una prueba de carga con baseline y cuello de botella. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye informe-carga para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 129 Capacidad, rendimiento y pruebas de carga

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega informe-carga con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 130 — Timeouts, retries, backoff, circuit breaker y bulkhead

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: reliability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar timeouts, retries, backoff, circuit breaker y bulkhead dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar timeouts, retries, backoff, circuit breaker y bulkhead con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cliente-resiliente con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                                         |
|----------|-----------------------------------------------------------------------------------------------------------------|
| timeouts | Define su papel en timeouts, retries, backoff, circuit breaker y bulkhead y cómo observarlo en un sistema real. |
| retries  | Define su papel en timeouts, retries, backoff, circuit breaker y bulkhead y cómo observarlo en un sistema real. |
| backoff  | Define su papel en timeouts, retries, backoff, circuit breaker y bulkhead y cómo observarlo en un sistema real. |
| circuit  | Define su papel en timeouts, retries, backoff, circuit breaker y bulkhead y cómo observarlo en un sistema real. |
| breaker  | Define su papel en timeouts, retries, backoff, circuit breaker y bulkhead y cómo observarlo en un sistema real. |
| bulkhead | Define su papel en timeouts, retries, backoff, circuit breaker y bulkhead y cómo observarlo en un sistema real. |

## Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Timeouts, retries, backoff, circuit breaker y bulkhead"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar timeouts, retries, backoff, circuit breaker y bulkhead. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/130-timeouts-retries-backoff-circuit-breaker-y-bulkhead/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa cliente-resiliente en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye cliente-resiliente para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 130 Timeouts, retries, backoff, circuit breaker y bulkhead

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega cliente-resiliente con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 131 — Chaos engineering y game days

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: chaos · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar chaos engineering y game days dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar chaos engineering y game days con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar experimento-caos con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                |
|-------------|----------------------------------------------------------------------------------------|
| chaos       | Define su papel en chaos engineering y game days y cómo observarlo en un sistema real. |
| engineering | Define su papel en chaos engineering y game days y cómo observarlo en un sistema real. |
| game        | Define su papel en chaos engineering y game days y cómo observarlo en un sistema real. |
| days        | Define su papel en chaos engineering y game days y cómo observarlo en un sistema real. |

## Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Chaos engineering y game days"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar chaos engineering y game days. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/131-chaos-engineering-y-game-days/lab.py
```

El laboratorio selecciona el motor de práctica chaos y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa experimento-caos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una hipótesis de resiliencia y criterio de abortar. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye experimento-caos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 131 Chaos engineering y game days

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega experimento-caos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 132 — Proyecto: operación SRE de CloudShop

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar proyecto: operación sre de cloudshop dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: operación sre de cloudshop con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-sre con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                       |
|-----------|-----------------------------------------------------------------------------------------------|
| proyecto  | Define su papel en proyecto: operación sre de cloudshop y cómo observarlo en un sistema real. |
| operación | Define su papel en proyecto: operación sre de cloudshop y cómo observarlo en un sistema real. |
| sre       | Define su papel en proyecto: operación sre de cloudshop y cómo observarlo en un sistema real. |
| cloudshop | Define su papel en proyecto: operación sre de cloudshop y cómo observarlo en un sistema real. |

## Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: operación SRE de CloudShop"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: operación sre de cloudshop. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/132-proyecto-operacion-sre-de-cloudshop/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-sre en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye plataforma-sre para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 132 Proyecto: operación SRE de CloudShop

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plataforma-sre con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 11 — Seguridad, gobierno, cumplimiento y FinOps

← Parte 10 · Índice completo · Parte 12 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Construir guardrails que hagan segura y económicamente sostenible la autonomía.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

## Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                        | Laboratorio  | Horas |
|-----|--------------------------------------------------------------|--------------|-------|
| 133 | Zero Trust y defensa en profundidad                          | security     | 4     |
| 134 | Mínimo privilegio, acceso temporal y separación de funciones | iam          | 4     |
| 135 | Segmentación, perímetro, WAF, DDoS y egress                  | network      | 4     |
| 136 | Cifrado, KMS, HSM, rotación y envelope encryption            | security     | 4     |
| 137 | Gestión de secretos y credenciales de workloads              | security     | 4     |
| 138 | Vulnerabilidades, imágenes y cadena de suministro            | supply-chain | 4     |
| 139 | CSPM, postura, policy as code y remediación                  | governance   | 4     |
| 140 | Threat modeling con STRIDE y attack paths                    | security     | 4     |
| 141 | Cumplimiento, residencia, privacidad y evidencia             | compliance   | 4     |
| 142 | FinOps: showback, chargeback, budgets y anomalías            | finops       | 4     |
| 143 | Optimización de costo, capacidad y sostenibilidad            | finops       | 4     |
| 144 | Proyecto: landing zone con guardrails                        | capstone     | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.

## 133 — Zero Trust y defensa en profundidad

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar zero trust y defensa en profundidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar zero trust y defensa en profundidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-zero-trust con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                      |
|-------------|----------------------------------------------------------------------------------------------|
| zero        | Define su papel en zero trust y defensa en profundidad y cómo observarlo en un sistema real. |
| trust       | Define su papel en zero trust y defensa en profundidad y cómo observarlo en un sistema real. |
| defensa     | Define su papel en zero trust y defensa en profundidad y cómo observarlo en un sistema real. |
| profundidad | Define su papel en zero trust y defensa en profundidad y cómo observarlo en un sistema real. |

### Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Zero Trust y defensa en profundidad"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar zero trust y defensa en profundidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/133-zero-trust-y-defensa-en-profundidad/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modelo-zero-trust en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye modelo-zero-trust para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 133 Zero Trust y defensa en profundidad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega modelo-zero-trust con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 134 — Mínimo privilegio, acceso temporal y separación de funciones

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar mínimo privilegio, acceso temporal y separación de funciones dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar mínimo privilegio, acceso temporal y separación de funciones con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar revision-accesos con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                               |
|------------|-----------------------------------------------------------------------------------------------------------------------|
| mínimo     | Define su papel en mínimo privilegio, acceso temporal y separación de funciones y cómo observarlo en un sistema real. |
| privilegio | Define su papel en mínimo privilegio, acceso temporal y separación de funciones y cómo observarlo en un sistema real. |
| acceso     | Define su papel en mínimo privilegio, acceso temporal y separación de funciones y cómo observarlo en un sistema real. |
| temporal   | Define su papel en mínimo privilegio, acceso temporal y separación de funciones y cómo observarlo en un sistema real. |
| separación | Define su papel en mínimo privilegio, acceso temporal y separación de funciones y cómo observarlo en un sistema real. |
| funciones  | Define su papel en mínimo privilegio, acceso temporal y separación de funciones y cómo observarlo en un sistema real. |

## Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Mínimo privilegio, acceso temporal y separación de funciones"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar mínimo privilegio, acceso temporal y separación de funciones. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/134-minimo-privilegio-acceso-temporal-y-separacion-
```

de-funciones/lab.py

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa revision-accesos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye revision-accesos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 134 Mínimo privilegio, acceso temporal y separación de funciones

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega revision-accesos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 135 — Segmentación, perímetro, WAF, DDoS y egress

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar segmentación, perímetro, waf, ddos y egress dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar segmentación, perímetro, waf, ddos y egress con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar arquitectura-defensiva con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                              |
|--------------|------------------------------------------------------------------------------------------------------|
| segmentación | Define su papel en segmentación, perímetro, waf, ddos y egress y cómo observarlo en un sistema real. |
| perímetro    | Define su papel en segmentación, perímetro, waf, ddos y egress y cómo observarlo en un sistema real. |
| waf          | Define su papel en segmentación, perímetro, waf, ddos y egress y cómo observarlo en un sistema real. |
| ddos         | Define su papel en segmentación, perímetro, waf, ddos y egress y cómo observarlo en un sistema real. |
| egress       | Define su papel en segmentación, perímetro, waf, ddos y egress y cómo observarlo en un sistema real. |

## Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Segmentación, perímetro, WAF, DDoS y egress"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar segmentación, perímetro, waf, ddos y egress. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/135-segmentacion-perimetro-waf-ddos-y-egress/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa arquitectura-defensiva en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye arquitectura-defensiva para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Practical Cloud Security — Chris Dotson.

- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 135 Segmentación, perímetro, WAF, DDoS y egress

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega arquitectura-defensiva con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 136 — Cifrado, KMS, HSM, rotación y envelope encryption

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar cifrado, kms, hsm, rotación y envelope encryption dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cifrado, kms, hsm, rotación y envelope encryption con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar jerarquía-claves con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                    |
|------------|------------------------------------------------------------------------------------------------------------|
| cifrado    | Define su papel en cifrado, kms, hsm, rotación y envelope encryption y cómo observarlo en un sistema real. |
| kms        | Define su papel en cifrado, kms, hsm, rotación y envelope encryption y cómo observarlo en un sistema real. |
| hsm        | Define su papel en cifrado, kms, hsm, rotación y envelope encryption y cómo observarlo en un sistema real. |
| rotación   | Define su papel en cifrado, kms, hsm, rotación y envelope encryption y cómo observarlo en un sistema real. |
| envelope   | Define su papel en cifrado, kms, hsm, rotación y envelope encryption y cómo observarlo en un sistema real. |
| encryption | Define su papel en cifrado, kms, hsm, rotación y envelope encryption y cómo observarlo en un sistema real. |

## Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Cifrado, KMS, HSM, rotación y envelope encryption"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cifrado, kms, hsm, rotación y envelope encryption. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/136-cifrado-kms-hsm-rotacion-y-envelope-encryption/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa jerarquía-claves en evidencia/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye jerarquía-claves para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 136 Cifrado, KMS, HSM, rotación y envelope encryption

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega jerarquía-claves con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 137 — Gestión de secretos y credenciales de workloads

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar gestión de secretos y credenciales de workloads dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar gestión de secretos y credenciales de workloads con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar ciclo-secretos con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                  |
|--------------|----------------------------------------------------------------------------------------------------------|
| gestión      | Define su papel en gestión de secretos y credenciales de workloads y cómo observarlo en un sistema real. |
| secretos     | Define su papel en gestión de secretos y credenciales de workloads y cómo observarlo en un sistema real. |
| credenciales | Define su papel en gestión de secretos y credenciales de workloads y cómo observarlo en un sistema real. |
| workloads    | Define su papel en gestión de secretos y credenciales de workloads y cómo observarlo en un sistema real. |

## Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Gestión de secretos y credenciales de workloads"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar gestión de secretos y credenciales de workloads. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/137-gestion-de-secretos-y-credenciales-de-workloads/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa ciclo-secretos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye ciclo-secretos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 137 Gestión de secretos y credenciales de workloads

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega ciclo-secretos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 138 — Vulnerabilidades, imágenes y cadena de suministro

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: supply-chain · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar vulnerabilidades, imágenes y cadena de suministro dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar vulnerabilidades, imágenes y cadena de suministro con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar informe-supply-chain con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto         | Comprensión verificable                                                                                    |
|------------------|------------------------------------------------------------------------------------------------------------|
| vulnerabilidades | Define su papel en vulnerabilidades, imágenes y cadena de suministro y cómo observarlo en un sistema real. |
| imágenes         | Define su papel en vulnerabilidades, imágenes y cadena de suministro y cómo observarlo en un sistema real. |
| cadena           | Define su papel en vulnerabilidades, imágenes y cadena de suministro y cómo observarlo en un sistema real. |
| suministro       | Define su papel en vulnerabilidades, imágenes y cadena de suministro y cómo observarlo en un sistema real. |

## Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Vulnerabilidades, imágenes y cadena de suministro"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar vulnerabilidades, imágenes y cadena de suministro. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/138-vulnerabilidades-imagenes-y-cadena-de-suministro/lab.py
```

El laboratorio selecciona el motor de práctica supply-chain y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa informe-supply-chain en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es procedencia, inventario y verificación del artefacto. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye informe-supply-chain para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 138 Vulnerabilidades, imágenes y cadena de suministro

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega informe-supply-chain con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 139 — CSPM, postura, policy as code y remediación

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar cspm, postura, policy as code y remediación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cspm, postura, policy as code y remediación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar guardrail-policy con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                              |
|-------------|------------------------------------------------------------------------------------------------------|
| csdm        | Define su papel en cspm, postura, policy as code y remediación y cómo observarlo en un sistema real. |
| postura     | Define su papel en cspm, postura, policy as code y remediación y cómo observarlo en un sistema real. |
| policy      | Define su papel en cspm, postura, policy as code y remediación y cómo observarlo en un sistema real. |
| as          | Define su papel en cspm, postura, policy as code y remediación y cómo observarlo en un sistema real. |
| code        | Define su papel en cspm, postura, policy as code y remediación y cómo observarlo en un sistema real. |
| remediación | Define su papel en cspm, postura, policy as code y remediación y cómo observarlo en un sistema real. |

## Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: CSPM, postura, policy as code y remediación"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cspm, postura, policy as code y remediación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/139-cspm-postura-policy-as-code-y-remediacion/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa guardrail-policy en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `guardrail-policy` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Practical Cloud Security — Chris Dotson.

- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 139 CSPM, postura, policy as code y remediación

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega guardrail-policy con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 140 — Threat modeling con STRIDE y attack paths

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar threat modeling con stride y attack paths dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar threat modeling con stride y attack paths con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-amenazas con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                            |
|----------|----------------------------------------------------------------------------------------------------|
| threat   | Define su papel en threat modeling con stride y attack paths y cómo observarlo en un sistema real. |
| modeling | Define su papel en threat modeling con stride y attack paths y cómo observarlo en un sistema real. |
| stride   | Define su papel en threat modeling con stride y attack paths y cómo observarlo en un sistema real. |
| attack   | Define su papel en threat modeling con stride y attack paths y cómo observarlo en un sistema real. |
| paths    | Define su papel en threat modeling con stride y attack paths y cómo observarlo en un sistema real. |

## Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Threat modeling con STRIDE y attack paths"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar threat modeling con stride y attack paths. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/140-threat-modeling-con-stride-y-attack-paths/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modelo-amenazas en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye modelo-amenazas para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Practical Cloud Security — Chris Dotson.

- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 140 Threat modeling con STRIDE y attack paths

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega modelo-amenazas con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 141 — Cumplimiento, residencia, privacidad y evidencia

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: compliance · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar cumplimiento, residencia, privacidad y evidencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cumplimiento, residencia, privacidad y evidencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-controles con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                   |
|--------------|-----------------------------------------------------------------------------------------------------------|
| cumplimiento | Define su papel en cumplimiento, residencia, privacidad y evidencia y cómo observarlo en un sistema real. |
| residencia   | Define su papel en cumplimiento, residencia, privacidad y evidencia y cómo observarlo en un sistema real. |
| privacidad   | Define su papel en cumplimiento, residencia, privacidad y evidencia y cómo observarlo en un sistema real. |
| evidencia    | Define su papel en cumplimiento, residencia, privacidad y evidencia y cómo observarlo en un sistema real. |

## Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Cumplimiento, residencia, privacidad y evidencia"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cumplimiento, residencia, privacidad y evidencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/141-cumplimiento-residencia-privacidad-y-evidencia/lab.py
```

El laboratorio selecciona el motor de práctica compliance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-controles en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control mapeado a evidencia y responsable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye matriz-controles para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 141 Cumplimiento, residencia, privacidad y evidencia

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega matriz-controles con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 142 — FinOps: showback, chargeback, budgets y anomalías

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: finops · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar finops: showback, chargeback, budgets y anomalías dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar finops: showback, chargeback, budgets y anomalías con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-asignacion-costos con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                    |
|------------|------------------------------------------------------------------------------------------------------------|
| finops     | Define su papel en finops: showback, chargeback, budgets y anomalías y cómo observarlo en un sistema real. |
| showback   | Define su papel en finops: showback, chargeback, budgets y anomalías y cómo observarlo en un sistema real. |
| chargeback | Define su papel en finops: showback, chargeback, budgets y anomalías y cómo observarlo en un sistema real. |
| budgets    | Define su papel en finops: showback, chargeback, budgets y anomalías y cómo observarlo en un sistema real. |
| anomalías  | Define su papel en finops: showback, chargeback, budgets y anomalías y cómo observarlo en un sistema real. |

## Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: FinOps: showback, chargeback, budgets y anomalías"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar finops: showback, chargeback, budgets y anomalías. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/142-finops-showback-chargeback-budgets-y-anomalias/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa modelo-asignacion-costos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye modelo-asignacion-costos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 142 FinOps: showback, chargeback, budgets y anomalías

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega modelo-asignacion-costos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 143 — Optimización de costo, capacidad y sostenibilidad

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: finops · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar optimización de costo, capacidad y sostenibilidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar optimización de costo, capacidad y sostenibilidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar backlog-optimizacion con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                                    |
|----------------|------------------------------------------------------------------------------------------------------------|
| optimización   | Define su papel en optimización de costo, capacidad y sostenibilidad y cómo observarlo en un sistema real. |
| costo          | Define su papel en optimización de costo, capacidad y sostenibilidad y cómo observarlo en un sistema real. |
| capacidad      | Define su papel en optimización de costo, capacidad y sostenibilidad y cómo observarlo en un sistema real. |
| sostenibilidad | Define su papel en optimización de costo, capacidad y sostenibilidad y cómo observarlo en un sistema real. |

## Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Optimización de costo, capacidad y sostenibilidad"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar optimización de costo, capacidad y sostenibilidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/143-optimizacion-de-costo-capacidad-y-sostenibilidad/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa backlog-optimizacion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye backlog-optimización para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 143 Optimización de costo, capacidad y sostenibilidad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega backlog-optimización con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 144 — Proyecto: landing zone con guardrails

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar proyecto: landing zone con guardrails dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: landing zone con guardrails con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar landing-zone-gobernada con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                        |
|------------|------------------------------------------------------------------------------------------------|
| proyecto   | Define su papel en proyecto: landing zone con guardrails y cómo observarlo en un sistema real. |
| landing    | Define su papel en proyecto: landing zone con guardrails y cómo observarlo en un sistema real. |
| zone       | Define su papel en proyecto: landing zone con guardrails y cómo observarlo en un sistema real. |
| guardrails | Define su papel en proyecto: landing zone con guardrails y cómo observarlo en un sistema real. |

## Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: landing zone con guardrails"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: landing zone con guardrails. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/144-proyecto-landing-zone-con-guardrails/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa landing-zone-gobernada en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye landing-zone-gobernada para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 144 Proyecto: landing zone con guardrails

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega landing-zone-gobernada con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 12 — Arquitectura cloud-native y sistemas distribuidos

← Parte 11 · Índice completo · Parte 13 →

Nivel: avanzado-experto · Clases: 12 · Duración sugerida: 6–8 semanas

Tomar decisiones de arquitectura con compensaciones explícitas y evidencia.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

## Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                            | Laboratorio  | Horas |
|-----|--------------------------------------------------|--------------|-------|
| 145 | Requisitos, restricciones y atributos de calidad | architecture | 4     |
| 146 | Twelve-Factor App y configuración cloud-native   | architecture | 4     |
| 147 | DDD, bounded contexts y ownership de datos       | architecture | 4     |
| 148 | Monolito modular, microservicios y función       | decision     | 4     |
| 149 | CAP, PACELC y consistencia por operación         | distributed  | 4     |
| 150 | Replicación, particionado y consenso             | distributed  | 4     |
| 151 | Fallos parciales y patrones de resiliencia       | reliability  | 4     |
| 152 | Service discovery, malla y comunicación          | network      | 4     |
| 153 | Contratos API, compatibilidad y evolución        | api          | 4     |
| 154 | Multi-tenancy, aislamiento y noisy neighbor      | architecture | 4     |
| 155 | Rendimiento, costo, seguridad y operabilidad     | decision     | 4     |
| 156 | Proyecto: revisión de arquitectura con ADR       | capstone     | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.



Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.

# 145 — Requisitos, restricciones y atributos de calidad

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4  
Laboratorio: architecture · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar requisitos, restricciones y atributos de calidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar requisitos, restricciones y atributos de calidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar quality-attribute-scenarios con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                                   |
|---------------|-----------------------------------------------------------------------------------------------------------|
| requisitos    | Define su papel en requisitos, restricciones y atributos de calidad y cómo observarlo en un sistema real. |
| restricciones | Define su papel en requisitos, restricciones y atributos de calidad y cómo observarlo en un sistema real. |
| atributos     | Define su papel en requisitos, restricciones y atributos de calidad y cómo observarlo en un sistema real. |
| calidad       | Define su papel en requisitos, restricciones y atributos de calidad y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Requisitos, restricciones y atributos de calidad"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar requisitos, restricciones y atributos de calidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/145-requisitos-restricciones-y-atributos-de-calidad/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa quality-attribute-scenarios en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye quality-attribute-scenarios para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 145 Requisitos, restricciones y atributos de calidad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega quality-attribute-scenarios con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 146 — Twelve-Factor App y configuración cloud-native

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4  
Laboratorio: architecture · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar twelve-factor app y configuración cloud-native dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar twelve-factor app y configuración cloud-native con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar revision-twelve-factor con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                                 |
|---------------|---------------------------------------------------------------------------------------------------------|
| twelve        | Define su papel en twelve-factor app y configuración cloud-native y cómo observarlo en un sistema real. |
| factor        | Define su papel en twelve-factor app y configuración cloud-native y cómo observarlo en un sistema real. |
| app           | Define su papel en twelve-factor app y configuración cloud-native y cómo observarlo en un sistema real. |
| configuración | Define su papel en twelve-factor app y configuración cloud-native y cómo observarlo en un sistema real. |
| cloud         | Define su papel en twelve-factor app y configuración cloud-native y cómo observarlo en un sistema real. |
| native        | Define su papel en twelve-factor app y configuración cloud-native y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Twelve-Factor App y configuración cloud-native"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar twelve-factor app y configuración cloud-native. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/146-twelve-factor-app-y-configuracion-cloud-native/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa revision-twelve-factor en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye revision-twelve-factor para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 146 Twelve-Factor App y configuración cloud-native

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega revision-twelve-factor con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 147 — DDD, bounded contexts y ownership de datos

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4  
Laboratorio: architecture · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar ddd, bounded contexts y ownership de datos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ddd, bounded contexts y ownership de datos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar context-map con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                             |
|-----------|-----------------------------------------------------------------------------------------------------|
| ddd       | Define su papel en ddd, bounded contexts y ownership de datos y cómo observarlo en un sistema real. |
| bounded   | Define su papel en ddd, bounded contexts y ownership de datos y cómo observarlo en un sistema real. |
| contexts  | Define su papel en ddd, bounded contexts y ownership de datos y cómo observarlo en un sistema real. |
| ownership | Define su papel en ddd, bounded contexts y ownership de datos y cómo observarlo en un sistema real. |
| datos     | Define su papel en ddd, bounded contexts y ownership de datos y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: DDD, bounded contexts y ownership de datos"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ddd, bounded contexts y ownership de datos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/147-ddd-bounded-contexts-y-ownership-de-datos/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa context-map en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye context-map para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 147 DDD, bounded contexts y ownership de datos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega context-map con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 148 — Monolito modular, microservicios y función

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4  
Laboratorio: decision · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar monolito modular, microservicios y función dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar monolito modular, microservicios y función con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar adr-descomposicion con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------|
| monolito       | Define su papel en monolito modular, microservicios y función y cómo observarlo en un sistema real. |
| modular        | Define su papel en monolito modular, microservicios y función y cómo observarlo en un sistema real. |
| microservicios | Define su papel en monolito modular, microservicios y función y cómo observarlo en un sistema real. |
| función        | Define su papel en monolito modular, microservicios y función y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Monolito modular, microservicios y función"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar monolito modular, microservicios y función. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/148-monolito-modular-microservicios-y-funcion/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa adr-descomposicion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye adr-descomposicion para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 148 Monolito modular, microservicios y función

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega adr-descomposicion con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 149 — CAP, PACELC y consistencia por operación

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4  
Laboratorio: distributed · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar cap, pacelc y consistencia por operación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cap, pacelc y consistencia por operación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-consistencia con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                           |
|--------------|---------------------------------------------------------------------------------------------------|
| cap          | Define su papel en cap, pacelc y consistencia por operación y cómo observarlo en un sistema real. |
| pacelc       | Define su papel en cap, pacelc y consistencia por operación y cómo observarlo en un sistema real. |
| consistencia | Define su papel en cap, pacelc y consistencia por operación y cómo observarlo en un sistema real. |
| operación    | Define su papel en cap, pacelc y consistencia por operación y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: CAP, PACELC y consistencia por operación"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cap, pacelc y consistencia por operación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/149-cap-pacelc-y-consistencia-por-operacion/lab.py
```

El laboratorio selecciona el motor de práctica distributed y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-consistencia en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una traza de consistencia, reintento o fallo parcial. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye matriz-consistencia para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 149 CAP, PACELC y consistencia por operación

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega matriz-consistencia con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 150 — Replicación, particionado y consenso

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4  
Laboratorio: distributed · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar replicación, particionado y consenso dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar replicación, particionado y consenso con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-replicacion con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                       |
|--------------|-----------------------------------------------------------------------------------------------|
| replicación  | Define su papel en replicación, particionado y consenso y cómo observarlo en un sistema real. |
| particionado | Define su papel en replicación, particionado y consenso y cómo observarlo en un sistema real. |
| consenso     | Define su papel en replicación, particionado y consenso y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Replicación, particionado y consenso"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar replicación, particionado y consenso. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/150-replicacion-particionado-y-consenso/lab.py
```

El laboratorio selecciona el motor de práctica distributed y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modelo-replicacion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una traza de consistencia, reintento o fallo parcial. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye modelo-replicación para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 150 Replicación, particionado y consenso

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega modelo-replicación con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 151 — Fallos parciales y patrones de resiliencia

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4  
Laboratorio: reliability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar fallos parciales y patrones de resiliencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar fallos parciales y patrones de resiliencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar mapa-fallos con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                             |
|-------------|-----------------------------------------------------------------------------------------------------|
| fallos      | Define su papel en fallos parciales y patrones de resiliencia y cómo observarlo en un sistema real. |
| parciales   | Define su papel en fallos parciales y patrones de resiliencia y cómo observarlo en un sistema real. |
| patrones    | Define su papel en fallos parciales y patrones de resiliencia y cómo observarlo en un sistema real. |
| resiliencia | Define su papel en fallos parciales y patrones de resiliencia y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Fallos parciales y patrones de resiliencia"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar fallos parciales y patrones de resiliencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/151-fallos-parciales-y-patrones-de-resiliencia/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa mapa-fallos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye mapa-fallos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 151 Fallos parciales y patrones de resiliencia

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega mapa-fallos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 152 — Service discovery, malla y comunicación

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4  
Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar service discovery, malla y comunicación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar service discovery, malla y comunicación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar topología-servicios con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                          |
|--------------|--------------------------------------------------------------------------------------------------|
| service      | Define su papel en service discovery, malla y comunicación y cómo observarlo en un sistema real. |
| discovery    | Define su papel en service discovery, malla y comunicación y cómo observarlo en un sistema real. |
| malla        | Define su papel en service discovery, malla y comunicación y cómo observarlo en un sistema real. |
| comunicación | Define su papel en service discovery, malla y comunicación y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Service discovery, malla y comunicación"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar service discovery, malla y comunicación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/152-service-discovery-malla-y-comunicacion/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa topología-servicios en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye topología-servicios para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 152 Service discovery, malla y comunicación

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega topología-servicios con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 153 — Contratos API, compatibilidad y evolución

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4  
Laboratorio: api · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar contratos api, compatibilidad y evolución dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar contratos api, compatibilidad y evolución con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar contrato-versionado con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                            |
|----------------|----------------------------------------------------------------------------------------------------|
| contratos      | Define su papel en contratos api, compatibilidad y evolución y cómo observarlo en un sistema real. |
| api            | Define su papel en contratos api, compatibilidad y evolución y cómo observarlo en un sistema real. |
| compatibilidad | Define su papel en contratos api, compatibilidad y evolución y cómo observarlo en un sistema real. |
| evolución      | Define su papel en contratos api, compatibilidad y evolución y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Contratos API, compatibilidad y evolución"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar contratos api, compatibilidad y evolución. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/153-contratos-api-compatibilidad-y-evolucion/lab.py
```

El laboratorio selecciona el motor de práctica api y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa contrato-versionado en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un contrato versionado con pruebas positivas y negativas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye contrato-versionado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 153 Contratos API, compatibilidad y evolución

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega contrato-versionado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 154 — Multi-tenancy, aislamiento y noisy neighbor

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4  
Laboratorio: architecture · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar multi-tenancy, aislamiento y noisy neighbor dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar multi-tenancy, aislamiento y noisy neighbor con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-tenancy con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                              |
|-------------|------------------------------------------------------------------------------------------------------|
| multi       | Define su papel en multi-tenancy, aislamiento y noisy neighbor y cómo observarlo en un sistema real. |
| tenancy     | Define su papel en multi-tenancy, aislamiento y noisy neighbor y cómo observarlo en un sistema real. |
| aislamiento | Define su papel en multi-tenancy, aislamiento y noisy neighbor y cómo observarlo en un sistema real. |
| noisy       | Define su papel en multi-tenancy, aislamiento y noisy neighbor y cómo observarlo en un sistema real. |
| neighbor    | Define su papel en multi-tenancy, aislamiento y noisy neighbor y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Multi-tenancy, aislamiento y noisy neighbor"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar multi-tenancy, aislamiento y noisy neighbor. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/154-multi-tenancy-aislamiento-y-noisy-neighbor/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa modelo-tenancy en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye modelo-tenancy para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 154 Multi-tenancy, aislamiento y noisy neighbor

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega modelo-tenancy con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 155 — Rendimiento, costo, seguridad y operabilidad

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4  
Laboratorio: decision · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar rendimiento, costo, seguridad y operabilidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar rendimiento, costo, seguridad y operabilidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar tradeoff-analysis con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                               |
|--------------|-------------------------------------------------------------------------------------------------------|
| rendimiento  | Define su papel en rendimiento, costo, seguridad y operabilidad y cómo observarlo en un sistema real. |
| costo        | Define su papel en rendimiento, costo, seguridad y operabilidad y cómo observarlo en un sistema real. |
| seguridad    | Define su papel en rendimiento, costo, seguridad y operabilidad y cómo observarlo en un sistema real. |
| operabilidad | Define su papel en rendimiento, costo, seguridad y operabilidad y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Rendimiento, costo, seguridad y operabilidad"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar rendimiento, costo, seguridad y operabilidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/155-rendimiento-costo-seguridad-y-operabilidad/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa tradeoff-analysis en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye tradeoff-analysis para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 155 Rendimiento, costo, seguridad y operabilidad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega tradeoff-analysis con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 156 — Proyecto: revisión de arquitectura con ADR

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 8  
Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar proyecto: revisión de arquitectura con adr dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: revisión de arquitectura con adr con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar architecture-review con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                             |
|--------------|-----------------------------------------------------------------------------------------------------|
| proyecto     | Define su papel en proyecto: revisión de arquitectura con adr y cómo observarlo en un sistema real. |
| revisión     | Define su papel en proyecto: revisión de arquitectura con adr y cómo observarlo en un sistema real. |
| arquitectura | Define su papel en proyecto: revisión de arquitectura con adr y cómo observarlo en un sistema real. |
| adr          | Define su papel en proyecto: revisión de arquitectura con adr y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: revisión de arquitectura con ADR"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: revisión de arquitectura con adr. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/156-proyecto-revision-de-arquitectura-con-adr/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa architecture-review en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye architecture-review para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 156 Proyecto: revisión de arquitectura con ADR

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega architecture-review con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 13 — Multi-cloud, híbrido, migración y recuperación

← Parte 12 · Índice completo · Parte 14 →

Nivel: experto · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar continuidad y portabilidad sin ocultar complejidad, latencia ni egress.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

## Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                              | Laboratorio   | Horas |
|-----|----------------------------------------------------|---------------|-------|
| 157 | Motivaciones y anti-patrones de multi-cloud        | decision      | 4     |
| 158 | Portabilidad, capas de abstracción y lock-in       | architecture  | 4     |
| 159 | Federación de identidad entre nubes                | iam           | 4     |
| 160 | Conectividad, tránsito, DNS y service discovery    | network       | 4     |
| 161 | Replicación de datos, soberanía y costos de egress | data          | 4     |
| 162 | Observabilidad y operación entre proveedores       | observability | 4     |
| 163 | Terraform multi-provider y separación de estados   | iac           | 4     |
| 164 | Flotas Kubernetes y políticas comunes              | kubernetes    | 4     |
| 165 | Nube híbrida, edge y conectividad privada          | hybrid        | 4     |
| 166 | Backup, RTO, RPO y patrones de disaster recovery   | reliability   | 4     |
| 167 | Las 7R de migración y oleadas                      | migration     | 4     |
| 168 | Proyecto: continuidad activa-pasiva entre nubes    | capstone      | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.

# 157 — Motivaciones y anti-patrones de multi-cloud

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: decision · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar motivaciones y anti-patrones de multi-cloud dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar motivaciones y anti-patrones de multi-cloud con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar decision-multicloud con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                              |
|--------------|------------------------------------------------------------------------------------------------------|
| motivaciones | Define su papel en motivaciones y anti-patrones de multi-cloud y cómo observarlo en un sistema real. |
| anti         | Define su papel en motivaciones y anti-patrones de multi-cloud y cómo observarlo en un sistema real. |
| patrones     | Define su papel en motivaciones y anti-patrones de multi-cloud y cómo observarlo en un sistema real. |
| multi        | Define su papel en motivaciones y anti-patrones de multi-cloud y cómo observarlo en un sistema real. |
| cloud        | Define su papel en motivaciones y anti-patrones de multi-cloud y cómo observarlo en un sistema real. |

## Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Motivaciones y anti-patrones de multi-cloud"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar motivaciones y anti-patrones de multi-cloud. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/157-motivaciones-y-anti-patrones-de-multi-cloud/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa decision-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye decision-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 157 Motivaciones y anti-patrones de multi-cloud

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega decision-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 158 — Portabilidad, capas de abstracción y lock-in

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar portabilidad, capas de abstracción y lock-in dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar portabilidad, capas de abstracción y lock-in con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-portabilidad con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                               |
|--------------|-------------------------------------------------------------------------------------------------------|
| portabilidad | Define su papel en portabilidad, capas de abstracción y lock-in y cómo observarlo en un sistema real. |
| capas        | Define su papel en portabilidad, capas de abstracción y lock-in y cómo observarlo en un sistema real. |
| abstracción  | Define su papel en portabilidad, capas de abstracción y lock-in y cómo observarlo en un sistema real. |
| lock         | Define su papel en portabilidad, capas de abstracción y lock-in y cómo observarlo en un sistema real. |
| in           | Define su papel en portabilidad, capas de abstracción y lock-in y cómo observarlo en un sistema real. |

## Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Portabilidad, capas de abstracción y lock-in"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar portabilidad, capas de abstracción y lock-in. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/158-portabilidad-capas-de-abstraccion-y-lock-in/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa matriz-portabilidad en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye matriz-portabilidad para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 158 Portabilidad, capas de abstracción y lock-in

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega matriz-portabilidad con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 159 — Federación de identidad entre nubes

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar federación de identidad entre nubes dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar federación de identidad entre nubes con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar federacion-multicloud con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                      |
|------------|----------------------------------------------------------------------------------------------|
| federación | Define su papel en federación de identidad entre nubes y cómo observarlo en un sistema real. |
| identidad  | Define su papel en federación de identidad entre nubes y cómo observarlo en un sistema real. |
| entre      | Define su papel en federación de identidad entre nubes y cómo observarlo en un sistema real. |
| nubes      | Define su papel en federación de identidad entre nubes y cómo observarlo en un sistema real. |

## Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Federación de identidad entre nubes"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar federación de identidad entre nubes. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/159-federacion-de-identidad-entre-nubes/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa federacion-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye federación-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 159 Federación de identidad entre nubes

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega federacion-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 160 — Conectividad, tránsito, DNS y service discovery

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar conectividad, tránsito, dns y service discovery dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar conectividad, tránsito, dns y service discovery con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar red-multicloud con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                  |
|--------------|----------------------------------------------------------------------------------------------------------|
| conectividad | Define su papel en conectividad, tránsito, dns y service discovery y cómo observarlo en un sistema real. |
| tránsito     | Define su papel en conectividad, tránsito, dns y service discovery y cómo observarlo en un sistema real. |
| dns          | Define su papel en conectividad, tránsito, dns y service discovery y cómo observarlo en un sistema real. |
| service      | Define su papel en conectividad, tránsito, dns y service discovery y cómo observarlo en un sistema real. |
| discovery    | Define su papel en conectividad, tránsito, dns y service discovery y cómo observarlo en un sistema real. |

## Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Conectividad, tránsito, DNS y service discovery"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar conectividad, tránsito, dns y service discovery. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/160-conectividad-transito-dns-y-service-discovery/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa red-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye red-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 160 Conectividad, tránsito, DNS y service discovery

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega red-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 161 — Replicación de datos, soberanía y costos de egress

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar replicación de datos, soberanía y costos de egress dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar replicación de datos, soberanía y costos de egress con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar estrategia-datos-multicloud con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                     |
|-------------|-------------------------------------------------------------------------------------------------------------|
| replicación | Define su papel en replicación de datos, soberanía y costos de egress y cómo observarlo en un sistema real. |
| datos       | Define su papel en replicación de datos, soberanía y costos de egress y cómo observarlo en un sistema real. |
| soberanía   | Define su papel en replicación de datos, soberanía y costos de egress y cómo observarlo en un sistema real. |
| costos      | Define su papel en replicación de datos, soberanía y costos de egress y cómo observarlo en un sistema real. |
| egress      | Define su papel en replicación de datos, soberanía y costos de egress y cómo observarlo en un sistema real. |

## Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Replicación de datos, soberanía y costos de egress"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar replicación de datos, soberanía y costos de egress. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/161-replicacion-de-datos-soberania-y-costos-de-egress/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa estrategia-datos-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye estrategia-datos-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 161 Replicación de datos, soberanía y costos de egress

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega estrategia-datos-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 162 — Observabilidad y operación entre proveedores

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar observabilidad y operación entre proveedores dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar observabilidad y operación entre proveedores con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar telemetría-multicloud con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                               |
|----------------|-------------------------------------------------------------------------------------------------------|
| observabilidad | Define su papel en observabilidad y operación entre proveedores y cómo observarlo en un sistema real. |
| operación      | Define su papel en observabilidad y operación entre proveedores y cómo observarlo en un sistema real. |
| entre          | Define su papel en observabilidad y operación entre proveedores y cómo observarlo en un sistema real. |
| proveedores    | Define su papel en observabilidad y operación entre proveedores y cómo observarlo en un sistema real. |

## Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Observabilidad y operación entre proveedores"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar observabilidad y operación entre proveedores. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/162-observabilidad-y-operacion-entre-proveedores/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa telemetría-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye telemetría-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 162 Observabilidad y operación entre proveedores

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega telemetria-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 163 — Terraform multi-provider y separación de estados

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar terraform multi-provider y separación de estados dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar terraform multi-provider y separación de estados con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar iac-multicloud con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                   |
|------------|-----------------------------------------------------------------------------------------------------------|
| terraform  | Define su papel en terraform multi-provider y separación de estados y cómo observarlo en un sistema real. |
| multi      | Define su papel en terraform multi-provider y separación de estados y cómo observarlo en un sistema real. |
| provider   | Define su papel en terraform multi-provider y separación de estados y cómo observarlo en un sistema real. |
| separación | Define su papel en terraform multi-provider y separación de estados y cómo observarlo en un sistema real. |
| estados    | Define su papel en terraform multi-provider y separación de estados y cómo observarlo en un sistema real. |

## Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Terraform multi-provider y separación de estados"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar terraform multi-provider y separación de estados. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/163-terraform-multi-provider-y-separacion-de-estados/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa iac-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye iac-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 163 Terraform multi-provider y separación de estados

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega iac-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 164 — Flotas Kubernetes y políticas comunes

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: kubernetes · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar flotas kubernetes y políticas comunes dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar flotas kubernetes y políticas comunes con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar fleet-kubernetes con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                        |
|------------|------------------------------------------------------------------------------------------------|
| flotas     | Define su papel en flotas kubernetes y políticas comunes y cómo observarlo en un sistema real. |
| kubernetes | Define su papel en flotas kubernetes y políticas comunes y cómo observarlo en un sistema real. |
| políticas  | Define su papel en flotas kubernetes y políticas comunes y cómo observarlo en un sistema real. |
| comunes    | Define su papel en flotas kubernetes y políticas comunes y cómo observarlo en un sistema real. |

### Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Flotas Kubernetes y políticas comunes"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar flotas kubernetes y políticas comunes. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/164-flotas-kubernetes-y-politicas-comunes/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa fleet-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye fleet-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 164 Flotas Kubernetes y políticas comunes

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega fleet-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 165 — Nube híbrida, edge y conectividad privada

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: hybrid · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar nube híbrida, edge y conectividad privada dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar nube híbrida, edge y conectividad privada con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar arquitectura-híbrida con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                            |
|--------------|----------------------------------------------------------------------------------------------------|
| nube         | Define su papel en nube híbrida, edge y conectividad privada y cómo observarlo en un sistema real. |
| híbrida      | Define su papel en nube híbrida, edge y conectividad privada y cómo observarlo en un sistema real. |
| edge         | Define su papel en nube híbrida, edge y conectividad privada y cómo observarlo en un sistema real. |
| conectividad | Define su papel en nube híbrida, edge y conectividad privada y cómo observarlo en un sistema real. |
| privada      | Define su papel en nube híbrida, edge y conectividad privada y cómo observarlo en un sistema real. |

## Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Nube híbrida, edge y conectividad privada"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar nube híbrida, edge y conectividad privada. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/165-nube-hibrida-edge-y-conectividad-privada/lab.py
```

El laboratorio selecciona el motor de práctica hybrid y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa arquitectura-hibrida en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una topología híbrida con dependencia y modo degradado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye arquitectura-hibrida para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 165 Nube híbrida, edge y conectividad privada

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega arquitectura-híbrida con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 166 — Backup, RTO, RPO y patrones de disaster recovery

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: reliability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar backup, rto, rpo y patrones de disaster recovery dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar backup, rto, rpo y patrones de disaster recovery con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plan-dr con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                                   |
|----------|-----------------------------------------------------------------------------------------------------------|
| backup   | Define su papel en backup, rto, rpo y patrones de disaster recovery y cómo observarlo en un sistema real. |
| rto      | Define su papel en backup, rto, rpo y patrones de disaster recovery y cómo observarlo en un sistema real. |
| rpo      | Define su papel en backup, rto, rpo y patrones de disaster recovery y cómo observarlo en un sistema real. |
| patrones | Define su papel en backup, rto, rpo y patrones de disaster recovery y cómo observarlo en un sistema real. |
| disaster | Define su papel en backup, rto, rpo y patrones de disaster recovery y cómo observarlo en un sistema real. |
| recovery | Define su papel en backup, rto, rpo y patrones de disaster recovery y cómo observarlo en un sistema real. |

## Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Backup, RTO, RPO y patrones de disaster recovery"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar backup, rto, rpo y patrones de disaster recovery. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/166-backup-rto-rpo-y-patrones-de-disaster-recovery/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa plan-dr en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye plan-dr para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 166 Backup, RTO, RPO y patrones de disaster recovery

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plan-dr con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 167 — Las 7R de migración y oleadas

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: migration · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar las 7r de migración y oleadas dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar las 7r de migración y oleadas con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar roadmap-migracion con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                |
|-----------|----------------------------------------------------------------------------------------|
| 7r        | Define su papel en las 7r de migración y oleadas y cómo observarlo en un sistema real. |
| migración | Define su papel en las 7r de migración y oleadas y cómo observarlo en un sistema real. |
| oleadas   | Define su papel en las 7r de migración y oleadas y cómo observarlo en un sistema real. |

## Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Las 7R de migración y oleadas"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar las 7r de migración y oleadas. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/167-las-7r-de-migracion-y-oleadas/lab.py
```

El laboratorio selecciona el motor de práctica migration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa roadmap-migracion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un inventario, dependencias, riesgo y oleadas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye roadmap-migración para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 167 Las 7R de migración y oleadas

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega roadmap-migración con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 168 — Proyecto: continuidad activa-pasiva entre nubes

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar proyecto: continuidad activa-pasiva entre nubes dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: continuidad activa-pasiva entre nubes con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-multicloud con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                  |
|-------------|----------------------------------------------------------------------------------------------------------|
| proyecto    | Define su papel en proyecto: continuidad activa-pasiva entre nubes y cómo observarlo en un sistema real. |
| continuidad | Define su papel en proyecto: continuidad activa-pasiva entre nubes y cómo observarlo en un sistema real. |
| activa      | Define su papel en proyecto: continuidad activa-pasiva entre nubes y cómo observarlo en un sistema real. |
| pasiva      | Define su papel en proyecto: continuidad activa-pasiva entre nubes y cómo observarlo en un sistema real. |
| entre       | Define su papel en proyecto: continuidad activa-pasiva entre nubes y cómo observarlo en un sistema real. |
| nubes       | Define su papel en proyecto: continuidad activa-pasiva entre nubes y cómo observarlo en un sistema real. |

## Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Proyecto: continuidad activa-pasiva entre nubes"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: continuidad activa-pasiva entre nubes. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/168-proyecto-continuidad-activa-pasiva-entre-nubes/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa plataforma-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye plataforma-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 168 Proyecto: continuidad activa-pasiva entre nubes

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega plataforma-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 14 — Plataformas avanzadas, capstones y carrera

← Parte 13 · Índice completo · Parte 15 →

Nivel: experto-frontera · Clases: 12 · Duración sugerida: 6–8 semanas

Integrar tecnología, operación, gobierno y comunicación en evidencia profesional.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

## Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                            | Laboratorio | Horas |
|-----|--------------------------------------------------|-------------|-------|
| 169 | Landing zones empresariales y vending de cuentas | governance  | 4     |
| 170 | Gobierno federado y policy as code a escala      | governance  | 4     |
| 171 | Platform as a Product y roadmap de capacidades   | platform    | 4     |
| 172 | Modelo operativo FinOps y economía unitaria      | finops      | 4     |
| 173 | Madurez SRE y confiabilidad organizacional       | sre         | 4     |
| 174 | Arquitectura de seguridad cloud empresarial      | security    | 4     |
| 175 | Workloads de IA, GPU, datos y MLOps multi-cloud  | ai          | 4     |
| 176 | Edge, IoT y procesamiento desconectado           | edge        | 4     |
| 177 | Soberanía digital y confidential computing       | security    | 4     |
| 178 | Capstone: descubrimiento y diseño                | capstone    | 8     |
| 179 | Capstone: implementación y operación             | capstone    | 8     |
| 180 | Capstone: defensa, portafolio y plan profesional | capstone    | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.

# 169 — Landing zones empresariales y vending de cuentas

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar landing zones empresariales y vending de cuentas dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar landing zones empresariales y vending de cuentas con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar landing-zone-empresarial con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                                   |
|---------------|-----------------------------------------------------------------------------------------------------------|
| landing       | Define su papel en landing zones empresariales y vending de cuentas y cómo observarlo en un sistema real. |
| zones         | Define su papel en landing zones empresariales y vending de cuentas y cómo observarlo en un sistema real. |
| empresariales | Define su papel en landing zones empresariales y vending de cuentas y cómo observarlo en un sistema real. |
| vending       | Define su papel en landing zones empresariales y vending de cuentas y cómo observarlo en un sistema real. |
| cuentas       | Define su papel en landing zones empresariales y vending de cuentas y cómo observarlo en un sistema real. |

## Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Landing zones empresariales y vending de cuentas"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar landing zones empresariales y vending de cuentas. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/169-landing-zones-empresariales-y-vending-de-cuentas/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa landing-zone-empresarial en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye landing-zone-empresarial para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 169 Landing zones empresariales y vending de cuentas

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega landing-zone-empresarial con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 170 — Gobierno federado y policy as code a escala

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar gobierno federado y policy as code a escala dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar gobierno federado y policy as code a escala con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-gobierno con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                              |
|----------|------------------------------------------------------------------------------------------------------|
| gobierno | Define su papel en gobierno federado y policy as code a escala y cómo observarlo en un sistema real. |
| federado | Define su papel en gobierno federado y policy as code a escala y cómo observarlo en un sistema real. |
| policy   | Define su papel en gobierno federado y policy as code a escala y cómo observarlo en un sistema real. |
| as       | Define su papel en gobierno federado y policy as code a escala y cómo observarlo en un sistema real. |
| code     | Define su papel en gobierno federado y policy as code a escala y cómo observarlo en un sistema real. |
| escala   | Define su papel en gobierno federado y policy as code a escala y cómo observarlo en un sistema real. |

## Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Gobierno federado y policy as code a escala"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar gobierno federado y policy as code a escala. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/170-gobierno-federado-y-policy-as-code-a-escala/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa modelo-gobierno en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye modelo-gobierno para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 170 Gobierno federado y policy as code a escala

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega modelo-gobierno con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 171 — Platform as a Product y roadmap de capacidades

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: platform · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar platform as a product y roadmap de capacidades dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar platform as a product y roadmap de capacidades con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar roadmap-plataforma con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                 |
|-------------|---------------------------------------------------------------------------------------------------------|
| platform    | Define su papel en platform as a product y roadmap de capacidades y cómo observarlo en un sistema real. |
| as          | Define su papel en platform as a product y roadmap de capacidades y cómo observarlo en un sistema real. |
| product     | Define su papel en platform as a product y roadmap de capacidades y cómo observarlo en un sistema real. |
| roadmap     | Define su papel en platform as a product y roadmap de capacidades y cómo observarlo en un sistema real. |
| capacidades | Define su papel en platform as a product y roadmap de capacidades y cómo observarlo en un sistema real. |

## Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Platform as a Product y roadmap de capacidades"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar platform as a product y roadmap de capacidades. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/171-platform-as-a-product-y-roadmap-de-capacidades/lab.py
```

El laboratorio selecciona el motor de práctica platform y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa roadmap-plataforma en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una capacidad autoservicio con contrato y golden path. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye roadmap-plataforma para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 171 Platform as a Product y roadmap de capacidades

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega roadmap-plataforma con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 172 — Modelo operativo FinOps y economía unitaria

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: finops · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar modelo operativo finops y economía unitaria dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar modelo operativo finops y economía unitaria con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar operating-model-finops con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                              |
|-----------|------------------------------------------------------------------------------------------------------|
| modelo    | Define su papel en modelo operativo finops y economía unitaria y cómo observarlo en un sistema real. |
| operativo | Define su papel en modelo operativo finops y economía unitaria y cómo observarlo en un sistema real. |
| finops    | Define su papel en modelo operativo finops y economía unitaria y cómo observarlo en un sistema real. |
| economía  | Define su papel en modelo operativo finops y economía unitaria y cómo observarlo en un sistema real. |
| unitaria  | Define su papel en modelo operativo finops y economía unitaria y cómo observarlo en un sistema real. |

## Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Modelo operativo FinOps y economía unitaria"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar modelo operativo finops y economía unitaria. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/172-modelo-operativo-finops-y-economia-unitaria/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa operating-model-finops en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye operating-model-finops para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 172 Modelo operativo FinOps y economía unitaria

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega operating-model-finops con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 173 — Madurez SRE y confiabilidad organizacional

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: sre · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar madurez sre y confiabilidad organizacional dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar madurez sre y confiabilidad organizacional con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar assessment-sre con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------|
| madurez        | Define su papel en madurez sre y confiabilidad organizacional y cómo observarlo en un sistema real. |
| sre            | Define su papel en madurez sre y confiabilidad organizacional y cómo observarlo en un sistema real. |
| confiabilidad  | Define su papel en madurez sre y confiabilidad organizacional y cómo observarlo en un sistema real. |
| organizacional | Define su papel en madurez sre y confiabilidad organizacional y cómo observarlo en un sistema real. |

## Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Madurez SRE y confiabilidad organizacional"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar madurez sre y confiabilidad organizacional. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/173-madurez-sre-y-confiabilidad-organizacional/lab.py
```

El laboratorio selecciona el motor de práctica sre y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa assessment-sre en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un SLO con presupuesto de error y política de acción. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye assessment-sre para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 173 Madurez SRE y confiabilidad organizacional

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega assessment-sre con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 174 — Arquitectura de seguridad cloud empresarial

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar arquitectura de seguridad cloud empresarial dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar arquitectura de seguridad cloud empresarial con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar security-reference-architecture con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                              |
|--------------|------------------------------------------------------------------------------------------------------|
| arquitectura | Define su papel en arquitectura de seguridad cloud empresarial y cómo observarlo en un sistema real. |
| seguridad    | Define su papel en arquitectura de seguridad cloud empresarial y cómo observarlo en un sistema real. |
| cloud        | Define su papel en arquitectura de seguridad cloud empresarial y cómo observarlo en un sistema real. |
| empresarial  | Define su papel en arquitectura de seguridad cloud empresarial y cómo observarlo en un sistema real. |

## Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Arquitectura de seguridad cloud empresarial"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar arquitectura de seguridad cloud empresarial. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/174-arquitectura-de-seguridad-cloud-empresarial/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa security-reference-architecture en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye security-reference-architecture para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 174 Arquitectura de seguridad cloud empresarial

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega security-reference-architecture con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 175 — Workloads de IA, GPU, datos y MLOps multi-cloud

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: ai · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar workloads de ia, gpu, datos y mlops multi-cloud dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar workloads de ia, gpu, datos y mlops multi-cloud con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar arquitectura-ia-cloud con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                                  |
|-----------|----------------------------------------------------------------------------------------------------------|
| workloads | Define su papel en workloads de ia, gpu, datos y mlops multi-cloud y cómo observarlo en un sistema real. |
| ia        | Define su papel en workloads de ia, gpu, datos y mlops multi-cloud y cómo observarlo en un sistema real. |
| gpu       | Define su papel en workloads de ia, gpu, datos y mlops multi-cloud y cómo observarlo en un sistema real. |
| datos     | Define su papel en workloads de ia, gpu, datos y mlops multi-cloud y cómo observarlo en un sistema real. |
| mlops     | Define su papel en workloads de ia, gpu, datos y mlops multi-cloud y cómo observarlo en un sistema real. |
| multi     | Define su papel en workloads de ia, gpu, datos y mlops multi-cloud y cómo observarlo en un sistema real. |

## Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Workloads de IA, GPU, datos y MLOps multi-cloud"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar workloads de ia, gpu, datos y mlops multi-cloud. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/175-workloads-de-ia-gpu-datos-y-mlops-multi-cloud/lab.py
```

El laboratorio selecciona el motor de práctica ai y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa arquitectura-ia-cloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una arquitectura de IA con datos, cómputo, costo y seguridad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye arquitectura-ia-cloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 175 Workloads de IA, GPU, datos y MLOps multi-cloud

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega arquitectura-ia-cloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 176 — Edge, IoT y procesamiento desconectado

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: edge · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar edge, iot y procesamiento desconectado dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar edge, iot y procesamiento desconectado con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar arquitectura-edge con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                         |
|---------------|-------------------------------------------------------------------------------------------------|
| edge          | Define su papel en edge, iot y procesamiento desconectado y cómo observarlo en un sistema real. |
| iot           | Define su papel en edge, iot y procesamiento desconectado y cómo observarlo en un sistema real. |
| procesamiento | Define su papel en edge, iot y procesamiento desconectado y cómo observarlo en un sistema real. |
| desconectado  | Define su papel en edge, iot y procesamiento desconectado y cómo observarlo en un sistema real. |

## Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Edge, IoT y procesamiento desconectado"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar edge, iot y procesamiento desconectado. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/176-edge-iot-y-procesamiento-desconectado/lab.py
```

El laboratorio selecciona el motor de práctica edge y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa arquitectura-edge en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un flujo edge que tolera desconexión y sincroniza estado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye arquitectura-edge para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 176 Edge, IoT y procesamiento desconectado

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega arquitectura-edge con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 177 — Soberanía digital y confidential computing

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar soberanía digital y confidential computing dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar soberanía digital y confidential computing con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar decision-soberania con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                             |
|--------------|-----------------------------------------------------------------------------------------------------|
| soberanía    | Define su papel en soberanía digital y confidential computing y cómo observarlo en un sistema real. |
| digital      | Define su papel en soberanía digital y confidential computing y cómo observarlo en un sistema real. |
| confidential | Define su papel en soberanía digital y confidential computing y cómo observarlo en un sistema real. |
| computing    | Define su papel en soberanía digital y confidential computing y cómo observarlo en un sistema real. |

## Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Soberanía digital y confidential computing"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar soberanía digital y confidential computing. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/177-soberania-digital-y-confidential-computing/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa decision-soberania en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye decision-soberania para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 177 Soberanía digital y confidential computing

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega decision-soberania con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 178 — Capstone: descubrimiento y diseño

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar capstone: descubrimiento y diseño dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone: descubrimiento y diseño con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar dossier-diseno-final con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                    |
|----------------|--------------------------------------------------------------------------------------------|
| capstone       | Define su papel en capstone: descubrimiento y diseño y cómo observarlo en un sistema real. |
| descubrimiento | Define su papel en capstone: descubrimiento y diseño y cómo observarlo en un sistema real. |
| diseño         | Define su papel en capstone: descubrimiento y diseño y cómo observarlo en un sistema real. |

## Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone: descubrimiento y diseño"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone: descubrimiento y diseño. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/178-capstone-descubrimiento-y-diseno/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa dossier-diseno-final en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye dossier-diseno-final para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 178 Capstone: descubrimiento y diseño

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega dossier-diseño-final con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 179 — Capstone: implementación y operación

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar capstone: implementación y operación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone: implementación y operación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar evidencia-implementacion-final con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                       |
|----------------|-----------------------------------------------------------------------------------------------|
| capstone       | Define su papel en capstone: implementación y operación y cómo observarlo en un sistema real. |
| implementación | Define su papel en capstone: implementación y operación y cómo observarlo en un sistema real. |
| operación      | Define su papel en capstone: implementación y operación y cómo observarlo en un sistema real. |

## Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone: implementación y operación"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone: implementación y operación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/179-capstone-implementacion-y-operacion/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa evidencia-implementacion-final en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye evidencia-implementacion-final para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 179 Capstone: implementación y operación

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega evidencia-implementacion-final con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 180 — Capstone: defensa, portafolio y plan profesional

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar capstone: defensa, portafolio y plan profesional dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone: defensa, portafolio y plan profesional con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar defensa-final con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                   |
|-------------|-----------------------------------------------------------------------------------------------------------|
| capstone    | Define su papel en capstone: defensa, portafolio y plan profesional y cómo observarlo en un sistema real. |
| defensa     | Define su papel en capstone: defensa, portafolio y plan profesional y cómo observarlo en un sistema real. |
| portafolio  | Define su papel en capstone: defensa, portafolio y plan profesional y cómo observarlo en un sistema real. |
| plan        | Define su papel en capstone: defensa, portafolio y plan profesional y cómo observarlo en un sistema real. |
| profesional | Define su papel en capstone: defensa, portafolio y plan profesional y cómo observarlo en un sistema real. |

## Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone: defensa, portafolio y plan profesional"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone: defensa, portafolio y plan profesional. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/180-capstone-defensa-portafolio-y-plan-profesional/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa defensa-final en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye defensa-final para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 180 Capstone: defensa, portafolio y plan profesional

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega defensa-final con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 15 — Arquitectura de sistemas e ingeniería de requisitos

← Parte 14 · Índice completo · Parte 16 →

Nivel: intermedio-avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Pasar de requisitos ambiguos a arquitecturas comprobables antes de elegir servicios cloud.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

## Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                        | Laboratorio  | Horas |
|-----|--------------------------------------------------------------|--------------|-------|
| 181 | Requisitos funcionales, restricciones y atributos de calidad | architecture | 4     |
| 182 | Contexto, contenedores, componentes y código con C4          | architecture | 4     |
| 183 | Acoplamiento, cohesión, modularidad y fronteras              | architecture | 4     |
| 184 | Arquitectura monolítica, modular y de microservicios         | decision     | 4     |
| 185 | Disponibilidad, confiabilidad y análisis de puntos de fallo  | reliability  | 4     |
| 186 | Capacidad, latencia, throughput y teoría de colas            | performance  | 4     |
| 187 | Consistencia, particiones, relojes y consenso                | distributed  | 4     |
| 188 | Contratos de API, eventos y compatibilidad evolutiva         | api          | 4     |
| 189 | Modelado de amenazas y arquitectura de confianza cero        | security     | 4     |
| 190 | ADRs, fitness functions y gobierno de decisiones             | architecture | 4     |
| 191 | Architecture review y comunicación con stakeholders          | governance   | 4     |
| 192 | Proyecto: arquitectura completa de CloudShop                 | capstone     | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.

# 181 — Requisitos funcionales, restricciones y atributos de calidad

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: architecture · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar requisitos funcionales, restricciones y atributos de calidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar requisitos funcionales, restricciones y atributos de calidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar quality-attribute-scenarios con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                                               |
|---------------|-----------------------------------------------------------------------------------------------------------------------|
| requisitos    | Define su papel en requisitos funcionales, restricciones y atributos de calidad y cómo observarlo en un sistema real. |
| funcionales   | Define su papel en requisitos funcionales, restricciones y atributos de calidad y cómo observarlo en un sistema real. |
| restricciones | Define su papel en requisitos funcionales, restricciones y atributos de calidad y cómo observarlo en un sistema real. |
| atributos     | Define su papel en requisitos funcionales, restricciones y atributos de calidad y cómo observarlo en un sistema real. |
| calidad       | Define su papel en requisitos funcionales, restricciones y atributos de calidad y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Requisitos funcionales, restricciones y atributos de calidad"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar requisitos funcionales, restricciones y atributos de calidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/181-requisitos-funcionales-restricciones-y-atributos-de-calidad/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa quality-attribute-scenarios en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye quality-attribute-scenarios para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 181 Requisitos funcionales, restricciones y atributos de calidad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega quality-attribute-scenarios con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 182 — Contexto, contenedores, componentes y código con C4

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: architecture · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar contexto, contenedores, componentes y código con c4 dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar contexto, contenedores, componentes y código con c4 con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar c4-model con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                      |
|--------------|--------------------------------------------------------------------------------------------------------------|
| contexto     | Define su papel en contexto, contenedores, componentes y código con c4 y cómo observarlo en un sistema real. |
| contenedores | Define su papel en contexto, contenedores, componentes y código con c4 y cómo observarlo en un sistema real. |
| componentes  | Define su papel en contexto, contenedores, componentes y código con c4 y cómo observarlo en un sistema real. |
| código       | Define su papel en contexto, contenedores, componentes y código con c4 y cómo observarlo en un sistema real. |
| c4           | Define su papel en contexto, contenedores, componentes y código con c4 y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Contexto, contenedores, componentes y código con C4"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar contexto, contenedores, componentes y código con c4. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/182-contexto-contenedores-componentes-y-codigo-con-c4/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa c4-model en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye c4-model para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 182 Contexto, contenedores, componentes y código con C4

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega c4-model con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 183 — Acoplamiento, cohesión, modularidad y fronteras

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: architecture · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar acoplamiento, cohesión, modularidad y fronteras dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar acoplamiento, cohesión, modularidad y fronteras con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar module-map con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                  |
|--------------|----------------------------------------------------------------------------------------------------------|
| acoplamiento | Define su papel en acoplamiento, cohesión, modularidad y fronteras y cómo observarlo en un sistema real. |
| cohesión     | Define su papel en acoplamiento, cohesión, modularidad y fronteras y cómo observarlo en un sistema real. |
| modularidad  | Define su papel en acoplamiento, cohesión, modularidad y fronteras y cómo observarlo en un sistema real. |
| fronteras    | Define su papel en acoplamiento, cohesión, modularidad y fronteras y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Acoplamiento, cohesión, modularidad y fronteras"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar acoplamiento, cohesión, modularidad y fronteras. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/183-acoplamiento-cohesion-modularidad-y-fronteras/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa module-map en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye module-map para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 183 Acoplamiento, cohesión, modularidad y fronteras

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega module-map con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 184 — Arquitectura monolítica, modular y de microservicios

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: decision · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar arquitectura monolítica, modular y de microservicios dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar arquitectura monolítica, modular y de microservicios con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar architecture-style-adr con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                                       |
|----------------|---------------------------------------------------------------------------------------------------------------|
| arquitectura   | Define su papel en arquitectura monolítica, modular y de microservicios y cómo observarlo en un sistema real. |
| monolítica     | Define su papel en arquitectura monolítica, modular y de microservicios y cómo observarlo en un sistema real. |
| modular        | Define su papel en arquitectura monolítica, modular y de microservicios y cómo observarlo en un sistema real. |
| microservicios | Define su papel en arquitectura monolítica, modular y de microservicios y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Arquitectura monolítica, modular y de microservicios"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar arquitectura monolítica, modular y de microservicios. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/184-arquitectura-monolitica-modular-y-de-microservicios/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa architecture-style-adr en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye architecture-style-adr para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 184 Arquitectura monolítica, modular y de microservicios

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega architecture-style-adr con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 185 — Disponibilidad, confiabilidad y análisis de puntos de fallo

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: reliability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar disponibilidad, confiabilidad y análisis de puntos de fallo dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar disponibilidad, confiabilidad y análisis de puntos de fallo con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar failure-mode-analysis con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                                              |
|----------------|----------------------------------------------------------------------------------------------------------------------|
| disponibilidad | Define su papel en disponibilidad, confiabilidad y análisis de puntos de fallo y cómo observarlo en un sistema real. |
| confiabilidad  | Define su papel en disponibilidad, confiabilidad y análisis de puntos de fallo y cómo observarlo en un sistema real. |
| análisis       | Define su papel en disponibilidad, confiabilidad y análisis de puntos de fallo y cómo observarlo en un sistema real. |
| puntos         | Define su papel en disponibilidad, confiabilidad y análisis de puntos de fallo y cómo observarlo en un sistema real. |
| fallo          | Define su papel en disponibilidad, confiabilidad y análisis de puntos de fallo y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Disponibilidad, confiabilidad y análisis de puntos de fallo"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar disponibilidad, confiabilidad y análisis de puntos de fallo. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/185-disponibilidad-confiabilidad-y-analisis-de-puntos-de-fallo/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa failure-mode-analysis en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye failure-mode-analysis para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 185 Disponibilidad, confiabilidad y análisis de puntos de fallo

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega failure-mode-analysis con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 186 — Capacidad, latencia, throughput y teoría de colas

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: performance · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar capacidad, latencia, throughput y teoría de colas dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capacidad, latencia, throughput y teoría de colas con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar capacity-model con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                    |
|------------|------------------------------------------------------------------------------------------------------------|
| capacidad  | Define su papel en capacidad, latencia, throughput y teoría de colas y cómo observarlo en un sistema real. |
| latencia   | Define su papel en capacidad, latencia, throughput y teoría de colas y cómo observarlo en un sistema real. |
| throughput | Define su papel en capacidad, latencia, throughput y teoría de colas y cómo observarlo en un sistema real. |
| teoría     | Define su papel en capacidad, latencia, throughput y teoría de colas y cómo observarlo en un sistema real. |
| colas      | Define su papel en capacidad, latencia, throughput y teoría de colas y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capacidad, latencia, throughput y teoría de colas"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capacidad, latencia, throughput y teoría de colas. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/186-capacidad-latencia-throughput-y-teoria-de-colas/lab.py
```

El laboratorio selecciona el motor de práctica performance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa capacity-model en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una prueba de carga con baseline y cuello de botella. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye capacity-model para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 186 Capacidad, latencia, throughput y teoría de colas

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega capacity-model con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 187 — Consistencia, particiones, relojes y consenso

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: distributed · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar consistencia, particiones, relojes y consenso dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar consistencia, particiones, relojes y consenso con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar consistency-model con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                |
|--------------|--------------------------------------------------------------------------------------------------------|
| consistencia | Define su papel en consistencia, particiones, relojes y consenso y cómo observarlo en un sistema real. |
| particiones  | Define su papel en consistencia, particiones, relojes y consenso y cómo observarlo en un sistema real. |
| relojes      | Define su papel en consistencia, particiones, relojes y consenso y cómo observarlo en un sistema real. |
| consenso     | Define su papel en consistencia, particiones, relojes y consenso y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Consistencia, particiones, relojes y consenso"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar consistencia, particiones, relojes y consenso. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/187-consistencia-particiones-relojes-y-consenso/lab.py
```

El laboratorio selecciona el motor de práctica distributed y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa consistency-model en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una traza de consistencia, reintento o fallo parcial. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye consistency-model para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 187 Consistencia, particiones, relojes y consenso

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega consistency-model con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 188 — Contratos de API, eventos y compatibilidad evolutiva

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: api · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar contratos de api, eventos y compatibilidad evolutiva dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar contratos de api, eventos y compatibilidad evolutiva con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar contract-evolution con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                                       |
|----------------|---------------------------------------------------------------------------------------------------------------|
| contratos      | Define su papel en contratos de api, eventos y compatibilidad evolutiva y cómo observarlo en un sistema real. |
| api            | Define su papel en contratos de api, eventos y compatibilidad evolutiva y cómo observarlo en un sistema real. |
| eventos        | Define su papel en contratos de api, eventos y compatibilidad evolutiva y cómo observarlo en un sistema real. |
| compatibilidad | Define su papel en contratos de api, eventos y compatibilidad evolutiva y cómo observarlo en un sistema real. |
| evolutiva      | Define su papel en contratos de api, eventos y compatibilidad evolutiva y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Contratos de API, eventos y compatibilidad evolutiva"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar contratos de api, eventos y compatibilidad evolutiva. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/188-contratos-de-api-eventos-y-compatibilidad-evolutiva/lab.py
```

El laboratorio selecciona el motor de práctica api y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa contract-evolution en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un contrato versionado con pruebas positivas y negativas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye contract-evolution para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 188 Contratos de API, eventos y compatibilidad evolutiva

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega contract-evolution con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 189 — Modelado de amenazas y arquitectura de confianza cero

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: security · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar modelado de amenazas y arquitectura de confianza cero dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar modelado de amenazas y arquitectura de confianza cero con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar system-threat-model con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                        |
|--------------|----------------------------------------------------------------------------------------------------------------|
| modelado     | Define su papel en modelado de amenazas y arquitectura de confianza cero y cómo observarlo en un sistema real. |
| amenazas     | Define su papel en modelado de amenazas y arquitectura de confianza cero y cómo observarlo en un sistema real. |
| arquitectura | Define su papel en modelado de amenazas y arquitectura de confianza cero y cómo observarlo en un sistema real. |
| confianza    | Define su papel en modelado de amenazas y arquitectura de confianza cero y cómo observarlo en un sistema real. |
| cero         | Define su papel en modelado de amenazas y arquitectura de confianza cero y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Modelado de amenazas y arquitectura de confianza cero"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar modelado de amenazas y arquitectura de confianza cero. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/189-modelado-de-amenazas-y-arquitectura-de-confianza-cero/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa system-threat-model en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye system-threat-model para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 189 Modelado de amenazas y arquitectura de confianza cero

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega system-threat-model con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 190 — ADRs, fitness functions y gobierno de decisiones

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: architecture · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar adrs, fitness functions y gobierno de decisiones dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar adrs, fitness functions y gobierno de decisiones con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar adr-library con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                   |
|------------|-----------------------------------------------------------------------------------------------------------|
| adrs       | Define su papel en adrs, fitness functions y gobierno de decisiones y cómo observarlo en un sistema real. |
| fitness    | Define su papel en adrs, fitness functions y gobierno de decisiones y cómo observarlo en un sistema real. |
| functions  | Define su papel en adrs, fitness functions y gobierno de decisiones y cómo observarlo en un sistema real. |
| gobierno   | Define su papel en adrs, fitness functions y gobierno de decisiones y cómo observarlo en un sistema real. |
| decisiones | Define su papel en adrs, fitness functions y gobierno de decisiones y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: ADRs, fitness functions y gobierno de decisiones"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar adrs, fitness functions y gobierno de decisiones. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/190-adrs-fitness-functions-y-gobierno-de-decisiones/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa adr-library en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye adr-library para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 190 ADRs, fitness functions y gobierno de decisiones

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega adr-library con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 191 — Architecture review y comunicación con stakeholders

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: governance · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar architecture review y comunicación con stakeholders dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar architecture review y comunicación con stakeholders con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar architecture-review con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                      |
|--------------|--------------------------------------------------------------------------------------------------------------|
| architecture | Define su papel en architecture review y comunicación con stakeholders y cómo observarlo en un sistema real. |
| review       | Define su papel en architecture review y comunicación con stakeholders y cómo observarlo en un sistema real. |
| comunicación | Define su papel en architecture review y comunicación con stakeholders y cómo observarlo en un sistema real. |
| stakeholders | Define su papel en architecture review y comunicación con stakeholders y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Architecture review y comunicación con stakeholders"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar architecture review y comunicación con stakeholders. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/191-architecture-review-y-comunicacion-con-stakeholders/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa architecture-review en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye architecture-review para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 191 Architecture review y comunicación con stakeholders

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega architecture-review con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 192 — Proyecto: arquitectura completa de CloudShop

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 8  
Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar proyecto: arquitectura completa de cloudshop dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: arquitectura completa de cloudshop con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloudshop-system-design con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                               |
|--------------|-------------------------------------------------------------------------------------------------------|
| proyecto     | Define su papel en proyecto: arquitectura completa de cloudshop y cómo observarlo en un sistema real. |
| arquitectura | Define su papel en proyecto: arquitectura completa de cloudshop y cómo observarlo en un sistema real. |
| completa     | Define su papel en proyecto: arquitectura completa de cloudshop y cómo observarlo en un sistema real. |
| cloudshop    | Define su papel en proyecto: arquitectura completa de cloudshop y cómo observarlo en un sistema real. |

## Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: arquitectura completa de CloudShop"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E["¿Cumple seguridad, SLO y costo?"]
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: arquitectura completa de cloudshop. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/192-proyecto-arquitectura-completa-de-cloudshop/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloudshop-system-design en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye cloudshop-system-design para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 192 Proyecto: arquitectura completa de CloudShop

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega cloudshop-system-design con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 16 — Redes cloud avanzadas, conectividad híbrida y edge

← Parte 15 · Índice completo · Parte 17 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar conectividad segura y observable entre regiones, proveedores, centros de datos y edge.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

## Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                      | Laboratorio   | Horas |
|-----|------------------------------------------------------------|---------------|-------|
| 193 | CIDR, subnetting y planificación IP a escala               | network       | 4     |
| 194 | Routing, BGP, tránsito y propagación de rutas              | network       | 4     |
| 195 | DNS autoritativo, recursivo, split-horizon y DNSSEC        | network       | 4     |
| 196 | Balanceo L4/L7, proxies, TLS y gestión de certificados     | network       | 4     |
| 197 | CDN, caché, origin shielding y edge compute                | performance   | 4     |
| 198 | VPN, Direct Connect, ExpressRoute e Interconnect           | network       | 4     |
| 199 | Transit Gateway, Virtual WAN y Network Connectivity Center | network       | 4     |
| 200 | Private endpoints, service networking y egress control     | security      | 4     |
| 201 | Service mesh, mTLS y gestión de tráfico este-oeste         | network       | 4     |
| 202 | eBPF, flow logs, packet capture y diagnóstico              | observability | 4     |
| 203 | SD-WAN, 5G, IoT y operación desconectada                   | architecture  | 4     |
| 204 | Proyecto: red multi-región y multi-cloud                   | capstone      | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.



Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.

# 193 — CIDR, subnetting y planificación IP a escala

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar cidr, subnetting y planificación ip a escala dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cidr, subnetting y planificación ip a escala con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar ipam-plan con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                               |
|---------------|-------------------------------------------------------------------------------------------------------|
| cidr          | Define su papel en cidr, subnetting y planificación ip a escala y cómo observarlo en un sistema real. |
| subnetting    | Define su papel en cidr, subnetting y planificación ip a escala y cómo observarlo en un sistema real. |
| planificación | Define su papel en cidr, subnetting y planificación ip a escala y cómo observarlo en un sistema real. |
| ip            | Define su papel en cidr, subnetting y planificación ip a escala y cómo observarlo en un sistema real. |
| escala        | Define su papel en cidr, subnetting y planificación ip a escala y cómo observarlo en un sistema real. |

## Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: CIDR, subnetting y planificación IP a escala"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cidr, subnetting y planificación ip a escala. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/193-cidr-subnetting-y-planificacion-ip-a-escala/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa ipam-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye ipam-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 193 CIDR, subnetting y planificación IP a escala

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega ipam-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 194 — Routing, BGP, tránsito y propagación de rutas

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar routing, bgp, tránsito y propagación de rutas dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar routing, bgp, tránsito y propagación de rutas con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar routing-lab con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                |
|-------------|--------------------------------------------------------------------------------------------------------|
| routing     | Define su papel en routing, bgp, tránsito y propagación de rutas y cómo observarlo en un sistema real. |
| bgp         | Define su papel en routing, bgp, tránsito y propagación de rutas y cómo observarlo en un sistema real. |
| tránsito    | Define su papel en routing, bgp, tránsito y propagación de rutas y cómo observarlo en un sistema real. |
| propagación | Define su papel en routing, bgp, tránsito y propagación de rutas y cómo observarlo en un sistema real. |
| rutas       | Define su papel en routing, bgp, tránsito y propagación de rutas y cómo observarlo en un sistema real. |

## Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Routing, BGP, tránsito y propagación de rutas"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar routing, bgp, tránsito y propagación de rutas. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/194-routing-bgp-transito-y-propagacion-de-rutas/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa routing-lab en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye routing-lab para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 194 Routing, BGP, tránsito y propagación de rutas

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega routing-lab con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 195 — DNS autoritativo, recursivo, split-horizon y DNSSEC

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar dns autoritativo, recursivo, split-horizon y dnssec dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar dns autoritativo, recursivo, split-horizon y dnssec con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar dns-design con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                      |
|--------------|--------------------------------------------------------------------------------------------------------------|
| dns          | Define su papel en dns autoritativo, recursivo, split-horizon y dnssec y cómo observarlo en un sistema real. |
| autoritativo | Define su papel en dns autoritativo, recursivo, split-horizon y dnssec y cómo observarlo en un sistema real. |
| recursivo    | Define su papel en dns autoritativo, recursivo, split-horizon y dnssec y cómo observarlo en un sistema real. |
| split        | Define su papel en dns autoritativo, recursivo, split-horizon y dnssec y cómo observarlo en un sistema real. |
| horizon      | Define su papel en dns autoritativo, recursivo, split-horizon y dnssec y cómo observarlo en un sistema real. |
| dnssec       | Define su papel en dns autoritativo, recursivo, split-horizon y dnssec y cómo observarlo en un sistema real. |

## Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: DNS autoritativo, recursivo, split-horizon y DNSSEC"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar dns autoritativo, recursivo, split-horizon y dnssec. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/195-dns-autoritativo-recursivo-split-horizon-y-dnssec/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa dns-design en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye dns-design para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 195 DNS autoritativo, recursivo, split-horizon y DNSSEC

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega dns-design con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 196 — Balanceo L4/L7, proxies, TLS y gestión de certificados

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar balanceo L4/L7, proxies, TLS y gestión de certificados dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar balanceo L4/L7, proxies, TLS y gestión de certificados con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar traffic-entry con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                                         |
|----------|-----------------------------------------------------------------------------------------------------------------|
| balanceo | Define su papel en balanceo L4/L7, proxies, TLS y gestión de certificados y cómo observarlo en un sistema real. |
| L4       | Define su papel en balanceo L4/L7, proxies, TLS y gestión de certificados y cómo observarlo en un sistema real. |
| L7       | Define su papel en balanceo L4/L7, proxies, TLS y gestión de certificados y cómo observarlo en un sistema real. |
| proxies  | Define su papel en balanceo L4/L7, proxies, TLS y gestión de certificados y cómo observarlo en un sistema real. |
| TLS      | Define su papel en balanceo L4/L7, proxies, TLS y gestión de certificados y cómo observarlo en un sistema real. |
| gestión  | Define su papel en balanceo L4/L7, proxies, TLS y gestión de certificados y cómo observarlo en un sistema real. |

## Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Balanceo L4/L7, proxies, TLS y gestión de certificados"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar balanceo l4/l7, proxies, tls y gestión de certificados. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/196-balanceo-l4-l7-proxies-tls-y-gestion-de-certificados/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa traffic-entry en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye traffic-entry para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 196 Balanceo L4/L7, proxies, TLS y gestión de certificados

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega traffic-entry con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 197 — CDN, caché, origin shielding y edge compute

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: performance · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar cdn, caché, origin shielding y edge compute dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cdn, caché, origin shielding y edge compute con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cdn-experiment con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                              |
|-----------|------------------------------------------------------------------------------------------------------|
| cdn       | Define su papel en cdn, caché, origin shielding y edge compute y cómo observarlo en un sistema real. |
| caché     | Define su papel en cdn, caché, origin shielding y edge compute y cómo observarlo en un sistema real. |
| origin    | Define su papel en cdn, caché, origin shielding y edge compute y cómo observarlo en un sistema real. |
| shielding | Define su papel en cdn, caché, origin shielding y edge compute y cómo observarlo en un sistema real. |
| edge      | Define su papel en cdn, caché, origin shielding y edge compute y cómo observarlo en un sistema real. |
| compute   | Define su papel en cdn, caché, origin shielding y edge compute y cómo observarlo en un sistema real. |

## Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: CDN, caché, origin shielding y edge compute"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E["¿Cumple seguridad, SLO y costo?"]
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cdn, caché, origin shielding y edge compute. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/197-cdn-cache-origin-shielding-y-edge-compute/lab.py
```

El laboratorio selecciona el motor de práctica performance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa `cdn-experiment` en `evidence/` usando la plantilla indicada.
5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una prueba de carga con baseline y cuello de botella. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `cdn-experiment` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 197 CDN, caché, origin shielding y edge compute

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega cdn-experiment con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 198 — VPN, Direct Connect, ExpressRoute e Interconnect

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar vpn, direct connect, expressroute e interconnect dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar vpn, direct connect, expressroute e interconnect con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar hybrid-connectivity con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                   |
|--------------|-----------------------------------------------------------------------------------------------------------|
| vpn          | Define su papel en vpn, direct connect, expressroute e interconnect y cómo observarlo en un sistema real. |
| direct       | Define su papel en vpn, direct connect, expressroute e interconnect y cómo observarlo en un sistema real. |
| connect      | Define su papel en vpn, direct connect, expressroute e interconnect y cómo observarlo en un sistema real. |
| expressroute | Define su papel en vpn, direct connect, expressroute e interconnect y cómo observarlo en un sistema real. |
| e            | Define su papel en vpn, direct connect, expressroute e interconnect y cómo observarlo en un sistema real. |
| interconnect | Define su papel en vpn, direct connect, expressroute e interconnect y cómo observarlo en un sistema real. |

## Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: VPN, Direct Connect, ExpressRoute e Interconnect"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar vpn, direct connect, expressroute e interconnect. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/198-vpn-direct-connect-expressroute-e-interconnect/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa hybrid-connectivity en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye hybrid-connectivity para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 198 VPN, Direct Connect, ExpressRoute e Interconnect

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega hybrid-connectivity con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 199 — Transit Gateway, Virtual WAN y Network Connectivity Center

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar transit gateway, virtual wan y network connectivity center dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar transit gateway, virtual wan y network connectivity center con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar transit-comparison con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                             |
|--------------|---------------------------------------------------------------------------------------------------------------------|
| transit      | Define su papel en transit gateway, virtual wan y network connectivity center y cómo observarlo en un sistema real. |
| gateway      | Define su papel en transit gateway, virtual wan y network connectivity center y cómo observarlo en un sistema real. |
| virtual      | Define su papel en transit gateway, virtual wan y network connectivity center y cómo observarlo en un sistema real. |
| wan          | Define su papel en transit gateway, virtual wan y network connectivity center y cómo observarlo en un sistema real. |
| network      | Define su papel en transit gateway, virtual wan y network connectivity center y cómo observarlo en un sistema real. |
| connectivity | Define su papel en transit gateway, virtual wan y network connectivity center y cómo observarlo en un sistema real. |

## Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Transit Gateway, Virtual WAN y Network Connectivity Center"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar transit gateway, virtual wan y network connectivity center. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/199-transit-gateway-virtual-wan-y-network-connectivity-center/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa transit-comparison en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye transit-comparison para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 199 Transit Gateway, Virtual WAN y Network Connectivity Center

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega transit-comparison con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 200 — Private endpoints, service networking y egress control

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar private endpoints, service networking y egress control dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar private endpoints, service networking y egress control con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar private-connectivity con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                                         |
|------------|-----------------------------------------------------------------------------------------------------------------|
| private    | Define su papel en private endpoints, service networking y egress control y cómo observarlo en un sistema real. |
| endpoints  | Define su papel en private endpoints, service networking y egress control y cómo observarlo en un sistema real. |
| service    | Define su papel en private endpoints, service networking y egress control y cómo observarlo en un sistema real. |
| networking | Define su papel en private endpoints, service networking y egress control y cómo observarlo en un sistema real. |
| egress     | Define su papel en private endpoints, service networking y egress control y cómo observarlo en un sistema real. |
| control    | Define su papel en private endpoints, service networking y egress control y cómo observarlo en un sistema real. |

### Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Private endpoints, service networking y egress control"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar private endpoints, service networking y egress control. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/200-private-endpoints-service-networking-y-egress-control/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa private-connectivity en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye private-connectivity para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 200 Private endpoints, service networking y egress control

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega private-connectivity con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 201 — Service mesh, mTLS y gestión de tráfico este-oeste

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar service mesh, mTLS y gestión de tráfico este-oeste dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar service mesh, mTLS y gestión de tráfico este-oeste con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar service-mesh con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                                     |
|----------|-------------------------------------------------------------------------------------------------------------|
| service  | Define su papel en service mesh, mTLS y gestión de tráfico este-oeste y cómo observarlo en un sistema real. |
| mesh     | Define su papel en service mesh, mTLS y gestión de tráfico este-oeste y cómo observarlo en un sistema real. |
| mTLS     | Define su papel en service mesh, mTLS y gestión de tráfico este-oeste y cómo observarlo en un sistema real. |
| gestión  | Define su papel en service mesh, mTLS y gestión de tráfico este-oeste y cómo observarlo en un sistema real. |
| tráfico  | Define su papel en service mesh, mTLS y gestión de tráfico este-oeste y cómo observarlo en un sistema real. |
| este     | Define su papel en service mesh, mTLS y gestión de tráfico este-oeste y cómo observarlo en un sistema real. |

## Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Service mesh, mTLS y gestión de tráfico este-oeste"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar service mesh, mTLS y gestión de tráfico este-oeste. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/201-service-mesh-mtls-y-gestion-de-trafico-este-oeste/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa service-mesh en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye service-mesh para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 201 Service mesh, mTLS y gestión de tráfico este-oeste

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega service-mesh con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 202 — eBPF, flow logs, packet capture y diagnóstico

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar ebpf, flow logs, packet capture y diagnóstico dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ebpf, flow logs, packet capture y diagnóstico con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar network-evidence con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                                |
|-------------|--------------------------------------------------------------------------------------------------------|
| ebpf        | Define su papel en ebpf, flow logs, packet capture y diagnóstico y cómo observarlo en un sistema real. |
| flow        | Define su papel en ebpf, flow logs, packet capture y diagnóstico y cómo observarlo en un sistema real. |
| logs        | Define su papel en ebpf, flow logs, packet capture y diagnóstico y cómo observarlo en un sistema real. |
| packet      | Define su papel en ebpf, flow logs, packet capture y diagnóstico y cómo observarlo en un sistema real. |
| capture     | Define su papel en ebpf, flow logs, packet capture y diagnóstico y cómo observarlo en un sistema real. |
| diagnóstico | Define su papel en ebpf, flow logs, packet capture y diagnóstico y cómo observarlo en un sistema real. |

### Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: eBPF, flow logs, packet capture y diagnóstico"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ebpf, flow logs, packet capture y diagnóstico. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/202-ebpf-flow-logs-packet-capture-y-diagnostico/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa network-evidence en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye network-evidence para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 202 eBPF, flow logs, packet capture y diagnóstico

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega network-evidence con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 203 — SD-WAN, 5G, IoT y operación desconectada

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar sd-wan, 5g, iot y operación desconectada dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sd-wan, 5g, iot y operación desconectada con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar edge-topology con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                                           |
|--------------|---------------------------------------------------------------------------------------------------|
| sd           | Define su papel en sd-wan, 5g, iot y operación desconectada y cómo observarlo en un sistema real. |
| wan          | Define su papel en sd-wan, 5g, iot y operación desconectada y cómo observarlo en un sistema real. |
| 5g           | Define su papel en sd-wan, 5g, iot y operación desconectada y cómo observarlo en un sistema real. |
| iot          | Define su papel en sd-wan, 5g, iot y operación desconectada y cómo observarlo en un sistema real. |
| operación    | Define su papel en sd-wan, 5g, iot y operación desconectada y cómo observarlo en un sistema real. |
| desconectada | Define su papel en sd-wan, 5g, iot y operación desconectada y cómo observarlo en un sistema real. |

### Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: SD-WAN, 5G, IoT y operación desconectada"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar sd-wan, 5g, iot y operación desconectada. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/203-sd-wan-5g-iot-y-operacion-desconectada/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa edge-topology en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye edge-topology para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 203 SD-WAN, 5G, IoT y operación desconectada

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega edge-topology con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 204 — Proyecto: red multi-región y multi-cloud

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar proyecto: red multi-región y multi-cloud dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: red multi-región y multi-cloud con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar multicloud-network con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                           |
|----------|---------------------------------------------------------------------------------------------------|
| proyecto | Define su papel en proyecto: red multi-región y multi-cloud y cómo observarlo en un sistema real. |
| red      | Define su papel en proyecto: red multi-región y multi-cloud y cómo observarlo en un sistema real. |
| multi    | Define su papel en proyecto: red multi-región y multi-cloud y cómo observarlo en un sistema real. |
| región   | Define su papel en proyecto: red multi-región y multi-cloud y cómo observarlo en un sistema real. |
| multi    | Define su papel en proyecto: red multi-región y multi-cloud y cómo observarlo en un sistema real. |
| cloud    | Define su papel en proyecto: red multi-región y multi-cloud y cómo observarlo en un sistema real. |

### Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: red multi-región y multi-cloud"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: red multi-región y multi-cloud. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/204-proyecto-red-multi-region-y-multi-cloud/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa multicloud-network en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye multicloud-network para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 204 Proyecto: red multi-región y multi-cloud

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega multicloud-network con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 17 — AWS: arquitectura, automatización y operación en producción

← Parte 16 · Índice completo · Parte 18 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Integrar los casos AWS existentes en una ruta de producción con seguridad, costo, evidencia y destrucción.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

## Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                    | Laboratorio   | Horas |
|-----|----------------------------------------------------------|---------------|-------|
| 205 | Hosting progresivo con Amplify, S3 y CloudFront          | delivery      | 4     |
| 206 | OIDC de GitHub y GitLab hacia AWS sin secretos           | iam           | 4     |
| 207 | SAM, Lambda, API Gateway y despliegue serverless         | serverless    | 4     |
| 208 | DynamoDB por patrones de acceso y single-table design    | data          | 4     |
| 209 | Cognito, JWT authorizers, WAF y defensa en profundidad   | security      | 4     |
| 210 | EventBridge, SQS, DLQ, replay e idempotencia             | messaging     | 4     |
| 211 | CloudWatch, X-Ray y observabilidad como código           | observability | 4     |
| 212 | ECR, ECS Fargate, ALB y autoscaling                      | container     | 4     |
| 213 | EKS, IRSA, GitOps y operación de clúster                 | kubernetes    | 4     |
| 214 | Budgets, Cost Explorer, etiquetado y FinOps automatizado | finops        | 4     |
| 215 | Multi-región, Route 53, failover y game day              | reliability   | 4     |
| 216 | Proyecto: CloudShop productivo en AWS                    | capstone      | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.

# 205 — Hosting progresivo con Amplify, S3 y CloudFront

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: delivery · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar hosting progresivo con amplify, s3 y cloudfront dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar hosting progresivo con amplify, s3 y cloudfront con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-static-platform con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                  |
|------------|----------------------------------------------------------------------------------------------------------|
| hosting    | Define su papel en hosting progresivo con amplify, s3 y cloudfront y cómo observarlo en un sistema real. |
| progresivo | Define su papel en hosting progresivo con amplify, s3 y cloudfront y cómo observarlo en un sistema real. |
| amplify    | Define su papel en hosting progresivo con amplify, s3 y cloudfront y cómo observarlo en un sistema real. |
| s3         | Define su papel en hosting progresivo con amplify, s3 y cloudfront y cómo observarlo en un sistema real. |
| cloudfront | Define su papel en hosting progresivo con amplify, s3 y cloudfront y cómo observarlo en un sistema real. |

## Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Hosting progresivo con Amplify, S3 y CloudFront"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar hosting progresivo con amplify, s3 y cloudfront. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/205-hosting-progresivo-con-amplify-s3-y-cloudfront/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-static-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye aws-static-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 205 Hosting progresivo con Amplify, S3 y CloudFront

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega aws-static-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 206 — OIDC de GitHub y GitLab hacia AWS sin secretos

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: iam · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar oidc de github y gitlab hacia aws sin secretos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar oidc de github y gitlab hacia aws sin secretos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-oidc-federation con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                                 |
|----------|---------------------------------------------------------------------------------------------------------|
| oidc     | Define su papel en oidc de github y gitlab hacia aws sin secretos y cómo observarlo en un sistema real. |
| github   | Define su papel en oidc de github y gitlab hacia aws sin secretos y cómo observarlo en un sistema real. |
| gitlab   | Define su papel en oidc de github y gitlab hacia aws sin secretos y cómo observarlo en un sistema real. |
| hacia    | Define su papel en oidc de github y gitlab hacia aws sin secretos y cómo observarlo en un sistema real. |
| aws      | Define su papel en oidc de github y gitlab hacia aws sin secretos y cómo observarlo en un sistema real. |
| sin      | Define su papel en oidc de github y gitlab hacia aws sin secretos y cómo observarlo en un sistema real. |

### Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: OIDC de GitHub y GitLab hacia AWS sin secretos"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar oidc de github y gitlab hacia aws sin secretos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/206-oidc-de-github-y-gitlab-hacia-aws-sin-secretos/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-oidc-federation en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye aws-oidc-federation para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 206 OIDC de GitHub y GitLab hacia AWS sin secretos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega aws-oidc-federation con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 207 — SAM, Lambda, API Gateway y despliegue serverless

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: serverless · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar sam, lambda, api gateway y despliegue serverless dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sam, lambda, api gateway y despliegue serverless con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-serverless-api con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                   |
|------------|-----------------------------------------------------------------------------------------------------------|
| sam        | Define su papel en sam, lambda, api gateway y despliegue serverless y cómo observarlo en un sistema real. |
| lambda     | Define su papel en sam, lambda, api gateway y despliegue serverless y cómo observarlo en un sistema real. |
| api        | Define su papel en sam, lambda, api gateway y despliegue serverless y cómo observarlo en un sistema real. |
| gateway    | Define su papel en sam, lambda, api gateway y despliegue serverless y cómo observarlo en un sistema real. |
| despliegue | Define su papel en sam, lambda, api gateway y despliegue serverless y cómo observarlo en un sistema real. |
| serverless | Define su papel en sam, lambda, api gateway y despliegue serverless y cómo observarlo en un sistema real. |

## Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: SAM, Lambda, API Gateway y despliegue serverless"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar sam, lambda, api gateway y despliegue serverless. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/207-sam-lambda-api-gateway-y-despliegue-serverless/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-serverless-api en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye aws-serverless-api para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 207 SAM, Lambda, API Gateway y despliegue serverless

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega aws-serverless-api con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 208 — DynamoDB por patrones de acceso y single-table design

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: data · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar dynamodb por patrones de acceso y single-table design dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar dynamodb por patrones de acceso y single-table design con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-dynamodb-model con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                                        |
|----------|----------------------------------------------------------------------------------------------------------------|
| dynamodb | Define su papel en dynamodb por patrones de acceso y single-table design y cómo observarlo en un sistema real. |
| patrones | Define su papel en dynamodb por patrones de acceso y single-table design y cómo observarlo en un sistema real. |
| acceso   | Define su papel en dynamodb por patrones de acceso y single-table design y cómo observarlo en un sistema real. |
| single   | Define su papel en dynamodb por patrones de acceso y single-table design y cómo observarlo en un sistema real. |
| table    | Define su papel en dynamodb por patrones de acceso y single-table design y cómo observarlo en un sistema real. |
| design   | Define su papel en dynamodb por patrones de acceso y single-table design y cómo observarlo en un sistema real. |

## Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: DynamoDB por patrones de acceso y single-table design"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar dynamodb por patrones de acceso y single-table design. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/208-dynamodb-por-patrones-de-acceso-y-single-table-design/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-dynamodb-model en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye aws-dynamodb-model para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 208 DynamoDB por patrones de acceso y single-table design

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega aws-dynamodb-model con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 209 — Cognito, JWT authorizers, WAF y defensa en profundidad

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: security · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar cognito, jwt authorizers, waf y defensa en profundidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cognito, jwt authorizers, waf y defensa en profundidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-identity-edge con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                                         |
|-------------|-----------------------------------------------------------------------------------------------------------------|
| cognito     | Define su papel en cognito, jwt authorizers, waf y defensa en profundidad y cómo observarlo en un sistema real. |
| jwt         | Define su papel en cognito, jwt authorizers, waf y defensa en profundidad y cómo observarlo en un sistema real. |
| authorizers | Define su papel en cognito, jwt authorizers, waf y defensa en profundidad y cómo observarlo en un sistema real. |
| waf         | Define su papel en cognito, jwt authorizers, waf y defensa en profundidad y cómo observarlo en un sistema real. |
| defensa     | Define su papel en cognito, jwt authorizers, waf y defensa en profundidad y cómo observarlo en un sistema real. |
| profundidad | Define su papel en cognito, jwt authorizers, waf y defensa en profundidad y cómo observarlo en un sistema real. |

### Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Cognito, JWT authorizers, WAF y defensa en profundidad"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cognito, jwt authorizers, waf y defensa en profundidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/209-cognito-jwt-authorizers-waf-y-defensa-en-profundidad/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-identity-edge en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye aws-identity-edge para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 209 Cognito, JWT authorizers, WAF y defensa en profundidad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega aws-identity-edge con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 210 — EventBridge, SQS, DLQ, replay e idempotencia

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: messaging · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar eventbridge, sqs, dlq, replay e idempotencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar eventbridge, sqs, dlq, replay e idempotencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-event-platform con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                               |
|--------------|-------------------------------------------------------------------------------------------------------|
| eventbridge  | Define su papel en eventbridge, sqs, dlq, replay e idempotencia y cómo observarlo en un sistema real. |
| sqs          | Define su papel en eventbridge, sqs, dlq, replay e idempotencia y cómo observarlo en un sistema real. |
| dlq          | Define su papel en eventbridge, sqs, dlq, replay e idempotencia y cómo observarlo en un sistema real. |
| replay       | Define su papel en eventbridge, sqs, dlq, replay e idempotencia y cómo observarlo en un sistema real. |
| e            | Define su papel en eventbridge, sqs, dlq, replay e idempotencia y cómo observarlo en un sistema real. |
| idempotencia | Define su papel en eventbridge, sqs, dlq, replay e idempotencia y cómo observarlo en un sistema real. |

## Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: EventBridge, SQS, DLQ, replay e idempotencia"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar eventbridge, sqs, dlq, replay e idempotencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/210-eventbridge-sqs-dlq-replay-e-idempotencia/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-event-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye aws-event-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 210 EventBridge, SQS, DLQ, replay e idempotencia

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega aws-event-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 211 — CloudWatch, X-Ray y observabilidad como código

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: observability · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar cloudwatch, x-ray y observabilidad como código dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloudwatch, x-ray y observabilidad como código con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-observability con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto       | Comprensión verificable                                                                                 |
|----------------|---------------------------------------------------------------------------------------------------------|
| cloudwatch     | Define su papel en cloudwatch, x-ray y observabilidad como código y cómo observarlo en un sistema real. |
| x              | Define su papel en cloudwatch, x-ray y observabilidad como código y cómo observarlo en un sistema real. |
| ray            | Define su papel en cloudwatch, x-ray y observabilidad como código y cómo observarlo en un sistema real. |
| observabilidad | Define su papel en cloudwatch, x-ray y observabilidad como código y cómo observarlo en un sistema real. |
| código         | Define su papel en cloudwatch, x-ray y observabilidad como código y cómo observarlo en un sistema real. |

### Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: CloudWatch, X-Ray y observabilidad como código"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloudwatch, x-ray y observabilidad como código. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/211-cloudwatch-x-ray-y-observabilidad-como-codigo/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-observability en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye aws-observability para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 211 CloudWatch, X-Ray y observabilidad como código

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega aws-observability con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 212 — ECR, ECS Fargate, ALB y autoscaling

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: container · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar ecr, ecs fargate, alb y autoscaling dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ecr, ecs fargate, alb y autoscaling con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-ecs-platform con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                      |
|-------------|----------------------------------------------------------------------------------------------|
| ecr         | Define su papel en ecr, ecs fargate, alb y autoscaling y cómo observarlo en un sistema real. |
| ecs         | Define su papel en ecr, ecs fargate, alb y autoscaling y cómo observarlo en un sistema real. |
| fargate     | Define su papel en ecr, ecs fargate, alb y autoscaling y cómo observarlo en un sistema real. |
| alb         | Define su papel en ecr, ecs fargate, alb y autoscaling y cómo observarlo en un sistema real. |
| autoscaling | Define su papel en ecr, ecs fargate, alb y autoscaling y cómo observarlo en un sistema real. |

### Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: ECR, ECS Fargate, ALB y autoscaling"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ecr, ecs fargate, alb y autoscaling. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/212-ecr-ecs-fargate-alb-y-autoscaling/lab.py
```

El laboratorio selecciona el motor de práctica container y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-ecs-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una imagen mínima, escaneada y ejecutada sin privilegios. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye aws-ecs-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 212 ECR, ECS Fargate, ALB y autoscaling

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega aws-ecs-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 213 — EKS, IRSA, GitOps y operación de clúster

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: kubernetes · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar eks, irsa, gitops y operación de clúster dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar eks, irsa, gitops y operación de clúster con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-eks-gitops con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                           |
|-----------|---------------------------------------------------------------------------------------------------|
| eks       | Define su papel en eks, irsa, gitops y operación de clúster y cómo observarlo en un sistema real. |
| irsa      | Define su papel en eks, irsa, gitops y operación de clúster y cómo observarlo en un sistema real. |
| gitops    | Define su papel en eks, irsa, gitops y operación de clúster y cómo observarlo en un sistema real. |
| operación | Define su papel en eks, irsa, gitops y operación de clúster y cómo observarlo en un sistema real. |
| clúster   | Define su papel en eks, irsa, gitops y operación de clúster y cómo observarlo en un sistema real. |

### Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: EKS, IRSA, GitOps y operación de clúster"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar eks, irsa, gitops y operación de clúster. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/213-eks-irsa-gitops-y-operacion-de-cluster/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-eks-gitops en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `aws-eks-gitops` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.

- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 213 EKS, IRSA, GitOps y operación de clúster

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega aws-eks-gitops con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 214 — Budgets, Cost Explorer, etiquetado y FinOps automatizado

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: finops · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar budgets, cost explorer, etiquetado y finops automatizado dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar budgets, cost explorer, etiquetado y finops automatizado con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-cost-control con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                                                           |
|--------------|-------------------------------------------------------------------------------------------------------------------|
| budgets      | Define su papel en budgets, cost explorer, etiquetado y finops automatizado y cómo observarlo en un sistema real. |
| cost         | Define su papel en budgets, cost explorer, etiquetado y finops automatizado y cómo observarlo en un sistema real. |
| explorer     | Define su papel en budgets, cost explorer, etiquetado y finops automatizado y cómo observarlo en un sistema real. |
| etiquetado   | Define su papel en budgets, cost explorer, etiquetado y finops automatizado y cómo observarlo en un sistema real. |
| finops       | Define su papel en budgets, cost explorer, etiquetado y finops automatizado y cómo observarlo en un sistema real. |
| automatizado | Define su papel en budgets, cost explorer, etiquetado y finops automatizado y cómo observarlo en un sistema real. |

### Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Budgets, Cost Explorer, etiquetado y FinOps automatizado"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar budgets, cost explorer, etiquetado y finops automatizado. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/214-budgets-cost-explorer-etiquetado-y-finops-automatizado/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-cost-control en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye aws-cost-control para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 214 Budgets, Cost Explorer, etiquetado y FinOps automatizado

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega aws-cost-control con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 215 — Multi-región, Route 53, failover y game day

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: reliability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar multi-región, route 53, failover y game day dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar multi-región, route 53, failover y game day con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-dr-drill con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                              |
|----------|------------------------------------------------------------------------------------------------------|
| multi    | Define su papel en multi-región, route 53, failover y game day y cómo observarlo en un sistema real. |
| región   | Define su papel en multi-región, route 53, failover y game day y cómo observarlo en un sistema real. |
| route    | Define su papel en multi-región, route 53, failover y game day y cómo observarlo en un sistema real. |
| 53       | Define su papel en multi-región, route 53, failover y game day y cómo observarlo en un sistema real. |
| failover | Define su papel en multi-región, route 53, failover y game day y cómo observarlo en un sistema real. |
| game     | Define su papel en multi-región, route 53, failover y game day y cómo observarlo en un sistema real. |

## Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Multi-región, Route 53, failover y game day"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E["¿Cumple seguridad, SLO y costo?"]
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar multi-región, route 53, failover y game day. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/215-multi-region-route-53-failover-y-game-day/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-dr-drill en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye aws-dr-drill para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 215 Multi-región, Route 53, failover y game day

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega aws-dr-drill con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 216 — Proyecto: CloudShop productivo en AWS

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 8  
Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar proyecto: cloudshop productivo en aws dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: cloudshop productivo en aws con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloudshop-aws con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                        |
|------------|------------------------------------------------------------------------------------------------|
| proyecto   | Define su papel en proyecto: cloudshop productivo en aws y cómo observarlo en un sistema real. |
| cloudshop  | Define su papel en proyecto: cloudshop productivo en aws y cómo observarlo en un sistema real. |
| productivo | Define su papel en proyecto: cloudshop productivo en aws y cómo observarlo en un sistema real. |
| aws        | Define su papel en proyecto: cloudshop productivo en aws y cómo observarlo en un sistema real. |

## Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: CloudShop productivo en AWS"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: cloudshop productivo en aws. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/216-proyecto-cloudshop-productivo-en-aws/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloudshop-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye cloudshop-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 216 Proyecto: CloudShop productivo en AWS

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega cloudshop-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 18 — Azure: arquitectura empresarial y operación en producción

← Parte 17 · Índice completo · Parte 19 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Operar CloudShop sobre landing zones, PaaS, datos, identidad y observabilidad de Azure.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

## Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                 | Laboratorio   | Horas |
|-----|-------------------------------------------------------|---------------|-------|
| 217 | Enterprise-scale landing zones y management groups    | governance    | 4     |
| 218 | Entra ID, workload identity, PIM y Conditional Access | iam           | 4     |
| 219 | Hub-spoke, Virtual WAN, Private Link y DNS privado    | network       | 4     |
| 220 | Bicep, deployment stacks y Azure Verified Modules     | iac           | 4     |
| 221 | App Service, Functions y Container Apps en producción | serverless    | 4     |
| 222 | AKS, workload identity, ingress y GitOps              | kubernetes    | 4     |
| 223 | Azure SQL, Cosmos DB y consistencia distribuida       | data          | 4     |
| 224 | Service Bus, Event Grid y Event Hubs                  | messaging     | 4     |
| 225 | Azure Monitor, Application Insights y OpenTelemetry   | observability | 4     |
| 226 | Defender for Cloud, Policy y Sentinel                 | security      | 4     |
| 227 | Cost Management, Advisor, resiliencia y Chaos Studio  | finops        | 4     |
| 228 | Proyecto: CloudShop productivo en Azure               | capstone      | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.

# 217 — Enterprise-scale landing zones y management groups

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: governance · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar enterprise-scale landing zones y management groups dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar enterprise-scale landing zones y management groups con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-landing-zone con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                     |
|------------|-------------------------------------------------------------------------------------------------------------|
| enterprise | Define su papel en enterprise-scale landing zones y management groups y cómo observarlo en un sistema real. |
| scale      | Define su papel en enterprise-scale landing zones y management groups y cómo observarlo en un sistema real. |
| landing    | Define su papel en enterprise-scale landing zones y management groups y cómo observarlo en un sistema real. |
| zones      | Define su papel en enterprise-scale landing zones y management groups y cómo observarlo en un sistema real. |
| management | Define su papel en enterprise-scale landing zones y management groups y cómo observarlo en un sistema real. |
| groups     | Define su papel en enterprise-scale landing zones y management groups y cómo observarlo en un sistema real. |

## Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Enterprise-scale landing zones y management groups"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar enterprise-scale landing zones y management groups. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/217-enterprise-scale-landing-zones-y-management-groups/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-landing-zone en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye azure-landing-zone para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 217 Enterprise-scale landing zones y management groups

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega azure-landing-zone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 218 — Entra ID, workload identity, PIM y Conditional Access

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: iam · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar entra id, workload identity, pim y conditional access dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar entra id, workload identity, pim y conditional access con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-identity con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                        |
|-------------|----------------------------------------------------------------------------------------------------------------|
| entra       | Define su papel en entra id, workload identity, pim y conditional access y cómo observarlo en un sistema real. |
| id          | Define su papel en entra id, workload identity, pim y conditional access y cómo observarlo en un sistema real. |
| workload    | Define su papel en entra id, workload identity, pim y conditional access y cómo observarlo en un sistema real. |
| identity    | Define su papel en entra id, workload identity, pim y conditional access y cómo observarlo en un sistema real. |
| pim         | Define su papel en entra id, workload identity, pim y conditional access y cómo observarlo en un sistema real. |
| conditional | Define su papel en entra id, workload identity, pim y conditional access y cómo observarlo en un sistema real. |

## Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Entra ID, workload identity, PIM y Conditional Access"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar entra id, workload identity, pim y conditional access. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/218-entra-id-workload-identity-pim-y-conditional-access/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-identity en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye azure-identity para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 218 Entra ID, workload identity, PIM y Conditional Access

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega azure-identity con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 219 — Hub-spoke, Virtual WAN, Private Link y DNS privado

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar hub-spoke, virtual wan, private link y dns privado dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar hub-spoke, virtual wan, private link y dns privado con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-network con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto | Comprensión verificable                                                                                     |
|----------|-------------------------------------------------------------------------------------------------------------|
| hub      | Define su papel en hub-spoke, virtual wan, private link y dns privado y cómo observarlo en un sistema real. |
| spoke    | Define su papel en hub-spoke, virtual wan, private link y dns privado y cómo observarlo en un sistema real. |
| virtual  | Define su papel en hub-spoke, virtual wan, private link y dns privado y cómo observarlo en un sistema real. |
| wan      | Define su papel en hub-spoke, virtual wan, private link y dns privado y cómo observarlo en un sistema real. |
| private  | Define su papel en hub-spoke, virtual wan, private link y dns privado y cómo observarlo en un sistema real. |
| link     | Define su papel en hub-spoke, virtual wan, private link y dns privado y cómo observarlo en un sistema real. |

## Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Hub-spoke, Virtual WAN, Private Link y DNS privado"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar hub-spoke, virtual wan, private link y dns privado. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/219-hub-spoke-virtual-wan-private-link-y-dns-privado/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-network en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye azure-network para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 219 Hub-spoke, Virtual WAN, Private Link y DNS privado

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega azure-network con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 220 — Bicep, deployment stacks y Azure Verified Modules

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: iac · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar bicep, deployment stacks y azure verified modules dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar bicep, deployment stacks y azure verified modules con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-bicep-stack con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                    |
|------------|------------------------------------------------------------------------------------------------------------|
| bicep      | Define su papel en bicep, deployment stacks y azure verified modules y cómo observarlo en un sistema real. |
| deployment | Define su papel en bicep, deployment stacks y azure verified modules y cómo observarlo en un sistema real. |
| stacks     | Define su papel en bicep, deployment stacks y azure verified modules y cómo observarlo en un sistema real. |
| azure      | Define su papel en bicep, deployment stacks y azure verified modules y cómo observarlo en un sistema real. |
| verified   | Define su papel en bicep, deployment stacks y azure verified modules y cómo observarlo en un sistema real. |
| modules    | Define su papel en bicep, deployment stacks y azure verified modules y cómo observarlo en un sistema real. |

## Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Bicep, deployment stacks y Azure Verified Modules"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar bicep, deployment stacks y azure verified modules. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/220-bicep-deployment-stacks-y-azure-verified-modules/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-bicep-stack en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye azure-bicep-stack para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 220 Bicep, deployment stacks y Azure Verified Modules

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega azure-bicep-stack con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 221 — App Service, Functions y Container Apps en producción

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: serverless · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar app service, functions y container apps en producción dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar app service, functions y container apps en producción con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-app-platform con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                        |
|------------|----------------------------------------------------------------------------------------------------------------|
| app        | Define su papel en app service, functions y container apps en producción y cómo observarlo en un sistema real. |
| service    | Define su papel en app service, functions y container apps en producción y cómo observarlo en un sistema real. |
| functions  | Define su papel en app service, functions y container apps en producción y cómo observarlo en un sistema real. |
| container  | Define su papel en app service, functions y container apps en producción y cómo observarlo en un sistema real. |
| apps       | Define su papel en app service, functions y container apps en producción y cómo observarlo en un sistema real. |
| producción | Define su papel en app service, functions y container apps en producción y cómo observarlo en un sistema real. |

## Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: App Service, Functions y Container Apps en producción"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar app service, functions y container apps en producción. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/221-app-service-functions-y-container-apps-en-produccion/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-app-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye azure-app-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 221 App Service, Functions y Container Apps en producción

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega azure-app-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 222 — AKS, workload identity, ingress y GitOps

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: kubernetes · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar aks, workload identity, ingress y gitops dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar aks, workload identity, ingress y gitops con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-aks-platform con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                           |
|----------|---------------------------------------------------------------------------------------------------|
| aks      | Define su papel en aks, workload identity, ingress y gitops y cómo observarlo en un sistema real. |
| workload | Define su papel en aks, workload identity, ingress y gitops y cómo observarlo en un sistema real. |
| identity | Define su papel en aks, workload identity, ingress y gitops y cómo observarlo en un sistema real. |
| ingress  | Define su papel en aks, workload identity, ingress y gitops y cómo observarlo en un sistema real. |
| gitops   | Define su papel en aks, workload identity, ingress y gitops y cómo observarlo en un sistema real. |

### Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: AKS, workload identity, ingress y GitOps"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar aks, workload identity, ingress y gitops. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/222-aks-workload-identity-ingress-y-gitops/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa azure-aks-platform en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `azure-aks-platform` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.

- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 222 AKS, workload identity, ingress y GitOps

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega azure-aks-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 223 — Azure SQL, Cosmos DB y consistencia distribuida

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: data · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar azure sql, cosmos db y consistencia distribuida dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar azure sql, cosmos db y consistencia distribuida con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-data-platform con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                  |
|--------------|----------------------------------------------------------------------------------------------------------|
| azure        | Define su papel en azure sql, cosmos db y consistencia distribuida y cómo observarlo en un sistema real. |
| sql          | Define su papel en azure sql, cosmos db y consistencia distribuida y cómo observarlo en un sistema real. |
| cosmos       | Define su papel en azure sql, cosmos db y consistencia distribuida y cómo observarlo en un sistema real. |
| db           | Define su papel en azure sql, cosmos db y consistencia distribuida y cómo observarlo en un sistema real. |
| consistencia | Define su papel en azure sql, cosmos db y consistencia distribuida y cómo observarlo en un sistema real. |
| distribuida  | Define su papel en azure sql, cosmos db y consistencia distribuida y cómo observarlo en un sistema real. |

## Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Azure SQL, Cosmos DB y consistencia distribuida"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar azure sql, cosmos db y consistencia distribuida. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/223-azure-sql-cosmos-db-y-consistencia-distribuida/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-data-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye azure-data-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 223 Azure SQL, Cosmos DB y consistencia distribuida

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega azure-data-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 224 — Service Bus, Event Grid y Event Hubs

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: messaging · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar service bus, event grid y event hubs dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar service bus, event grid y event hubs con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-event-platform con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                       |
|----------|-----------------------------------------------------------------------------------------------|
| service  | Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real. |
| bus      | Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real. |
| event    | Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real. |
| grid     | Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real. |
| event    | Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real. |
| hubs     | Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real. |

### Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Service Bus, Event Grid y Event Hubs"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar service bus, event grid y event hubs. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/224-service-bus-event-grid-y-event-hubs/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa azure-event-platform en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `azure-event-platform` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.

- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 224 Service Bus, Event Grid y Event Hubs

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega azure-event-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 225 — Azure Monitor, Application Insights y OpenTelemetry

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: observability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar azure monitor, application insights y opentelemetry dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar azure monitor, application insights y opentelemetry con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-observability con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                                      |
|---------------|--------------------------------------------------------------------------------------------------------------|
| azure         | Define su papel en azure monitor, application insights y opentelemetry y cómo observarlo en un sistema real. |
| monitor       | Define su papel en azure monitor, application insights y opentelemetry y cómo observarlo en un sistema real. |
| application   | Define su papel en azure monitor, application insights y opentelemetry y cómo observarlo en un sistema real. |
| insights      | Define su papel en azure monitor, application insights y opentelemetry y cómo observarlo en un sistema real. |
| opentelemetry | Define su papel en azure monitor, application insights y opentelemetry y cómo observarlo en un sistema real. |

## Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Azure Monitor, Application Insights y OpenTelemetry"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar azure monitor, application insights y opentelemetry. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/225-azure-monitor-application-insights-y-opentelemetry/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-observability en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye azure-observability para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 225 Azure Monitor, Application Insights y OpenTelemetry

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega azure-observability con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 226 — Defender for Cloud, Policy y Sentinel

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: security · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar defender for cloud, policy y sentinel dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar defender for cloud, policy y sentinel con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-security-operations con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                        |
|----------|------------------------------------------------------------------------------------------------|
| defender | Define su papel en defender for cloud, policy y sentinel y cómo observarlo en un sistema real. |
| for      | Define su papel en defender for cloud, policy y sentinel y cómo observarlo en un sistema real. |
| cloud    | Define su papel en defender for cloud, policy y sentinel y cómo observarlo en un sistema real. |
| policy   | Define su papel en defender for cloud, policy y sentinel y cómo observarlo en un sistema real. |
| sentinel | Define su papel en defender for cloud, policy y sentinel y cómo observarlo en un sistema real. |

### Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Defender for Cloud, Policy y Sentinel"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar defender for cloud, policy y sentinel. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/226-defender-for-cloud-policy-y-sentinel/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa azure-security-operations en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `azure-security-operations` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.

- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 226 Defender for Cloud, Policy y Sentinel

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega azure-security-operations con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 227 — Cost Management, Advisor, resiliencia y Chaos Studio

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: finops · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar cost management, advisor, resiliencia y chaos studio dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cost management, advisor, resiliencia y chaos studio con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-optimization con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto    | Comprensión verificable                                                                                       |
|-------------|---------------------------------------------------------------------------------------------------------------|
| cost        | Define su papel en cost management, advisor, resiliencia y chaos studio y cómo observarlo en un sistema real. |
| management  | Define su papel en cost management, advisor, resiliencia y chaos studio y cómo observarlo en un sistema real. |
| advisor     | Define su papel en cost management, advisor, resiliencia y chaos studio y cómo observarlo en un sistema real. |
| resiliencia | Define su papel en cost management, advisor, resiliencia y chaos studio y cómo observarlo en un sistema real. |
| chaos       | Define su papel en cost management, advisor, resiliencia y chaos studio y cómo observarlo en un sistema real. |
| studio      | Define su papel en cost management, advisor, resiliencia y chaos studio y cómo observarlo en un sistema real. |

## Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Cost Management, Advisor, resiliencia y Chaos Studio"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cost management, advisor, resiliencia y chaos studio. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/227-cost-management-advisor-resiliencia-y-chaos-studio/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-optimization en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye azure-optimization para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 227 Cost Management, Advisor, resiliencia y Chaos Studio

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega azure-optimization con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 228 — Proyecto: CloudShop productivo en Azure

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 8  
Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar proyecto: cloudshop productivo en azure dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: cloudshop productivo en azure con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloudshop-azure con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                          |
|------------|--------------------------------------------------------------------------------------------------|
| proyecto   | Define su papel en proyecto: cloudshop productivo en azure y cómo observarlo en un sistema real. |
| cloudshop  | Define su papel en proyecto: cloudshop productivo en azure y cómo observarlo en un sistema real. |
| productivo | Define su papel en proyecto: cloudshop productivo en azure y cómo observarlo en un sistema real. |
| azure      | Define su papel en proyecto: cloudshop productivo en azure y cómo observarlo en un sistema real. |

### Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: CloudShop productivo en Azure"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: cloudshop productivo en azure. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/228-proyecto-cloudshop-productivo-en-azure/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloudshop-azure en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye cloudshop-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 228 Proyecto: CloudShop productivo en Azure

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega cloudshop-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 19 — Google Cloud: arquitectura de datos y operación en producción

← Parte 18 · Índice completo · Parte 20 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Operar CloudShop con la red global, servicios administrados, seguridad y plataforma de datos de Google Cloud.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

## Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                 | Laboratorio   | Horas |
|-----|-------------------------------------------------------|---------------|-------|
| 229 | Resource Manager, folders, Shared VPC y guardrails    | governance    | 4     |
| 230 | Workload Identity Federation, IAM Conditions y PAM    | iam           | 4     |
| 231 | Red global, load balancing, PSC y Cloud DNS           | network       | 4     |
| 232 | Terraform, Infrastructure Manager y policy validation | iac           | 4     |
| 233 | Cloud Run, Functions, API Gateway y Workflows         | serverless    | 4     |
| 234 | GKE Autopilot, Workload Identity y Config Sync        | kubernetes    | 4     |
| 235 | Cloud SQL, Spanner, Firestore y Bigtable              | data          | 4     |
| 236 | BigQuery, Dataflow, Dataproc y gobernanza de datos    | data          | 4     |
| 237 | Pub/Sub, Eventarc y entrega exactamente-una-vez       | messaging     | 4     |
| 238 | Cloud Operations, Trace y OpenTelemetry               | observability | 4     |
| 239 | SCC, VPC Service Controls, KMS y FinOps               | security      | 4     |
| 240 | Proyecto: CloudShop productivo en Google Cloud        | capstone      | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.

## 229 — Resource Manager, folders, Shared VPC y guardrails

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: governance · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar resource manager, folders, shared vpc y guardrails dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar resource manager, folders, shared vpc y guardrails con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-foundation con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                                     |
|------------|-------------------------------------------------------------------------------------------------------------|
| resource   | Define su papel en resource manager, folders, shared vpc y guardrails y cómo observarlo en un sistema real. |
| manager    | Define su papel en resource manager, folders, shared vpc y guardrails y cómo observarlo en un sistema real. |
| folders    | Define su papel en resource manager, folders, shared vpc y guardrails y cómo observarlo en un sistema real. |
| shared     | Define su papel en resource manager, folders, shared vpc y guardrails y cómo observarlo en un sistema real. |
| vpc        | Define su papel en resource manager, folders, shared vpc y guardrails y cómo observarlo en un sistema real. |
| guardrails | Define su papel en resource manager, folders, shared vpc y guardrails y cómo observarlo en un sistema real. |

### Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Resource Manager, folders, Shared VPC y guardrails"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar resource manager, folders, shared vpc y guardrails. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/229-resource-manager-folders-shared-vpc-y-guardrails/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-foundation en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye gcp-foundation para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 229 Resource Manager, folders, Shared VPC y guardrails

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega gcp-foundation con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 230 — Workload Identity Federation, IAM Conditions y PAM

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: iam · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar workload identity federation, iam conditions y pam dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar workload identity federation, iam conditions y pam con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-identity con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                     |
|------------|-------------------------------------------------------------------------------------------------------------|
| workload   | Define su papel en workload identity federation, iam conditions y pam y cómo observarlo en un sistema real. |
| identity   | Define su papel en workload identity federation, iam conditions y pam y cómo observarlo en un sistema real. |
| federation | Define su papel en workload identity federation, iam conditions y pam y cómo observarlo en un sistema real. |
| iam        | Define su papel en workload identity federation, iam conditions y pam y cómo observarlo en un sistema real. |
| conditions | Define su papel en workload identity federation, iam conditions y pam y cómo observarlo en un sistema real. |
| pam        | Define su papel en workload identity federation, iam conditions y pam y cómo observarlo en un sistema real. |

## Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Workload Identity Federation, IAM Conditions y PAM"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar workload identity federation, iam conditions y pam. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/230-workload-identity-federation-iam-conditions-y-pam/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-identity en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye gcp-identity para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 230 Workload Identity Federation, IAM Conditions y PAM

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega gcp-identity con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 231 — Red global, load balancing, PSC y Cloud DNS

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: network · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar red global, load balancing, psc y cloud dns dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar red global, load balancing, psc y cloud dns con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-network con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                              |
|-----------|------------------------------------------------------------------------------------------------------|
| red       | Define su papel en red global, load balancing, psc y cloud dns y cómo observarlo en un sistema real. |
| global    | Define su papel en red global, load balancing, psc y cloud dns y cómo observarlo en un sistema real. |
| load      | Define su papel en red global, load balancing, psc y cloud dns y cómo observarlo en un sistema real. |
| balancing | Define su papel en red global, load balancing, psc y cloud dns y cómo observarlo en un sistema real. |
| psc       | Define su papel en red global, load balancing, psc y cloud dns y cómo observarlo en un sistema real. |
| cloud     | Define su papel en red global, load balancing, psc y cloud dns y cómo observarlo en un sistema real. |

## Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Red global, load balancing, PSC y Cloud DNS"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar red global, load balancing, psc y cloud dns. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/231-red-global-load-balancing-psc-y-cloud-dns/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-network en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye gcp-network para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 231 Red global, load balancing, PSC y Cloud DNS

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega gcp-network con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 232 — Terraform, Infrastructure Manager y policy validation

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: iac · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar terraform, infrastructure manager y policy validation dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar terraform, infrastructure manager y policy validation con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-iac-stack con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto       | Comprensión verificable                                                                                        |
|----------------|----------------------------------------------------------------------------------------------------------------|
| terraform      | Define su papel en terraform, infrastructure manager y policy validation y cómo observarlo en un sistema real. |
| infrastructure | Define su papel en terraform, infrastructure manager y policy validation y cómo observarlo en un sistema real. |
| manager        | Define su papel en terraform, infrastructure manager y policy validation y cómo observarlo en un sistema real. |
| policy         | Define su papel en terraform, infrastructure manager y policy validation y cómo observarlo en un sistema real. |
| validation     | Define su papel en terraform, infrastructure manager y policy validation y cómo observarlo en un sistema real. |

### Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Terraform, Infrastructure Manager y policy validation"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar terraform, infrastructure manager y policy validation. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/232-terraform-infrastructure-manager-y-policy-validation/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-iac-stack en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye gcp-iac-stack para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 232 Terraform, Infrastructure Manager y policy validation

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega gcp-iac-stack con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 233 — Cloud Run, Functions, API Gateway y Workflows

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: serverless · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar cloud run, functions, api gateway y workflows dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud run, functions, api gateway y workflows con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-serverless con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                                |
|-----------|--------------------------------------------------------------------------------------------------------|
| cloud     | Define su papel en cloud run, functions, api gateway y workflows y cómo observarlo en un sistema real. |
| run       | Define su papel en cloud run, functions, api gateway y workflows y cómo observarlo en un sistema real. |
| functions | Define su papel en cloud run, functions, api gateway y workflows y cómo observarlo en un sistema real. |
| api       | Define su papel en cloud run, functions, api gateway y workflows y cómo observarlo en un sistema real. |
| gateway   | Define su papel en cloud run, functions, api gateway y workflows y cómo observarlo en un sistema real. |
| workflows | Define su papel en cloud run, functions, api gateway y workflows y cómo observarlo en un sistema real. |

### Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Cloud Run, Functions, API Gateway y Workflows"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud run, functions, api gateway y workflows. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/233-cloud-run-functions-api-gateway-y-workflows/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-serverless en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye gcp-serverless para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 233 Cloud Run, Functions, API Gateway y Workflows

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega gcp-serverless con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 234 — GKE Autopilot, Workload Identity y Config Sync

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: kubernetes · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar gke autopilot, workload identity y config sync dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que expicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar gke autopilot, workload identity y config sync con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-gke-platform con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                                 |
|-----------|---------------------------------------------------------------------------------------------------------|
| gke       | Define su papel en gke autopilot, workload identity y config sync y cómo observarlo en un sistema real. |
| autopilot | Define su papel en gke autopilot, workload identity y config sync y cómo observarlo en un sistema real. |
| workload  | Define su papel en gke autopilot, workload identity y config sync y cómo observarlo en un sistema real. |
| identity  | Define su papel en gke autopilot, workload identity y config sync y cómo observarlo en un sistema real. |
| config    | Define su papel en gke autopilot, workload identity y config sync y cómo observarlo en un sistema real. |
| sync      | Define su papel en gke autopilot, workload identity y config sync y cómo observarlo en un sistema real. |

### Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: GKE Autopilot, Workload Identity y Config Sync"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar gke autopilot, workload identity y config sync. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/234-gke-autopilot-workload-identity-y-config-sync/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-gke-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye gcp-gke-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 234 GKE Autopilot, Workload Identity y Config Sync

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega gcp-gke-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 235 — Cloud SQL, Spanner, Firestore y Bigtable

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: data · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar cloud sql, spanner, firestore y bigtable dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud sql, spanner, firestore y bigtable con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-operational-data con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                           |
|-----------|---------------------------------------------------------------------------------------------------|
| cloud     | Define su papel en cloud sql, spanner, firestore y bigtable y cómo observarlo en un sistema real. |
| sql       | Define su papel en cloud sql, spanner, firestore y bigtable y cómo observarlo en un sistema real. |
| spanner   | Define su papel en cloud sql, spanner, firestore y bigtable y cómo observarlo en un sistema real. |
| firestore | Define su papel en cloud sql, spanner, firestore y bigtable y cómo observarlo en un sistema real. |
| bigtable  | Define su papel en cloud sql, spanner, firestore y bigtable y cómo observarlo en un sistema real. |

## Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Cloud SQL, Spanner, Firestore y Bigtable"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud sql, spanner, firestore y bigtable. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/235-cloud-sql-spanner-firestore-y-bigtable/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-operational-data en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `gcp-operational-data` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.

- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 235 Cloud SQL, Spanner, Firestore y Bigtable

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega gcp-operational-data con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 236 — BigQuery, Dataflow, Dataproc y gobernanza de datos

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: data · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar bigquery, dataflow, dataproc y gobernanza de datos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar bigquery, dataflow, dataproc y gobernanza de datos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-analytics-platform con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                                     |
|------------|-------------------------------------------------------------------------------------------------------------|
| bigquery   | Define su papel en bigquery, dataflow, dataproc y gobernanza de datos y cómo observarlo en un sistema real. |
| dataflow   | Define su papel en bigquery, dataflow, dataproc y gobernanza de datos y cómo observarlo en un sistema real. |
| dataproc   | Define su papel en bigquery, dataflow, dataproc y gobernanza de datos y cómo observarlo en un sistema real. |
| gobernanza | Define su papel en bigquery, dataflow, dataproc y gobernanza de datos y cómo observarlo en un sistema real. |
| datos      | Define su papel en bigquery, dataflow, dataproc y gobernanza de datos y cómo observarlo en un sistema real. |

### Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: BigQuery, Dataflow, Dataproc y gobernanza de datos"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar bigquery, dataflow, dataproc y gobernanza de datos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/236-bigquery-dataflow-dataproc-y-gobernanza-de-datos/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-analytics-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye gcp-analytics-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 236 BigQuery, Dataflow, Dataproc y gobernanza de datos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega gcp-analytics-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 237 — Pub/Sub, Eventarc y entrega exactamente-una-vez

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: messaging · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar pub/sub, eventarc y entrega exactamente-una-vez dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar pub/sub, eventarc y entrega exactamente-una-vez con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-event-platform con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                                  |
|-------------|----------------------------------------------------------------------------------------------------------|
| pub         | Define su papel en pub/sub, eventarc y entrega exactamente-una-vez y cómo observarlo en un sistema real. |
| sub         | Define su papel en pub/sub, eventarc y entrega exactamente-una-vez y cómo observarlo en un sistema real. |
| eventarc    | Define su papel en pub/sub, eventarc y entrega exactamente-una-vez y cómo observarlo en un sistema real. |
| entrega     | Define su papel en pub/sub, eventarc y entrega exactamente-una-vez y cómo observarlo en un sistema real. |
| exactamente | Define su papel en pub/sub, eventarc y entrega exactamente-una-vez y cómo observarlo en un sistema real. |
| vez         | Define su papel en pub/sub, eventarc y entrega exactamente-una-vez y cómo observarlo en un sistema real. |

### Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Pub/Sub, Eventarc y entrega exactamente-una-vez"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E["¿Cumple seguridad, SLO y costo?"]
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar pub/sub, eventarc y entrega exactamente-una-vez. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/237-pub-sub-eventarc-y-entrega-exactamente-una-vez/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-event-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye gcp-event-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 237 Pub/Sub, Eventarc y entrega exactamente-una-vez

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega gcp-event-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 238 — Cloud Operations, Trace y OpenTelemetry

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: observability · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar cloud operations, trace y opentelemetry dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud operations, trace y opentelemetry con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-observability con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto      | Comprensión verificable                                                                          |
|---------------|--------------------------------------------------------------------------------------------------|
| cloud         | Define su papel en cloud operations, trace y opentelemetry y cómo observarlo en un sistema real. |
| operations    | Define su papel en cloud operations, trace y opentelemetry y cómo observarlo en un sistema real. |
| trace         | Define su papel en cloud operations, trace y opentelemetry y cómo observarlo en un sistema real. |
| opentelemetry | Define su papel en cloud operations, trace y opentelemetry y cómo observarlo en un sistema real. |

### Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Cloud Operations, Trace y OpenTelemetry"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud operations, trace y opentelemetry. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/238-cloud-operations-trace-y-opentelemetry/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-observability en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye gcp-observability para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 238 Cloud Operations, Trace y OpenTelemetry

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega gcp-observability con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 239 — SCC, VPC Service Controls, KMS y FinOps

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4  
Laboratorio: security · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar scc, vpc service controls, kms y finops dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar scc, vpc service controls, kms y finops con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-security-finops con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                          |
|----------|--------------------------------------------------------------------------------------------------|
| scc      | Define su papel en scc, vpc service controls, kms y finops y cómo observarlo en un sistema real. |
| vpc      | Define su papel en scc, vpc service controls, kms y finops y cómo observarlo en un sistema real. |
| service  | Define su papel en scc, vpc service controls, kms y finops y cómo observarlo en un sistema real. |
| controls | Define su papel en scc, vpc service controls, kms y finops y cómo observarlo en un sistema real. |
| kms      | Define su papel en scc, vpc service controls, kms y finops y cómo observarlo en un sistema real. |
| finops   | Define su papel en scc, vpc service controls, kms y finops y cómo observarlo en un sistema real. |

### Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: SCC, VPC Service Controls, KMS y FinOps"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar scc, vpc service controls, kms y finops. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/239-scc-vpc-service-controls-kms-y-finops/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-security-finops en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `gcp-security-finops` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.

- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 239 SCC, VPC Service Controls, KMS y FinOps

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega gcp-security-finops con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 240 — Proyecto: CloudShop productivo en Google Cloud

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 8  
Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar proyecto: cloudshop productivo en google cloud dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: cloudshop productivo en google cloud con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloudshop-gcp con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                                 |
|------------|---------------------------------------------------------------------------------------------------------|
| proyecto   | Define su papel en proyecto: cloudshop productivo en google cloud y cómo observarlo en un sistema real. |
| cloudshop  | Define su papel en proyecto: cloudshop productivo en google cloud y cómo observarlo en un sistema real. |
| productivo | Define su papel en proyecto: cloudshop productivo en google cloud y cómo observarlo en un sistema real. |
| google     | Define su papel en proyecto: cloudshop productivo en google cloud y cómo observarlo en un sistema real. |
| cloud      | Define su papel en proyecto: cloudshop productivo en google cloud y cómo observarlo en un sistema real. |

### Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Proyecto: CloudShop productivo en Google Cloud"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: cloudshop productivo en google cloud. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/240-proyecto-cloudshop-productivo-en-google-cloud/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa cloudshop-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye cloudshop-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 240 Proyecto: CloudShop productivo en Google Cloud

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega cloudshop-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 20 — Plataformas cloud de datos, analítica, IA y agentes

← Parte 19 · Índice completo · Parte 21 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar plataformas gobernadas para batch, streaming, ML, IA generativa y agentes.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

## Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                    | Laboratorio   | Horas |
|-----|----------------------------------------------------------|---------------|-------|
| 241 | Lakehouse, warehouse, mesh y contratos de datos          | data          | 4     |
| 242 | Ingesta batch, CDC y streaming                           | data          | 4     |
| 243 | Orquestación, calidad, lineage y observabilidad de datos | observability | 4     |
| 244 | Feature stores, training pipelines y experiment tracking | platform      | 4     |
| 245 | Serving online, batch inference y escalado de modelos    | performance   | 4     |
| 246 | MLOps, registro, promoción, drift y rollback             | delivery      | 4     |
| 247 | Modelos fundacionales, tokens, embeddings y RAG          | architecture  | 4     |
| 248 | Bedrock, Azure AI Foundry y Vertex AI                    | decision      | 4     |
| 249 | Agentes, tools, memoria, permisos y guardrails           | security      | 4     |
| 250 | Evaluación de IA, red teaming y observabilidad           | testing       | 4     |
| 251 | Privacidad, gobernanza, sostenibilidad y costo de IA     | governance    | 4     |
| 252 | Proyecto: asistente operativo de CloudShop               | capstone      | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.



Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

### **Pauta bibliográfica**

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.

## 241 — Lakehouse, warehouse, mesh y contratos de datos

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar lakehouse, warehouse, mesh y contratos de datos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar lakehouse, warehouse, mesh y contratos de datos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar data-architecture con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                                  |
|-----------|----------------------------------------------------------------------------------------------------------|
| lakehouse | Define su papel en lakehouse, warehouse, mesh y contratos de datos y cómo observarlo en un sistema real. |
| warehouse | Define su papel en lakehouse, warehouse, mesh y contratos de datos y cómo observarlo en un sistema real. |
| mesh      | Define su papel en lakehouse, warehouse, mesh y contratos de datos y cómo observarlo en un sistema real. |
| contratos | Define su papel en lakehouse, warehouse, mesh y contratos de datos y cómo observarlo en un sistema real. |
| datos     | Define su papel en lakehouse, warehouse, mesh y contratos de datos y cómo observarlo en un sistema real. |

### Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Lakehouse, warehouse, mesh y contratos de datos"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar lakehouse, warehouse, mesh y contratos de datos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/241-lakehouse-warehouse-mesh-y-contratos-de-datos/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa data-architecture en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye data-architecture para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 241 Lakehouse, warehouse, mesh y contratos de datos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega data-architecture con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 242 — Ingesta batch, CDC y streaming

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar ingesta batch, cdc y streaming dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ingesta batch, cdc y streaming con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar ingestion-pipeline con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                 |
|-----------|-----------------------------------------------------------------------------------------|
| ingesta   | Define su papel en ingesta batch, cdc y streaming y cómo observarlo en un sistema real. |
| batch     | Define su papel en ingesta batch, cdc y streaming y cómo observarlo en un sistema real. |
| cdc       | Define su papel en ingesta batch, cdc y streaming y cómo observarlo en un sistema real. |
| streaming | Define su papel en ingesta batch, cdc y streaming y cómo observarlo en un sistema real. |

### Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Ingesta batch, CDC y streaming"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ingesta batch, cdc y streaming. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/242-ingesta-batch-cdc-y-streaming/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa ingestion-pipeline en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye ingestion-pipeline para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 242 Ingesta batch, CDC y streaming

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega ingestion-pipeline con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 243 — Orquestación, calidad, lineage y observabilidad de datos

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar orquestación, calidad, lineage y observabilidad de datos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar orquestación, calidad, lineage y observabilidad de datos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar data-reliability con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto       | Comprensión verificable                                                                                           |
|----------------|-------------------------------------------------------------------------------------------------------------------|
| orquestación   | Define su papel en orquestación, calidad, lineage y observabilidad de datos y cómo observarlo en un sistema real. |
| calidad        | Define su papel en orquestación, calidad, lineage y observabilidad de datos y cómo observarlo en un sistema real. |
| lineage        | Define su papel en orquestación, calidad, lineage y observabilidad de datos y cómo observarlo en un sistema real. |
| observabilidad | Define su papel en orquestación, calidad, lineage y observabilidad de datos y cómo observarlo en un sistema real. |
| datos          | Define su papel en orquestación, calidad, lineage y observabilidad de datos y cómo observarlo en un sistema real. |

### Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Orquestación, calidad, lineage y observabilidad de datos"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar orquestación, calidad, lineage y observabilidad de datos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/243-orquestacion-calidad-lineage-y-observabilidad-de-datos/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa data-reliability en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye data-reliability para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 243 Orquestación, calidad, lineage y observabilidad de datos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega data-reliability con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 244 — Feature stores, training pipelines y experiment tracking

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: platform · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar feature stores, training pipelines y experiment tracking dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar feature stores, training pipelines y experiment tracking con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar ml-platform con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                                           |
|------------|-------------------------------------------------------------------------------------------------------------------|
| feature    | Define su papel en feature stores, training pipelines y experiment tracking y cómo observarlo en un sistema real. |
| stores     | Define su papel en feature stores, training pipelines y experiment tracking y cómo observarlo en un sistema real. |
| training   | Define su papel en feature stores, training pipelines y experiment tracking y cómo observarlo en un sistema real. |
| pipelines  | Define su papel en feature stores, training pipelines y experiment tracking y cómo observarlo en un sistema real. |
| experiment | Define su papel en feature stores, training pipelines y experiment tracking y cómo observarlo en un sistema real. |
| tracking   | Define su papel en feature stores, training pipelines y experiment tracking y cómo observarlo en un sistema real. |

### Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Feature stores, training pipelines y experiment tracking"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar feature stores, training pipelines y experiment tracking. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/244-feature-stores-training-pipelines-y-experiment-tracking/lab.py
```

El laboratorio selecciona el motor de práctica platform y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa ml-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es una capacidad autoservicio con contrato y golden path. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye ml-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 244 Feature stores, training pipelines y experiment tracking

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega ml-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 245 — Serving online, batch inference y escalado de modelos

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: performance · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar serving online, batch inference y escalado de modelos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar serving online, batch inference y escalado de modelos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar model-serving con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto  | Comprensión verificable                                                                                        |
|-----------|----------------------------------------------------------------------------------------------------------------|
| serving   | Define su papel en serving online, batch inference y escalado de modelos y cómo observarlo en un sistema real. |
| online    | Define su papel en serving online, batch inference y escalado de modelos y cómo observarlo en un sistema real. |
| batch     | Define su papel en serving online, batch inference y escalado de modelos y cómo observarlo en un sistema real. |
| inference | Define su papel en serving online, batch inference y escalado de modelos y cómo observarlo en un sistema real. |
| escalado  | Define su papel en serving online, batch inference y escalado de modelos y cómo observarlo en un sistema real. |
| modelos   | Define su papel en serving online, batch inference y escalado de modelos y cómo observarlo en un sistema real. |

## Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Serving online, batch inference y escalado de modelos"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar serving online, batch inference y escalado de modelos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/245-serving-online-batch-inference-y-escalado-de-modelos/lab.py
```

El laboratorio selecciona el motor de práctica performance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa model-serving en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una prueba de carga con baseline y cuello de botella. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye model-serving para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 245 Serving online, batch inference y escalado de modelos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega model-serving con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 246 — MLOps, registro, promoción, drift y rollback

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: delivery · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar mlops, registro, promoción, drift y rollback dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar mlops, registro, promoción, drift y rollback con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar mlops-pipeline con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                               |
|-----------|-------------------------------------------------------------------------------------------------------|
| mlops     | Define su papel en mlops, registro, promoción, drift y rollback y cómo observarlo en un sistema real. |
| registro  | Define su papel en mlops, registro, promoción, drift y rollback y cómo observarlo en un sistema real. |
| promoción | Define su papel en mlops, registro, promoción, drift y rollback y cómo observarlo en un sistema real. |
| drift     | Define su papel en mlops, registro, promoción, drift y rollback y cómo observarlo en un sistema real. |
| rollback  | Define su papel en mlops, registro, promoción, drift y rollback y cómo observarlo en un sistema real. |

### Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: MLOps, registro, promoción, drift y rollback"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar mlops, registro, promoción, drift y rollback. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/246-mlops-registro-promocion-drift-y-rollback/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa mlops-pipeline en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

### Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye mlops-pipeline para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

### Referencias

- Designing Machine Learning Systems — Chip Huyen.



- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 246 MLOps, registro, promoción, drift y rollback

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega mlops-pipeline con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 247 — Modelos fundacionales, tokens, embeddings y RAG

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar modelos fundacionales, tokens, embeddings y rag dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar modelos fundacionales, tokens, embeddings y rag con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar rag-system con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                                  |
|---------------|----------------------------------------------------------------------------------------------------------|
| modelos       | Define su papel en modelos fundacionales, tokens, embeddings y rag y cómo observarlo en un sistema real. |
| fundacionales | Define su papel en modelos fundacionales, tokens, embeddings y rag y cómo observarlo en un sistema real. |
| tokens        | Define su papel en modelos fundacionales, tokens, embeddings y rag y cómo observarlo en un sistema real. |
| embeddings    | Define su papel en modelos fundacionales, tokens, embeddings y rag y cómo observarlo en un sistema real. |
| rag           | Define su papel en modelos fundacionales, tokens, embeddings y rag y cómo observarlo en un sistema real. |

## Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Modelos fundacionales, tokens, embeddings y RAG"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar modelos fundacionales, tokens, embeddings y rag. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/247-modelos-fundacionales-tokens-embeddings-y-rag/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa rag-system en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye rag-system para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 247 Modelos fundacionales, tokens, embeddings y RAG

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega rag-system con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 248 — Bedrock, Azure AI Foundry y Vertex AI

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: decision · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar bedrock, azure ai foundry y vertex ai dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar bedrock, azure ai foundry y vertex ai con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar genai-provider-adr con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                        |
|----------|------------------------------------------------------------------------------------------------|
| bedrock  | Define su papel en bedrock, azure ai foundry y vertex ai y cómo observarlo en un sistema real. |
| azure    | Define su papel en bedrock, azure ai foundry y vertex ai y cómo observarlo en un sistema real. |
| ai       | Define su papel en bedrock, azure ai foundry y vertex ai y cómo observarlo en un sistema real. |
| foundry  | Define su papel en bedrock, azure ai foundry y vertex ai y cómo observarlo en un sistema real. |
| vertex   | Define su papel en bedrock, azure ai foundry y vertex ai y cómo observarlo en un sistema real. |
| ai       | Define su papel en bedrock, azure ai foundry y vertex ai y cómo observarlo en un sistema real. |

### Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Bedrock, Azure AI Foundry y Vertex AI"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar bedrock, azure ai foundry y vertex ai. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/248-bedrock-azure-ai-foundry-y-vertex-ai/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa genai-provider-adr en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `genai-provider-adr` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Machine Learning Systems — Chip Huyen.



- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 248 Bedrock, Azure AI Foundry y Vertex AI

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega genai-provider-adr con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 249 — Agentes, tools, memoria, permisos y guardrails

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar agentes, tools, memoria, permisos y guardrails dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar agentes, tools, memoria, permisos y guardrails con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar agent-architecture con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                                 |
|------------|---------------------------------------------------------------------------------------------------------|
| agentes    | Define su papel en agentes, tools, memoria, permisos y guardrails y cómo observarlo en un sistema real. |
| tools      | Define su papel en agentes, tools, memoria, permisos y guardrails y cómo observarlo en un sistema real. |
| memoria    | Define su papel en agentes, tools, memoria, permisos y guardrails y cómo observarlo en un sistema real. |
| permisos   | Define su papel en agentes, tools, memoria, permisos y guardrails y cómo observarlo en un sistema real. |
| guardrails | Define su papel en agentes, tools, memoria, permisos y guardrails y cómo observarlo en un sistema real. |

### Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Agentes, tools, memoria, permisos y guardrails"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar agentes, tools, memoria, permisos y guardrails. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/249-agentes-tools-memoria-permisos-y-guardrails/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa agent-architecture en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `agent-architecture` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- `Designing Machine Learning Systems` — Chip Huyen.

- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 249 Agentes, tools, memoria, permisos y guardrails

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega agent-architecture con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 250 — Evaluación de IA, red teaming y observabilidad

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: testing · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar evaluación de ia, red teaming y observabilidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar evaluación de ia, red teaming y observabilidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar ai-evaluation con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto       | Comprensión verificable                                                                                 |
|----------------|---------------------------------------------------------------------------------------------------------|
| evaluación     | Define su papel en evaluación de ia, red teaming y observabilidad y cómo observarlo en un sistema real. |
| ia             | Define su papel en evaluación de ia, red teaming y observabilidad y cómo observarlo en un sistema real. |
| red            | Define su papel en evaluación de ia, red teaming y observabilidad y cómo observarlo en un sistema real. |
| teaming        | Define su papel en evaluación de ia, red teaming y observabilidad y cómo observarlo en un sistema real. |
| observabilidad | Define su papel en evaluación de ia, red teaming y observabilidad y cómo observarlo en un sistema real. |

### Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Evaluación de IA, red teaming y observabilidad"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar evaluación de ia, red teaming y observabilidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/250-evaluacion-de-ia-red-teaming-y-observabilidad/lab.py
```

El laboratorio selecciona el motor de práctica testing y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa ai-evaluation en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es pruebas automatizadas con fallos diagnósticos. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye ai-evaluation para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 250 Evaluación de IA, red teaming y observabilidad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega ai-evaluation con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 251 — Privacidad, gobernanza, sostenibilidad y costo de IA

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar privacidad, gobernanza, sostenibilidad y costo de ia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar privacidad, gobernanza, sostenibilidad y costo de ia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar responsible-ai-controls con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto       | Comprensión verificable                                                                                       |
|----------------|---------------------------------------------------------------------------------------------------------------|
| privacidad     | Define su papel en privacidad, gobernanza, sostenibilidad y costo de ia y cómo observarlo en un sistema real. |
| gobernanza     | Define su papel en privacidad, gobernanza, sostenibilidad y costo de ia y cómo observarlo en un sistema real. |
| sostenibilidad | Define su papel en privacidad, gobernanza, sostenibilidad y costo de ia y cómo observarlo en un sistema real. |
| costo          | Define su papel en privacidad, gobernanza, sostenibilidad y costo de ia y cómo observarlo en un sistema real. |
| ia             | Define su papel en privacidad, gobernanza, sostenibilidad y costo de ia y cómo observarlo en un sistema real. |

## Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Privacidad, gobernanza, sostenibilidad y costo de IA"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar privacidad, gobernanza, sostenibilidad y costo de ia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/251-privacidad-gobernanza-sostenibilidad-y-costo-de-ia/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa responsible-ai-controls en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye responsible-ai-controls para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 251 Privacidad, gobernanza, sostenibilidad y costo de IA

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega responsible-ai-controls con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 252 — Proyecto: asistente operativo de CloudShop

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar proyecto: asistente operativo de cloudshop dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: asistente operativo de cloudshop con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloudshop-ai-assistant con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                             |
|-----------|-----------------------------------------------------------------------------------------------------|
| proyecto  | Define su papel en proyecto: asistente operativo de cloudshop y cómo observarlo en un sistema real. |
| asistente | Define su papel en proyecto: asistente operativo de cloudshop y cómo observarlo en un sistema real. |
| operativo | Define su papel en proyecto: asistente operativo de cloudshop y cómo observarlo en un sistema real. |
| cloudshop | Define su papel en proyecto: asistente operativo de cloudshop y cómo observarlo en un sistema real. |

### Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: asistente operativo de CloudShop"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: asistente operativo de cloudshop. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/252-proyecto-asistente-operativo-de-cloudshop/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloudshop-ai-assistant en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye cloudshop-ai-assistant para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 252 Proyecto: asistente operativo de CloudShop

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega cloudshop-ai-assistant con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 21 — Operación cloud, automatización y respuesta a incidentes

← Parte 20 · Índice completo · Parte 22 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Convertir la operación diaria en procesos observables, repetibles y progresivamente automatizados.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

## Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                 | Laboratorio | Horas |
|-----|-------------------------------------------------------|-------------|-------|
| 253 | Inventario, etiquetado, CMDB y ownership              | governance  | 4     |
| 254 | Patching, imágenes doradas y gestión de configuración | operations  | 4     |
| 255 | Backups, restore testing, vaults e inmutabilidad      | reliability | 4     |
| 256 | Administración remota sin SSH permanente              | security    | 4     |
| 257 | Alertas, on-call, escalamiento y comunicación         | incident    | 4     |
| 258 | Triage de red, cómputo, datos y dependencias          | incident    | 4     |
| 259 | Runbooks ejecutables y auto-remediation               | operations  | 4     |
| 260 | Change management, ventanas y rollback                | delivery    | 4     |
| 261 | Game days, chaos engineering y aprendizaje            | chaos       | 4     |
| 262 | Capacity planning, cuotas y gestión de demanda        | capacity    | 4     |
| 263 | AIOps, automatización asistida y límites humanos      | governance  | 4     |
| 264 | Proyecto: centro de operaciones de CloudShop          | capstone    | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.

## 253 — Inventario, etiquetado, CMDB y ownership

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4  
Laboratorio: governance · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar inventario, etiquetado, cmdb y ownership dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar inventario, etiquetado, cmdb y ownership con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloud-inventory con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                           |
|------------|---------------------------------------------------------------------------------------------------|
| inventario | Define su papel en inventario, etiquetado, cmdb y ownership y cómo observarlo en un sistema real. |
| etiquetado | Define su papel en inventario, etiquetado, cmdb y ownership y cómo observarlo en un sistema real. |
| cmdb       | Define su papel en inventario, etiquetado, cmdb y ownership y cómo observarlo en un sistema real. |
| ownership  | Define su papel en inventario, etiquetado, cmdb y ownership y cómo observarlo en un sistema real. |

### Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Inventario, etiquetado, CMDB y ownership"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar inventario, etiquetado, cmdb y ownership. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/253-inventario-etiquetado-cmdb-y-ownership/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloud-inventory en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye cloud-inventory para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 253 Inventario, etiquetado, CMDB y ownership

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega cloud-inventory con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 254 — Patching, imágenes doradas y gestión de configuración

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4

Laboratorio: operations · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar patching, imágenes doradas y gestión de configuración dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar patching, imágenes doradas y gestión de configuración con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar patch-strategy con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                                        |
|---------------|----------------------------------------------------------------------------------------------------------------|
| patching      | Define su papel en patching, imágenes doradas y gestión de configuración y cómo observarlo en un sistema real. |
| imágenes      | Define su papel en patching, imágenes doradas y gestión de configuración y cómo observarlo en un sistema real. |
| doradas       | Define su papel en patching, imágenes doradas y gestión de configuración y cómo observarlo en un sistema real. |
| gestión       | Define su papel en patching, imágenes doradas y gestión de configuración y cómo observarlo en un sistema real. |
| configuración | Define su papel en patching, imágenes doradas y gestión de configuración y cómo observarlo en un sistema real. |

## Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Patching, imágenes doradas y gestión de configuración"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar patching, imágenes doradas y gestión de configuración. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/254-patching-imagenes-doradas-y-gestion-de-configuracion/lab.py
```

El laboratorio selecciona el motor de práctica operations y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa patch-strategy en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es un runbook probado por otra persona. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye patch-strategy para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 254 Patching, imágenes doradas y gestión de configuración

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega patch-strategy con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 255 — Backups, restore testing, vaults e inmutabilidad

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4  
Laboratorio: reliability · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar backups, restore testing, vaults e inmutabilidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar backups, restore testing, vaults e inmutabilidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar backup-restore con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto      | Comprensión verificable                                                                                   |
|---------------|-----------------------------------------------------------------------------------------------------------|
| backups       | Define su papel en backups, restore testing, vaults e inmutabilidad y cómo observarlo en un sistema real. |
| restore       | Define su papel en backups, restore testing, vaults e inmutabilidad y cómo observarlo en un sistema real. |
| testing       | Define su papel en backups, restore testing, vaults e inmutabilidad y cómo observarlo en un sistema real. |
| vaults        | Define su papel en backups, restore testing, vaults e inmutabilidad y cómo observarlo en un sistema real. |
| e             | Define su papel en backups, restore testing, vaults e inmutabilidad y cómo observarlo en un sistema real. |
| inmutabilidad | Define su papel en backups, restore testing, vaults e inmutabilidad y cómo observarlo en un sistema real. |

## Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Backups, restore testing, vaults e inmutabilidad"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar backups, restore testing, vaults e inmutabilidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/255-backups-restore-testing-vaults-e-inmutabilidad/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa backup-restore en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye backup-restore para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 255 Backups, restore testing, vaults e inmutabilidad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega backup-restore con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 256 — Administración remota sin SSH permanente

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4  
Laboratorio: security · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar administración remota sin ssh permanente dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar administración remota sin ssh permanente con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar secure-operations con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto       | Comprensión verificable                                                                           |
|----------------|---------------------------------------------------------------------------------------------------|
| administración | Define su papel en administración remota sin ssh permanente y cómo observarlo en un sistema real. |
| remota         | Define su papel en administración remota sin ssh permanente y cómo observarlo en un sistema real. |
| sin            | Define su papel en administración remota sin ssh permanente y cómo observarlo en un sistema real. |
| ssh            | Define su papel en administración remota sin ssh permanente y cómo observarlo en un sistema real. |
| permanente     | Define su papel en administración remota sin ssh permanente y cómo observarlo en un sistema real. |

### Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Administración remota sin SSH permanente"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E["¿Cumple seguridad, SLO y costo?"]
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar administración remota sin ssh permanente. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/256-administracion-remota-sin-ssh-permanente/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa secure-operations en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `secure-operations` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Site Reliability Workbook — Beyer et al..

- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 256 Administración remota sin SSH permanente

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega secure-operations con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 257 — Alertas, on-call, escalamiento y comunicación

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4  
Laboratorio: incident · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar alertas, on-call, escalamiento y comunicación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar alertas, on-call, escalamiento y comunicación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar oncall-system con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                                                |
|--------------|--------------------------------------------------------------------------------------------------------|
| alertas      | Define su papel en alertas, on-call, escalamiento y comunicación y cómo observarlo en un sistema real. |
| on           | Define su papel en alertas, on-call, escalamiento y comunicación y cómo observarlo en un sistema real. |
| call         | Define su papel en alertas, on-call, escalamiento y comunicación y cómo observarlo en un sistema real. |
| escalamiento | Define su papel en alertas, on-call, escalamiento y comunicación y cómo observarlo en un sistema real. |
| comunicación | Define su papel en alertas, on-call, escalamiento y comunicación y cómo observarlo en un sistema real. |

### Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Alertas, on-call, escalamiento y comunicación"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar alertas, on-call, escalamiento y comunicación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/257-alertas-on-call-escalamiento-y-comunicacion/lab.py
```

El laboratorio selecciona el motor de práctica incident y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa oncall-system en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una cronología, roles, comunicación y aprendizaje. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye oncall-system para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 257 Alertas, on-call, escalamiento y comunicación

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega oncall-system con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 258 — Triage de red, cómputo, datos y dependencias

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4  
Laboratorio: incident · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar triage de red, cómputo, datos y dependencias dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar triage de red, cómputo, datos y dependencias con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar diagnostic-tree con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                                               |
|--------------|-------------------------------------------------------------------------------------------------------|
| triage       | Define su papel en triage de red, cómputo, datos y dependencias y cómo observarlo en un sistema real. |
| red          | Define su papel en triage de red, cómputo, datos y dependencias y cómo observarlo en un sistema real. |
| cómputo      | Define su papel en triage de red, cómputo, datos y dependencias y cómo observarlo en un sistema real. |
| datos        | Define su papel en triage de red, cómputo, datos y dependencias y cómo observarlo en un sistema real. |
| dependencias | Define su papel en triage de red, cómputo, datos y dependencias y cómo observarlo en un sistema real. |

### Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Triage de red, cómputo, datos y dependencias"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar triage de red, cómputo, datos y dependencias. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/258-triage-de-red-computo-datos-y-dependencias/lab.py
```

El laboratorio selecciona el motor de práctica incident y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa diagnostic-tree en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una cronología, roles, comunicación y aprendizaje. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye diagnostic-tree para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 258 Triage de red, cómputo, datos y dependencias

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega diagnostic-tree con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 259 — Runbooks ejecutables y auto-remediation

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4  
Laboratorio: operations · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar runbooks ejecutables y auto-remediation dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar runbooks ejecutables y auto-remediation con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar executable-runbook con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                          |
|-------------|--------------------------------------------------------------------------------------------------|
| runbooks    | Define su papel en runbooks ejecutables y auto-remediation y cómo observarlo en un sistema real. |
| ejecutables | Define su papel en runbooks ejecutables y auto-remediation y cómo observarlo en un sistema real. |
| auto        | Define su papel en runbooks ejecutables y auto-remediation y cómo observarlo en un sistema real. |
| remediation | Define su papel en runbooks ejecutables y auto-remediation y cómo observarlo en un sistema real. |

### Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Runbooks ejecutables y auto-remediation"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar runbooks ejecutables y auto-remediation. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/259-runbooks-ejecutables-y-auto-remediation/lab.py
```

El laboratorio selecciona el motor de práctica operations y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa executable-runbook en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un runbook probado por otra persona. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye executable-runbook para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 259 Runbooks ejecutables y auto-remediation

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega ejecutable-runbook con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 260 — Change management, ventanas y rollback

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4  
Laboratorio: delivery · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar change management, ventanas y rollback dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar change management, ventanas y rollback con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar change-plan con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                         |
|------------|-------------------------------------------------------------------------------------------------|
| change     | Define su papel en change management, ventanas y rollback y cómo observarlo en un sistema real. |
| management | Define su papel en change management, ventanas y rollback y cómo observarlo en un sistema real. |
| ventanas   | Define su papel en change management, ventanas y rollback y cómo observarlo en un sistema real. |
| rollback   | Define su papel en change management, ventanas y rollback y cómo observarlo en un sistema real. |

### Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Change management, ventanas y rollback"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar change management, ventanas y rollback. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/260-change-management-ventanas-y-rollback/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa change-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye change-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 260 Change management, ventanas y rollback

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega change-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 261 — Game days, chaos engineering y aprendizaje

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4  
Laboratorio: chaos · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar game days, chaos engineering y aprendizaje dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que expicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar game days, chaos engineering y aprendizaje con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gameday-report con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                             |
|-------------|-----------------------------------------------------------------------------------------------------|
| game        | Define su papel en game days, chaos engineering y aprendizaje y cómo observarlo en un sistema real. |
| days        | Define su papel en game days, chaos engineering y aprendizaje y cómo observarlo en un sistema real. |
| chaos       | Define su papel en game days, chaos engineering y aprendizaje y cómo observarlo en un sistema real. |
| engineering | Define su papel en game days, chaos engineering y aprendizaje y cómo observarlo en un sistema real. |
| aprendizaje | Define su papel en game days, chaos engineering y aprendizaje y cómo observarlo en un sistema real. |

### Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Game days, chaos engineering y aprendizaje"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar game days, chaos engineering y aprendizaje. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/261-game-days-chaos-engineering-y-aprendizaje/lab.py
```

El laboratorio selecciona el motor de práctica chaos y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gameday-report en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una hipótesis de resiliencia y criterio de abortar. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye gameday-report para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 261 Game days, chaos engineering y aprendizaje

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega gameday-report con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 262 — Capacity planning, cuotas y gestión de demanda

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4  
Laboratorio: capacity · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar capacity planning, cuotas y gestión de demanda dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capacity planning, cuotas y gestión de demanda con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar capacity-plan con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                                 |
|----------|---------------------------------------------------------------------------------------------------------|
| capacity | Define su papel en capacity planning, cuotas y gestión de demanda y cómo observarlo en un sistema real. |
| planning | Define su papel en capacity planning, cuotas y gestión de demanda y cómo observarlo en un sistema real. |
| cuotas   | Define su papel en capacity planning, cuotas y gestión de demanda y cómo observarlo en un sistema real. |
| gestión  | Define su papel en capacity planning, cuotas y gestión de demanda y cómo observarlo en un sistema real. |
| demanda  | Define su papel en capacity planning, cuotas y gestión de demanda y cómo observarlo en un sistema real. |

### Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capacity planning, cuotas y gestión de demanda"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capacity planning, cuotas y gestión de demanda. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/262-capacity-planning-cuotas-y-gestion-de-demanda/lab.py
```

El laboratorio selecciona el motor de práctica capacity y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa capacity-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es requests, límites y una decisión de escalado medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye capacity-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 262 Capacity planning, cuotas y gestión de demanda

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega capacity-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 263 — AIOps, automatización asistida y límites humanos

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4  
Laboratorio: governance · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar aiops, automatización asistida y límites humanos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar aiops, automatización asistida y límites humanos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aiops-control con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto       | Comprensión verificable                                                                                   |
|----------------|-----------------------------------------------------------------------------------------------------------|
| aiops          | Define su papel en aiops, automatización asistida y límites humanos y cómo observarlo en un sistema real. |
| automatización | Define su papel en aiops, automatización asistida y límites humanos y cómo observarlo en un sistema real. |
| asistida       | Define su papel en aiops, automatización asistida y límites humanos y cómo observarlo en un sistema real. |
| límites        | Define su papel en aiops, automatización asistida y límites humanos y cómo observarlo en un sistema real. |
| humanos        | Define su papel en aiops, automatización asistida y límites humanos y cómo observarlo en un sistema real. |

### Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: AIOps, automatización asistida y límites humanos"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar aiops, automatización asistida y límites humanos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/263-aiops-automatizacion-asistida-y-limites-humanos/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aiops-control en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye aiops-control para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 263 AIOps, automatización asistida y límites humanos

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega aiops-control con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 264 — Proyecto: centro de operaciones de CloudShop

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 8  
Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar proyecto: centro de operaciones de cloudshop dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: centro de operaciones de cloudshop con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloudshop-operations con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                               |
|-------------|-------------------------------------------------------------------------------------------------------|
| proyecto    | Define su papel en proyecto: centro de operaciones de cloudshop y cómo observarlo en un sistema real. |
| centro      | Define su papel en proyecto: centro de operaciones de cloudshop y cómo observarlo en un sistema real. |
| operaciones | Define su papel en proyecto: centro de operaciones de cloudshop y cómo observarlo en un sistema real. |
| cloudshop   | Define su papel en proyecto: centro de operaciones de cloudshop y cómo observarlo en un sistema real. |

### Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: centro de operaciones de CloudShop"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.



## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: centro de operaciones de cloudshop. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/264-proyecto-centro-de-operaciones-de-cloudshop/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloudshop-operations en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye cloudshop-operations para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 264 Proyecto: centro de operaciones de CloudShop

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega cloudshop-operations con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 22 — Especializaciones, certificaciones y práctica profesional

← Parte 21 · Índice completo · Parte 23 →

Nivel: intermedio-avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Transformar el conocimiento transversal en perfiles demostrables, preparación certificable y comunicación profesional.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

## Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                  | Laboratorio | Horas |
|-----|--------------------------------------------------------|-------------|-------|
| 265 | Ruta Cloud Engineer y mapa de competencias             | decision    | 4     |
| 266 | Ruta DevOps y Delivery Engineer                        | decision    | 4     |
| 267 | Ruta Platform Engineer                                 | decision    | 4     |
| 268 | Ruta Site Reliability Engineer                         | decision    | 4     |
| 269 | Ruta Cloud Security Engineer                           | decision    | 4     |
| 270 | Ruta FinOps Practitioner                               | decision    | 4     |
| 271 | Ruta Cloud Data y AI Engineer                          | decision    | 4     |
| 272 | Ruta Cloud Solutions Architect                         | decision    | 4     |
| 273 | Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps    | assessment  | 4     |
| 274 | Preguntas de escenario y estrategia de examen          | assessment  | 4     |
| 275 | Portafolio, evidencia, README y entrevista de sistemas | assessment  | 4     |
| 276 | Proyecto: defensa técnica ante panel                   | capstone    | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## **Pauta bibliográfica**

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.

## 265 — Ruta Cloud Engineer y mapa de competencias

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: decision · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar ruta cloud engineer y mapa de competencias dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta cloud engineer y mapa de competencias con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloud-engineer-plan con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                                             |
|--------------|-----------------------------------------------------------------------------------------------------|
| ruta         | Define su papel en ruta cloud engineer y mapa de competencias y cómo observarlo en un sistema real. |
| cloud        | Define su papel en ruta cloud engineer y mapa de competencias y cómo observarlo en un sistema real. |
| engineer     | Define su papel en ruta cloud engineer y mapa de competencias y cómo observarlo en un sistema real. |
| mapa         | Define su papel en ruta cloud engineer y mapa de competencias y cómo observarlo en un sistema real. |
| competencias | Define su papel en ruta cloud engineer y mapa de competencias y cómo observarlo en un sistema real. |

### Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Ruta Cloud Engineer y mapa de competencias"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta cloud engineer y mapa de competencias. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/265-ruta-cloud-engineer-y-mapa-de-competencias/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa cloud-engineer-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye cloud-engineer-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 265 Ruta Cloud Engineer y mapa de competencias

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega cloud-engineer-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 266 — Ruta DevOps y Delivery Engineer

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: decision · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar ruta devops y delivery engineer dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta devops y delivery engineer con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar devops-plan con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                  |
|----------|------------------------------------------------------------------------------------------|
| ruta     | Define su papel en ruta devops y delivery engineer y cómo observarlo en un sistema real. |
| devops   | Define su papel en ruta devops y delivery engineer y cómo observarlo en un sistema real. |
| delivery | Define su papel en ruta devops y delivery engineer y cómo observarlo en un sistema real. |
| engineer | Define su papel en ruta devops y delivery engineer y cómo observarlo en un sistema real. |

### Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Ruta DevOps y Delivery Engineer"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta devops y delivery engineer. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/266-ruta-devops-y-delivery-engineer/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa devops-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye devops-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 266 Ruta DevOps y Delivery Engineer

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega devops-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 267 — Ruta Platform Engineer

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: decision · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar ruta platform engineer dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta platform engineer con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar platform-engineer-plan con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                         |
|----------|---------------------------------------------------------------------------------|
| ruta     | Define su papel en ruta platform engineer y cómo observarlo en un sistema real. |
| platform | Define su papel en ruta platform engineer y cómo observarlo en un sistema real. |
| engineer | Define su papel en ruta platform engineer y cómo observarlo en un sistema real. |

### Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Ruta Platform Engineer"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta platform engineer. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/267-ruta-platform-engineer/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa platform-engineer-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye platform-engineer-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 267 Ruta Platform Engineer

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega platform-engineer-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 268 — Ruta Site Reliability Engineer

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: decision · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar ruta site reliability engineer dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta site reliability engineer con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar sre-plan con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                 |
|-------------|-----------------------------------------------------------------------------------------|
| ruta        | Define su papel en ruta site reliability engineer y cómo observarlo en un sistema real. |
| site        | Define su papel en ruta site reliability engineer y cómo observarlo en un sistema real. |
| reliability | Define su papel en ruta site reliability engineer y cómo observarlo en un sistema real. |
| engineer    | Define su papel en ruta site reliability engineer y cómo observarlo en un sistema real. |

### Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Ruta Site Reliability Engineer"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta site reliability engineer. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/268-ruta-site-reliability-engineer/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa sre-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye sre-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 268 Ruta Site Reliability Engineer

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega sre-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 269 — Ruta Cloud Security Engineer

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: decision · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar ruta cloud security engineer dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta cloud security engineer con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar security-engineer-plan con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                               |
|----------|---------------------------------------------------------------------------------------|
| ruta     | Define su papel en ruta cloud security engineer y cómo observarlo en un sistema real. |
| cloud    | Define su papel en ruta cloud security engineer y cómo observarlo en un sistema real. |
| security | Define su papel en ruta cloud security engineer y cómo observarlo en un sistema real. |
| engineer | Define su papel en ruta cloud security engineer y cómo observarlo en un sistema real. |

### Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Ruta Cloud Security Engineer"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta cloud security engineer. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/269-ruta-cloud-security-engineer/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa security-engineer-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye security-engineer-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 269 Ruta Cloud Security Engineer

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega security-engineer-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 270 — Ruta FinOps Practitioner

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: decision · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar ruta finops practitioner dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta finops practitioner con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar finops-plan con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                           |
|--------------|-----------------------------------------------------------------------------------|
| ruta         | Define su papel en ruta finops practitioner y cómo observarlo en un sistema real. |
| finops       | Define su papel en ruta finops practitioner y cómo observarlo en un sistema real. |
| practitioner | Define su papel en ruta finops practitioner y cómo observarlo en un sistema real. |

### Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Ruta FinOps Practitioner"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta finops practitioner. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/270-ruta-finops-practitioner/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa finops-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye finops-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 270 Ruta FinOps Practitioner

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega finops-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 271 — Ruta Cloud Data y AI Engineer

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: decision · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar ruta cloud data y ai engineer dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta cloud data y ai engineer con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar data-ai-plan con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                |
|----------|----------------------------------------------------------------------------------------|
| ruta     | Define su papel en ruta cloud data y ai engineer y cómo observarlo en un sistema real. |
| cloud    | Define su papel en ruta cloud data y ai engineer y cómo observarlo en un sistema real. |
| data     | Define su papel en ruta cloud data y ai engineer y cómo observarlo en un sistema real. |
| ai       | Define su papel en ruta cloud data y ai engineer y cómo observarlo en un sistema real. |
| engineer | Define su papel en ruta cloud data y ai engineer y cómo observarlo en un sistema real. |

### Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Ruta Cloud Data y AI Engineer"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta cloud data y ai engineer. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/271-ruta-cloud-data-y-ai-engineer/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa data-ai-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye data-ai-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.



## Evaluación — 271 Ruta Cloud Data y AI Engineer

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega data-ai-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 272 — Ruta Cloud Solutions Architect

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: decision · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar ruta cloud solutions architect dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta cloud solutions architect con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar architect-plan con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                 |
|-----------|-----------------------------------------------------------------------------------------|
| ruta      | Define su papel en ruta cloud solutions architect y cómo observarlo en un sistema real. |
| cloud     | Define su papel en ruta cloud solutions architect y cómo observarlo en un sistema real. |
| solutions | Define su papel en ruta cloud solutions architect y cómo observarlo en un sistema real. |
| architect | Define su papel en ruta cloud solutions architect y cómo observarlo en un sistema real. |

### Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Ruta Cloud Solutions Architect"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta cloud solutions architect. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/272-ruta-cloud-solutions-architect/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa architect-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye architect-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 272 Ruta Cloud Solutions Architect

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega architect-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 273 — Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: assessment · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar mapeo aws, azure, google cloud, kubernetes y finops dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar mapeo aws, azure, google cloud, kubernetes y finops con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar certification-map con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                      |
|------------|--------------------------------------------------------------------------------------------------------------|
| mapeo      | Define su papel en mapeo aws, azure, google cloud, kubernetes y finops y cómo observarlo en un sistema real. |
| aws        | Define su papel en mapeo aws, azure, google cloud, kubernetes y finops y cómo observarlo en un sistema real. |
| azure      | Define su papel en mapeo aws, azure, google cloud, kubernetes y finops y cómo observarlo en un sistema real. |
| google     | Define su papel en mapeo aws, azure, google cloud, kubernetes y finops y cómo observarlo en un sistema real. |
| cloud      | Define su papel en mapeo aws, azure, google cloud, kubernetes y finops y cómo observarlo en un sistema real. |
| kubernetes | Define su papel en mapeo aws, azure, google cloud, kubernetes y finops y cómo observarlo en un sistema real. |

## Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar mapeo aws, azure, google cloud, kubernetes y finops. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/273-mapeo-aws-azure-google-cloud-kubernetes-y-finops/lab.py
```

El laboratorio selecciona el motor de práctica assessment y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa certification-map en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es una evaluación por escenarios con rúbrica y evidencia trazable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye certification-map para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 273 Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega certification-map con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 274 — Preguntas de escenario y estrategia de examen

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: assessment · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar preguntas de escenario y estrategia de examen dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar preguntas de escenario y estrategia de examen con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar exam-simulation con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                                |
|------------|--------------------------------------------------------------------------------------------------------|
| preguntas  | Define su papel en preguntas de escenario y estrategia de examen y cómo observarlo en un sistema real. |
| escenario  | Define su papel en preguntas de escenario y estrategia de examen y cómo observarlo en un sistema real. |
| estrategia | Define su papel en preguntas de escenario y estrategia de examen y cómo observarlo en un sistema real. |
| examen     | Define su papel en preguntas de escenario y estrategia de examen y cómo observarlo en un sistema real. |

### Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Preguntas de escenario y estrategia de examen"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

### Desarrollo

#### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

## 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

## 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar preguntas de escenario y estrategia de examen. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/274-preguntas-de-escenario-y-estrategia-de-examen/lab.py
```

El laboratorio selecciona el motor de práctica assessment y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa exam-simulation en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una evaluación por escenarios con rúbrica y evidencia trazable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye exam-simulation para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 274 Preguntas de escenario y estrategia de examen

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega exam-simulation con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 275 — Portafolio, evidencia, README y entrevista de sistemas

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4  
Laboratorio: assessment · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar portafolio, evidencia, readme y entrevista de sistemas dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar portafolio, evidencia, readme y entrevista de sistemas con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar professional-portfolio con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto   | Comprensión verificable                                                                                         |
|------------|-----------------------------------------------------------------------------------------------------------------|
| portafolio | Define su papel en portafolio, evidencia, readme y entrevista de sistemas y cómo observarlo en un sistema real. |
| evidencia  | Define su papel en portafolio, evidencia, readme y entrevista de sistemas y cómo observarlo en un sistema real. |
| readme     | Define su papel en portafolio, evidencia, readme y entrevista de sistemas y cómo observarlo en un sistema real. |
| entrevista | Define su papel en portafolio, evidencia, readme y entrevista de sistemas y cómo observarlo en un sistema real. |
| sistemas   | Define su papel en portafolio, evidencia, readme y entrevista de sistemas y cómo observarlo en un sistema real. |

## Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Portafolio, evidencia, README y entrevista de sistemas"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar portafolio, evidencia, readme y entrevista de sistemas. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/275-portafolio-evidencia-readme-y-entrevista-de-sistemas/lab.py
```

El laboratorio selecciona el motor de práctica assessment y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa professional-portfolio en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es una evaluación por escenarios con rúbrica y evidencia trazable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye professional-portfolio para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?



## Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 275 Portafolio, evidencia, README y entrevista de sistemas

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega professional-portfolio con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 276 — Proyecto: defensa técnica ante panel

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 8  
Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar proyecto: defensa técnica ante panel dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: defensa técnica ante panel con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar technical-defense con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                       |
|----------|-----------------------------------------------------------------------------------------------|
| proyecto | Define su papel en proyecto: defensa técnica ante panel y cómo observarlo en un sistema real. |
| defensa  | Define su papel en proyecto: defensa técnica ante panel y cómo observarlo en un sistema real. |
| técnica  | Define su papel en proyecto: defensa técnica ante panel y cómo observarlo en un sistema real. |
| ante     | Define su papel en proyecto: defensa técnica ante panel y cómo observarlo en un sistema real. |
| panel    | Define su papel en proyecto: defensa técnica ante panel y cómo observarlo en un sistema real. |

### Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Proyecto: defensa técnica ante panel"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: defensa técnica ante panel. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/276-proyecto-defensa-tecnica-ante-panel/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa technical-defense en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye technical-defense para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 276 Proyecto: defensa técnica ante panel

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega technical-defense con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# Parte 23 — Capstones por industria y defensa final

← Parte 22 · Índice completo · Fin

Nivel: experto · Clases: 12 · Duración sugerida: 6–8 semanas

Integrar arquitectura, implementación, operación, costo y comunicación en casos industriales completos.

## Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

## Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

## Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

## Clases

| ID  | Clase                                                  | Laboratorio  | Horas |
|-----|--------------------------------------------------------|--------------|-------|
| 277 | Capstone retail: comercio multi-región                 | capstone     | 8     |
| 278 | Capstone financiero: pagos, auditoría y recuperación   | capstone     | 8     |
| 279 | Capstone salud: privacidad e interoperabilidad         | capstone     | 8     |
| 280 | Capstone sector público: soberanía y continuidad       | capstone     | 8     |
| 281 | Capstone media: streaming y distribución global        | capstone     | 8     |
| 282 | Capstone industria: IoT, edge y operación desconectada | capstone     | 8     |
| 283 | Capstone SaaS: multi-tenancy y unit economics          | capstone     | 8     |
| 284 | Capstone datos e IA: plataforma gobernada              | capstone     | 8     |
| 285 | Game day integrado y respuesta a incidentes            | chaos        | 4     |
| 286 | Revisión Well-Architected multi-proveedor              | architecture | 4     |
| 287 | Paquete de evidencia, costos y riesgos residuales      | assessment   | 4     |
| 288 | Defensa final, retrospectiva y plan de 12 meses        | capstone     | 8     |

## Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

## Pauta bibliográfica

- Building Evolutionary Architectures — Ford, Parsons y Kua.
- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.

## 277 — Capstone retail: comercio multi-región

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar capstone retail: comercio multi-región dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone retail: comercio multi-región con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar retail-capstone con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto | Comprensión verificable                                                                         |
|----------|-------------------------------------------------------------------------------------------------|
| capstone | Define su papel en capstone retail: comercio multi-región y cómo observarlo en un sistema real. |
| retail   | Define su papel en capstone retail: comercio multi-región y cómo observarlo en un sistema real. |
| comercio | Define su papel en capstone retail: comercio multi-región y cómo observarlo en un sistema real. |
| multi    | Define su papel en capstone retail: comercio multi-región y cómo observarlo en un sistema real. |
| región   | Define su papel en capstone retail: comercio multi-región y cómo observarlo en un sistema real. |

### Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Capstone retail: comercio multi-región"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone retail: comercio multi-región. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/277-capstone-retail-comercio-multi-region/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa retail-capstone en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye retail-capstone para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 277 Capstone retail: comercio multi-región

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega retail-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 278 — Capstone financiero: pagos, auditoría y recuperación

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar capstone financiero: pagos, auditoría y recuperación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone financiero: pagos, auditoría y recuperación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar finance-capstone con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                                                       |
|--------------|---------------------------------------------------------------------------------------------------------------|
| capstone     | Define su papel en capstone financiero: pagos, auditoría y recuperación y cómo observarlo en un sistema real. |
| financiero   | Define su papel en capstone financiero: pagos, auditoría y recuperación y cómo observarlo en un sistema real. |
| pagos        | Define su papel en capstone financiero: pagos, auditoría y recuperación y cómo observarlo en un sistema real. |
| auditoría    | Define su papel en capstone financiero: pagos, auditoría y recuperación y cómo observarlo en un sistema real. |
| recuperación | Define su papel en capstone financiero: pagos, auditoría y recuperación y cómo observarlo en un sistema real. |

### Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone financiero: pagos, auditoría y recuperación"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone financiero: pagos, auditoría y recuperación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/278-capstone-financiero-pagos-auditoria-y-recuperacion/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa finance-capstone en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye finance-capstone para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.
- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 278 Capstone financiero: pagos, auditoría y recuperación

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega finance-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 279 — Capstone salud: privacidad e interoperabilidad

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar capstone salud: privacidad e interoperabilidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone salud: privacidad e interoperabilidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar health-capstone con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto          | Comprensión verificable                                                                                 |
|-------------------|---------------------------------------------------------------------------------------------------------|
| capstone          | Define su papel en capstone salud: privacidad e interoperabilidad y cómo observarlo en un sistema real. |
| salud             | Define su papel en capstone salud: privacidad e interoperabilidad y cómo observarlo en un sistema real. |
| privacidad        | Define su papel en capstone salud: privacidad e interoperabilidad y cómo observarlo en un sistema real. |
| e                 | Define su papel en capstone salud: privacidad e interoperabilidad y cómo observarlo en un sistema real. |
| interoperabilidad | Define su papel en capstone salud: privacidad e interoperabilidad y cómo observarlo en un sistema real. |

### Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone salud: privacidad e interoperabilidad"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone salud: privacidad e interoperabilidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/279-capstone-salud-privacidad-e-interoperabilidad/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa health-capstone en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `health-capstone` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 279 Capstone salud: privacidad e interoperabilidad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega health-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 280 — Capstone sector público: soberanía y continuidad

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar capstone sector público: soberanía y continuidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone sector público: soberanía y continuidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar public-sector-capstone con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                                   |
|-------------|-----------------------------------------------------------------------------------------------------------|
| capstone    | Define su papel en capstone sector público: soberanía y continuidad y cómo observarlo en un sistema real. |
| sector      | Define su papel en capstone sector público: soberanía y continuidad y cómo observarlo en un sistema real. |
| público     | Define su papel en capstone sector público: soberanía y continuidad y cómo observarlo en un sistema real. |
| soberanía   | Define su papel en capstone sector público: soberanía y continuidad y cómo observarlo en un sistema real. |
| continuidad | Define su papel en capstone sector público: soberanía y continuidad y cómo observarlo en un sistema real. |

### Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone sector público: soberanía y continuidad"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone sector público: soberanía y continuidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/280-capstone-sector-publico-soberania-y-continuidad/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa public-sector-capstone en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye public-sector-capstone para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 280 Capstone sector público: soberanía y continuidad

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega public-sector-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

# 281 — Capstone media: streaming y distribución global

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

## Propósito

Comprender y aplicar capstone media: streaming y distribución global dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

## Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone media: streaming y distribución global con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar media-capstone con evidencia, límites y criterio de reversión.

## Conceptos centrales

| Concepto     | Comprensión verificable                                                                                  |
|--------------|----------------------------------------------------------------------------------------------------------|
| capstone     | Define su papel en capstone media: streaming y distribución global y cómo observarlo en un sistema real. |
| media        | Define su papel en capstone media: streaming y distribución global y cómo observarlo en un sistema real. |
| streaming    | Define su papel en capstone media: streaming y distribución global y cómo observarlo en un sistema real. |
| distribución | Define su papel en capstone media: streaming y distribución global y cómo observarlo en un sistema real. |
| global       | Define su papel en capstone media: streaming y distribución global y cómo observarlo en un sistema real. |

## Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

## ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Capstone media: streaming y distribución global"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone media: streaming y distribución global. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/281-capstone-media-streaming-y-distribucion-global/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa media-capstone en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

### Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye media-capstone para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

### Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 281 Capstone media: streaming y distribución global

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega media-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 282 — Capstone industria: IoT, edge y operación desconectada

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar capstone industria: iot, edge y operación desconectada dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone industria: iot, edge y operación desconectada con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar industry-capstone con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto     | Comprensión verificable                                                                                         |
|--------------|-----------------------------------------------------------------------------------------------------------------|
| capstone     | Define su papel en capstone industria: iot, edge y operación desconectada y cómo observarlo en un sistema real. |
| industria    | Define su papel en capstone industria: iot, edge y operación desconectada y cómo observarlo en un sistema real. |
| iot          | Define su papel en capstone industria: iot, edge y operación desconectada y cómo observarlo en un sistema real. |
| edge         | Define su papel en capstone industria: iot, edge y operación desconectada y cómo observarlo en un sistema real. |
| operación    | Define su papel en capstone industria: iot, edge y operación desconectada y cómo observarlo en un sistema real. |
| desconectada | Define su papel en capstone industria: iot, edge y operación desconectada y cómo observarlo en un sistema real. |

### Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone industria: IoT, edge y operación desconectada"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone industria: iot, edge y operación desconectada. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/282-capstone-industria-iot-edge-y-operacion-desconectada/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa industry-capstone en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye industry-capstone para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.
- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 282 Capstone industria: IoT, edge y operación desconectada

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega industry-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 283 — Capstone SaaS: multi-tenancy y unit economics

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar capstone saas: multi-tenancy y unit economics dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone saas: multi-tenancy y unit economics con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar saas-capstone con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto  | Comprensión verificable                                                                                |
|-----------|--------------------------------------------------------------------------------------------------------|
| capstone  | Define su papel en capstone saas: multi-tenancy y unit economics y cómo observarlo en un sistema real. |
| saas      | Define su papel en capstone saas: multi-tenancy y unit economics y cómo observarlo en un sistema real. |
| multi     | Define su papel en capstone saas: multi-tenancy y unit economics y cómo observarlo en un sistema real. |
| tenancy   | Define su papel en capstone saas: multi-tenancy y unit economics y cómo observarlo en un sistema real. |
| unit      | Define su papel en capstone saas: multi-tenancy y unit economics y cómo observarlo en un sistema real. |
| economics | Define su papel en capstone saas: multi-tenancy y unit economics y cómo observarlo en un sistema real. |

### Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Capstone SaaS: multi-tenancy y unit economics"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone saas: multi-tenancy y unit economics. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/283-capstone-saas-multi-tenancy-y-unit-economics/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa saas-capstone en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `saas-capstone` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 283 Capstone SaaS: multi-tenancy y unit economics

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega saas-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 284 — Capstone datos e IA: plataforma gobernada

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar capstone datos e ia: plataforma gobernada dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone datos e ia: plataforma gobernada con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar data-ai-capstone con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                            |
|------------|----------------------------------------------------------------------------------------------------|
| capstone   | Define su papel en capstone datos e ia: plataforma gobernada y cómo observarlo en un sistema real. |
| datos      | Define su papel en capstone datos e ia: plataforma gobernada y cómo observarlo en un sistema real. |
| e          | Define su papel en capstone datos e ia: plataforma gobernada y cómo observarlo en un sistema real. |
| ia         | Define su papel en capstone datos e ia: plataforma gobernada y cómo observarlo en un sistema real. |
| plataforma | Define su papel en capstone datos e ia: plataforma gobernada y cómo observarlo en un sistema real. |
| gobernada  | Define su papel en capstone datos e ia: plataforma gobernada y cómo observarlo en un sistema real. |

### Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Capstone datos e IA: plataforma gobernada"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone datos e ia: plataforma gobernada. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/284-capstone-datos-e-ia-plataforma-gobernada/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa data-ai-capstone en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye data-ai-capstone para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

### Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 284 Capstone datos e IA: plataforma gobernada

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega data-ai-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 285 — Game day integrado y respuesta a incidentes

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 4 Laboratorio: chaos · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar game day integrado y respuesta a incidentes dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar game day integrado y respuesta a incidentes con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar final-gameday con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                              |
|------------|------------------------------------------------------------------------------------------------------|
| game       | Define su papel en game day integrado y respuesta a incidentes y cómo observarlo en un sistema real. |
| day        | Define su papel en game day integrado y respuesta a incidentes y cómo observarlo en un sistema real. |
| integrado  | Define su papel en game day integrado y respuesta a incidentes y cómo observarlo en un sistema real. |
| respuesta  | Define su papel en game day integrado y respuesta a incidentes y cómo observarlo en un sistema real. |
| incidentes | Define su papel en game day integrado y respuesta a incidentes y cómo observarlo en un sistema real. |

### Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Game day integrado y respuesta a incidentes"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar game day integrado y respuesta a incidentes. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/285-game-day-integrado-y-respuesta-a-incidentes/lab.py
```

El laboratorio selecciona el motor de práctica chaos y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa final-gameday en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

### Evidencia esperada

El artefacto principal es una hipótesis de resiliencia y criterio de abortar. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye final-gameday para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

### Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 285 Game day integrado y respuesta a incidentes

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega final-gameday con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 286 — Revisión Well-Architected multi-proveedor

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar revisión well-architected multi-proveedor dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar revisión well-architected multi-proveedor con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar final-architecture-review con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto    | Comprensión verificable                                                                            |
|-------------|----------------------------------------------------------------------------------------------------|
| revisión    | Define su papel en revisión well-architected multi-proveedor y cómo observarlo en un sistema real. |
| well        | Define su papel en revisión well-architected multi-proveedor y cómo observarlo en un sistema real. |
| architected | Define su papel en revisión well-architected multi-proveedor y cómo observarlo en un sistema real. |
| multi       | Define su papel en revisión well-architected multi-proveedor y cómo observarlo en un sistema real. |
| proveedor   | Define su papel en revisión well-architected multi-proveedor y cómo observarlo en un sistema real. |

### Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Revisión Well-Architected multi-proveedor"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar revisión well-architected multi-proveedor. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/286-revision-well-architected-multi-proveedor/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa final-architecture-review en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `final-architecture-review` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 286 Revisión Well-Architected multi-proveedor

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega final-architecture-review con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 287 — Paquete de evidencia, costos y riesgos residuales

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 4 Laboratorio: assessment · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar paquete de evidencia, costos y riesgos residuales dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar paquete de evidencia, costos y riesgos residuales con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar final-evidence-pack con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto   | Comprensión verificable                                                                                    |
|------------|------------------------------------------------------------------------------------------------------------|
| paquete    | Define su papel en paquete de evidencia, costos y riesgos residuales y cómo observarlo en un sistema real. |
| evidencia  | Define su papel en paquete de evidencia, costos y riesgos residuales y cómo observarlo en un sistema real. |
| costos     | Define su papel en paquete de evidencia, costos y riesgos residuales y cómo observarlo en un sistema real. |
| riesgos    | Define su papel en paquete de evidencia, costos y riesgos residuales y cómo observarlo en un sistema real. |
| residuales | Define su papel en paquete de evidencia, costos y riesgos residuales y cómo observarlo en un sistema real. |

### Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Paquete de evidencia, costos y riesgos residuales"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```



## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar paquete de evidencia, costos y riesgos residuales. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/287-paquete-de-evidencia-costos-y-riesgos-residuales/lab.py
```

El laboratorio selecciona el motor de práctica assessment y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa final-evidence-pack en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

## Evidencia esperada

El artefacto principal es una evaluación por escenarios con rúbrica y evidencia trazable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

## Reto verificable

Construye `final-evidence-pack` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

## ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

## ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

## ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

## ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

## Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 287 Paquete de evidencia, costos y riesgos residuales

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega final-evidence-pack con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |

## 288 — Defensa final, retrospectiva y plan de 12 meses

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

### Propósito

Comprender y aplicar defensa final, retrospectiva y plan de 12 meses dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

### Resultados de aprendizaje

Al finalizar podrás:

1. Explicar defensa final, retrospectiva y plan de 12 meses con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar final-defense con evidencia, límites y criterio de reversión.

### Conceptos centrales

| Concepto      | Comprensión verificable                                                                                  |
|---------------|----------------------------------------------------------------------------------------------------------|
| defensa       | Define su papel en defensa final, retrospectiva y plan de 12 meses y cómo observarlo en un sistema real. |
| final         | Define su papel en defensa final, retrospectiva y plan de 12 meses y cómo observarlo en un sistema real. |
| retrospectiva | Define su papel en defensa final, retrospectiva y plan de 12 meses y cómo observarlo en un sistema real. |
| plan          | Define su papel en defensa final, retrospectiva y plan de 12 meses y cómo observarlo en un sistema real. |
| 12            | Define su papel en defensa final, retrospectiva y plan de 12 meses y cómo observarlo en un sistema real. |
| meses         | Define su papel en defensa final, retrospectiva y plan de 12 meses y cómo observarlo en un sistema real. |

### Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

### ■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Defensa final, retrospectiva y plan de 12 meses"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E["¿Cumple seguridad, SLO y costo?"]
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

## Desarrollo

### 1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

### 2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

### 3. Compensaciones que deben quedar visibles

| Dimensión     | Pregunta de diseño                                                    |
|---------------|-----------------------------------------------------------------------|
| Confiabilidad | ¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?            |
| Seguridad     | ¿Qué identidad actúa y cuál es el mínimo privilegio necesario?        |
| Costo         | ¿Cuál es la unidad de consumo y qué hace crecer la factura?           |
| Operación     | ¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor? |
| Portabilidad  | ¿Qué contrato es estándar y qué decisión es específica del proveedor? |

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

## Ejemplo trabajado

Una plataforma de pedidos necesita aplicar defensa final, retrospectiva y plan de 12 meses. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

## Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/288-defensa-final-retrospectiva-y-plan-de-12-meses/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa final-defense en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

### Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

### Reto verificable

Construye final-defense para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

### ■ Criterio de aceptación

- [ ] `lab.py` termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

### ■ ■ Errores frecuentes

| Síntoma                                            | Causa probable                                   | Corrección                                                       |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| El diseño enumera servicios pero no requisitos     | Se comenzó por el catálogo del proveedor         | Reescribe primero escenarios y restricciones medibles.           |
| La demo funciona una vez y se declara lista        | Se confundió ejecución con evidencia operacional | Añade repetición, fallo, telemetría y recuperación.              |
| Todo tiene permisos administrativos                | El laboratorio heredó credenciales humanas       | Usa identidad de workload y prueba explícitamente la denegación. |
| No se puede explicar la factura                    | Faltan unidades y ownership de costo             | Etiqueta, estima por unidad y define presupuesto o alerta.       |
| La solución se llama multi-cloud pero replica todo | Portabilidad se confundió con duplicación        | Define qué riesgo se mitiga y porta solo el contrato necesario.  |

### ■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

### ■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

### Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

## Evaluación — 288 Defensa final, retrospectiva y plan de 12 meses

### Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

### Reto verificable

Entrega final-defense con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

### Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

### Escala

| Nivel             | Evidencia                                                |
|-------------------|----------------------------------------------------------|
| A — competente    | Cumple todo y defiende trade-offs con datos.             |
| B — en desarrollo | Cumple lo esencial; falta profundidad en una dimensión.  |
| C — inicial       | Artefacto parcial o conclusión sin evidencia suficiente. |
| 0                 | Sin entrega reproducible.                                |